

Desarrollo de funcionalidad para el cliente HTML



© 2013 Meta4 Spain, S.A. Se reservan todos los derechos.

AVISO: Este manual está protegido por la legislación referente a propiedad intelectual e industrial y por tratados internacionales. La utilización permitida de esta documentación queda limitada a su uso en conexión con el producto, y todo uso no autorizado será perseguido de acuerdo con la legislación aplicable. Se prohíbe su copia, modificación, reproducción o distribución sin permiso del titular.

Meta4 PeopleNet © 1999 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 KnowNet © 1996 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 e-mind © 2001 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 PeopleNet Ksystem © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 t.innova © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4[®], Meta4Mind[®], Meta4 PeopleNet[®], Meta4 KnowNet[®], Meta4 e-mind[®], Meta4 PeopleNet Ksystem[®] y Meta4 t.innova[®] son marcas registradas propiedad de Meta4Spain, S.A.

Otros nombres de compañías, productos o servicios son marcas registradas o nombres comerciales de sus respectivos propietarios.

Meta4 Spain, S.A.
Centro Europa Empresarial
Edificio Roma
C/ Rozabella, 8
Ctra. de La Coruña, km 24,200
28290 Las Rozas, Madrid
ESPAÑA
<http://www.meta4.com>

Fecha de creación: enero de 2003.

Fecha de la última publicación: diciembre de 2013.

Tabla de contenidos

1 Clientes HTML

■ Introducción	9
■ La arquitectura del cliente HTML	13
■ Tareas	14
Estructura de directorios	15
■ Subsesiones	18
■ Primeros pasos	19
Comprobación del entorno	19
Acceso a la aplicación	23

2 Diseño de páginas básicas

■ Páginas JSP y funcionalidad	25
■ Estructura básica de las páginas JSP	28
Comienzo de página	28
Servicio de datos	29
Presentación de datos	30
Fin de página	30
■ Recuperación y presentación de datos	31
Paso 1: Definir el entorno de la transacción	31
Paso 2: Definir el puesto	32
Paso 3: Presentación de datos	37
Paso 4: Presentación	41
■ Almacenamiento de registros	42
■ Sugerencias de diseño	46
Sugerencias de diseño JSP	46
Las sugerencias son aplicables a la tecnología Meta4	46

3 Diseño de páginas avanzadas

■ Introducción	47
■ Introducción y lectura de parámetros	48
■ Cambio de la posición del registro actual	51
■ Creación de bucles con M4Tags	53
■ Filtros y ordenación	58

■ Cambio de la presentación de los datos	65
■ Diseño de una página de inicio de sesión	69
Página de inicio de sesión	69
Inicio de sesión anónimo	71
Inicio de sesión para dispositivos WML	73
Opciones de sesión	75
■ M4Tags de bolsas de sesión	77
■ Gestión de archivos	79
Almacenamiento de archivos	82
Recuperación de archivos	83
■ Gestión de errores	86
■ Herramientas de presentación	88
■ Propiedades de transacciones avanzadas	90
■ Referencia de traducciones	92
Definir la traducción	92
Traducir	94
Presentar el campo traducido	95
■ Diseño de una página JSP con enlaces entre sitios	100
■ Depuración de páginas JSP	101
■ Sugerencias de diseño	104
Recomendaciones de seguridad	105

4 Filtros para el entorno Web

■ Acerca de este capítulo	111
■ Introducción	112
■ Filtros HTML	113
■ Filtros HTML dinámicos	123

5 Uso de gráficos para desarrollar páginas en el entorno Web

■ Acerca de este capítulo	127
■ Introducción	128
Gráficos de uno, dos y tres ejes	128
■ Creación de gráficos de dos ejes en HTML	129
Caso 1: Gráfico de columnas del número de hombres y mujeres en cada país	129
Caso 2: Gráfico de tarta de hombres y mujeres por países	131
Caso 3: Conmutación de los ejes X e Y	132
Caso 4: Rotación del texto de los ejes	133
Caso 5: Gráfico de categorías	133
Caso 6: Configuración de valores en el eje Y	136

Caso 7: Control de escala del eje Y	137
Caso 8: Tipos de fuentes	139

6 Salida HTML de informes

■ Acerca de este capítulo	141
Introducción	141
■ Características de la salida HTML	143
Navegación	144
Tabla de contenidos	145
Zoom	147
■ Tipos de salida HTML	148
■ Parametrización de la salida HTML	149
Plantilla HTML	149
Salida HTML	159
■ Referencia rápida de parámetros HTML	161
Descripción general	161
Ejecución del informe desde presentaciones	163

7 Parametrización del sistema de publicación en cliente HTML

■ Acerca de este capítulo	167
■ Introducción	168
Procedimiento general	168
Publicación de documentos	168
Disponibilidad de documentos en el entorno HTML	170
Gestión de informes publicados en distintos entornos de empresa	171
■ Parametrización de la publicación de documentos	174
Parametrización del Meta4Object de datos	174
Parametrización del listado, visualización y borrado de documentos	176
■ Parametrización del acceso a la publicación en HTML	181
Tarea TCPUBDOCS	181
Páginas JSP suministradas	181
Tarea DEL_EXPIRED_DOCS	186

8 Integración con el gestor documental

■ Acerca de este capítulo	187
■ Introducción	188
■ Integración del include para gestión documental en el SSE/SSM	189

Configuración del include en modo formulario.....	193
Configuración del include en modo tabla.....	196
Configuración del borrado de documentos	200
Funciones para mostrar y descargar documentos.....	202
■ Uso de la tarea para ver un documento y subirlo al gestor documental	204

9 Referencia de la biblioteca M4TagLib

■ Semántica de referencia	207
■ M4Tags de servicio de datos.....	208
M4Tags de servicio de datos básicos	208
M4Tags de filtros y ordenación	219
M4Tags de asignación	224
M4Tags de herramientas de servicio de datos.....	226
■ M4Tags de presentación	229
M4Tags de presentación básicos.....	229
M4Tags de iteración.....	237
M4Tags de consulta de traducciones.....	241
■ M4Tags de seguridad y sesión	244
M4Tags de seguridad.....	244
M4Tags de bolsas de sesión	247
■ Etiquetas de informes	251
Informes.....	251
■ M4Tags de herramientas	255
■ M4Tags de configuración de la aplicación.....	262
■ Referencia rápida de M4Tags	267

10 Referencia del paquete M4Operations

■ Introducción	275
Uso de m4operations en páginas JSP.....	276
■ Clase M4Operations	280
Campos	280
Constructores.....	284
Métodos	284
■ Clase OperationException.....	307
Constructores.....	307
Métodos	307
■ Clase LogMessage	308
Constructores.....	308
Métodos	308

■ Clase SortElement	310
Campos	310
Constructores	310
Métodos	310

A Apéndice: Especificación de datos

B Apéndice: Referencia de sintaxis del formato

■ Fuentes	313
Sintaxis del formato de horas	313
Sintaxis del formato de números	313
Sintaxis del formato de expresiones de cadena	313

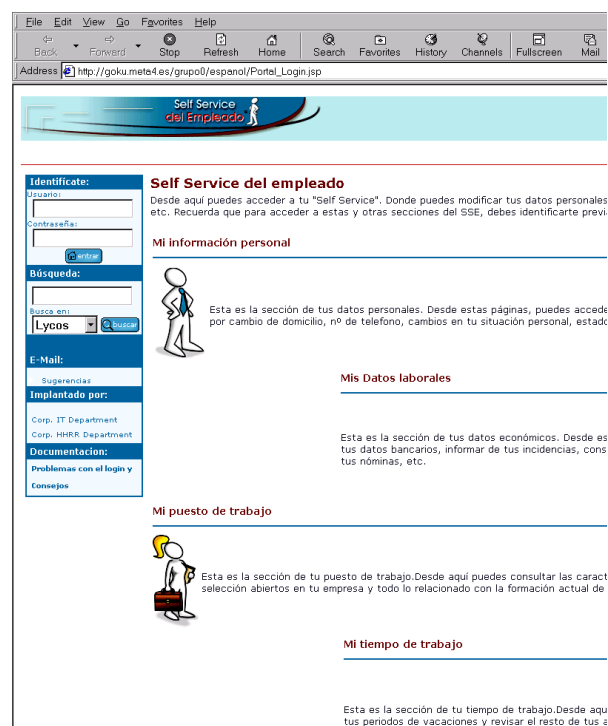
Cientes HTML

1

Introducción

En la arquitectura de tres capas de las aplicaciones Meta4, el cliente HTML (o cliente ligero) representa una de las soluciones tecnológicas más avanzadas que se pueden encontrar actualmente en el mercado. A diferencia de otros clientes Meta4 (como Windows), el único requisito es disponer de un explorador estándar (como Internet Explorer) instalado en el cliente.

Figura 1. Cliente HTML de Meta4



No hay instalaciones en cliente, por lo que se reduce el coste total de propiedad (TCO). Esto se debe a que la mayoría del proceso se realiza en el servidor, dado que el cliente HTML usa páginas JavaServer (JSP).

De este modo, cuando el usuario necesita realizar una operación o consulta, el navegador estándar emite una petición al servidor Web que, a su vez, y dependiendo de la consulta recibida, realiza una petición de datos al servidor de aplicaciones.

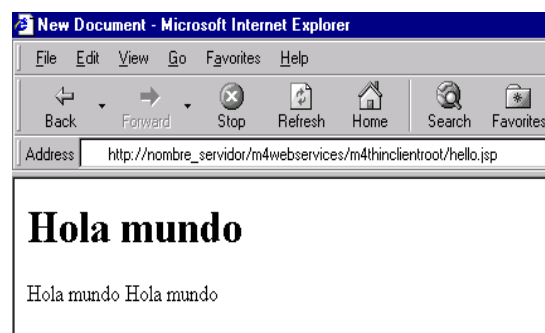
Los datos de respuesta van entonces desde el servidor de aplicaciones al servidor Web, donde se genera la página HTML que se envía al navegador.

EJEMPLO

Para explicar este concepto, utilizaremos una página de ejemplo llamada *hola.jsp*. Esta página contiene las siguientes líneas de código:

```
<HTML>
...
<BODY>
<H1> Hola mundo </H1>
<%
    int i;
    for(i=1;i<3;i++) out.println("Hola mundo");
%>
</BODY>
...
</HTML>
```

Figura 2. Resultado de la ejecución de la página *hola.jsp*



Cuando en tiempo de ejecución el servidor Web recibe por primera vez la petición de la página *hola.jsp*, el entorno JSP construye y compila un *servlet* para esta página.

Las etiquetas HTML estándar se codifican directamente en la página HTML que devuelve el servidor Web. El código que hay entre las etiquetas `<%` y `%>` es un código Java para el *servlet*. De esta forma, en peticiones posteriores de la misma página, el entorno JSP tan sólo necesitará invocar al *servlet* creado.

Además de los códigos Java y las etiquetas HTML, los desarrolladores JSP pueden usar etiquetas personalizadas para ejecutar tareas específicas. Por ejemplo, enviar la fecha y hora del servidor Web al explorador del cliente.



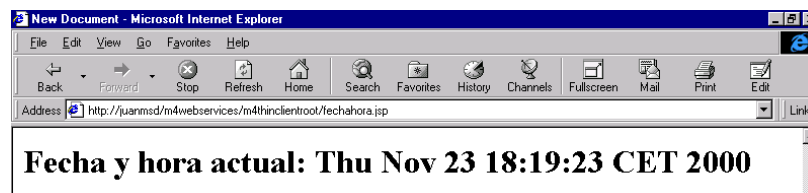
Las etiquetas dinámicas, también llamadas *taglets* en versiones anteriores de clientes HTML, cumplen las especificaciones JSP 1.1 estándar. Estos estándares se desarrollaron siguiendo el *Java Community Process*. Según este estándar, el uso de etiquetas dinámicas consiste en definir las e incluirlas dentro de una biblioteca.

EJEMPLO

Para explicar este concepto, utilizaremos una página de ejemplo llamada *DateTime.jsp*. Esta página contiene el siguiente código:

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<HTML>
...
<BODY>
<H1> Fecha y hora actual: <m4:date/> </H1>
</BODY>
...
</HTML>
```

Figura 3. Resultado de la ejecución de la página *datetime.jsp*



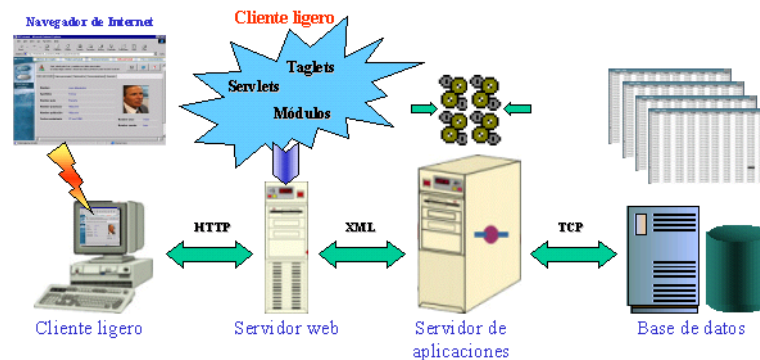
La etiqueta `<m4:date>` es una de las etiquetas personalizadas que facilita Meta4. En apartados posteriores se describen estas etiquetas.

Junto con el cliente HTML, Meta4 facilita un amplio juego de etiquetas personalizadas conocidas como *M4Tags*, así como clases Java específicas que facilitan su funcionalidad. Esto reduce al mínimo la complejidad de diseñar páginas HTML dinámicas y permite a los desarrolladores JSP obtener acceso al servidor de aplicaciones de Meta4. Para obtener más información, consulte los capítulos **Referencia de la biblioteca M4TagLib** y **Referencia del paquete M4Operations** de este manual.

La arquitectura del cliente HTML

El entorno necesario para ejecutar un cliente HTML es un conjunto de servicios (*servlets* y *etiquetas personalizadas*) asociados al servidor Web para ofrecer un sistema de comunicación abierto a cualquier navegador.

Figura 4. Arquitectura del cliente HTML en Meta4



Para obtener acceso a la información que contiene la base de datos, tan sólo es necesario disponer de un navegador que reconozca el lenguaje HTML (como Internet Explorer o Netscape Navigator) y se comunique directamente con el servidor Web.

Las páginas JSP del servidor Web contienen M4Tags que en última instancia representan peticiones al servidor de aplicaciones. La comunicación entre el servidor Web y el servidor de aplicaciones se realiza a través de XML.

Una vez en la base de datos, y ya realizadas las tareas solicitadas, el servidor de aplicaciones devuelve el resultado al servidor Web, donde el servicio de presentaciones se encarga de transformar esta respuesta en código HTML. Este código es interpretado por el navegador Web, que muestra al usuario la información solicitada a través de una interfaz gráfica de fácil acceso.

Tareas

Una tarea es una entidad que contiene páginas agrupadas por funcionalidad así como el juego de Meta4Objects con los que trabajan dichas páginas. Cuando las JSP están agrupadas por funcionalidad, el cliente HTML de Meta4 puede ejecutar una serie de acciones relacionadas.

Una aplicación está diseñada para facilitar funcionalidad específica con ciertas acciones que producen funcionalidad adicional. Cada tipo de subfuncionalidad o función especializada es una tarea.

EJEMPLO

Las páginas que actualizan o muestran información correspondiente al estado civil o al domicilio de una persona están relacionadas por el hecho de mostrar datos personales.

Sin embargo, los datos relevantes a los cursos de formación impartidos por la empresa no son significativos para los datos personales indicados, por lo que constituyen una tarea independiente de la anterior.

Las tareas se usan para conceder permisos de uso de estas páginas que se agrupan por funcionalidad. Cuando se conecta un usuario y se valida su contraseña, el servidor Web genera una lista de control de acceso (ACL). Esta lista muestra los roles disponibles para ese usuario. Cuando éste solicita una página, su solicitud se envía al servicio de seguridad, que comprueba si alguno de los roles del usuario puede consultar la tarea o el grupo de páginas relacionado.

EJEMPLO

Se toma como ejemplo una aplicación sencilla diseñada para gestionar las vacaciones de los empleados. Los usuarios que obtengan acceso a la misma con el rol de empleado tendrán permisos tan sólo para consultar o solicitar vacaciones, mientras que el responsable de aprobarlas también podrá aprobarlas.

Estructura de directorios

Las tareas se organizan en una jerarquía de carpetas que representa la estructura interna de la aplicación.

Cada carpeta corresponde a una tarea, y agrupa un conjunto de páginas con funciones afines. De este modo se puede, por ejemplo, distinguir la tarea que gestiona los datos personales de los individuos, la que gestiona los cursos de formación, etc.

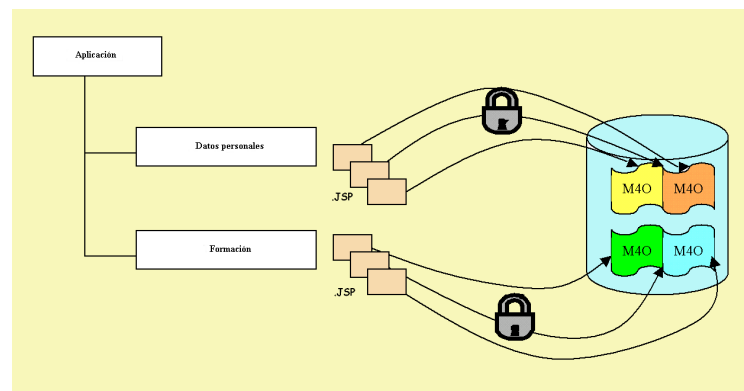
EJEMPLO

La siguiente imagen muestra un ejemplo de la estructura de carpetas de una aplicación que sólo tiene dos tareas.

Las dos carpetas corresponden a la tarea de gestión de datos personales y a la de gestión de la formación.

Cada tarea agrupa a un conjunto de JSP que solicitan servicios de datos y, en ejecución, inician Meta4Objects en el servidor.

Figura 5. Estructura jerárquica de tareas sencilla



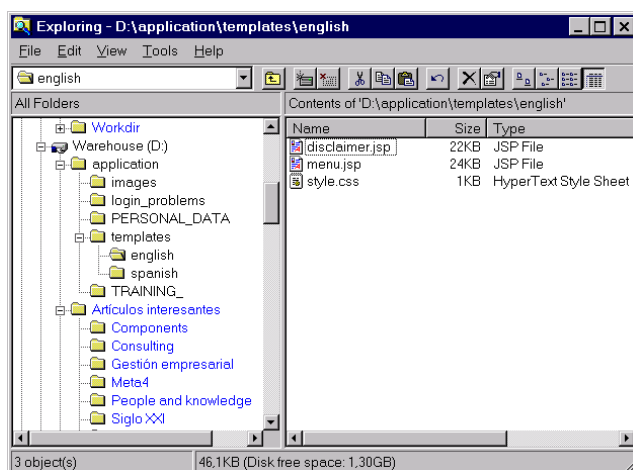
Además de tareas, una aplicación puede tener carpetas o directorios adicionales, con archivos que se van a compartir entre el conjunto de páginas como, por ejemplo, imágenes, hojas de estilo, encabezamientos y pies de página, etc.

Recuerde que cualquier aplicación de cliente HTML está diseñada para funcionar con diversos idiomas, incluso si sólo utiliza uno. Por lo tanto, se recomienda que distribuya en carpetas los recursos. De este modo, no se mezclan en una misma carpeta páginas de distintos idiomas, lo que puede dar lugar a confusiones y complicar el proceso de traducción.

EJEMPLO

Siguiendo con la estructura de nuestro ejemplo, se tendría el siguiente conjunto de carpetas:

Figura 6. Ejemplo de estructuración por idiomas



En la imagen superior, observará que se han agregado diversas carpetas a PERSONAL_DATA y TRAINING:

- La primera contiene las imágenes incluidas en las páginas.
- La segunda incluye las páginas HTML que aparecen si hay problemas de conexión.
- La tercera almacena plantillas de diseño, organizadas en los idiomas que admite la aplicación (en este caso, inglés y español).

Cuando se define una tarea, se le asocia una página de inicio. Suele ser la página donde comienza la navegación por la tarea. También puede incluir funciones de índice de tareas desde las que se puede acceder a páginas más específicas de otras tareas.

Se puede usar la página de inicio de la tarea para declarar y cargar aquellos Meta4Objects que van a ser denominador común de todas las páginas correspondientes a dicha tarea.

Se recomienda que cada página remita a un nodo de Meta4Object. Aunque no hay restricciones técnicas en cuanto a obtener acceso a distintos nodos de un Meta4Object, este diseño mejora los tiempos globales de respuesta. Asimismo, facilita la codificación de la página y mejora también la sencillez de presentación de los datos.

Para lograr este objetivo, se utilizan varias páginas para arropar una misma funcionalidad. Por ejemplo, una página para mostrar los datos, otra para actualizarlos, una tercera para información acerca de la actualización, etc. En

particular, es aconsejable nombrar las páginas de modo que se puedan localizar rápidamente según su funcionalidad.

EJEMPLO

Siguiendo con la aplicación, la carpeta de español de la tarea PERSONAL_DATA incluye las siguientes páginas:

- menu_person.jsp, que llama al Meta4Object STD_PERSON.
- visualize_data.jsp, que utiliza el Meta4Object STD_PERSON y llama al Meta4Object STD_GENDER.
- edit_status.jsp, que permite actualizar el estado civil y hace uso de los Meta4Objects cargados.
- actualize_data.jsp, que realiza los cambios contra el repositorio y vuelve a la página de visualización.

Estas páginas, con el mismo nombre, se repetirán en los distintos directorios de idiomas que cuelguen de la tarea seleccionada. Los contenidos se traducen a los idiomas correspondientes.

La página de inicio de la tarea será menu_person.jsp.

Subsesiones

Cuando el servidor Web inicia una transacción (una solicitud de datos o de ejecución de un método) con el servicio XML del servidor de aplicaciones, este servidor guarda la información con la solicitud. Así se mejora la eficacia de futuras transacciones. Esto se consigue a través de subsesiones, que son espacios de memoria del servidor de aplicaciones que guardan información sobre los objetos usados en las transacciones. Las subsesiones funcionan como cachés: la información se guarda en el disco duro del servidor de aplicaciones una vez se alcanza en la memoria el número máximo de subsesiones.

Hay dos formas de limitar los recursos del servidor de aplicaciones que consumen las subsesiones:

- Los desarrolladores pueden controlar el uso de subsesiones con un parámetro en cada página.
- Los administradores pueden limitar el número máximo de subsesiones de la memoria (consulte el manual del servidor de aplicaciones y el sitio del servidor Meta4).

Primeros pasos

Comprobación del entorno

El cliente HTML sólo necesita tener instalado un navegador estándar. En cambio, el servidor requiere tener en funcionamiento los siguientes componentes:

- Servidor de aplicaciones de Meta4
- Motor de servlets (contenedor web J2EE) con componentes Meta4
- Servidor Web

El servidor de aplicaciones no necesita estar en la misma máquina que el motor de *servlets* y el servidor Web. Debe iniciar cada componente en el orden indicado arriba. Es decir, inicie primero el servidor de aplicaciones Meta4, a continuación el motor de *servlets* y, por último, el servidor Web. Es muy importante que compruebe que el servidor de aplicaciones está preparado antes de realizar peticiones de datos.

Tras iniciar todos estos componentes, compruebe que funcionan correctamente. Para ello, cargue la página de prueba principal para acceder a todas las pruebas.

- *http://server_name/sitetest/language/testmain.jsp*

donde *server_name* es el nombre del servidor Web y *language* el idioma correspondiente.

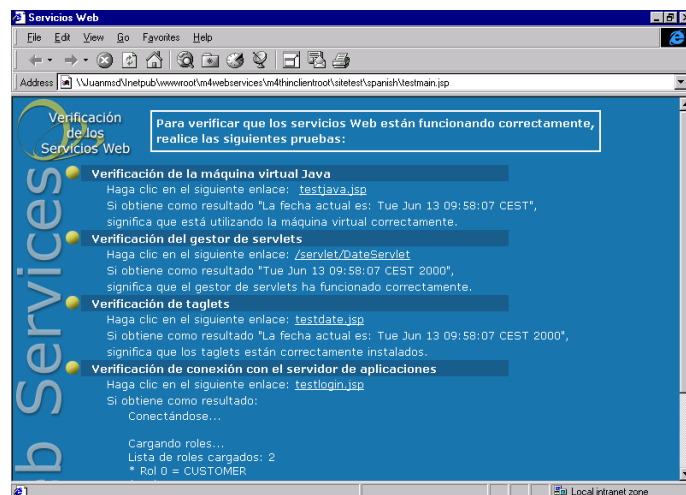
Las siguientes pruebas le permiten comprobar el entorno del servidor Web antes de empezar el desarrollo:

- *testjava.jsp*: comprueba que el código Java del JSP funciona.
- *testdate.jsp*: comprueba que las M4Tags están configurados correctamente.
- *testlogin.jsp*: comprueba que es posible conectarse al servidor de aplicaciones.
- *testgraph.jsp*: comprueba que la herramienta de gráficos funciona correctamente.
- *reports.jsp*: comprueba el funcionamiento del Lanzador de informes.
- *login.jsp*: comprueba que el proceso de inicio de sesión se puede realizar correctamente.

Si encuentra problemas al ejecutar algunas pruebas, debe configurar correctamente el entorno antes de empezar a desarrollar nuevas páginas.

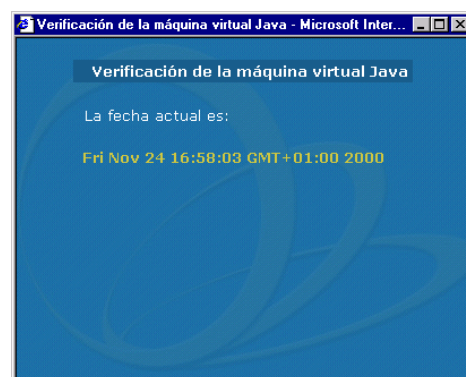
A continuación se incluye un ejemplo de la ejecución correcta de las pruebas:

Figura 7. Resultado de la página testmain.jsp



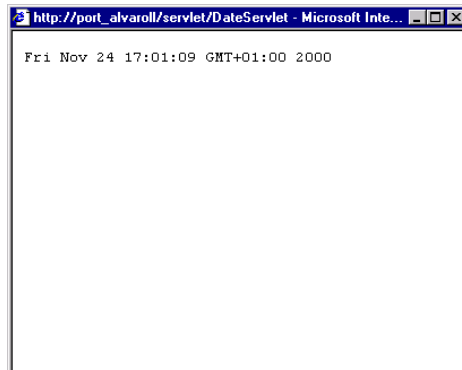
Haga clic en *testjava.jsp* para comprobar si la máquina virtual Java está funcionando correctamente. Para ello, el programa Java contenido en esta página JSP debe compilarse y ejecutarse correctamente. En tal caso, se obtendrá este resultado:

Figura 8. Resultado de la página testjava.jsp



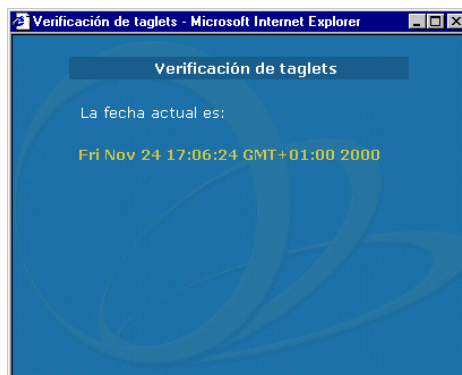
A continuación, haga clic en `/servlet/DateServlet` para comprobar el funcionamiento del gestor de *servlets* con la ejecución del *servlet* que nos da la fecha y hora actuales.

Figura 9. Resultado del enlace `/servlet/DateServlet`



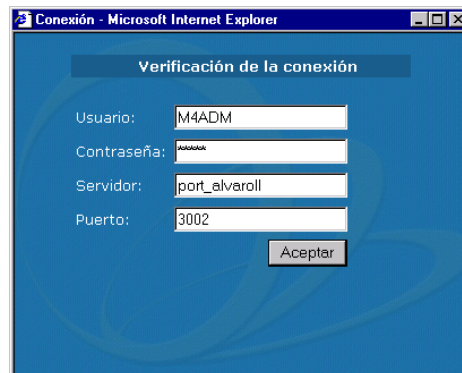
Seguidamente, para verificar la correcta instalación y funcionamiento de los M4Tags, debemos hacer clic en `testdate.jsp`.

Figura 10. Resultado de la página `testdate.jsp`



Para comprobar que la conexión con el servidor de aplicaciones de Meta4 funciona correctamente, haga clic en `testlogin.jsp`.

Figura 11. Resultado de la página testlogin.jsp



Verificación de la conexión

Usuario: M4ADM

Contraseña: [masked]

Servidor: port_alvaroll

Puerto: 3002

Aceptar

Tras introducir el usuario, la contraseña, el servidor y el puerto, haga clic en **Aceptar** para obtener los roles asociados a ese usuario. Se cierra la conexión con el servidor de aplicaciones.

Figura 12. Resultado de la página testlogin.jsp



Verificación de la conexión

Conectándose...

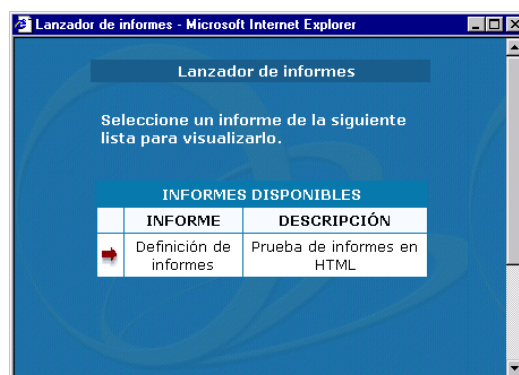
Cargando roles ...

Lista de roles cargados: 1

* Rol 0 = M4ADM

Por último, haga clic en *reports.jsp* para comprobar el Lanzador de informes. Tras abrir la ventana, haga clic en la flecha roja para ejecutar el informe de prueba.

Figura 13. Resultado de la página reports.jsp tras la conexión



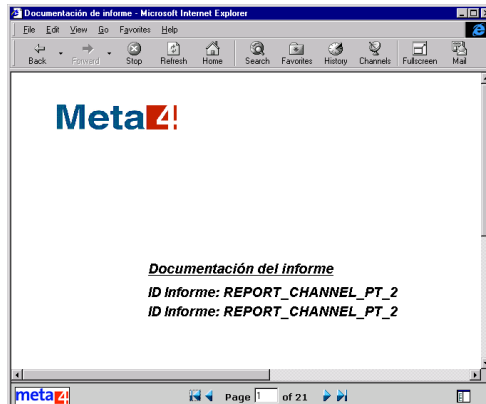
Lanzador de informes

Seleccione un informe de la siguiente lista para visualizarlo.

INFORMES DISPONIBLES	
INFORME	DESCRIPCIÓN
 Definición de informes	Prueba de informes en HTML

Si el Lanzador de informes funciona correctamente, debe aparecer en pantalla el informe de prueba. Puede recorrer cada una de sus páginas mediante los botones de navegación que aparecen en el panel inferior de la ventana.

Figura 14. Informe



Acceso a la aplicación

Una vez comprobado que todo funciona correctamente, se puede conectar y comenzar a usar las aplicaciones diseñadas para el cliente HTML.

El último componente que probó en el apartado anterior fue el proceso de inicio de sesión. El proceso de inicio de sesión y autenticación de usuario se realiza una única vez, al principio de la sesión, mediante un mecanismo típico de nombre de usuario/contraseña.

Tras introducir los datos de inicio de sesión, el sistema envía una solicitud de conexión al servidor Web. Una vez autenticados los datos de inicio de sesión, un servlet (el servlet de inicio de sesión) abre una nueva sesión de usuario.

En este proceso se recuperan de la aplicación la información de seguridad y los roles correspondientes al usuario que está obteniendo acceso al sistema. Concluido éste, se envía al navegador de la máquina cliente una cookie, que se utilizará para identificar al usuario durante las siguientes peticiones de páginas.

El usuario puede terminar una sesión mediante el proceso de cierre de sesión. Las sesiones también se terminan cuando no ha habido actividad del usuario en el servidor Web durante un determinado intervalo de tiempo que puede especificar el administrador del sitio Web.

Las páginas desarrolladas para el cliente HTML sólo se pueden ver si el usuario tiene una sesión activa.

Con respecto a la autorización, se debería acceder siempre a las páginas a través del servlet que comprueba si el usuario está autorizado para realizar las tareas con los roles que le corresponden (el servlet CheckSecurity).

Diseño de páginas básicas

2

Páginas JSP y funcionalidad

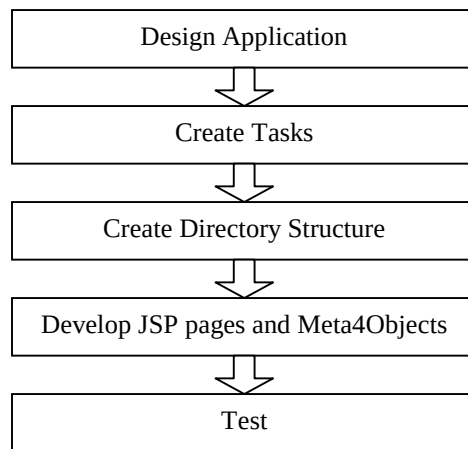
El desarrollo de funcionalidad para clientes HTML implica la creación de páginas JSP en las que implantar dicha funcionalidad. Para optimizar el rendimiento, dicho desarrollo debe procurar la reducción del número de transacciones entre el servidor Web y el de aplicaciones al mínimo.

Para desarrollar una aplicación para el cliente HTML, siga estos pasos:

1. Diseñe la aplicación y divídala en tareas funcionales relacionadas, pero independientes entre sí.
2. Mediante el Editor de tareas del cliente de desarrollo o el Editor de páginas JSP (analizados en capítulos posteriores), registre en la base de datos cada tarea funcional identificada. Esta página debe estar asociada a un archivo JSP, que se convertirá en la página de inicio de la tarea.
3. Para cada tarea creada en el paso anterior, cree en el servidor Web una carpeta con el mismo nombre.
4. Diseñe las páginas Web que compondrán cada una de las tareas, asegurándose siempre de que el número de transacciones sea el mínimo posible.
5. Estructure y organice las páginas basándose en los conceptos indicados en el apartado **Estructura de directorios** del capítulo **Clientes HTML**.
6. Compruebe que las páginas funcionan al abrirlas desde un navegador estándar o desde enlaces de otras páginas.

En este capítulo se describe el marco de este desarrollo y se explican los conceptos implicados. Se incluyen ejemplos de la estructura básica y las funciones de las páginas JSP.

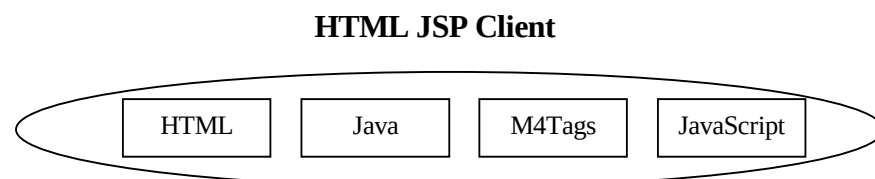
Figura 15. Pasos para desarrollar una aplicación de cliente HTML



En general, estas páginas JSP son documentos HTML ampliados con otros tipos de código interno y contienen:

- Etiquetas propias de HTML que permiten estructurar el documento por apartados o aplicar formatos concretos al cuerpo del documento.
- Etiquetas desarrolladas por Meta4® basadas en el JSP 1.1 estándar. Son las llamadas M4Tags, incluidas en la biblioteca M4TagLib. Estas etiquetas se utilizan para recuperar y presentar datos del repositorio de Meta4, entre otras posibilidades.
- Código JavaScript, que tiene un cometido similar al de las etiquetas HTML y, además, puede:
 - Controlar acciones del usuario
 - Controlar campos de texto y otros elementos HTML
 - Modificar el formato de valores y resultados
 - Mejorar la presentación
 - Levantar otras instancias del navegador
- Código Java, para realizar tareas más complejas. Se puede utilizar, pero conviene evitarlo pues dificulta en gran medida la comprensión de la página JSP y su traducción.

Figura 16. JSP





Las M4Tags no tienen ningún estilo gráfico especial, por lo que, para modificar el aspecto gráfico de una sección determinada, debe utilizar las etiquetas HTML.

Este manual está dirigido a usuarios con conocimientos básicos de las etiquetas y la estructura de una página HTML, por lo que sólo se indicará la codificación correcta de una página con M4Tags. Si bien no es necesario tener conocimientos sobre JavaScript o Java, puede resultar conveniente. Es preciso conocer el funcionamiento básico de los Meta4Objects.

Estructura básica de las páginas JSP

Por lo general, la sección de M4Tags de cada JSP del cliente HTML se divide en:

- Definición del comienzo de la página
- Servicio de datos
- Presentación de datos
- Definición del final de la página

La sintaxis completa de las M4Tags se puede consultar en el capítulo **Referencia de la biblioteca M4TagLib** de este mismo manual; así, en los capítulos donde se explica el diseño de páginas, sólo se presentarán los conceptos de M4Tag que sean necesarios.

Comienzo de página

En una página JSP la parte específica de las M4Tags empieza con la definición o declaración de tarea mediante la M4Tag **page** o **startpage**. Por ejemplo:

```
<m4:page subsessionid="PERSON">
o
<m4:startpage m4task="PERSON"/>
```



Es mejor utilizar la etiqueta **page** que la **startpage**.

El parámetro *subsessionid* (o *m4task*) debe indicar un identificador. Para simplificar, éste coincide con el nombre de la tarea a la que pertenece la página.

Esta etiqueta realiza varias operaciones antes de mostrar al usuario la página solicitada:

- Verifica la autenticidad del usuario por medio de la *cookie* enviada al navegador al principio de la sesión.
- Define las estructuras de datos necesarias en el servidor de aplicaciones para ejecutar una transacción.
- Inicia el contador de tiempo que controla el tiempo de inactividad del usuario.

- Redirige a una página de error cuando se producen errores de seguridad.



La declaración de tarea inicia controles para detectar la inactividad del usuario durante cierto tiempo. Si el tiempo entre dos llamadas page o startpage supera el tiempo máximo de inactividad permitido, se cierra la conexión con el servidor de aplicación y el usuario tendrá que volver a iniciar una sesión.

Servicio de datos

El servicio o la recuperación de datos es un conjunto de M4Tags situadas dentro de la etiqueta **job** o entre el par de M4Tags **beginjob** y **endjob**. Esas etiquetas definen una transacción opuesta a la base de datos, es decir, la definición y recuperación de la información. Esa petición se envía al servidor de aplicaciones en formato XML.



Es mejor utilizar la etiqueta **job** que la pareja de **beginjob** y **endjob**.

En el entorno Web, una transacción origina un intercambio de datos entre el servidor Web, el servidor de aplicaciones y el equipo del usuario.



Este concepto es similar al comando BEGINTRANS y ENDTRANS de las bases de datos, aunque aquí no se puede deshacer su ejecución.

Entre las etiquetas mencionadas, se incluye un conjunto de M4Tags especializadas en la petición de datos. Estas M4Tags especifican una serie de acciones secuenciales que se realizarán conjuntamente al cerrarse la etiqueta **job** o ejecutarse la M4Tag **endjob**.

Como resultado de esa ejecución, se obtendrán unos datos y estructuras que sólo se podrán utilizar desde el bloque Presentación de datos.

Éstas son las operaciones que se utilizan con más frecuencia durante la solicitud del servicio de datos al servidor de aplicaciones y sus correspondientes etiquetas de servicio de datos:

- Definir los Meta4Objects en el servidor de aplicaciones (< **m4:datadef**...>)
- Ejecutar métodos o conceptos (<**m4:exec**...>)
- Recuperar datos (<**m4:outputdef**...>)
- Filtrar datos (<**m4:filter**>)

- Situar en registros (**<m4:move>**)
- Modificar el valor de un grupo de elementos (**<m4:setitems>**)

Antes de solicitar una operación, debe conocer la estructura y las propiedades del Meta4Object al que quiere acceder. Esto incluye los nodos, los campos y los métodos que se van a invocar.

Es recomendable declarar sólo el rango de registros y los elementos que desea ver. De este modo, reducirá el intercambio de datos y aumentará el rendimiento de la aplicación.

Asimismo, es aconsejable controlar el resultado de la transacción mediante los atributos especiales que proporciona **<m4:endjob...>** que gestionan el control de errores y mostrar siempre al usuario la información positiva o negativa sobre el resultado de la aplicación.

Presentación de datos

A diferencia de los bloques de declaración de tarea de servicio y el cierre de declaración de tarea (todos ellos obligatorios), no siempre existe un bloque de presentación de datos. Por ejemplo, si quiere guardar registros, puede utilizar una página intermedia que contiene una secuencia de **<m4:setitems>** seguido por **<m4:exec>**, que ejecuta un método de almacenamiento. No hay ningún dato que mostrar en esta operación.

Sin embargo, en aquellos casos en los que se realiza la presentación de los datos, lo más habitual es que se vea la siguiente secuencia de acciones:

- En primer lugar, se utiliza la etiqueta **move** para controlar la posición de los registros.
- A continuación, utilice etiquetas como **<m4:item>** para mostrar los datos.
- Finalmente, se usan variables para controlar los datos que se van a ver o el conjunto de acciones que se van a seguir.

Fin de página

Los bloques de M4Tags finalizan obligatoriamente con el cierre de la declaración de tarea usando las etiquetas **<m4:page>** o **<m4:endpage/>**, según la que se haya utilizado para iniciar la declaración de tarea.

Recuperación y presentación de datos

A continuación, se analizan detalladamente los apartados antes mencionados, ilustrándolos también con ejemplos sencillos.

En el primer ejemplo, se explica paso a paso cómo crear el código JSP que recupera el primer registro de un campo de la base de datos, concretamente el campo **Nombre de país**.

A continuación, este código se modifica para mostrar todos los registros del mismo campo. Se obtiene el número total de registros existentes y la posición que ocupa cada uno de ellos.

Finalmente, en el tercer ejemplo, se incluyen más campos: **ID País**, **Nombre de nacionalidad** e **ID Nacionalidad**.

En los tres casos, se utilizará solamente el nodo **SCO_GEO_STRUCTURE** del Meta4Object **SCO_GEO_STRUCTURE**.

La primera diferencia respecto a una página HTML estándar consiste en que carga la biblioteca de M4Tags (o *taglib*) para poder incluir posteriormente las M4Tags necesarias. Esto se hará en la primera línea de código del ejemplo 1.

EJEMPLO

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<HTML>
<HEAD>
...
...
</HTML>
```

Paso 1: Definir el entorno de la transacción

En todas las páginas JSP con M4Tags, la primera M4Tag que debe aparecer es la de declaración de tarea (**<m4:startpage>**), que se complementará al final del código con la M4Tag de cierre de declaración (**<m4:endpage/>**).

En el siguiente ejemplo se utiliza una M4Tag de página que define un entorno de transacción llamado "subsession_alias", como se muestra en **<m4:page subsessionid="subsession_alias">**.

EJEMPLO

```

<%@ taglib uri="M4Tags" prefix="m4" %>
...
<m4:page subsessionId="subsession_alias">
...
</m4:page>
...

```

Paso 2: Definir el puesto

Empezar y finalizar la transacción

Tras la declaración de tarea, hay que incluir una M4Tag de inicio de transacción con el servidor de aplicaciones. Esta M4Tag es **<m4:job>**, y funciona con **</m4:job>** para delimitar la transacción. Entre las dos etiquetas se definen los datos y los métodos de la transacción.

Esa agrupación de datos y métodos recibe el nombre de **trabajo**; es el conjunto de operaciones que se van a realizar en el servidor de aplicaciones. En este apartado, se especifica el conjunto de Meta4Objects que se van a utilizar, los métodos que se van a ejecutar y los datos que se desea recuperar desde el servidor de aplicaciones.

EJEMPLO

```

<%@ taglib uri="M4Tags" prefix="m4" %>
...
<m4:page subsessionId="subsession_alias">
...
<m4:job>
...
</m4:job>
...
</m4:page>

```

Definición de Meta4Objects

Dentro del bloque de la transacción, es necesario algún mecanismo que ofrezca acceso a los datos del servidor de aplicaciones. Esto se consigue con la M4Tag **datadef**, que crea en el servidor Web la estructura de los datos y solicita estos al servidor de aplicaciones.

En este caso, las estructuras de datos son Meta4Objects, es decir, cada M4Tag **datadef** define un Meta4Object que se especifica dentro del parámetro **m4o**. El nombre de **<m4:datadef...>**, especificado en el parámetro *m4name*, es el alias del Meta4Object. Si se produce alguna excepción de seguridad, esta M4Tag genera una página de error.

El Meta4Object que se utiliza en este ejemplo es **SCO_GEO_STRUCTURE**, y se accederá a él a través del alias *geo_alias*.

EJEMPLO

```
<%@ taglib uri="M4Tags" prefix="m4" %>
...
<m4:page subsessionid="subsession_alias">
  <m4:job>
    <m4:datadef m4o="SCO_GEO_STRUCTURE" m4name="geo_alias"/>
    ...
  </m4:job>
  ...
</m4:page>
```

Ejecución de métodos

Después de obtener acceso a los datos del servidor de aplicaciones, puede ejecutar en él un método o un concepto. Para ello, utilice la M4Tag **exec**, que recoge el tipo de elemento para ejecutarlo como un parámetro obligatorio.

La ejecución de métodos o conceptos con esta M4Tag tiene dos restricciones:

- No se permiten los cuadros de mensajes LN4 ni otros métodos LN4 que muestren información en pantalla.
- No es posible mezclar argumentos pasados por referencia y por nombre. Si es necesario hacer esto, se debe utilizar la clase M4Operations.

En el siguiente ejemplo, se ejecuta el método LOAD que carga el Meta4Object **SCO_GEO_STRUCTURE**.

EJEMPLO

```

<%@ taglib uri="M4Tags" prefix="m4" %>
...
<m4:page subsessionid="subsession_alias">
<m4:job>
  <m4:datadef m4o="SCO_GEO_STRUCTURE" m4name="geo_alias"/>
  <m4:exec m4method="load_alias:geo_alias!LOAD"/>
  ...
</m4:job>
...
</m4:page>

```

Recuperación de datos

Cuando haya terminado las operaciones anteriores, puede definir los datos que quiere extraer del servidor. Para ello, utilice la etiqueta **outputdef**. Los datos solicitados pueden ser todos los de un nodo o un elemento concreto de un registro determinado.

Puede utilizar los atributos de etiqueta *node* y *records* para indicar que quiere recuperar un número determinado de registros de todos los campos de un nodo. Por ejemplo, puede utilizar la siguiente notación para recuperar todos los registros del nodo **NODE_ID**.

```

<m4:outputdef
  m4alias="outputdef_alias" m4object="alias"
  node="NODE_ID" records="*>
  .....

```

También se puede utilizar la etiqueta **outputdef** para solicitar cada grupo de datos con su propio parámetro dentro de la etiqueta. Esto permite recuperar los registros de un campo específico.

Por ejemplo, puede usar el siguiente código para recuperar todos los registros del campo **FIELD_ID** del nodo **NODE_ID** del Meta4Object cuyo alias es **m4oalias**:

```

:<m4:param
  .....

```

```
value="m4oalias!NODE_ID[*].FIELD_ID"/>
```

La sintaxis siguiente permite recuperar un nodo completo, es decir, todos los registros de todos sus campos.

```
<m4:param  
.....  
value="m4oalias!NODE_ID[*]"/>
```

Para recuperar todos los campos asociados a un registro determinado, utilice `[]` en vez de `[*]`. De este modo, obtendrá todos los campos asociados a "j".

```
<m4:param  
.....  
value="m4oalias!NODE_ID[j]"/>
```

En el siguiente ejemplo se muestra cómo usar los atributos *node* y *record* para recuperar todos los registros del nodo **STD_MT_COUNTRY**.

EJEMPLO

```
<%@ taglib uri="M4Tags" prefix="m4" %>  
...  
<m4:page subsessionid="subsession_alias">  
<m4:job>  
  <m4:datadef m4o="SCO_GEO_STRUCTURE" m4name="geo_alias"/>  
  <m4:exec m4method="load_alias:geo_alias!LOAD"/>  
  <m4:outputdef m4alias="mt_alias" m4object="geo_alias"  
    node="STD_MT_COUNTRY" records="*">  
  </m4:outputdef>  
</m4:job>  
...  
</m4:page>
```

El atributo *m4alias* del ejemplo anterior se puede utilizar para hacer referencia a los datos solicitados cuando se haya terminado la transacción.

A continuación se muestra el mismo ejemplo utilizando la etiqueta **param**.

EJEMPLO

```
<%@ taglib uri="M4Tags" prefix="m4" %>
...
<m4:page subsessionId="subsession_alias">
<m4:job>
  <m4:datadef m4o="SCO_GEO_STRUCTURE" m4name="geo_alias"/>
  <m4:exec m4method="load_alias:geo_alias!LOAD"/>
  <m4:outputdef m4alias="mt_alias">
    <m4:param
      name="M4NAME0"
      value="geo_alias!STD_MT_COUNTRY[*]" />
  </m4:outputdef>
</m4:job>
...
</m4:page>
```

Los atributos *node* y *records* son mutuamente excluyentes con las etiquetas **param**; es decir: puede utilizar una notación o la otra, pero no ambas.

En los demás ejemplos, utilizaremos la notación con la etiqueta **param**.

Paso 3: Presentación de datos

Tras establecer la conexión con el servidor de aplicaciones, se muestra la información recuperada de la base de datos.

Para ello se cuenta con varias M4Tags, que se agrupan en categorías según la función que ejecutan. Los ejemplos cubren los siguientes casos:

- Presentación de un registro de un campo
- Presentación de todos los registros de un campo
- Presentación de todos los registros de varios campos

Presentación de un registro de un campo

Si sólo se muestra información relativa a un campo, la M4Tag **item** presentará su valor en la página HTML de salida. El siguiente código refleja el nombre del país del primer registro que se especifica por medio del parámetro *m4name* de la M4Tag.

EJEMPLO¹

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:page subsessionId="subsession_alias">
<m4:job>
  <m4:datadef m4o="SCO_GEO_STRUCTURE" m4name="geo_alias"/>
  <m4:exec m4method="load_alias:geo_alias!LOAD"/>
  <m4:outputdef m4alias="mt_alias">
    <m4:param
      name="M4NAME0"
      value="geo_alias!STD_MT_COUNTRY[*]"/>
    </m4:outputdef>
  </m4:job>
  <P>MY DATA:
  <m4:item
    m4name="mt_alias:geo_alias!STD_MT_COUNTRY[0].STD_N_COUNTRY"/>
  </m4:page>
```

1. Este código no está optimizado, porque la etiqueta `outputdef` recupera más elementos y registros de los que aparecen en la página. No obstante, esta notación será adecuada para el resto de la sección.

Para ver el resultado de la ejecución de este ejemplo, guarde el contenido del archivo como `ejemplo1.jsp`. Coloque el archivo en una carpeta cuyo nombre coincida con el nombre de una tarea existente, dentro del directorio raíz de la aplicación Web. La carpeta de la tarea debe contener distintas carpetas para cada ejemplo, y `ejemplo1.jsp` debe estar en la carpeta del idioma seleccionado al iniciar la sesión.

EJEMPLO

Observe el siguiente caso:

- El directorio raíz de la aplicación Web es: "`C:\Archivos de programa\Meta4\M4WS\default`"
- El nombre de la tarea definido en la base de datos es "`task1`".
- Si el usuario inició la sesión en inglés, el nombre de la carpeta será "`inglés`".

En este caso, `ejemplo1.jsp` debería guardarse en el siguiente directorio:

```
C:\Archivos de
programa\Meta4\M4WS\default\task1\inglés\ejemplo1.jsp
```

Después de guardar la página, puede introducir la siguiente dirección:

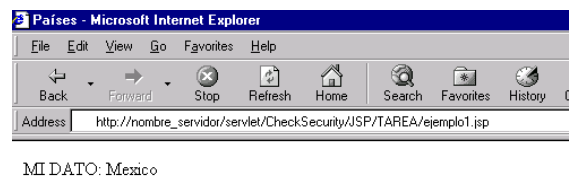
`http://nombre_servidore/servlet/CheckSecurity/JSP/task1/ejemplo1.jsp`

El `nombre_servidor` es el nombre del servidor Web y `task1`, el de la carpeta en la que se encuentra la página. Como se explicó anteriormente, `task1` debe estar en la base de datos y estar definida sin seguridad o con una lista de roles autorizados que coincidan al menos con uno de los roles del usuario.

La parte de la dirección que está entre ambas variables, `/servlet/CheckSecurity/JSP/`, es siempre la misma para cualquier página. Esta parte hace que el servlet `CheckSecurity` examine la seguridad de la página. Si la tarea no existe o el usuario no está autorizado para ejecutarla, aparecerá un mensaje de error.

En el ejemplo, el primer registro de país corresponde a Francia.

Figura 17. Resultado de la página `ejemplo1.jsp`



Presentación de todos los registros de un campo

Para mostrar más de un registro, utilice la etiqueta **dataloop**.

Esta etiqueta creará un bucle con todos los registros disponibles en la solicitud de datos hecha con la etiqueta **outputdef**.

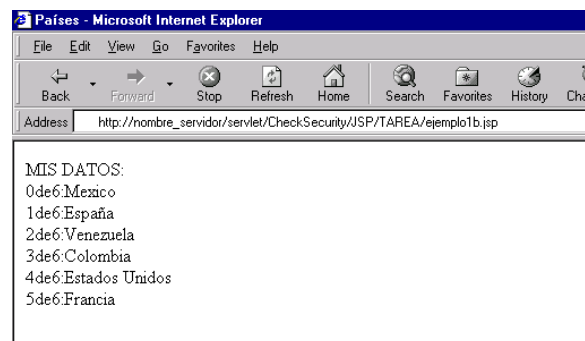
Puede utilizar las etiquetas **current** y **count** para mostrar el número de registros correspondientes, de entre todos los registros.

EJEMPLO²

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:page subsessionid="subsession_alias">
<m4:job>
  <m4:datadef m4o="SCO_GEO_STRUCTURE" m4name="geo_alias"/>
  <m4:exec m4method="load_alias:geo_alias!LOAD"/>
  <m4:outputdef m4alias="mt_alias">
    <m4:param name="M4NAME0"
      value="geo_alias!STD_MT_COUNTRY[*]" />
  </m4:outputdef>
</m4:job>
<P>MIS DATOS:<br>
<m4:dataloop outputdef="mt_alias" count="*">
  <m4:current m4name="mt_alias:geo_alias!STD_MT_COUNTRY"> of
  <m4:count m4name="mt_alias:geo_alias!STD_MT_COUNTRY">:
  <m4:item
    m4name="mt_alias:geo_alias!STD_MT_COUNTRY.STD_N_COUNTRY">
  <br>
</m4:dataloop>
</m4:page>
```

2. El código de este ejemplo no está optimizado, porque la etiqueta **outputdef** recupera más elementos de los que se muestran finalmente.

Figura 18. Resultado de la ejecución de la página ejemplo1B.jsp



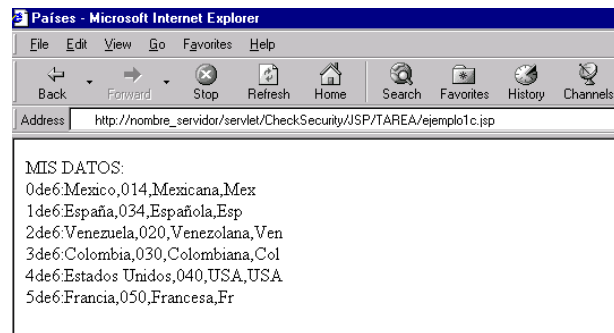
Presentación de todos los registros de varios campos

Para mostrar registros de varios campos, utilice varias etiquetas **item** dentro de **dataloop**.

EJEMPLO

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:page subsessionid="subsession_alias">
<m4:job>
  <m4:datadef m4o="SCO_GEO_STRUCTURE" m4name="geo_alias"/>
  <m4:exec m4method="load_alias:geo_alias!LOAD"/>
  <m4:outputdef m4alias="mt_alias">
    <m4:param name="M4NAME0" value="geo_alias!STD_MT_COUNTRY[*]" />
  </m4:outputdef>
</m4:job>
<P>MIS DATOS:<br>
<m4:dataloop outputdef="mt_alias" count="*">
  <m4:current m4name="mt_alias:geo_alias!STD_MT_COUNTRY"> of
  <m4:count m4name="mt_alias:geo_alias!STD_MT_COUNTRY">:
  <m4:item
    m4name="mt_alias:geo_alias!STD_MT_COUNTRY.STD_N_COUNTRY">,
  <m4:item
    m4name="mt_alias:geo_alias!STD_MT_COUNTRY.STD_ID_COUNTRY">,
  <m4:item
    m4name="mt_alias:geo_alias!STD_MT_COUNTRY.STD_N_NATIONALITY">,
  <m4:item
    m4name="mt_alias:geo_alias!STD_MT_COUNTRY.STD_ID_NATIONALITY">
</m4:dataloop>
</m4:page>
```


Figura 19. Resultado de la página ejemplo1C.jsp



Paso 4: Presentación

Puede añadir elementos de presentación HTML para mejorar el diseño de la página.

EJEMPLO

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<HTML>
<HEAD>
<TITLE> Países </TITLE>
</HEAD>
<BODY>
<m4:page subsessionId="subsession_alias"/>
...
</m4:page>
</BODY>
</HTML>
```

Almacenamiento de registros

En el apartado anterior, se ha mostrado cómo recuperar datos guardados. En éste, se va a analizar cómo guardar datos nuevos en la base de datos.

En el siguiente ejemplo (guardar.jsp), se agregará un nuevo país a los ya existentes en los ejemplos anteriores.

Para ello, tendrá que crear un nuevo registro, introducir los valores de los campos de dicho registro (los códigos identificativos del país y la nacionalidad, y el nombre del país y la nacionalidad) y guardar el registro.

Se van a seguir estos pasos:

- Creación de un nuevo registro

Para crear un nuevo registro, tiene que ejecutar el método `NEWRECORD`.

- Asignación de valores a los campos del registro

Para dar valores a los distintos campos del nuevo registro, utilice la etiqueta `setitems`. Esta etiqueta lleva asociado un parámetro por cada valor que se desee asignar. De este modo, hay cuatro parámetros para los campos asociados al código identificativo y el nombre de país y nacionalidad. Cada parámetro tiene dos variables:

- *name*: indica el campo al que se quiere asignar el valor. Debe indicar el *m4name* asociado a la instancia del `Meta4Object` y el código identificativo del nodo al que pertenece el campo.
- *value*: valor que se desea asociar al campo.

- Almacenamiento de los registros

Una vez asignados los valores a los campos del nuevo registro, guárdelos. Para ello, utilice la `M4Tagexec`, pero ahora con la palabra reservada `PERSIST`.

En el siguiente ejemplo se muestran los tres pasos.

Si ejecuta el ejemplo, verá que no muestra ningún dato porque no se han recuperado.

En su lugar, para verificar que el nuevo registro se ha guardado correctamente, se utilizará un enlace a la página `Ejemplo1c.jsp` (del apartado anterior), que muestra los campos **ID País**, **Nombre país**, **ID Nacionalidad** y **Nombre nacionalidad** de todos los registros almacenados en la base de datos.

Además, por lo general la página de este ejemplo recibirá valores como parámetros de un cliente en vez de haberlos codificado directamente en la página.

EJEMPLO GUARDAR.JSP

```
<%@ taglib uri="M4Tags" prefix="m4" %>

<m4:page subsessionId="subsession_alias">

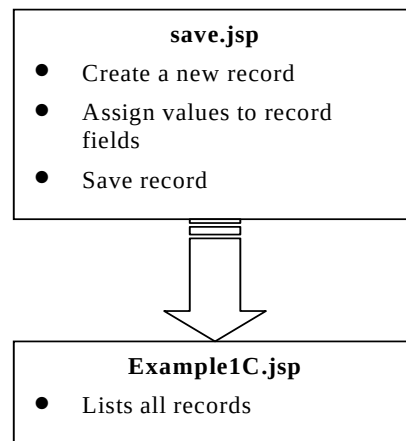
<m4:job>
  <m4:datadef m4o="SCO_GEO_STRUCTURE" m4name="geo_alias"/>
  <m4:exec
    m4method="newrecord_alias:geo_alias!STD_MT_COUNTRY.NEWRECORD"/>

  <m4:setitems>
    <m4:param
      name="GEO!STD_MT_COUNTRY.STD_ID_COUNTRY"
      value="073"/>
    <m4:param
      name="GEO!STD_MT_COUNTRY.STD_N_COUNTRY"
      value="Polonia"/>
    <m4:param
      name="GEO!STD_MT_COUNTRY.STD_ID_NATIONALITY"
      value="Pol"/>
    <m4:param
      name="GEO!STD_MT_COUNTRY.STD_N_NATIONALITY"
      value="Polaco"/>
  </m4:setitems>

  <m4:exec m4method="save_alias:geo_alias.PERSIST"/>
</m4:job>

<P>Los datos se han guardado:<br>
Vaya a
<a href="/servlet/CheckSecurity/JSP/task1/
ejemplo1c.jsp">ejemplo1c.jsp</a>
para comprobarlo
<m4:endpage/>
</BODY>
</HTML>
```

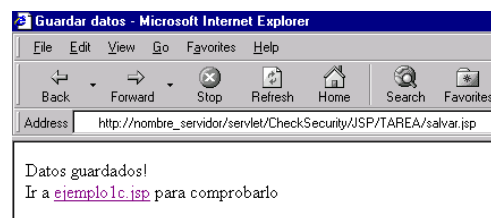
Figura 20. Esquema de ejemplo



Para ejecutar la página, abra la siguiente dirección con el navegador:

`/servlet/CheckSecurity/JSP/task1/save.jsp`

Figura 21. Resultado de la ejecución de la página guardar.jsp



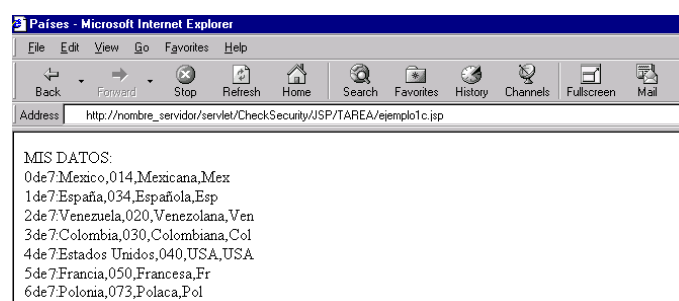
Para mostrar todos los registros, hay un enlace estándar HTML con *href* que lleva a:

`/servlet/CheckSecurity/JSP/task1/example1c.jsp`

Ejemplo1c.jsp tendrá que pertenecer al directorio *task1/idioma* (*idioma* usado en la conexión).

Haga clic en el enlace para mostrar el nuevo registro introducido.

Figura 22. Resultado de la ejecución de la página guardar.jsp



Al crear un nuevo registro, tendrá que cambiar el código JSP de la página JSP y asignar otros valores a cada campo del nuevo registro.

Ésta no es la forma más eficiente de guardar nuevos registros, pero sirve para comprender el proceso que hay que seguir con ejemplos más complejos en los que el usuario, a través de un formulario, tendrá que introducir los valores de los campos del nuevo registro.

Sugerencias de diseño

Para terminar este capítulo, se incluyen unas recomendaciones de diseño para optimizar el desarrollo de la aplicación.

Sugerencias de diseño JSP

- Para asegurarse de que la aplicación trabaja independientemente de la plataforma, escriba siempre en minúsculas los nombres de los archivos y las carpetas.
- Para ganar claridad, se recomienda empaquetar las funciones JavaScript en archivos auxiliares.
- No utilice la programación en Java dentro de las páginas JSP, excepto para ejecutar operaciones avanzadas.
- Cuando añada elementos de presentación HTML a la página, realice un diseño funcional. Por ejemplo, los iconos resultan útiles para representar mensajes. De este modo, se evitan los títulos y textos que pueden complicar la traducción de la imagen.

Las sugerencias son aplicables a la tecnología Meta4

- Procure no mezclar distintas funcionalidades en la misma página, de modo que resulte clara la separación de tareas. Por ejemplo, no asocie el mismo Meta4Object a tareas distintas.
- Siempre que sea posible, separe en páginas distintas la funcionalidad de consulta y la de actualización.
- Cada tarea debe tener un único punto de entrada, es decir, una página principal por tarea.
- El diseño ideal en un cliente HTML no incluye más de una transacción por página y utiliza un único Meta4Object.
- En cada página JSP no debe existir más de una M4Tag de declaración de tarea (**<m4:page...>**) y su nombre debe coincidir con el nombre de la tarea a la que se encuentra asociada la página.
- Es aconsejable que una página represente un nodo (y así no se complica la lógica asociada). No obstante, en algunas ocasiones esto no es posible; por ejemplo, con Meta4Objects de varios nodos.

Diseño de páginas avanzadas

3

Introducción

En el capítulo anterior se explicaron los conceptos básicos para crear una página JSP, como la recuperación y presentación de datos, y el almacenamiento de registros.

En este capítulo se explican concepto más avanzados como:

- Introducción y lectura de parámetros
- Cambio de la posición del registro actual
- Creación de bucles con etiquetas
- Filtros y ordenación
- Cambio de la presentación de los datos
- Diseño de una página de inicio de sesión

También se tratan otros temas importantes como:

- M4Tags de bolsas de sesión
- Gestión de archivos
- Herramientas de presentación
- Gestión de errores
- Gráficos HTML
- Propiedades de transacciones avanzadas
- Referencia de traducciones

Finalmente, se facilitan sugerencias de diseño y se explican modos de depuración de archivos JSP.

Introducción y lectura de parámetros

En este apartado se explica cómo pasar parámetros a M4Tags.

Se pueden pasar los siguientes parámetros a M4Tags:

- **Estáticos** o procedentes de la plantilla JSP: los parámetros permanecen invariables a lo largo del tiempo, pues están codificados en la página JSP.

EJEMPLO

```
<m4:item m4name = "M4:PERSON!SCO_PERSON.SCO_PERSON_ID"/>
```

- **Dinámicos** o procedentes de una petición HTTP: el cliente introduce el valor del parámetro. Ese valor puede cambiar en diferentes ejecuciones de la página.

EJEMPLO

El siguiente formulario es un ejemplo del cliente que realiza la petición HTTP. La página que recibe los parámetros es "action.jsp".

```
<form method="POST" name="formname" action="action.jsp">
<table>
<tr>
<td>Nombre de persona:</td>
<td><input type="text" name="sPersonName" size="16"></td>
</tr>
<tr>
<td>ID Persona:</td>
<td><input type="text" name="sPersonID" size="16"></td>
</tr>
<tr>
<td colspan="2"><input type="submit" value="Accept"></td>
</tr>
</table>
</form>
```

Si quiere pasar parámetros dinámicos desde el formulario del cliente a los M4Tags, tiene dos opciones distintas:

- **Pasar parámetros por valor:** el valor del parámetro, que generalmente procede de una petición HTTP, se almacena en una variable Java y después se pasa a las etiquetas.



Para obtener más información sobre la **petición HTTP**, los métodos que admite y otros datos que le puedan ayudar a desarrollar páginas JSP, visite el sitio Web de Javasoft (<http://java.sun.com>) y consulte los documentos de referencia del servlet 2.2 y JSP 1.1.

EJEMPLO

En el siguiente fragmento de código, la variable Java `PersonName` se obtiene por medio de una petición HTTP. De este modo, cuando se llama al M4Tag **setitems**, el entorno JSP sustituye el valor `PersonName` por el valor real de `PersonName`:

```
<%
    String  PersonName = request.getParameter("sPersonName");
    String  PersonID   = request.getParameter("sPersonID");
%>
<m4:setitems>
    <m4:param
      name="M4:PERSON!SCO_PERSON.SCO_PERSON_NAME"
      value="<%=PersonName%>" />
    <m4:param
      name="M4:PERSON!SCO_PERSON.SCO_PERSON_ID"
      value="<%=PersonID%>" />
</m4:setitems>
```



Si pasa parámetros por valor, no olvide las comillas dobles entre las que debe ir la variable Java que se pasa a la etiqueta.

- **Por referencia:** el valor del parámetro es la dirección de la que se obtiene el valor apropiado. En este caso, se utilizará siempre el carácter `&` y siempre dentro de un M4Tag. Meta4 le brinda cuatro posibilidades:
 - `&BAG.var`: da acceso a las variables definidas en las bolsas (vea el apartado *M4Tags de bolsas de sesión*).
 - `&REQUEST.var`: recoge las variables introducidas por el cliente.
 - `&VAR.varname`: permite el acceso a las variables definidas en las etiquetas de presentación y a otras variables Java definidas en la aplicación de la siguiente manera:

```
<%  pageContext.setAttribute("varname", "value",
    pageContext.PAGE_SCOPE); %>
```
 - `&aliasitem:aliasm4o!node.item`: permite el acceso a los elementos de los Meta4Objects.

El símbolo & permite las concatenaciones de variables en la página JSP¹:

```
<m4:move>
  <m4:param name="DataTagAlias"
    value="OutputDefAlias:NODE[&VAR.varname] />
</m4:move>
```

Por otro lado, con las variables por valor, la tecnología JSP no permite ese tipo de casos:

```
<m4:move>
  <m4:param name="DataTagAlias"
    value="OutputDefAlias:NODE[<%= varname %>] />
</m4:move>
```

EJEMPLO

El siguiente fragmento de código muestra la forma más sencilla de obtener el valor de un parámetro de una petición HTTP:

```
<m4:setitems>
  <m4:param name="M4:PERSON!SCO_PERSON.SCO_PERSON_NAME"
    value="&REQUEST.sPersonName" />
  <m4:param name="M4:PERSON!SCO_PERSON.SCO_PERSON_ID"
    value="&REQUEST.sPersonID" />
</m4:setitems>
```

Para que el símbolo & no se interprete como concatenación, agregue otro símbolo &:

```
<m4:setitems>
  <m4:param name="M4:PERSON!SCO_PERSON.SCO_PERSON_NAME"
    value="&&REQUEST.sPersonName" />
  <m4:param name="M4:PERSON!SCO_PERSON.SCO_PERSON_ID"
    value="&&REQUEST.sPersonID" />
</m4:setitems>
```

En este segundo caso, el valor del elemento será *&REQUEST.sPersonID*.



Hay varios nombres de variables que no debería utilizar (como "sesión" o "solicitud") porque es posible que ya hayan sido definidos por el motor de servlets. Para evitar este problema puede añadir un prefijo (por ejemplo, "m4") a todas las variables que defina.

1. Están permitidas las concatenaciones en los tres primeros casos de parámetros por referencia, pero no en el acceso a los elementos de los Meta4Objects.

Cambio de la posición del registro actual

Cuando se recupera un grupo de registros del servidor de aplicaciones, las etiquetas de servicio colocan por defecto el registro actual al final del grupo de registros. Puede seleccionar el registro actual utilizando la etiqueta **move** después de **job** o **endjob**. Esto se conoce como “movimiento en el cliente” (o “*movimiento local*”).

Cuando se usa la etiqueta **move** antes de **job** o **endjob**, puede modificar la posición dentro de los conjuntos de datos existentes en el Meta4Object. Esto se conoce como “movimiento en el servidor” (o “*movimiento remoto*”), y se utiliza para pasar Meta4Objects con varios nodos.

La diferencia queda patente en el siguiente ejemplo, en el que se solicitan datos de un Meta4Object con un nodo padre **STD_MT_COUNTRY** (País), un nodo hijo **STD_GEO_DIV** (Estado) y un nodo nieto **STD_SUB_GEO_DIV** (Condado).

EJEMPLO: MOVE.JSP

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:page subsessionid="subsession_alias">
  <m4:job>
    <m4:datadef m4o="SCO_GEO_STRUCTURE" m4name="geo_alias"/>
    <m4:exec m4method="load_alias:geo_alias!LOAD"/>
    <m4:outputdef m4alias="mt_alias">
      <m4:param name="M4NAME0" value="geo_alias!STD_MT_COUNTRY[*]" />
    </m4:outputdef>
    <m4:move>
      <m4:param name="geo_alias" value="mt_alias:STD_MT_COUNTRY[2]" />
    </m4:move>
    <m4:outputdef m4alias="mt_state_alias">
      <m4:param name="M4NAME0" value="geo_alias!STD_MT_GEO_DIV[*]" />
    </m4:outputdef>
    <m4:move>
      <m4:param name="geo_alias"
        value="mt_state_alias:STD_MT_GEO_DIV[12]" />
    </m4:move>
    <m4:outputdef m4alias="mt_town_alias">
      <m4:param name="M4NAME0"
        value="geo_alias!STD_MT_SUB_GEO_DIV[*]" />
    </m4:outputdef>
  </m4:job>
```

```

....
Registro actual:
<m4:item outputdef="mt_alias" item="STD_N_COUNTRY"/>.
<m4:item outputdef="mt_alias_state" item="STD_N_GEO_DIV"/>
<m4:item outputdef="mt_alias_town" item="STD_N_SUB_GEO_DIV"/>
Registro solicitado:
<m4:move>
  <m4:param name="geo_alias" value="mt_alias:STD_MT_COUNTRY[2]"/>
</m4:move>
<m4:move>
  <m4:param name="geo_alias"
    value="mt_alias_state:STD_MT_GEO_DIV[12]"/>
</m4:move>
<m4:move>
  <m4:param name="geo_alias"
    value="mt_alias_town:STD_MT_SUB_GEO_DIV[3]"/>
</m4:move>
<m4:item outputdef="mt_alias" item="STD_N_COUNTRY"/>.
<m4:item outputdef="mt_alias_state" item="STD_N_GEO_DIV"/>
<m4:item outputdef="mt_alias_town" item="STD_N_SUB_GEO_DIV"/>
</m4:page>

```

En este ejemplo, el desarrollador quiere saber el nombre del condado en la posición 3 (p. ej., Middlesex) del estado en la posición 12 (p. ej., Massachussets) del país en la posición 2 (p. ej., EE.UU.).

Se utiliza una etiqueta **move** dentro de la etiqueta **job** (*en el servidor*) para colocar el servidor de aplicaciones en el registro número 2 del nodo Países. Después, se utiliza la etiqueta **outputdef** para solicitar datos del nodo Divisiones geográficas (que ahora contiene los estados de EE.UU.). Se usa otra etiqueta **move** dentro de **job** para colocar el servidor de aplicaciones en el registro número 12 del nodo Divisiones geográficas, y después se usa la etiqueta **outputdef** para solicitar datos del nodo Subdivisiones geográficas (que ahora contiene los condados de Massachussets).

Si después de cerrar la etiqueta **job** se solicita un elemento desde cualquiera de las solicitudes de datos creadas con **outputdef**, se mostrarán los últimos elementos de la solicitud de datos ("Registro actual"), pero ése no es el objetivo del ejemplo.

El desarrollador tiene que volver a utilizar la etiqueta **move**, pero ahora fuera de **job** (*en el cliente*), para que se muestre el nombre del país de la posición 2, el estado de la posición 12 y el condado de la posición 3 ("Registro solicitado").

Creación de bucles con M4Tags

En numerosas ocasiones, resulta necesario usar bucles dentro de una página JSP para repetir determinadas operaciones. En general, puede usar cualquiera de los bucles permitidos en programación Java. No obstante, si quiere evitar el código Java en las páginas JSP, puede utilizar el M4Tag **loop** para crear bucles.

Concretamente hay un tipo de bucle especial que es muy común en los JSP de presentación. Este bucle es necesario para repetir operaciones por todos los registros recuperados de un nodo, definiendo la posición de cada repetición. Para este fin, puede utilizar la etiqueta **dataloop**.

Loop

Los parámetros de la etiqueta **loop** son los siguientes:

Parámetros obligatorios:

- *from*: índice inicial
- *to*: índice final

Parámetros opcionales:

- *step*: incremento
 - *+N*: incremento positivo
 - *-N*: incremento negativo
- *name*: nombre de la variable de índice del bucle
 - El nombre por defecto es *m4lix* (que corresponde a las siglas de *Meta4 Loop Index*, índice de bucle de *Meta4*).

Dentro del M4Tag **loop**, puede utilizar la variable de índice de bucle. Esta variable da el valor de iteración de cada paso del bucle desde el valor especificado en **from** hasta el indicado en *to*.

Para recoger su valor, utilice la sintaxis *&VAR.m4lix* indicada en el apartado *Parámetros*.

EJEMPLO

```
<m4:loop from="0" to="5">
    <m4:item m4name="geo!STD_MT_COUNTRY[&VAR.m4lix].STD_N_COUNTRY"/>
</m4:loop>
```

Si desea recorrer todos los registros del nodo, primero deberá contar el número de registros. Para ello, utilice el M4Tag **count**, cuyos parámetros son:

Parámetros obligatorios:

- *m4name*: nombre del nodo que se va a contar

Parámetros opcionales:

- *m4place*: especifica si la cuenta se hará sobre el conjunto local (*LOCAL*) o remoto (*REMOTE*). Por defecto, es sobre el remoto.
- *m4varname*: nombre de variable destino para obtener el valor

EJEMPLO

```
<m4:count m4varname="count"
    m4name="NOSORT:PERSON!STC_MT_TEST_PERSON"/>
```

Si el parámetro opcional *m4varname* no aparece, el número de registros se mostrará en la pantalla del navegador del cliente. Si dicho parámetro está presente, el número de registros no se mostrará en pantalla y se almacenará en la variable indicada en *m4varname*. Utilice esta variable en las páginas JSP como cualquier otro valor.

EJEMPLO

```
<% String count_minus_one = new Integer(new
Integer(count).intValue()-1).toString(); %>
<m4:loop from="0" to="<%= count_minus_one %>">
    <br><%= m4lix %>
    <m4:item m4name="PERSON!STC_MT_TEST_PERSON[&VAR.m4lix].LAST_NAME"/>,
    <m4:item m4name="PERSON!STC_MT_TEST_PERSON[&VAR.m4lix].FIRST_NAME"/>
</m4:loop>
```

Si el número de registros es 6, el bucle irá de 0 a 5. La variable auxiliar resta 1 del valor del recuento.

Dataloop

La etiqueta **dataloop** se puede usar para hacer repeticiones totales o parciales en los registros recuperados de un nodo, ya sea hacia delante o hacia atrás. Los parámetros de la etiqueta **dataloop** son los siguientes:

- *start*: índice inicial
- *stop*: índice final
- *count*: incremento
 - +N: incremento positivo (+*, todos los registros)
 - -N: incremento negativo (-*, todos los registros)

EJEMPLO

En el siguiente ejemplo se muestran los registros 20 a 10 del nodo.

```
<m4:dataloop outputdef="nosort" start="20" stop="10">
  <tr>
    <td><m4:item outputdef="nosort" item="LAST_NAME"/>
    <td><m4:item outputdef="nosort" item="FIRST_NAME"/>
  </tr>
</m4:dataloop>
```

Iterator

La etiqueta **iterator** es muy similar a **dataloop**, pero antes de usarla debe colocarse en el primer registro. Utilice el M4Tag **move** para situarse en el primer registro.

Dentro del M4Tag **move**, indique de la siguiente manera el registro en el que situarse:

```
<m4:move>
  <m4:param name="m4name" value="m4alias:node_id[i]" />
</m4:move>
```

donde *m4name* es el alias del Meta4Object en el M4Tag **datadef**, *m4alias* es el alias del *outputdef* e *id_nodo[i]* son los campos *i* del nodo cuyo código identificativo (ID) es *id_nodo[0]*. El *id_nodo[0]* proporciona el primer registro.

Al usar el M4Tag **iterator** hay que especificar una serie de variables y parámetros.

- El valor dado a la variable **m4rows** indica el número de registros que se quieren mostrar. Si se da el valor "*", se mostrarán todos los registros. Para mostrar sólo dos registros, hay que escribir **2**.
- El valor dado a la variable **m4node** indica el nodo del que se leen los datos.
- **m4:param** define el campo elegido.

Los datos que se presentarán en el navegador del cliente se indican después de la etiqueta **param** con el nombre dado al parámetro entre dos símbolos de \$: **\$M4ITEM0\$**

Para presentar el número del registro que se está mostrando, escriba: **\$CURRENT\$**

Para mostrar el número total de registros, escriba: **\$COUNT\$**.

En el siguiente ejemplo se indica cómo mostrar todos los registros de un campo:

EJEMPLO 1B CON ITERATOR²

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:page subsessionid="subsession_alias">
<m4:job><m4:datadef m4o="SCO_GEO_STRUCTURE" m4name="geo_alias"/>
  <m4:exec m4method="load_alias:geo_alias!LOAD"/>
  <m4:outputdef m4alias="country_alias">
    <m4:param
      name="M4NAME0"
      value="geo_alias!STD_MT_COUNTRY[*]" />
  </m4:outputdef>
</m4:job>
<P>MIS DATOS:<br>
<m4:move>
  <m4:param name="geo_alias"
    value="country_alias:STD_MT_COUNTRY[0]" />
</m4:move>
<m4:iterator
  m4rows="*" m4node="country_alias:geo_alias!STD_MT_COUNTRY">
  <m4:param name="M4ITEM0" value="STD_N_COUNTRY"/>
  $CURRENT$ of $COUNT$: $M4ITEM0$<br>
</m4:iterator>
</m4:page>
```

2. Éste es el Ejemplo 1B del apartado de diseño de páginas básicas, que se ha reescrito usando la etiqueta **iterator**.

Para mostrar los registros asociados a varios campos, se tendrá que asociar un parámetro a cada campo dentro del M4Tag **iterator**:

```
<m4:param name="M4ITEMi" value="field_id"/>
```

A continuación, dentro también del M4Tag **iterator**, se presentará el valor de cada parámetro, como se ha indicado en el apartado anterior, usando la siguiente notación:

```
$M4ITEMi$.
```

EJEMPLO1C CON ITERATOR

```
...
<m4:page subsessionid="subsession_alias">
<m4:job>
  <m4:datadef m4o="SCO_GEO_STRUCTURE" m4name="geo_alias"/>
  <m4:exec m4method="load_alias:geo_alias!LOAD"/>
  <m4:outputdef m4alias="country_alias">
    <m4:param
      name="M4NAME0"
      value="geo_alias!STD_MT_COUNTRY[*]"/>
    </m4:outputdef>
  </m4:job>
  <P>MIS DATOS:<br>
  <m4:move>
    <m4:param name="geo_alias"
      value="country_alias:STD_MT_COUNTRY[0]"/>
  </m4:move>
  <P>MIS DATOS:<br>
  <m4:iterator
    m4rows="*"
    m4node="country_alias:geo_alias!STD_MT_COUNTRY">
    <m4:param name="M4ITEM0" value="STD_N_COUNTRY"/>
    <m4:param name="M4ITEM1" value="STD_ID_COUNTRY"/>
    <m4:param name="M4ITEM2" value="STD_N_NACIONALITY"/>
    <m4:param name="M4ITEM3" value="STD_ID_NACIONALITY"/>
    $CURRENT$ de $COUNT$: $M4ITEM0$, $M4ITEM1$, $M4ITEM2$,
    $M4ITEM3$<br>
  </m4:iterator>
  <m4:endpage/>
```

Filtros y ordenación

Cuando se recuperan varios registros del servidor de aplicaciones, no aparecen en ningún orden concreto, a menos que se especifique que han de ordenarse.

Los filtros optimizan la transacción con el servidor de aplicaciones. Al aplicar un filtro, este servidor no envía al servidor Web todos los registros de nodos, sino sólo los necesarios.

Ordenación

Para ordenar los datos de los nodos, utilice el M4Tag **sortitems**. Esta etiqueta incluye el atributo *m4name* con el alias **datadef**, el código identificativo del nodo y el nombre dado a la ordenación. Cada elemento por el que se ordena estará definido por un **param** dentro del M4Tag **sortitems**. En él se indicará:

- *name*: nombre del elemento
- *value*: tipo de ordenación
 - *ASC*: ascendente
 - *DESC*: descendente

El M4Tag **sortitems** se coloca antes del M4Tag **outputdef** correspondiente.

En el siguiente ejemplo se ordenarán los registros del nodo STC_MT_TEST_PERSON del Meta4Object TC_MT_TEST_PERSON por el nombre de cada persona (campo de identificador FIRST_NAME). Se hacen dos ordenaciones: una ascendente y otra descendente.

EJEMPLO ORDER.JSP

```
<html><head><TITLE> Personas </TITLE></head><body>
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:startpage m4task="TC_TEST"/>
<m4:beginjob/>
<m4:datadef m4name="PERSON" m4o="TC_MT_TEST_PERSON"/>
<m4:exec m4method="EXECLOAD:PERSON.LOADONLY"/>
  <m4:sortitems m4name="PERSON!STC_MT_TEST_PERSON.ORDER">
    <m4:param name="FIRST_NAME" value="ASC"/>
  </m4:sortitems>
  <m4:outputdef m4alias="asc">
    <m4:param name="M4NAME0" value="PERSON!STC_MT_TEST_PERSON[0]"/>
  </m4:outputdef>
```

```

    <m4:sortitems m4name="PERSON!STC_MT_TEST_PERSON.ORDER">
      <m4:param name="FIRST_NAME" value="DESC"/>
    </m4:sortitems>
    <m4:outputdef m4alias="desc">
      <m4:param name="M4NAME0" value="PERSON!STC_MT_TEST_PERSON[0]"/>
    </m4:outputdef>
  <m4:endjob/>
  <m4:move>
    <m4:param name="PERSON" value="asc:STC_MT_TEST_PERSON[FIRST]"/>
    <m4:param name="PERSON" value="desc:STC_MT_TEST_PERSON[FIRST]"/>
  </m4:move>

  <h3>Orden ascendente</h3>
  <table border="1">
    <tr><td><b>NOMBRE</b></td>
    <td><b>APELLIDOS</b></td>
    <td><b>N°</b></td>
    </tr>
    <m4:dataloop outputdef="asc" count="*">
      <tr>
        <td><m4:item outputdef="asc" item="FIRST_NAME"/></td>
        <td><m4:item outputdef="asc" item="LAST_NAME"/></td>
      </tr>
    </m4:dataloop>
  </table>
  <br>

  <h3>Orden descendente<br>
  <table border="1">
    <tr>
      <td><b>NOMBRE</b></td>
      <td><b>APELLIDOS</b></td>
      <td><b>N°</b></td>
    </tr>
    <m4:dataloop outputdef="desc" count="*">
      <tr>
        <td><m4:item outputdef="desc" item="FIRST_NAME"/></td>
        <td><m4:item outputdef="desc" item="LAST_NAME"/></td>
      </tr>
    </m4:dataloop>
  </body>

```

```
</html>
```

Los dos registros incluidos en la base de datos se ordenarán por el nombre, primero en sentido ascendente y después descendente.

Filtrado

Los M4Tags filter permiten aplicar filtros a los datos cargados en el servidor de aplicaciones. De este modo, el acceso a los datos es más preciso y flexible.

Para filtrar los datos de un nodo, utilice el M4Tag **filter**. Consta de los siguientes parámetros obligatorios:

- *m4name*: nombre del filtro
 - *DATADEF* y *NODE*: se especifican del modo habitual
 - *Filtro*: nombre asignado al filtro
- **m4filter**: filtro que se va a aplicar

Los filtros se especifican usando código LN4.

EJEMPLO

```
<% String strM4Filter="IF STD_N_FIRST_NAME=\"JAMES\" THEN RETURN(1)";
%>

<m4:filter m4name="PERSON!STD_TR_PERSON.FILT" m4filter="<%=
strM4Filter %>" />
```

Defina el filtro antes del M4Tag **outputdef**. Después de él, se cancelará el filtro por medio del M4Tag **removefilter**. Si aplica varios filtros a un mismo Meta4Object, deberá borrar cada filtro antes de aplicar uno nuevo, a menos que desee aplicarlo de nuevo a los datos ya filtrados.

EJEMPLO

```
<m4:removefilter m4name="PERSON!RESULT.FILT" />
```

En el siguiente ejemplo, los registros del nodo STC_MT_TEST_PERSON del Meta4Object TC_MT_TEST_PERSON se filtran por el campo del código identificativo de persona. Se crean dos filtros: el primero usará el parámetro estático **ID_PERSON=2**; el segundo, un parámetro dinámico con la variable *index* dada por el cliente. Esta variable se asociará al mismo campo **ID_PERSON**.

EJEMPLO FILTER.JSP

```
...
<body>
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:startpage m4task="TC_TEST" />
<m4:beginjob />
    <m4:datadef m4name="PERSON" m4o="TC_MT_TEST_PERSON" />
    <m4:exec m4method="EXECLOAD:PERSON.LOADONLY" />
    <m4:outputdef m4alias="ALL">
        <m4:param name="M4NAME0" value="PERSON!STC_MT_TEST_PERSON[0]" />
    </m4:outputdef>

    <% String filter1 = "IF ID_PERSON = 2 THEN RETURN (1)"; %>
    <m4:filter
        m4name="PERSON!STC_MT_TEST_PERSON.FILTER1"
        m4filter="<%= filter1 %>" />
    <m4:outputdef m4alias="FILTER1">
```

```

        <m4:param name="M4NAME0" value="PERSON!STC_MT_TEST_PERSON[0]"/>
    </m4:outputdef>
    <m4:removefilter
        m4name="PERSON!STC_MT_TEST_PERSON.FILTER1"/>

    <%
String reg = request.getParameter("index");
String filter2 = "IF ID_PERSON = "+reg+" THEN RETURN (1)";
%>
    <m4:filter
        m4name="PERSON!STC_MT_TEST_PERSON.FILTER2"
        m4filter="<%= filtro2 %>"/>
    <m4:outputdef m4alias="FILTER2">
        <m4:param name="M4NAME0" value="PERSON!STC_MT_TEST_PERSON[0]"/>
    </m4:outputdef>
    <m4:removefilter
        m4name="PERSON!STC_MT_TEST_PERSON.FILTER2"/>

<m4:endjob/>

<m4:move>
    <m4:param name="PERSON" value="ALL:STC_MT_TEST_PERSON[FIRST]"/>
    <m4:param name="PERSON"
value="FILTER1:STC_MT_TEST_PERSON[FIRST]"/>
    <m4:param name="PERSON"
value="FILTER2:STC_MT_TEST_PERSON[FIRST]"/>
</m4:move>

<h3>1. Todos los elementos sin filtrar </h3>
<table border="1">
    <tr>
        <td><b>NOMBRE</b></td>
        <td><b>APELLIDOS</b></td>
        <td><b>n</b></td>
    </tr>
    <m4:iterator m4node="ALL:PERSON!STC_MT_TEST_PERSON">
        <m4:param name="M4ITEM0" value="FIRST_NAME"/>
        <m4:param name="M4ITEM1" value="LAST_NAME"/>
        <tr>
            <td>$M4ITEM0$</td>
            <td>$M4ITEM1$</td>

```

```

        <td>${CURRENT$ of $COUNT$</td></tr>
    </m4:iterator>
</table><br>

```

<h3>2. Elementos filtrados 1 por rec=2:

```

<br><br>
<table border="1">
    <tr>
        <td><b>NOMBRE</b></td>
        <td><b>APELLIDOS</b></td>
        <td><b>n</b></td>
    </tr>
    <m4:iterator
        m4node="FILTER1:PERSON!STC_MT_TEST_PERSON">
        <m4:param name="M4ITEM0" value="FIRST_NAME"/>
        <m4:param name="M4ITEM1" value="LAST_NAME"/>
        <tr>
            <td>${M4ITEM0$</td>
            <td>${M4ITEM1$</td>
            <td>${CURRENT$ of $COUNT$</td>
        </tr>
    </m4:iterator>
</table><br>

```

<h3>3. Elementos filtrados 2 por rec=index=<%= reg %>:

```

<br><br>
<table border="1">
    <tr>
        <td><b>NOMBRE</b></td>
        <td><b>APELLIDOS</b></td>
        <td><b>n</b></td>
    </tr>
    <m4:iterator
        m4node="FILTER2:PERSON!STC_MT_TEST_PERSON">
        <m4:param name="M4ITEM0" value="FIRST_NAME"/>
        <m4:param name="M4ITEM1" value="LAST_NAME"/>
        <tr>
            <td>${M4ITEM0$</td>
            <td>${M4ITEM1$</td>
            <td>${CURRENT$ of $COUNT$</td>
        </tr>

```

```

        </m4:iterator>
    </table>

    <m4:endpage/>
</body>
</html>

```

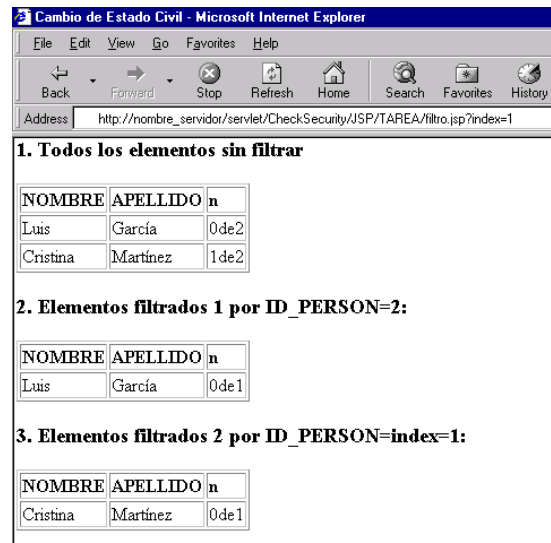
La variable *index* se pasa en el navegador tras el nombre de la página por medio del símbolo ?

```

http://nombre_servidor_Web/servlet/CheckSecurity/JSP/TAREA/
filter.jsp?index=1

```

Figura 23. Resultado en el navegador del cliente de filter.jsp



Cambio de la presentación de los datos

Cambiar la presentación de los datos que devuelve M4Tag puede ser útil en distintas situaciones.

En el caso de que a una misma aplicación Web vayan a acceder usuarios con distintas configuraciones regionales, parece lógico que las fechas, números o valores de moneda aparezcan en el formato al que está acostumbrado cada usuario. En ocasiones será necesario tener en cuenta las excepciones. Los M4Tags de presentación utilizan un mecanismo de **formateo de salida** predeterminado que lo hacen posible.

A veces, los elementos que se van a mostrar pueden contener **caracteres especiales** que pueden dificultar el funcionamiento de las herramientas de presentación HTML o JavaScript. Algunos M4Tags incluyen atributos especiales para no tener en cuenta los caracteres especiales o convertirlos a sus equivalentes en HTML, más seguros.

Formateo de salida

La mayoría de los tags de presentación, como **item**, **count**, **current** o **outputexec**, pueden formatear determinados tipos de elementos en función de las máscaras de formateo predeterminadas que haya en el Meta4Object Parametrización.

Las máscaras predeterminadas se pueden consultar usando la opción de menú Configuración de la aplicación de la Estación de trabajo de desarrollo (la aplicación es THINCLIENT, y la sección es FORMAT).

Además, existen atributos especiales para algunos M4Tags, con lo que es posible tener en cuenta las excepciones de formato de la salida.

Los atributos opcionales son:

- *m4format*: indica una máscara que especifica la transformación de los datos de salida.
 - La sintaxis de la máscara se describe en el Apéndice A.
- *typename*: indica que se debería utilizar un tipo de formato distinto.
 - La lista de tipos existentes es la lista predeterminada del Meta4Object Parametrización.

De este modo, si se va a mostrar un elemento concreto en un formato distinto al predeterminado, el desarrollador puede comprobar si hay otro tipo de formato que pueda servir (y hacer referencia a él usando el atributo *typename*), o escribir una máscara directamente en la etiqueta (usando el atributo *m4format*).

Si se desea hacer un cambio general, habrá que pensar en cambiar las máscaras predeterminadas.

A modo de ejemplo, imagine un método que devuelve una fecha que se ha de mostrar en formato largo (incluyendo los nombres del día de la semana y del mes).

Se puede utilizar la etiqueta **outputexec** para mostrar el valor obtenido al ejecutar un método. Esta etiqueta se emplea después de la etiqueta de cierre **beginjob** o **job**. Su único parámetro obligatorio es el alias de la solicitud de ejecución del método.

EJEMPLO

```
<m4:outputexec m4alias="method_alias"/>
```

Por defecto, el resultado que se muestra coincidirá con el valor de Configuración de la aplicación de la clave DATE en la sección FORMAT de la aplicación THINCLIENT (por ejemplo, *M/d/yyy* cuando el idioma de conexión sea inglés).

Como alternativa al formato DATE por defecto, se puede elegir el formato DATETEXT (por ejemplo, *EEEE, MMMM d, yyyy* cuando el idioma de conexión sea inglés).

Si ninguno satisface las necesidades del desarrollador, se puede especificar una máscara personalizada, siguiendo las reglas de sintaxis del Apéndice A.

EJEMPLO: DATEFORMAT.JSP

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:page subsessionid="example">
<m4:job>
<m4:datadef m4name="PERSON" m4o="TC_MT_TEST_PERSON"/>
<m4:exec m4object="PERSON" method="LOAD" alias="alias_load"/>
<m4:move>
  <m4:param name="PERSON" value="STC_MT_TEST_PERSON[0]"/>
</m4:move>
<m4:exec m4object="PERSON" node="STC_MT_TEST_PERSON"
  method="RETURN_NEXT_BIRTHDAY" alias="alias_return"/>
</m4:job>
...
```

Ejemplos de formateo:

```
(1) <m4:outputexec m4alias="alias_return"/><br>
```

```
(2)<m4:outputexec m4alias="alias_return" typename="datetext"/><br>
(3)<m4:outputexec m4alias="alias_return" m4format="EEE dd MMM, yy"/>
</m4:page>
```

Si al ejecutar el método se obtiene 2003-09-17 00:00:00, los resultados de las etiquetas son los siguientes:

- (1) (M/d/yyyy): 9/17/2003.
 - (2) (EEEE, MMMM d, yyyy): Wednesday, September 17, 2003.
 - (3) (EEE dd MMM, yy): Wed Sep 17, 2003.
-

Caracteres especiales

Las etiquetas **item** y **label** incluyen atributos para poder presentar correctamente los caracteres especiales contenidos en el resultado. Estos atributos son:

- *htmlsafe*: los valores pueden ser (true|false), el valor por defecto es false.
 - Si es true, algunos caracteres (como los caracteres no ASCII, el signo mayor o menor, y en general todos aquellos que tengan significado dentro de HTML/WML) se convierten a su equivalente en código Unicode.
- *jsafe*: los valores pueden ser (true|false), el valor por defecto es false.
 - Si es true, algunos caracteres (como la comilla doble y simple, y en general todos aquellos que tengan significado dentro de una función JavaScript) son precedidos por el carácter de escape, una barra invertida ("\").

En el siguiente ejemplo se muestra el efecto que tiene usar estos atributos.

EJEMPLO: SPECIALCHARACTERS.JSP

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:page subsessionId="example">
<m4:job>
<m4:datadef m4name="PERSON" m4o="TC_MT_TEST_PERSON"/>
<m4:exec m4object="PERSON" method="LOAD" alias="alias_load"
<m4:outputdef m4alias="out">
    <m4:param name="M4NAME0" value="PERSON!STC_MT_TEST_PERSON[*]"/>
</m4:outputdef>
</m4:job>
```

Caracteres especiales:

```
(1) <m4:item outputdef="out" item="LAST_NAME"><br>
(2) <m4:item outputdef="out" item="LAST_NAME" htmlspecialchars="true"/><br>
(3) <m4:item outputdef="out" item="LAST_NAME" jsafe="true"/><br>
(4) <m4:item outputdef="out" item="LAST_NAME" jsafe="true"
    htmlspecialchars="true"/>
</m4:page>
```

Un ejemplo sería el siguiente:

- (1) Eugène "El Niño" D'Orleans
- (2) (htmlspecialchars="false" jsafe="true"): Eugène \"El Niño\" D'Orleans
- (3) (htmlspecialchars="true" jsafe="false"): Eugène "El Niño" D'Orleans



Para obtener más detalles sobre los caracteres que se convierten, consulte la referencia [M4 TagLib](#).

Diseño de una página de inicio de sesión

Página de inicio de sesión

Para ejecutar los ejemplos expuestos en los apartados anteriores, antes debe haber iniciado una sesión introduciendo un nombre de usuario y una contraseña válidos. Puede utilizar alguna de las páginas de inicio de ejemplo que se incluyen con la tecnología. No obstante, si prefiere diseñar su propia página de inicio, debe saber cómo funciona el proceso de inicio de sesión.

La forma más sencilla de iniciar sesión es ejecutar directamente el servlet *login* introduciendo los dos parámetros del nombre de usuario (**_USER**) y la contraseña (**_PASSWD**) en la barra de dirección del navegador, con la siguiente sintaxis:

```
http://nombre_servidor/servlet/login?_USER=usuario&_PASSWD=contraseña
```

Sustituya así las cadenas de este ejemplo:

- **nombre_servidor** es el nombre del servidor Web que está utilizando.
- **usuario** es el usuario de la aplicación Meta4.
- **contraseña** es la contraseña de la aplicación Meta4.

De esta forma, si el usuario y la contraseña son correctos, tendría ya abierta una sesión con el servidor de aplicaciones de Meta4. El nombre de usuario y la contraseña son indispensables para el correcto registro del usuario en el sistema. Si no se proporcionan, el *servlet* mostrará el correspondiente mensaje de error.

Opcionalmente, el servlet *login* puede recibir parámetros correspondientes al idioma (**_LANG**) y a la página de inicio de la aplicación (**_URL**), también denominada **Página del portal**. De nuevo, la forma más sencilla de dar valores a estos dos parámetros y pasárselos al *servlet* es a través de la barra de direcciones del propio navegador:

```
http://nombre_servidor/servlet/login?_USER=usuario&_PASSWD=contraseña  
&_LANG=número&_URL=/servlet/CheckSecurity/JSP/tarea/página.jsp
```

- **número**: identificador del idioma; por ejemplo, 3 para español y 2 para inglés.

- **página.jsp**: página inicial que se va a cargar
- **tarea**: tarea donde se encuentra la página

Tras cargar el nombre de usuario y la contraseña, el *servlet* comprueba si se han proporcionado los parámetros de idioma y página inicial. Si el *servlet login* no recibe estos dos parámetros, se utilizarán como página inicial e idioma los definidos por defecto en los archivos de configuración. Finalmente, el *servlet* intentará registrar al usuario con los parámetros proporcionados tanto por el usuario como por el archivo de configuración. Si se produjese algún error en este proceso, aparecería el correspondiente mensaje de error.

Iniciar una sesión como se ha indicado anteriormente (pasando parámetros al *servlet login* desde la cadena de consulta) no es la forma más recomendable (la contraseña está visible, lo que reduce enormemente la seguridad al iniciar la sesión). Se aconseja que el usuario introduzca estos parámetros en un formulario HTML, la **página de inicio de sesión**.

En el siguiente ejemplo, *basic_login.jsp*, se muestra al usuario del sistema una página de inicio de sesión como un formulario, para que el usuario introduzca el nombre de usuario y la contraseña. El código de este archivo es el código HTML necesario para mostrar un formulario. La petición de servicio se efectúa desde esta página de entrada, como resultado de invocar el *servlet login* desde la URL del formulario. Por ejemplo:

```
<form method="POST" name="logon" action="/servlet/login/">
```

A través de este formulario, el *servlet* recibe los parámetros correspondientes al nombre y la contraseña del usuario.

También se especifican la página de inicio y el idioma. No obstante, estos parámetros no los puede introducir el usuario porque en este ejemplo se envían como variables ocultas.

EJEMPLO: BASIC_LOGIN.JSP

El siguiente código corresponde a una página de inicio de sesión básica.

```
<form method="POST" name="logon" action="/servlet/login/">
  <table>
    <tr><td><b>Nombre:</b></td></tr>
    <tr><td><input type="text" name="_USER" size="16"></td></tr>
    <tr><td><b>Contraseña:</b></td></tr>
    <tr><td><input type="password" name="_PASSWD" size="16"></td></tr>
    <input type="hidden" name="_LANG" value="3">
    <input type="hidden" id="_URL" name="_URL"
    value="/servlet/CheckSecurity/JSP/tctools/portal.jsp">
    <tr><td align="center"><input type="submit" value="Enter">
    </td></tr>
  </table>
</form>
```

Inicio de sesión anónimo

En algunos casos, puede ser necesario realizar algunas operaciones para conocer un nombre de usuario o una contraseña. Por ejemplo, si se conecta por primera vez al módulo Ofertas de empleo, debe registrarse y crear un nombre de usuario y una contraseña para poder insertar su currículum, y ver o seleccionar ofertas de empleo. Pero al registrarse como nuevo usuario debe realizar determinadas operaciones para las que es necesario abrir una sesión de la aplicación Meta4.

Esta conexión se realiza por medio del usuario anónimo o invitado. Una vez dado de alta el nuevo usuario con su contraseña correspondiente, las páginas deberían finalizar sesión con el usuario anónimo e iniciar sesión de nuevo con el usuario recién creado. Este proceso debería ser transparente. Es decir, que las páginas deben estar diseñadas de tal forma que para realizar la desconexión y posterior conexión no haga falta volver a la página de inicio de sesión.

El usuario anónimo debe ser un usuario de la aplicación Meta4 creado en la aplicación.



Es importante subrayar que se debe crear dicho usuario con los permisos de seguridad mínimos necesarios para realizar las tareas encomendadas al usuario invitado.

Una vez creados el usuario, el nombre de usuario y la contraseña del usuario invitado, se deberían introducir en el archivo de configuración *configclient.xml* y se reiniciará el motor de servlets.

EJEMPLO DE INTRODUCCIÓN DEL NOMBRE Y LA CONTRASEÑA DE USUARIO INVITADO EN CONFIGCLIENT.XML.

```
<M4CLIENT>
  <ANONYMOUS>
    <USER>ANONYMOUS_USER</USER>
    <PASSWORD>ANONYMOUS_USER</PASSWORD>
  </ANONYMOUS>
</M4CLIENT>
```

Página de inicio de sesión anónimo

Para entrar en la aplicación con un usuario anónimo, se debe crear en primer lugar un formulario con un campo oculto, **TYPE_LOGON**, que tenga el valor *ANONYMOUS*. El usuario no verá esta conexión y en los valores no se distinguirá entre mayúsculas y minúsculas. Este parámetro debe enviarse al *servlet* de conexión.

EJEMPLO

```
<form id="login_new" name="login" action="/servlet/login/"
method="post">
  <input size="15" type="hidden" id="_TYPE_LOGON"
    name="_TYPE_LOGON" value="ANONYMOUS"></input>
  Creación de nuevos usuarios
  <input type="hidden" id="_URL" name="_URL"
    value="/servlet/CheckSecurity/JSP/jobsite_g/
    alta_usuarios_nuevos.jsp?estado=0"> </input>
  <input type="image" src="/icons/spanish/alta_usuario.gif"
    alt="Nuevo usuario"></input>
  Haga clic aquí para registrar un nuevo usuario.
</form>
```

Inicio de sesión para dispositivos WML

La tecnología de Meta4 permite el desarrollo de una aplicación Web de Meta4 que soporte dispositivos que entiendan el protocolo WML, que se utilizan en los teléfonos móviles.

El *servlet* de login permite que un dispositivo móvil pueda redirigirse, tras una autenticación con éxito, a las páginas JSP securizadas típicas de Meta4. Utilizando un parámetro del *servlet* de login, el desarrollador puede diseñar una página de conexión que acepta usuario y contraseña y realiza la primera redirección de un modo válido para un agente de usuario WML.

Página de inicio de sesión WML

El *servlet* de login acepta un nuevo parámetro, **_REDIRECT**, que se utiliza cuando se quiera realizar una conexión autenticada con un dispositivo que no soporte HTML, como por ejemplo, un teléfono móvil.

El parámetro **_REDIRECT** se comporta de tres maneras distintas, dependiendo de su valor:

1. Valor es `sendRedirect`:

El *servlet* de login redirige a la página de bienvenida enviando un código HTTP 302. Si el dispositivo móvil implementa correctamente el protocolo HTTP, conseguirá redirigir correctamente a dicha página. Este es el valor recomendado.

2. Valor es `timerCard`:

El *servlet* de login envía código WML al dispositivo para que se realice la redirección.

3. Valor nulo o vacío:

El *servlet* de login redirige a la página de bienvenida enviando HTML que contiene un tag meta `http-equiv="refresh"`. Este tipo de redirección es válido para un cliente HTML, pero no para otros dispositivos

Para que la primera página muestre correctamente información protegida por el esquema de seguridad de las páginas de Meta4, el agente de usuario tiene que soportar *cookies*.

EJEMPLO

```
<?xml version="1.0"?>
<!DOCTYPE wml PUBLIC "-//WAPFORUM//DTD WML 1.3//EN" "http://
www.wapforum.org/DTD/wml13.dtd">
<wml>
```

```
<card id="login" title="Connect to Meta4">
  <p align="center">
    Enter your identification
    <br/>
    Username: <input name="Meta4Username" type="text"
    emptyok="false" value=""/>
    <br/>
    Password: <input name="Meta4Password" type="password"
    emptyok="false" value=""/>
  </p>
  <do type="accept" label="Accept">
    <go href="/servlet/login" method="post">
      <postfield name="_LANG" value="2"/>
      <postfield name="_URL" value="/servlet/CheckSecurity/
      JSP/tctools/wmlportal.jsp"/>
      <postfield name="_REDIRECT" value="sendRedirect"/>
      <postfield name="_USER" value="$ (Meta4Username)"/>
      <postfield name="_PASSWD" value="$ (Meta4Password)"/>
    </go>
  </do>
</card>
</wml>
```

Opciones de sesión

Cuando inicie una sesión, usando una de las técnicas descritas en los apartados anteriores, conservará el rol y la organización predeterminados para su usuario. No obstante, es posible que tenga que trabajar con un rol o una organización distintos de los predeterminados.

Hay dos formas distintas de diseñar una página de inicio de sesión que permita cambiar el rol o la organización:

- **Introducción de parámetros de cambio:** Puede pasar distintos parámetros al servlet de conexión para especificar si quiere un cambio de rol (M4_CHANGE_ROLE) o un cambio de organización (M4_CHANGE_ORGANIZATION).
 - Se deben incluir códigos especiales en la Página de portal para detectar que se ha ejecutado el servlet de conexión con estos parámetros.
 - Este código de la página del portal se encargará de mostrar una lista de roles u organizaciones entre los que puede elegir. Se recomienda utilizar este método cuando los usuarios tienen permiso para ver la lista de todos sus roles y organizaciones.

En el siguiente ejemplo se muestra un formulario de conexión desde el que se pasan los parámetros:

EJEMPLO: LOGIN_CHANGE.JSP

```
<form method="POST" name="logon" action="/servlet/login/">
  Name: <input type="text" name="_USER" size="16"><br>
  Password: <input type="password" name="_PASSWD" size="16"><br>
  Change Role:
    <input type="checkbox" name="M4_CHANGE_ROLE"><br>
  Change Organisation:
    <input type="checkbox" name="M4_CHANGE_ORGANIZATION"><br>
  <input type="submit" value="Enter">
</form>
```

-
- **Introducción de parámetros directos:** El rol (M4_ROLE) o la organización se pueden pasar directamente como parámetros del servlet de conexión (M4_ORGANIZATION).

Cuando se envía el parámetro **M4_ROLE** al *servlet* de conexión, un método del Meta4Object de sesión comprueba si el usuario tiene este rol.

- En caso afirmativo, se asigna el rol al usuario y se realiza la conexión.

- Si no es así, el usuario se desconecta y es redirigido a la página de error.

Cuando se envía el parámetro **M4_ORGANIZATION** al *servlet* de conexión, un método del Meta4Object de sesión comprueba que sólo hay un rol asignado a la organización.

- Si hay más de un rol para la organización seleccionada, el usuario se desconecta y es redirigido a la página de error.
- Si no hay organización para los roles existentes, el usuario se desconecta y es redirigido a la página de error.
- Si sólo hay un rol, se asigna la organización al usuario y se realiza la conexión.

Al definir los parámetros **M4_ROLE** y **M4_ORGANIZATION**, un método del Meta4Object de sesión comprueba si el usuario tiene este rol y organización.

- En caso afirmativo, se asigna el rol y la organización al usuario, y se realiza la conexión.
- De lo contrario, hay que introducir un código especial en la página de portal para mostrar la lista de organizaciones disponibles para el usuario.

En el siguiente ejemplo se muestra un formulario de conexión en el que el usuario puede introducir un rol y una organización.

EJEMPLO: LOGIN_DIRECT.JSP

```
<form method="POST" name="logon" action="/servlet/login/">
  Username: <input type="text" name="_USER" size="16"><br>
  Password: <input type="password" name="_PASSWD" size="16"><br>
  Role:
    <input type="text" name="M4_ROLE" size="16"><br>
  Organisation:
    <input type="text" name="M4_ORGANIZATION" size="16"><br>
  <input type="submit" value="Enter">
</form>
```

M4Tags de bolsas de sesión

Los M4Tags de bolsas permiten trabajar con contenedores que almacenan pares de elemento - valor. Las bolsas se mantienen en toda la sesión del usuario. Los M4Tags relacionados con bolsas son:

- *putbagvalue*: asigna un valor (parámetro *m4value*) a un elemento (parámetro *m4key*). Por ejemplo:

```
<m4:putbagvalue m4key="USER" m4value="JOHN"/>
```

- *getbagvalue*: recupera el valor de un elemento. Por ejemplo:

```
<m4:getbagvalue m4key="USER"/>
```

- *clearbagvalue*: vacía la bolsa. Por ejemplo:

```
<m4:clearbag/>
```

- *removebagvalue*: elimina un elemento de la bolsa. Por ejemplo:

```
<m4:removebagvalue m4key="USER"/>
```

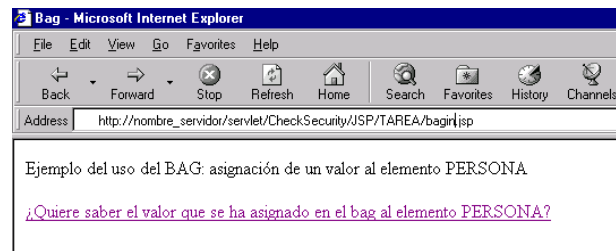
En el siguiente ejemplo, se asigna un valor a un elemento de una página. Posteriormente, este valor se recupera en otra página:

- *bagin.jsp*: es la página que agrega un nuevo valor a la bolsa.
- *bagout.jsp*: es la página que recupera el valor anterior de la bolsa.

EJEMPLO BAGIN.JSP

```
<HTML>
<HEAD>
<TITLE>Bolsa </TITLE>
</HEAD>
<BODY BGCOLOR="#FFFFFF">
Ejemplo: asignación de un valor al elemento PERSONA mediante una
bolsa.< BR><BR>
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:startpage m4task="TC_TEST"/>
<m4:putbagvalue m4key="PERSONA" m4value="Laura García"/>
<A HREF="my_bag_03.jsp">
¿Desea conocer el valor que se ha asignado en la bolsa al elemento
PERSONA?</A>
<m4:endpage/>
</BODY>
</HTML>
```

Figura 24. Página bagin.jsp

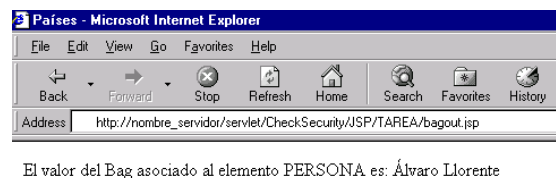


Al hacer clic en el enlace, se abre la página bagout.jsp. Esta página dará el valor asignado.

EJEMPLO BAGOUT.JSP

```
<HTML>
<HEAD>
<TITLE>Bolsa </TITLE>
</HEAD>
<BODY BGCOLOR="#FFFFFF">
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:startpage m4task="TC_TEST"/>
Valor de la bolsa asociado al elemento PERSONA:
<m4:getbagvalue m4key="PERSONA"/>
<m4:endpage/>
</BODY>
</HTML>
```

Figura 25. Página bagout.jsp



Las páginas asociadas a todas las tareas pueden obtener acceso a los valores de la bolsa, siempre que estén en la misma sesión. El número máximo de valores que pueden almacenarse en la bolsa es de 20.

Gestión de archivos

El cliente HTML permite descargar archivos almacenados en los campos de un Meta4Object y guardar un archivo en un campo de un Meta4Object. La transferencia de archivos desde una página JSP implica dos procesos diferentes:

- Almacenamiento de archivos: se transfieren desde el navegador del cliente al servidor de aplicaciones para que se almacenen en un elemento de un Meta4Object.
- Recuperación de archivos: es el proceso inverso al anterior; un archivo almacenado en un elemento de un Meta4Object se descarga del servidor de aplicaciones al cliente para que se vea en el navegador.

En el siguiente ejemplo se muestran ambos procesos. El sistema guarda una copia del archivo en un campo BLOB (MISC_DOC) de un Meta4Object con información personal (TC_MT_TEST_PERSON). En este campo se guarda información adicional sobre cada persona; por ejemplo, los currículos. A continuación, podrá recuperar este archivo y verlo en el navegador.

El ejemplo tiene dos páginas:

1. *fileform.html*: formulario HTML estándar. Para transferir archivos, el parámetro *enctype* del formulario debe ser *multipart/form-data*. El tipo de entrada del formulario tiene que ser *file* para abrir el cuadro de diálogo de selección de archivos. En el formulario, hay que introducir el nombre completo del archivo que se desea almacenar en la base de datos. En el parámetro *action* se debe indicar la página JSP que recogerá la variable del formulario.



Para leer los parámetros del formulario en vez de los nombres de archivo, utilice `pageContext.getAttribute("parámetro")` y no `request.getParameter("parámetro")`. Estas lecturas se deben hacer siempre bajo `<m4:startpage />` ya que en cualquier otro caso, no se obtendrá ningún valor.



Para que el contexto `<%String zarchivo = (String)pageContext.getAttribute("archivo");%>` no se lea antes de estar disponible, el código debe ser llamado siempre después del `M4Tag startpage`, ya que de lo contrario en el `pageContext` el atributo no está devolviendo el valor null.

EJEMPLO FILEFORM.HTML

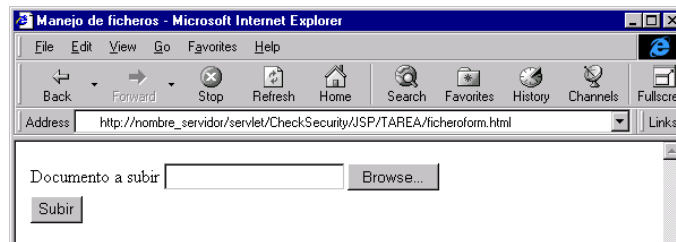
```
<html>

<head>
<meta http-equiv="Content-Type" content="text/html">
<title>Control de archivos</title>
</head>

<body>
<table>
  <form method="post" name="formDocs" enctype="multipart/form-data"
    action="/servlet/CheckSecurity/JSP/CAL_TASK/archivoupdown.jsp">
    <tr><td>Documento por subir</td>
    <td><input type="file" name="doc" size=21></td></tr>
    <tr><td><input type="submit" value ="Subir"></td></tr>
  </form>
</table>
...
```

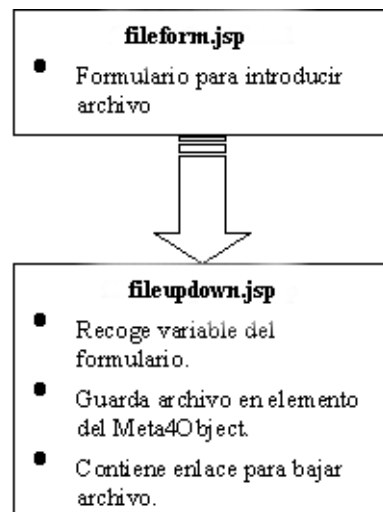
Al cargar esta página en el navegador, el resultado en pantalla será:

Figura 26. Formulario fileform.html



-
2. *fileupdown.jsp*: es la página que recoge el archivo introducido en el formulario y lo guarda en el campo MISC_DOC del Meta4Object TC_MT_TEST_PERSON. Además, tendrá un enlace para bajar ese archivo y mostrarlo en el navegador. Se analiza el código de esta página.

Figura 27. Esquema del ejemplo



Almacenamiento de archivos

Para guardar los archivos en un elemento de Meta4Object, dicho elemento debe ser de tipo BLOB. Además, debe guardar el M4Tag **setfile** en la página JSP.

```
<m4:setfile
    m4blob="ALIASM4O!NODO [REGISTRO] .ITEM"
    m4path="archivo" />
```

En este caso, el parámetro *m4blob* especifica el elemento de Meta4Object en el que el sistema va a guardar el archivo. El parámetro *m4path* indica el nombre completo del archivo.

En el ejemplo, *m4blob* indica que el archivo se va a guardar en el primer registro (o registro [0]) del campo MISC_DOC, en el nodo STC_MT_TEST_PERSON del Meta4Object cuyo alias es PERSON. Además, el valor de *m4path* se recoge del formulario y almacena en la variable *doc*, que es un parámetro dinámico que se pasa por referencia.

Tras asociar el archivo al elemento del Meta4Object, guarde los valores asignados mediante la ejecución del método *persist*.

EJEMPLO FILEUPDOWN.JSP

```
<html>
<head>
<meta http-equiv="Content-Type" content="text/html">
<title>Control de archivos</title>
</head>
<body>
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:startpage m4task="CAL_TASK"/>
<m4:beginjob/>
    <m4:datadef m4name="PERSON" m4o="TC_MT_TEST_PERSON"/>
    <m4:exec m4method="EXECLOAD:KU.LOADONLY"/>
    <m4:setfile
        m4blob="PERSON!STC_MT_TEST_PERSON[0].MISC_DOC"
        m4path="&REQUEST.doc" />
    <m4:exec m4method="PERSIST:PERSON.PERSIST"/>
    <m4:outputdef>
        <m4:param name="M4NAME0" value="PERSON!STC_MT_TEST_PERSON[0]"/>
    </m4:outputdef>
<m4:endjob/>
```

```
...

<m4:endpage/>
</body>
</html>
```

Recuperación de archivos

Para ver un archivo de la base de datos con el navegador del cliente, en primer lugar debe recuperarlo del elemento BLOB del Meta4Object que contiene el archivo. A continuación, envíelo al cliente mediante el servidor de aplicaciones. Para ello, debe llamar al servlet *download_blob* desde la página JSP. El servlet recibe dos parámetros: *task* (con la *m4task* correspondiente al M4Tag **startpage**) e *item* (con el elemento del Meta4Object que almacena el archivo):

```
http://nombre_servidor/servlet/download_blob?task=M4_TASK
&item=ALIASM4O!NODO[REGISTRO].ITEM
```

En el ejemplo, se selecciona el registro con la variable *\$CURRENT\$*. Previamente se ha movido dicha variable al primer registro del nodo con el M4Tag **move**. El *servlet* se llama dentro de un enlace HTML.

El código completo del ejemplo es el siguiente:

EJEMPLO FILEUPDOWN.JSP

```
<html>
<head>
<meta http-equiv="Content-Type" content="text/html">
<title>Control de archivos</title>
</head>
<body>
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:startpage m4task="CAL_TASK"/>
<m4:beginjob/>
  <m4:datadef m4name="PERSON" m4o="TC_MT_TEST_PERSON"/>
  <m4:exec m4method="EXECLOAD:KU.LOADONLY"/>
  <m4:setfile
    m4blob="PERSON!STC_MT_TEST_PERSON[0].MISC_DOC"
    m4path="&REQUEST.doc" />
  <m4:exec m4method="PERSIST:PERSON.PERSIST"/>
<m4:outputdef>
```

```

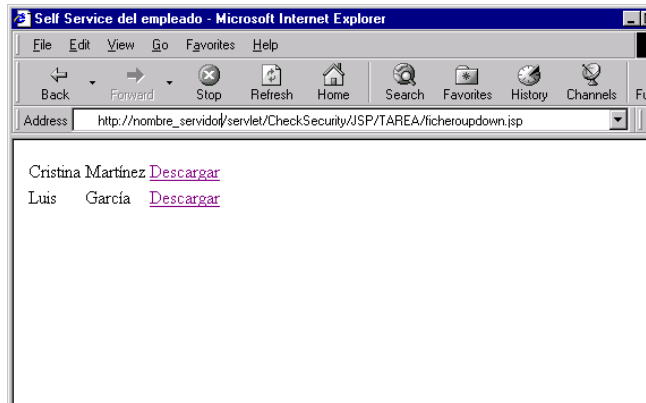
        <m4:param name="M4NAME0" value="PERSON!STC_MT_TEST_PERSON[0]" />
    </m4:outputdef>
<m4:endjob/>

<m4:move>
    <m4:param name="PERSON" value="STC_MT_TEST_PERSON[FIRST]" />
</m4:move>
<table>
<m4:iterator m4node="PERSON!STC_MT_TEST_PERSON">
    <m4:param name="M4ITEM0" value="FIRST_NAME" />
    <m4:param name="M4ITEM1" value="LAST_NAME" />
    <tr>
        <td>$M4ITEM0$</td>
        <td>$M4ITEM1$</td>
        <td>
            <a href="/servlet/download_blob?task=CAL_TASK&item=PERSON!
STC_MT_TEST_PERSON[$CURRENT$].MISC_DOC">Descargar</a>
        </td>
    </tr>
</m4:iterator>
</table>
<m4:endpage/>
</body>
</html>

```

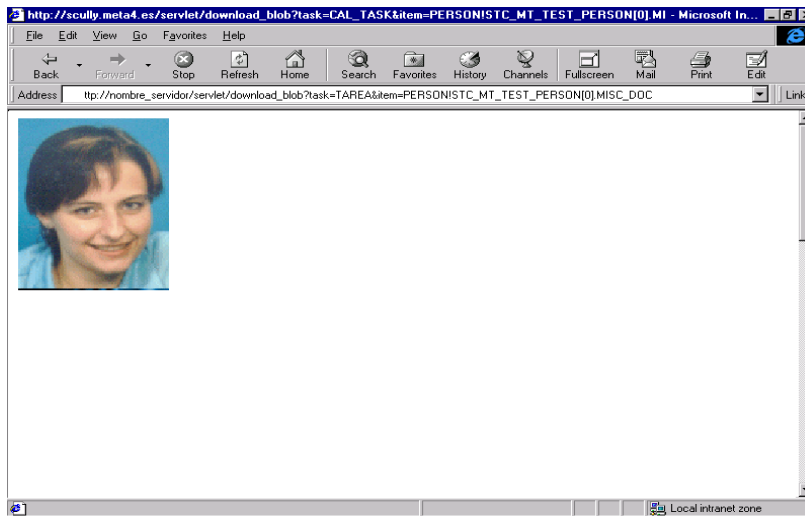
Cuando en el formulario **fileform.html** introduce el nombre completo de un archivo y hace clic en el botón *Subir*, se muestra en el navegador la página `fileupdown.jsp` que se ha procesado previamente en el servidor. El archivo se guardará en el elemento del `Meta4Object` correspondiente.

Figura 28. Página fileupdown.jsp



Al hacer clic en el primer enlace, aparecerá en el navegador el archivo almacenado anteriormente en el primer registro.

Figura 29. Archivo descargado



Para que el navegador no solicite la ruta para guardar en disco el archivo recuperado, sino que reconozca su formato y lo abra, tiene que reconocer su MIME-Type.

En el archivo que se muestra a continuación, se pueden especificar las extensiones de los archivos asociados a cada MIME-Type:

```
m4mimetypes.properties
```

Gestión de errores

En este apartado se explica cómo gestionar los errores del servidor de aplicaciones.

El M4Tag **endjob** marca el fin de la traducción. Además, controla el resultado de una traducción o método en los siguientes casos: error (*ERROR*), aviso (*WARNING*) y éxito (*SUCCESS*).

El control consiste en indicar una dirección URL hacia la que se enviará el navegador del cliente, según el resultado del método o transacción.

Los parámetros opcionales que se incluyen en el M4Tag **endjob** son:

- *m4alias*: alias del método por comprobar. Si está presente este parámetro, se controlará el resultado de ese método concreto. Si este parámetro no está presente, se controlará el resultado de toda la transacción.
- *m4url*: parámetro que indica la URL que se debe ejecutar para comprobar el resultado de la traducción.
- *m4check*: parámetro que indica el resultado de la traducción que desea comprobar:
 - ERROR
 - WARNING
 - SUCCESS

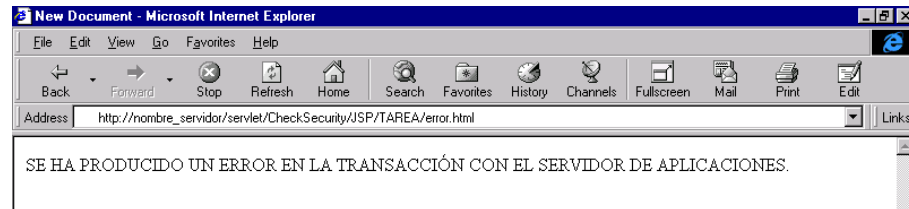
Sólo puede usar uno de los tres valores del parámetro.

EJEMPLO

```
<m4:beginjob/>
...
<m4:endjob
  m4check="ERROR"
  m4url="error.html"/>
```

Una vez cargada la página del ejemplo anterior, si se produce un error de transacción, aparecerá la correspondiente página error.html en el navegador del cliente. La siguiente figura muestra dicha página:

Figura 30. Página error.html



En caso de que no haya errores, se seguirá ejecutando el resto de la página. Vaya a otra página mediante la etiqueta *meta* de HTML.

EJEMPLO

```
...  
<meta  
    http-equiv='refresh'  
    content="3; URL='servlet/CheckSecurity/JSP/TC_TEST/testindex.jsp'>  
...
```

Herramientas de presentación

En este apartado se describen varias herramientas que permiten al diseñador de la página presentar los datos de un modo más adecuado.

Mostrar texto en función de la autorización

Se puede usar la etiqueta **checklink** para mostrar texto en función de la autorización. Esto impide que el usuario final vea las excepciones de seguridad de tareas para las que no tiene autorización.

En el siguiente ejemplo se muestra cómo usar la etiqueta **checklink**. Puede consultar más información sobre el uso de esta etiqueta en la sección **M4Tags de seguridad y sesión** del capítulo **Referencia de la biblioteca M4TagLib** de este manual.

EJEMPLO: CHECKLINK.JSP

```
<%@ taglib uri="M4Tags" prefix="m4" %>
...
<m4:checklink taskname="manager">
http://servlet/CheckSecurity/JSP/manager/approve_holidays.jsp
Haga clic en este enlace para aprobar las vacaciones de los
empleados</a>
(Si no tiene autorización para la tarea de gestión, no debería ver
esto) .
</m4:checklink>
...
```

El enlace y el texto sólo aparecerán si el usuario tiene autorización para la tarea de gestión.

Almacenamiento del resultado en variables

Puede ser conveniente almacenar el resultado de la ejecución de las etiquetas **item**, **outputexec**, **label**, **count** o **current** en variables Java del código de la página. Esto puede facilitar la labor de los diseñadores HTML que deben utilizar JavaScript para mostrar la información procedente de M4Tags.

El resultado de las etiquetas de presentación se puede almacenar en dos tipos de variables distintos:

- **Variables autodeclaradas:** La etiqueta crea la variable Java con el nombre especificado.
 - Esto se consigue usando el atributo *m4varname* de las etiquetas de presentación.
- **Variables predeclaradas:** La variable tiene que haberse declarado en la página .JSP y la etiqueta utiliza una variable con ese nombre. La etiqueta de presentación espera que la variable esté declarada.

Esto se consigue usando el atributo *var* de las etiquetas de presentación.

En el siguiente ejemplo se muestra cómo almacenar la descripción de un elemento en una variable predeclarada (usando la etiqueta **label**), y cómo almacenar el valor de un registro determinado en una variable autodeclarada (usando la etiqueta **item**).

EJEMPLO: VARIABLES.JSP

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:page subsessionid="example">
<m4:job>
<m4:datadef m4name="PERSON" m4o="TC_MT_TEST_PERSON"/>
<m4:exec m4object="PERSON" method="LOAD" alias="alias_load"
<m4:outputdef m4alias="out">
    <m4:param name="M4NAME0" value="PERSON!STC_MT_TEST_PERSON[*]"/>
</m4:outputdef>
</m4:job>
```

Uso de variables autodeclaradas:

```
<m4:item outputdef="out" item="DATE_BIRTH" m4varname="sDateBirth"/>
El valor de DATE_BIRTH para el registro actual es:
<%=sDateBirth%><br>
```

Uso de variables predeclaradas:

```
<% String sLabel_ID_PERSON = ""; %>
<m4:label outputdef="out" item="ID_PERSON" var="sLabel_ID_PERSON"/>
El valor de la etiqueta del elemento ID_PERSON es:
<%=sLabel_ID_PERSON%><br>
```

```
</m4:page>
```

Propiedades de transacciones avanzadas

En este apartado se explican las propiedades de transacciones avanzadas que pueden ser útiles para determinadas operaciones de servicios de datos.

Parámetros especiales para solicitar objetos

La etiqueta **datadef** tiene un atributo adicional que permite especial si se ha de extraer el Meta4Object de una instancia existente del servidor de aplicaciones. Es útil cuando se hacen llamadas de nivel 2 usando alias estáticos.

En el siguiente ejemplo, el Meta4Object TEST_M4O_1 tiene un alias de nivel 2 a TEST_M4O_2. En la primera transacción es necesario ejecutar un método simulado de TEST_M4O_1 para que se pueda resolver la instancia aunque sea la primera vez que se accede al objeto. Ese método simulado puede tener un código tan sencillo como:

```
ALIAS_M4O_2!EX_M4O_2_N.MoveTo(0)
return 0
```

La segunda transacción contiene la llamada de nivel 2 real. Observe el parámetro *m4find* que se ha pasado a la etiqueta **datadef**.

EJEMPLO DE LLAMADAS DE NIVEL 2:

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:startpage m4task="level2"/>
<m4:beginjob/>
    <m4:datadef m4o="TEST_M4O_1" m4name="ALIAS1"/>
    <m4:exec m4method="define:ALIAS1!TEST_M4O_1_N.METHOD_1"/>
<m4:endjob/>

<m4:beginjob/>
<m4:datadef m4o="TEST_M4O_1.IA" m4name="ALIAS2" m4find="TRUE"/>
<m4:outputdef m4alias="this_outputdef">
    <m4:param name="m4name0" value="ALIAS2!TEST_M4O_2_N[0-10]"/>
</m4:outputdef>
<m4:endjob/>
...
<m4:move>
    <m4:param name="ALIAS2"
value="this_outputdef:ALIAS2!TEST_M4O_2_N[0]"/>
```

```
</m4:move>
```

Repetición en los 10 registros:


```
<m4:iterator m4node="this_outputdef:ALIAS2!TEST_M4O_2_N"
m4rows="*">
    <m4:param name="m4item0" value="STD_ID_PERSON"/>
    <m4:param name="m4item1" value="STD_EMAIL"/>
    <tr>
        <td>STD_ID_PERSON: $M4ITEM0$, <br></td>
        <td>STD_EMAIL $M4ITEM1$ <br></td>
    </tr>
    <br>
</m4:iterator>
<m4:endpage/>
```

Referencia de traducciones

En algunos casos, puede ser conveniente utilizar tablas de referencia para obtener nombres de elementos asociados a una presentación. La información que relaciona los identificadores o códigos con sus nombres se puede almacenar en tablas auxiliares, llamadas también tablas de referencia.

Las tablas de referencia son especialmente útiles cuando una tabla tiene una relación establecida a través del ID con una tabla padre que tiene campos calculados. En este apartado se indica cómo usar esas tablas para realizar una traducción de los valores de los elementos.

Los M4Tags **translation** que se muestran a continuación permiten recuperar los nombres asociados a los identificadores de un Meta4Object, en el idioma en el que se ha producido la conexión al cliente HTML.

El proceso de traducción consta de varias etapas:

- Definir la traducción
- Traducir
- Presentar el campo traducido

El Meta4Object TC_MT_TEST_PERSON del ejemplo siguiente incluye información personal, como el código o identificador de género (ID_GENDER, con valor 0 o 1). Se mostrará un nombre de género en el idioma del usuario en vez del código numérico. Para ello, necesita utilizar los M4Tags **translation** para recoger los valores en el campo GENDER_NAME del Meta4Object auxiliar, TC_TR_TEST_GENDER. Este Meta4Object relaciona los códigos identificativos con los nombres de género.

Definir la traducción

En primer lugar, debe definir la traducción. Para eso, utilice el M4Tag **addtranslation**. Este M4Tag tiene el parámetro *m4translation*. Dicho parámetro contiene información acerca del Meta4Object de la tabla auxiliar, de donde el sistema obtiene el campo traducido asociado al código identificativo del usuario. Se deben especificar los elementos que se citan a continuación. (Los valores que están entre paréntesis son los que aparecen en los ejemplos.)

- El alias de traducción (GENDERTRANS)
- El Meta4Object asociado a la tabla auxiliar (TC_TR_TEST_GENDER)
- El nodo (STC_TR_TEST_GENDER)
- El método que hace la traducción (VALIDATE)

- El elemento en el que se almacena el nombre traducido (GENDER_NAME)

Defina la traducción con el M4Tag **outputdef**. Se incluye en el bloque de código de servicio de datos, es decir, en la transacción del servidor de aplicaciones entre los M4Tags **beginjob** y **endjob**, y después de cargar los datos del Meta4Object cuyo código identificativo desea traducir. El Meta4Object de la tabla auxiliar no necesita cargarse.

EJEMPLO TRANSLATION.JSP

```
<html>
<head>
<title>Nombre de género</title>
</head>
<body>

<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:startpage m4task="TC_TEST"/>

<m4:beginjob/>
  <m4:datadef
    m4name="PERSON"
    m4o="TC_MT_TEST_PERSON"/>
  <m4:exec m4method="LOAD:PERSON.LOAD"/>
  <m4:addtranslation
    m4translation ="GENDERTRANS:TC_TR_TEST_GENDER!
    STC_TR_TEST_GENDER.VALIDATE#GENDER_NAME"/>
    ...
  <m4:endjob/>
<m4:endpage/>
</body>
</html>
```

Traducir

Para traducir los códigos identificativos, utilice el M4Tag **firetranslation**. Se incluye en el bloque de código de servicio de datos tras la definición de traducción explicada en el apartado anterior. Se pueden traducir uno o varios códigos identificativos en el mismo M4Tag. Para cada uno de ellos, hay que especificar un parámetro *param* con los siguientes valores:

- **Name:** campo en el que se traduce el código identificativo, en este caso:
`name="PERSON!STC_MT_TEST_PERSON.ID_GENDER"`
- **Valor:** tras el alias de la traducción especificado en la definición de la traducción, se pasarán los valores de los argumentos del método que realiza la traducción. Los argumentos del método se separan entre sí por el símbolo #. Cada argumento llevará su código identificativo y valor, y estará separado por el símbolo @. En el ejemplo que se muestra, el método **VALIDATE** que realiza la traducción tiene que recibir dos argumentos: **ARG_SEC_TI**, al que se pasa un valor nulo (tras @ hay un espacio en blanco) y **ARG_ID_GENDER**, al que se pasa el campo por traducir (**PERSON!STC_MT_TEST_PERSON.ID_GENDER**)/

```
value="GENDERTRANS:ARG_SEC_TI@ #
ARG_ID_GENDER@PERSON!STC_MT_TEST_PERSON.ID_GENDER"
```

EJEMPLO TRANSLATION.JSP

```
<m4:beginjob/>
  <m4:datadef
    m4name="PERSON"
    m4o="TC_MT_TEST_PERSON"/>
  <m4:exec m4method="LOAD:PERSON.LOAD"/>
  <m4:addtranslation
    m4translation ="GENDERTRANS:TC_TR_TEST_GENDER!
    STC_TR_TEST_GENDER.VALIDATE#GENDER_NAME"/>
  </m4:firetranslation>
  <m4:param
    name="PERSON!STC_MT_TEST_PERSON.ID_GENDER"
    value="GENDERTRANS:ARG_SEC_TI@ #ARG_ID_GENDER@PERSON!
    STC_MT_TEST_PERSON.ID_GENDER"/>
  </m4:firetranslation>
  ...
<m4:endjob/>
<m4:endpage/>
```

Presentar el campo traducido

Para presentar en el navegador del cliente el campo traducido asociado al código identificativo, se usa el M4Tag **gettranslation** en el bloque de código de presentación de datos. En su parámetro *m4nametranslation*, se especifica el código identificativo que hay que traducir. Además, en la pantalla del cliente aparecerá el valor ya traducido. En el siguiente ejemplo, se muestra el campo traducido del primer registro.

EJEMPLO TRANSLATION.JSP

```
...
<m4:beginjob/>
  <m4:datadef
    m4name="PERSON"
    m4o="TC_MT_TEST_PERSON"/>
  <m4:exec m4method="LOAD:PERSON.LOAD"/>
  <m4:addtranslation
    m4translation = "GENDERTRANS:TC_TR_TEST_GENDER!
    STC_TR_TEST_GENDER.VALIDATE#GENDER_NAME"/>
    ...
  </m4:firetranslation>
  <m4:param
    name="PERSON!STC_MT_TEST_PERSON.ID_GENDER"
    value="GENDERTRANS:ARG_SEC_TI@ #ARG_ID_GENDER@PERSON!
    STC_MT_TEST_PERSON.ID_GENDER"/>
  </m4:firetranslation>

  <m4:outputdef m4alias="out">
    <m4:param name="M4NAME0" value="PERSON!STC_MT_TEST_PERSON[0]"/>
  </m4:outputdef>

<m4:endjob/>

<m4:move>
  <m4:param name="PERSON" value="out:STC_MT_TEST_PERSON[0]"/>
</m4:move>

<table border="1">
  <tr><td><b>Nombre</b></td><td><b>Apellido</b></td><td><b>Género</b></td></tr>
```

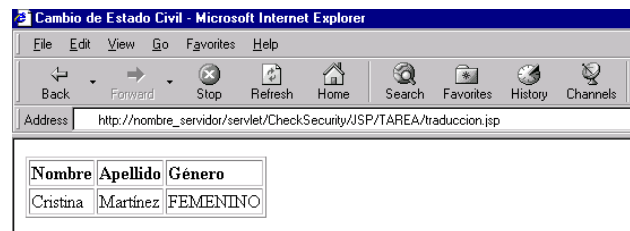
```

<tr><td><m4:item
m4name="out:PERSON!STC_MT_TEST_PERSON.FIRST_NAME"/>
<tr><td><m4:item
m4name="out:PERSON!STC_MT_TEST_PERSON.FIRST_NAME"/>
<td>
    <m4:gettranslation
    m4nametranslation=
    "out:PERSON!STC_MT_TEST_PERSON.ID_GENDER"/>
</tr>
</table>
...

```

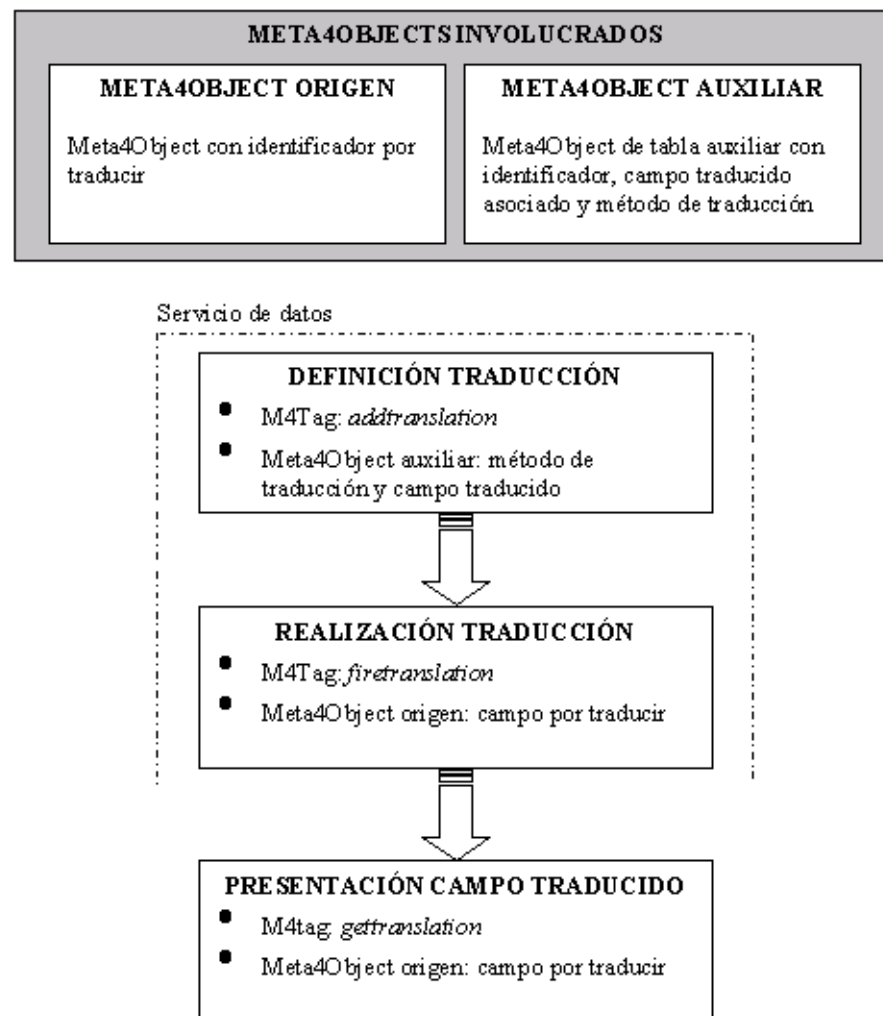
Al ejecutar la página *translation.jsp*, se muestra el campo traducido asociado al código identificativo de género.

Figura 31. Resultado en el navegador del cliente de *translation.jsp*



En la siguiente figura se resume el proceso de traducción:

Figura 32. Proceso de traducción



El siguiente código es la continuación del ejemplo *translation.html* del apartado anterior. En vez de escribir el campo traducido del primer registro, se escribe el de todos los registros incluidos en el nodo `STC_MT_TEST_PERSON` del Meta4Object `TC_MT_TEST_PERSON`. Para ello, utilice los M4Tags **count** y **loop**.

EJEMPLO TRANSLATION2.JSP

```
<html>
<head>
<title>Nombre de género</title>
</head>
<body>

<%@ taglib uri="M4Tags" prefix="m4" %>
<m4:startpage m4task="TC_TEST"/>

<m4:beginjob/>
  <m4:datadef
    m4name="PERSON"
    m4o="TC_MT_TEST_PERSON"/>
  <m4:exec m4method="LOAD:PERSON.LOAD"/>
  <m4:addtranslation
    m4translation ="GENDERTRANS:TC_TR_TEST_GENDER!
    STC_TR_TEST_GENDER.VALIDATE#GENDER_NAME"/>
    ...
  </m4:firetranslation>
  <m4:param
    name="PERSON!STC_MT_TEST_PERSON.ID_GENDER"
    value="GENDERTRANS:ARG_SEC_TI@ #ARG_ID_GENDER@PERSON!
    STC_MT_TEST_PERSON.ID_GENDER"/>
  </m4:firetranslation>

  <m4:outputdef m4alias="out">
    <m4:param name="M4NAME0" value="PERSON!STC_MT_TEST_PERSON[*]"/>
  </m4:outputdef>

<m4:endjob/>

<m4:count
  m4varname="count"
  m4name="out:PERSON!STC_MT_TEST_PERSON"/>
```

```

<m4:move>
    <m4:param name="PERSON" value="out:STC_MT_TEST_PERSON[0]" />
</m4:move>

<table border="1">
    <tr><td><b>Nombre</b></td><td><b>Apellido</b></td><td><b>Género</b></td></tr>

    <m4:loop
        from="0"
        to="<%= new Integer(new Integer(count).intValue()-1).toString()
            %>">

        <tr><td><m4:item
            m4name="out:PERSON!STC_MT_TEST_PERSON[&VAR.m4lix].FIRST_NAME" />
        <td><m4:item
            m4name="out:PERSON!STC_MT_TEST_PERSON[&VAR.m4lix].LAST_NAME" />
        <td><m4:gettranslation
            m4nametranslation="out:PERSON!STC_MT_TEST_PERSON[&VAR.m4lix].ID_GENDER" />
        </tr>

    </m4:loop>

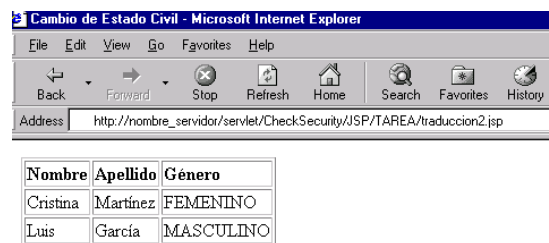
</table>

<m4:endpage/>
</body>
</html>

```

Al ejecutar la página translation2.jsp, se muestran los campos traducidos asociados a los códigos identificativos de género de todos los registros.

Figura 33. Resultado en el navegador del cliente de translation2.jsp



Diseño de una página JSP con enlaces entre sitios

Edite la página JSP del servidor origen desde la que desea realizar un enlace al servidor destino. Para ello, utilice la M4tag personalizada **crosslink**.

Si desea obtener la página `/servlet/CheckSecurity/JSP/some_task/some_page.jsp`, utilice la siguiente sintaxis:

```
<m4:crosslink uri="/servlet/CheckSecurity/JSP/some_tarea/
some_page.jsp" provider="0"/>
```

Este tag creará en su página una cadena que por defecto será la siguiente:

```
/servlet/CrossPoster?_URI=/servlet/CheckSecurity/JSP/some_task/
some_page.jsp
```

Así que si desea realizar un enlace con el tag A de HTML puede emplear la siguiente sintaxis:

```
<a href=<m4:crosslink
uri="/servlet/CheckSecurity/JSP/task/page.jsp"
provider="0"/>>haga clic en este enlace para ir al otro sitio</a>
```

Si suprime el *atributo provider* se tomará por defecto el servidor remoto indicado en el parámetro `URL_0` del fichero de configuración.

```
<a href=<m4:crosslink
uri="/servlet/CheckSecurity/JSP/task/page.jsp"/>>haga clic en este
enlace para ir al otro sitio</a>
```

Si indica otro identificador de proveedor, se proporcionará un enlace con el servidor que se haya incluido en el parámetro `URL_N` del archivo `configclient.xml`.

```
<a href=<m4:crosslink
uri="/servlet/CheckSecurity/JSP/task/page.jsp"
idprovider="1"/>>haga clic en este enlace para ir al otro sitio</a>
```

En ese caso, el enlace que se creará será como el siguiente:

```
/servlet/CrossPoster?_URI=/servlet/CheckSecurity/JSP/some_task/
some_page.jsp&_PROVIDER=1
```

Por supuesto, para poder ejecutar una página que contenga un M4tag **crosslink** es necesario tener una sesión Web activa. Por lo cual si intenta ejecutar una página JSP con dicho M4tag sin haberse conectado o después de un periodo de inactividad que haya hecho que su sesión haya expirado, el tag no generará la cadena de texto deseada en la página HTML, sino que no generará nada.

Depuración de páginas JSP

Es posible que, después de desarrollar una página JSP, ésta no funcione correctamente o como se esperaba. En ese caso, será necesario depurarla.

En primer lugar, compruebe que la página no contiene errores de sintaxis (o compilación). Para ello, llámela directamente (sin necesidad de llamar al *servlet* CheckSecurity). Si no se compila, consulte los archivos de traza del motor del *servlet* para averiguar en qué líneas hay errores de sintaxis.

El proceso se simplifica con JRun, ya que permite al navegador mostrar los motivos por los que no se produce la compilación. Si el navegador es Internet Explorer 5, deshabilite la opción para mostrar mensajes de error HTTP descriptivos. De este modo, podrá ver directamente la pila de errores.

EJEMPLO

Si la página `index.jsp` tiene una directriz errónea, aparecerá el siguiente mensaje de error:

```
Error de servidor interno 500
```

```
/index.jsp:
```

```
javax.servlet.ServletException: Error de análisis en la página JSP:  
La directriz de página tiene en la línea 17 un atributo no válido.
```

En segundo lugar, si se compila la página, aunque no funcione como esperaba, consulte los siguientes archivos de traza del servidor Web para obtener información adicional:

- **Archivo de traza de servicios Web:** este archivo contiene las excepciones y errores de código Java en el cliente HTML.
 - La ruta de acceso al archivo en Windows es:
`<m4log-dir>\m4webservices.log`
 - Y en Solaris es:
`<m4log-dir>/m4webservices.log`
donde:
 - `<m4log-dir>` es el parámetro `M4_LOG_DIR` del archivo `m4logsystem.cfg` que indica la ubicación de los archivos de traza.

El número de trazas de este archivo depende del parámetro `M4_TRACE_LEVEL` de `m4logsystem.cfg`. Los valores de este parámetro de nivel de traza son, de mayor a menor información de traza: **debug** | **info** | **warn** | **error** | **fatal**.

EJEMPLO

El M4Tag **saveconfig** ha detectado una excepción al guardar en disco. Esta información procede de `webservices.log`.

```
Jun 27 15:31:22 [ - jcp-6] FATAL: [EXCEPTION] para guardar el nuevo
archivo:
java.io.FileNotFoundException: c:\program
files\meta4\m4ws\default\classes\properties\configclient.xml (Access
is denied)
at java.io.FileOutputStream.open(Native Method)
at java.io.FileOutputStream.<init>(FileOutputStream.java:102)
at java.io.FileOutputStream.<init>(FileOutputStream.java:62)
at com.meta4.configuration.M4ConfigClient.createConfigClient(Unknown
Source)
at com.meta4.taglib.M4TagSaveConfig.doEndTag(M4TagSaveConfig.java:41)
at
jrun__config__espanol__config2ejsp1a._jspService(jrun__config__espano
l__config2ejsp1a.java:56)
[...]
```

- **Archivo de traza XML:** En este archivo se guardan detalles de las transacciones realizadas entre el servidor de aplicaciones y el servidor Web. Estas trazas resultan especialmente útiles cuando se detecta un comportamiento inusual en el acceso a los datos a través de las etiquetas del cliente HTML.
 - La ruta de acceso al archivo en Windows es:


```
<m4log-dir>\m4connect.log
```
 - Y en Solaris es:


```
<m4log-dir>/m4connect.log
```
- **Archivos de traza del motor de *servlets*:** estos archivos contienen excepciones y errores de JSP y *servlet* API. Su ubicación depende del motor de *servlets*.
 - La ruta de acceso a JRun en Windows es:


```
<raíz-jrun>\<servidor-jrun>\nivel-<pordefecto>.log
```
 - Y en Solaris es:


```
<raíz-jrun>/<servidor-jrun>/nivel-<pordefecto>.log
```

 donde:
 - `<raíz-jrun>` es el directorio de instalación de JRun. El directorio por defecto en Windows es `c:\program files\allaire\jrun`.
 - `<servidor-jrun>` es el identificador de la instancia de Jrun, que por defecto es `"default"`.

-
- <nivel> es "out" para las trazas enviadas a la salida estándar, "err" para las dirigidas al error estándar y "event" para las excepciones de registro.
 - La ruta de iWS es:
`<raíz-iws>/https-<servidor-iws>/config/<nivel>.`
donde:
 - <raíz-iws> es el directorio de instalación de iWS.
 - <servidor-iws> es el identificador de la instancia de iWS. Por defecto, es el nombre del servidor.
 - <nivel> es "access" para el archivo que registra los accesos al servidor Web y "errors" para el que tiene las trazas de compilación y ejecución del motor de *servlets*.

Finalmente, los mensajes de registro del servidor de aplicaciones asociados de una transacción determinada le pueden ayudar a depurar el acceso a los datos en los clientes HTML. Esta información de traza se envía al registro de servicios Web (*webservices.log*) y a archivos de traza XML (*connect.log*), pero si es necesario se le pueden mostrar al usuario en una página de error, como se explica en el apartado **Gestión de errores**.

Sugerencias de diseño

Para concluir este capítulo, se indican algunas recomendaciones de diseño de páginas avanzadas. Utilícelas para optimizar el desarrollo de sus aplicaciones. Estas sugerencias complementan las **Sugerencias de diseño** del capítulo **Diseño de páginas básicas** y se recomienda seguirlas.

- Utilice toda la lógica posible en los Meta4Objects para simplificar las páginas.
- Procure filtrar con métodos los Meta4Objects.
- Utilice Meta4Objects que sólo contienen unos pocos nodos, y si debe utilizar Meta4Objects con muchos nodos, intente no cargarlos todos en una sola página.
- Evite enviar las variables recogidas en un formulario mediante el método *get* cuando la consulta tenga más de 256 caracteres. En su lugar, use el método *post* y recoja las variables dentro del cuerpo de la página para que así no haya límite. El servidor Web impone esta limitación.
- Evite recoger todos los datos de un nodo. Es mejor que tome sólo los que vaya a necesitar.
- Para canales multinodo, utilice sólo argumentos variables y un método para guardar. Procure no programar en Java y que la página no sea compleja.
- Utilice bolsas sólo cuando sea necesario, ya que cada una tiene un límite de 20 elementos.
- Recuerde que en el cliente HTML se cargan los Meta4Objects como no separables. (Para más información sobre el comportamiento cliente/servidor de los Meta4Objects, consulte el manual de arquitectura cliente/servidor.)
- Evite usar tablas, en especial las anidadas, ya que ralentizan el funcionamiento de algunos navegadores como Netscape.
- Si sus páginas .JSP son muy grandes, puede llegar al límite de Java que prohíbe que un código de bytes binario de un único método sea mayor de 64K. La única solución para este problema es reducir el tamaño del código de la página.
- Tras una transacción de datos, utilice el M4Tag **move** para seleccionar el registro correspondiente.
- Al emplear variables en Java, utilice un prefijo concreto, como M4. Si no lo hace, podrán producirse errores por la duplicación de variables. Por ejemplo, en JRun se utiliza "session" como nombre de variable.

-
- En `<m4:outputdef...>`, declare sólo aquellos elementos y rango de registros que realmente desee ver. De este modo, se reduce la transferencia de datos y aumenta el rendimiento de la aplicación.

Si un Meta4Object sólo se va a cargar una vez, utilice el parámetro `preserve="false"` en el M4Tag `datadef`.

Recomendaciones de seguridad

Introducción

Un usuario malicioso que quiera realizar acciones no autorizadas en una aplicación web puede recurrir a manipular los parámetros de una petición para la que sí está autorizado.

Por ejemplo, en una aplicación en la que se pase de una página a otra un identificador de usuario, el atacante podría intentar introducir identificadores de otros usuarios para acceder a sus datos (`http://server.com/view_salaries?userid=1234`).

Los casos más frecuentes de manipulación de parámetros consisten en alterar los parámetros de la cadena de consulta (Query String Parameter Manipulation) y los parámetros escondidos en formularios (Hidden Form Parameter Manipulation), aunque existen aplicaciones web vulnerables a la manipulación de cookies o de cabeceras HTTP.

Así, en una aplicación web mal diseñada, si un atacante encuentra un parámetro de la petición que alterar con éxito, podría acceder a información no autorizada e incluso modificarla, abriendo la puerta a ataques más complejos.

Cómo explotar el problema

Existen dos posibilidades:

- **Manipulación de los parámetros de la cadena de consulta:** Una petición HTTP puede realizarse por GET o por POST. Si se realiza por GET, todos los valores de los parámetros aparecerán en la barra del navegador o en el título de la página, haciendo que el usuario los pueda ver y por tanto le sea fácil manipularlos.
- **Manipulación de los parámetros escondidos en formularios:** En las peticiones HTTP que se realizan por POST, se pueden pasar valores escondidos en formularios. Existen herramientas que permiten espiar los nombres y los valores de dichos parámetros escondidos. El atacante puede introducir los valores manipulados de los parámetros escondidos como

parte de la cadena de consulta, realizando una petición de tipo GET, o enviarlos dentro de una petición tipo POST utilizando una de estas herramientas o un desarrollo propio.

Cómo evitar el problema: API Java para encriptar valores de parámetros de páginas jsp sujetos a manipulación

Para evitar este problema, se sugiere que los parámetros que se pasen entre páginas sean encriptados a nivel de aplicación con un algoritmo criptográfico adecuado. En las aplicaciones web de Meta4, se dispone (a partir del HF1 de 7.1SP5) de unas funciones de encriptación basadas en el algoritmo AES, que trabajan con una clave aleatoria asociada a la sesión del usuario.

Funciones de encriptación y desencriptación

Los prototipos de las funciones de encriptar y desencriptar para el uso de las aplicaciones web de Meta4 se incluyen en m4taglib.jar/m4common.jar a partir de 7.1SP5 HF1. Estas funciones utilizan un algoritmo criptográfico AES de 128 bits y codifican en un derivado de Base64.

```
// encriptar
sValue =
com.meta4.taglib.util.M4PresentationUtilTaglib.secureEncrypt(request,
"prefix", sValue); %>

// desencriptar
sValue =
com.meta4.taglib.util.M4PresentationUtilTaglib.secureDecrypt(request,
"prefix", sValue);
```

El segundo argumento sirve para establecer compartimentos entre funcionalidades, para que lo encriptado en un grupo de páginas, no se pueda desencriptar en otro de seguridad distinta.

A continuación se muestra un ejemplo de manipulación de parámetros de la cadena de consulta y la forma de evitarlo.

Ejemplo de manipulación de parámetros de la cadena de consulta

Podría producirse un intento de manipulación de parámetros en la siguiente cadena de consulta en el entorno del SSE/SSM:

1. La página /sse_g1/espanol/ssco_g1_p6.jsp envía el parámetro (parámetro IDDoc)
2. El servlet /servlet/DocumentViewer recoge el parámetro (parámetro IDDoc)

Para ello, habría que localizar, a través de una herramienta de testeo, o mediante inspección de código, una URL que reciba parámetros por la cadena

de consulta.

Por ejemplo, en los datos personales del empleado, el empleado puede acceder a “Mis documentos”. Al pulsar “Abrir el documento” ve sus documentos asociados.

Supongamos que esta persona tiene asociados los documentos 77 y 110. Vemos que estos identificadores se introducen como parámetro en /servlet/DocumentViewer?IDDoc=110.

A continuación solo habría que invocar la página manipulando el parámetro de la cadena de consulta, por ejemplo para ver el documento de código 77, no autorizado:

Solución

Para impedir la manipulación, hay que realizar dos sustituciones:

1. La página que llama tiene que encriptar el parámetro manipulable.

EJEMPLO

El código sin encriptar:

```
<td class="fuentevalor<%=zPaint%>">
<input type="hidden" id="<%=sgtc_zNMInputIDDOC%>"
name="<%=sgtc_zNMInputIDDOC%>" value="<%=sgtc_zIDDOC %>"/>
```

El código que encripta:

```
<% sgtc_zIDDOC =
com.meta4.taglib.util.M4PresentationUtilTaglib.secureEncrypt
(request, "tc_docs", sgtc_zIDDOC); %>

<td class="fuentevalor<%=zPaint%>">
<input type="hidden" id="<%=sgtc_zNMInputIDDOC%>"
name="<%=sgtc_zNMInputIDDOC%>" value="<%=sgtc_zIDDOC %>"/>
```

-
2. La que recibe la llamada tiene que desencriptar el parámetro manipulable. Para ello es necesario introducir el mismo prefijo de la página que envía el parámetro para que desencripte.

EJEMPLO

El código sin encriptar:

```
String sIDDoc = request.getParameter(ID_DOC_PARAM);
```

El código que descripta:

```
String sIDDoc = M4SafeRequest.getParameter(request, ID_DOC_PARAM);  
sIDDoc =  
com.meta4.taglib.util.M4PresentationUtilTaglib.secureDecrypt(request,  
"tc_docs", sIDDoc);
```

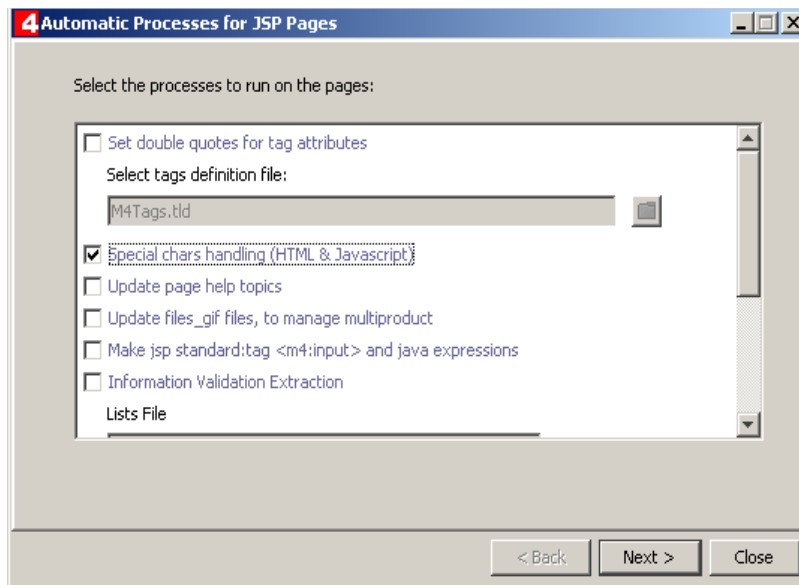
-
3. A continuación hay que verificar que los parámetros antes manipulables, ahora no lo son, y que la introducción de identificadores antiguos conocidos no produce ningún resultado, de forma que un usuario no pueda manipular aleatoriamente un valor encriptado.

Uso del robot m4jsprobot2k2 para evitar ataques de tipo XSS

Para evitar la vulnerabilidad conocida como XSS (del inglés "cross-site scripting"), puede ejecutar el robot m4jsprobot2k2, publicado en la carpeta m4toolkit del CD de producto.

Si decide ejecutar el robot para proteger la aplicación contra este tipo de ataques en caso de que el diseño de sus páginas sea vulnerable a ellos, debe tener seleccionada la opción de gestión de caracteres especiales (HTML y Javascript "Special chars handling (HTML & Javascript)", tal y como se muestra en la siguiente ventana:

Figura 34. Selección de procesos automáticos para páginas JSP



La metodología se basa en añadir “htmlsafe” y “jsafe” a las M4Tags empleadas para recuperar los datos del lado del servidor. Esto permite escapar cualquier valor peligroso que un atacante pudiera intentar utilizar, lo cual debe tenerse en cuenta en cualquier desarrollo.

El robot también sustituye la función `getParameter()` por otra similar de la clase `M4SafeRequest`, que escapa los valores de entrada para proteger la aplicación contra ataques de tipo XSS.

Filtros para el entorno Web

4

Acerca de este capítulo

En este capítulo se describen las principales características de dos tipos de filtros disponibles en la aplicación para ser utilizados en el entorno HTML.

Estos filtros están desarrollados a través de páginas JSP y pueden ser invocados desde otras páginas JSP personalizadas para filtrar datos de Meta4objects.

Se describen este tipo de filtros y se especifica cómo invocarlos desde páginas JSP.

Introducción

En el entorno html, a la hora de filtrar los datos de Meta4Objects desde páginas JSP, no sólo pueden usarse los taglets de Meta4Objects, sino que también se pueden utilizar un conjunto de páginas JSP definidas en el sistema con este fin.

Hay dos tipos de filtros disponibles:

- **Filtro HTML:** permite crear sentencias complejas de filtro basándose en una tabla o conjunto de tablas (escenario).
- **Filtro HTML dinámico:** permite definir y aplicar filtros sobre Meta4Objects que tienen definidos nodos con filtros dinámicos habilitados. Por ejemplo, estableciendo la propiedad avanzada del nodo **Filtro dinámico** a **Sí** o creando en el nodo una propiedad de ámbito bloque y con tipo interno igual a 34.

Filtros HTML

El sistema de Filtro HTML parte de los siguientes elementos:

- un ID Sentencia, que será nulo cuando sea una sentencia nueva y
- una tabla o conjunto de tablas (ID Escenario) sobre las que se define el filtro.

Una vez definido el filtro, el sistema retorna los siguientes elementos:

- el ID Sentencia generado en el caso de sentencia nueva o el ID Sentencia correspondiente al filtro que se está editando.
- el API SQL y
- el lenguaje natural que es la traducción del SQL asociado al filtro al idioma del usuario final.

La página de definición del filtro permite generar una sentencia de filtro que estará compuesto de uno o varios detalles que irán unidos con el operador lógico correspondiente, así como utilizando los paréntesis que sean necesarios para conseguir el orden de preferencia adecuado.

A la hora de definir un detalle, el sistema permite navegar entre la tabla o tablas del escenario indicado en la entrada generando los JOINS correspondientes entre ellas. Además cuando se trabaja con escenarios de más de una tabla se permitirá definir detalles utilizando la cláusula EXISTS sobre tablas distintas de la tabla base del escenario.

El sistema en el filtro final sólo creará un tipo de join para todas las tablas, que podrá ser parametrizado. Más información en los parámetros de entrada del apartado Cómo llamar a la función de filtro HTML.

Figura 35. Ejemplo de página de definición del filtro

DEFINICIÓN DE FILTRO HTML Correo electrónico ?

Nuevo detalle

* ID Detalle: 8

☒ Todos los registro que cumplen ☐ Existe algún registro que cumple

(	* Tabla/Campo	* Relación	Tabla/Campo	* Valor	)	y/o
	Correo electrónico	Es				
	ID Persona					

✓

Definición del filtro

(El campo ID Persona de la tabla Correo electrónico No empieza por M100) y (El campo Correo elect. de la tabla Correo electrónico No contiene com)

ID Detalle	Descripción
1	(El campo ID Persona de la tabla Correo electrónico No empieza por M100) y
2	(El campo Correo elect. de la tabla Correo electrónico No contiene com)

Entrada del sistema

Guardar filtro como predefinido

Salida del sistema

Las condiciones creadas en el entorno WEB no son válidas para el entorno Windows y viceversa.

Cómo llamar a la función de filtro HTML

Todo lo que se debe hacer es invocar a la página .JSP de filtro (shco_gen_htmlfilter.jsp) usando la tarea shco_g0 de los servicios Web de Meta4 y utilizando los parámetros adecuados en función de la operación que se le solicite realizar al filtro.

La funciones principales del filtro HTML son:

- Edición de filtros
- Grabación de filtros

Edición de filtros

Para esta función de edición se detallan a continuación los parámetros de entrada y de salida.

Parámetros de entrada

Los parámetros que debe pasar en la invocación a la página de filtro HTML son los siguientes:

Parámetros	Descripción	Obligatorio
zidoperation	Identificador de la operación a realizar sobre el filtro. Para definir, editar un filtro indicar "API_GET_FILTER" :	Sí
zreturnpage	Página de retorno a la que volver una vez definido el filtro	Sí
zidsentence	Identifica la sentencia a editar. Va vacío si la sentencia es nueva	No
zidscenariio	Identifica el escenario, conjunto de tablas, sobre que el que se define el filtro	No*
zidtable	Identifica la tabla sobre la que se va a definir el filtro.	No ¹
zidrelationtype	Identifica el tipo de relación. Si no se indica ninguna, el valor por defecto es 1, que corresponde a EQUAL_JOIN. Valores posibles: ■ 1: corresponde a EQUAL_JOIN ■ 2: corresponde a LEFT_JOIN ■ 3: corresponde a RIGHT_JOIN	No
zretmode	Modo de retorno a la página zreturnpage una vez establecido el filtro. Valores posibles: ■ 1: se retorna con m4returnvalues sobre los tres input indicados en la llamada con el siguiente orden : IdSentence, Lenguaje Natural y APISQL. Esto requiere haber invocado al filtro con m4window pasando en el parámetro de array de argumentos los identificadores de los inputs que recibirán los resultados del filtro. ■ Otro: se invoca a la página de retorno a través de un submit dejando accesibles los datos del filtro bajo los siguientes parámetros del request: - txtIdSentence (Identificador de la sentencia) - txtLanguage (lenguaje natural del filtro) - txtApiSql (APISQL del filtro)	

Parámetros	Descripción	Obligatorio
zhtmlfilterinstance	Identifica la instancia del filtro. Indica la instancia del Meta4object de API de filtro HTML que se va a usar para el caso de que este Meta4Object esté ya levantado.	No [†]
zsubsesion	Identifica la subsesión sobre la que se quiere trabajar. Esto es obligatorio cuando se trabaja sobre una instancia del Meta4object de API del filtro HTML (parámetro zhtmlfilterinstancier).	No ²
zshowsavebutton	Indica si mostrar o no el botón de grabar filtro para posibilitar guardarlo como predefinido y que se pueda reutilizar en otras ocasiones. Pasar el parámetro con valor 1 para que se muestre.	No

*. Es obligatorio introducir un valor para el ID escenario o el ID Tabla indistintamente

†. Los parámetros zhtmlfilterinstance y zsubsesion se utilizan menos. Los valores que toman por defecto corresponden a la instancia de filtro y la subsesión actuales. Para información más detallada, véase el apartado Cómo usar la función de filtro HTML con instancias" de este mismo capítulo.

El siguiente formulario utiliza los parámetros de filtro:

EJEMPLO

```
<form action="/servlet/CheckSecurity/JSP/shco_g0/
shco_gen_htmlfilter.jsp" name="frmcallfilter" id="frmcallfilter"
method="post" >
<input type="hidden" id="zidoperation" name="zidoperation"
value=""/>
<input type="hidden" id="zidsentence" name="zidsentence" value=""/>
<input type="hidden" id="zidescenario" name="zidescenario"
value=""/>
<input type="hidden" id="zidtable" name="zidtable" value = "" />
<input type="hidden" id="zsubsesion" name="zsubsesion" value = "" />
<input type="hidden" id="zreturnpage" name="zreturnpage" value = ""
/>
<input type="hidden" id="zidrelationtype" name="zidrelationtype"
value = "" />
<input type="hidden" id="zhtmlfilterinstance"
name="zhtmlfilterinstance" value = ""/>
<input type="hidden" id="znnode" name="znnode" value = "" />
<input type="hidden" id="zretmode" name="zretmode" value = "" />
```

```



```

Parámetros de salida

El sistema devolverá los siguientes parámetros. La forma de devolverlos dependerá del parámetro de entrada `zreturnmode`.

Tabla 1. Parámetros de salida

Parámetros	Descripción
ID Sentencia	Identificador único de sentencia
Lenguaje natural	Traducción del SQL del filtro al idioma del usuario final
API SQL	Contiene la sentencia SQL completa

EJEMPLO

```

//Gathering values to return
String sIdSentence =
getStringValue(request.getParameter("txtIdSentence"));

String sLanguage =
getStringValue(request.getParameter("txtLanguage"));

String sApiSql =
getStringValue(request.getParameter("txtApiSql"));

...
<TR>
    <TD><LABEL><%=Tran.getProperty("Api.SentenceID")%></LABEL></TD>
    <TD><input name=txtIdSentenceBack value="<%= sIdSentence%>"/></TD>
</TR>
<TR>
    <TD><LABEL><%=Tran.getProperty("Api.RelationType")%></LABEL></TD>
    <TD><input name=txtIdRelationType value="1"/></TD>
</TR>
<TR>
    <TD><LABEL><%=Tran.getProperty("Api.ApiSql")%></LABEL></TD>
    <TD><TEXTAREA name=txtApiSqlBack cols=100 rows = 5 > <%= sApiSql%>
</TEXTAREA> </TD>
</TR>

```

```

<TR>
  <TD><LABEL><%=Tran.getProperty("Api.Lang")%></LABEL></TD>
  <TD> <TEXTAREA name=txtLanguageBack cols=100 rows = 5 > <%=
    sLanguage%>
  </TEXTAREA> </TD>
  ...

```

Función EditFilter()

Para invocar la edición de filtro se utiliza la función javascript editFilter(). El ejemplo que se muestra a continuación tiene el siguiente comportamiento:

- Se edita la sentencia MY_SENTENCE, que va sobre el escenario PERSON_SCENARIO.
- Se indica que la página de retorno es mi_página.jsp, y que retorne los datos del filtro sobre los input Input_id_sentence, Input_natural_lang e Input_APISQL que están definidos bajo el formulario "NombreFormulario".

EJEMPLO

```

function editFilter(){
  var swindow = "htmlfilterwindow";
  var arg_array = new Array;
  arg_array[0] = "Input_id_sentence";
  arg_array[1] = "Input_natural_lang";
  arg_array[2] = "Input_APISQL";
  m4window(swindow,"",arg_array,"NombreFormulario","800","600","no",
    "no",true,"yes");
  document.forms.frmcallfilter.target= swindow;
  m4valor("frmcallfilter","zidoperation","API_GET_FILTER","set");
  m4valor("frmcallfilter","zidescenario","PERSON_SCENARIO","set");
  m4valor("frmcallfilter","zidtable","", "set");
  m4valor("frmcallfilter","zidsentence","MY_SENTENCE","set");
  m4valor("frmcallfilter","zreturnpage","mi_página.jsp","set");
  m4valor("frmcallfilter","zretmode","1","set");

  m4submit("frmcallfilter");
}

```

Grabación de filtros

Además de la función de edición tratada anteriormente, el filtro html permite la grabación de los mismos. Concretamente permite guardar todas las sentencias asociadas a los filtros definidos para una subsesión. Si previamente ha introducido un ID Subsesión (zsubsesion), podrá grabar juntos todos los filtros creados en la misma. Sólo se necesita invocar a la página del filtro indicando como zidoperation: API_PERSIT_FILTERS.

EJEMPLO

La función de grabación es la siguiente:

```
function persistFilters() {  
    m4valor("frmcallfilter", "zidoperation", "API_PERSIST_FILTERS", "set");  
  
    m4submit("frmcallfilter");  
}
```

Llamada a las funciones de edición y grabado de filtros

Para ejecutar las funciones javascript desde el código HTML, utilice la siguiente sintaxis:

```
<input type="button" name="GetFilter" value="Edit Statement"  
onclick="javascript:Edit_filter(')"/>  
  
<input type="button" name = "Persist All" value="Save"  
onclick="javascript:persistFilters()"/>
```

Cómo usar la función de filtro HTML con instancias

Si quiere que el usuario pueda guardar un conjunto específico de filtros HTML que se han definido bajo una funcionalidad concreta, necesitará crear una instancia propia del Meta4object API del filtro HTML : API_FILTER_HTML_CL en lugar de usar una instancia creada automáticamente por el sistema, dado que en este último caso al invocar a la persistencia se grabarían todos los filtros generados en la sesión.

Éstos son los pasos que debe seguir:

1. En el Meta4Object, debe crear una instancia dinámica del Meta4Object API_FILTER_HTML_CL.

Si su Meta4Object se llama EX_DEFINE_FILTER_INSTANCE, cree un método LN4_DEFINE_INSTANCE para almacenar el alias generado de la instancia en un elemento llamado ALIAS_X.

EJEMPLO

```
sAlias = "API_FILTER_HTML_CL"
    DefineInstance(sAlias, "API_FILTER_HTML_CL", M4_AUTOLOAD_NODESAYS,
        M4_INSTANCE_NOT_SHARED)
sCode = sAlias+ "!Addaspartner()"
ClcExecuteLN4Jit (sCode)
ALIAS_X = sAlias
```

2. Desde una página .JSP, ejecute este método antes que el filtro HTML.

EJEMPLO

```
<%
...
//Constants
    String zm4defineinstance = "EX_DEFINE_FILTER_INSTANCE ";
    String zoutputdef1 = "ALIAS_ EX_DEFINE_FILTER_INSTANCE ";
    String znodedefineinstance = " EX_DEFINE_FILTER_INSTANCE_NODE ";
    String zitemdefineinstance="DEFINE_INSTANCE";
%>
<m4:beginjob/>
    <m4:datadef m4o=" EX_DEFINE_FILTER_INSTANCE" m4name=
        "EX_DEFINE_FILTER_INSTANCE" />
    <m4:exec alias = "DEFINE_INSTANCE " m4object =
        "EX_DEFINE_FILTER_INSTANCE" node = "EX_DEFINE_FILTER_INSTANCE_NODE
        " method=" DEFINE_INSTANCE">
    </m4:exec>
    <m4:outputdef m4alias="ALIAS_ EX_DEFINE_FILTER_INSTANCE ";
        m4object=" EX_DEFINE_FILTER_INSTANCE "
        node="EX_DEFINE_FILTER_INSTANCE_NODE " records="*/>
<m4:endjob/>
```


-
3. Desde la página que llama al filtro, pase la instancia creada en una llamada a la página shco_gen_htmlfilter.jsp. Para ello, utilice una función de javascript que abra la página shco_gen_htmlfilter.jsp en un formulario.

EJEMPLO

```
<form action="/servlet/CheckSecurity/JSP/shco_g0/
shco_gen_htmlfilter.jsp" name="frmcallfilter" id="frmcallfilter"
method="post" >
<input type="hidden" id="zidoperation" name="zidoperation" value=""/>
<input type="hidden" id="zidsentence" name="zidsentence" value=""/>
<input type="hidden" id="zidescenario" name="zidescenario" value=""/>
<input type="hidden" id="zidtable" name="zidtable" value = "" />
<input type="hidden" id="zsubsesion" name="zsubsesion" value = "" />
<input type="hidden" id="zreturnpage" name="zreturnpage" value = "" />
<input type="hidden" id="zidrelationtype" name="zidrelationtype"
value = "" />
<input type="hidden" id="zhtmlfilterinstance"
name="zhtmlfilterinstance" value = ""/>
<input type="hidden" id="znode" name="znode" value = "" />
<input type="hidden" id="zretmode" name="zretmode" value = "" />
<input type="hidden" id="zshowsavobutton" name="zshowsavobutton"
value = "1" />
</form>
function editFilter(){
    var swindow = "htmlfilterwindow";
    var arg_array = new Array;
    arg_array[0] = "Input_id_sentence";
    arg_array[1] = "Input_natural_lang";
    arg_array[2] = "Input_APIISQL";
    m4window(swindow,"",arg_array,"NombreFormulario","800","600","no","
no",true,"yes");
    document.forms.frmcallfilter.target= swindow;
    m4valor("frmcallfilter","zidoperation", "API_GET_FILTER", "", "set");
    m4valor("frmcallfilter","zidescenario","PERSON_SCENARIO","set");
    m4valor("frmcallfilter","zidtable","", "set");
    m4valor("frmcallfilter","zidsentence","MY_SENTENCE","set");
    m4valor("frmcallfilter","zreturnpage","mi_pagina.jsp","set");
    m4valor("frmcallfilter","zretmode","1","set");
    m4valor("frmcallfilter","zhtmlfilterinstance","<%=zsAlias%>","set");
    m4valor("frmcallfilter","zretmode","<%=zsubsesion%>","set");
    m4submit("frmcallfilter"); }
```

4. Una vez realizadas todas estas operaciones, el usuario puede utilizar la instancia creada para guardar el Meta4Object API_FILTER_HTML_CL y todos los filtros HTML creados.

Para ello, cree un método llamado PERSIST_FILTERS en el Meta4Object con el siguiente código LN4:

EJEMPLO

```
sCode = "iRet = " + ALIAS_X + "!" +
"API_FILTER_HTML.API_PERSIST_FILTERS():return(iRet) "
iResult = clcExecuteLN4JIT(sCode)
if (iResult <> M4_SUCCESS) then
    messagebox ("ERROR")
endif
```

Puede ejecutar este método desde una página .JSP.

EJEMPLO

```
<m4:beginjob/>
  <m4:datadef m4o=" EX_DEFINE_FILTER_INSTANCE " m4name="
EX_DEFINE_FILTER_INSTANCE " />
  <m4:exec alias="PERSIST_FILTERS" m4object="
EX_DEFINE_FILTER_INSTANCE " node=" EX_DEFINE_FILTER_INSTANCE_node"
method="PERSIST_FILTERS">
  </m4:exec>
<m4:endjob/>
```

Filtros HTML dinámicos

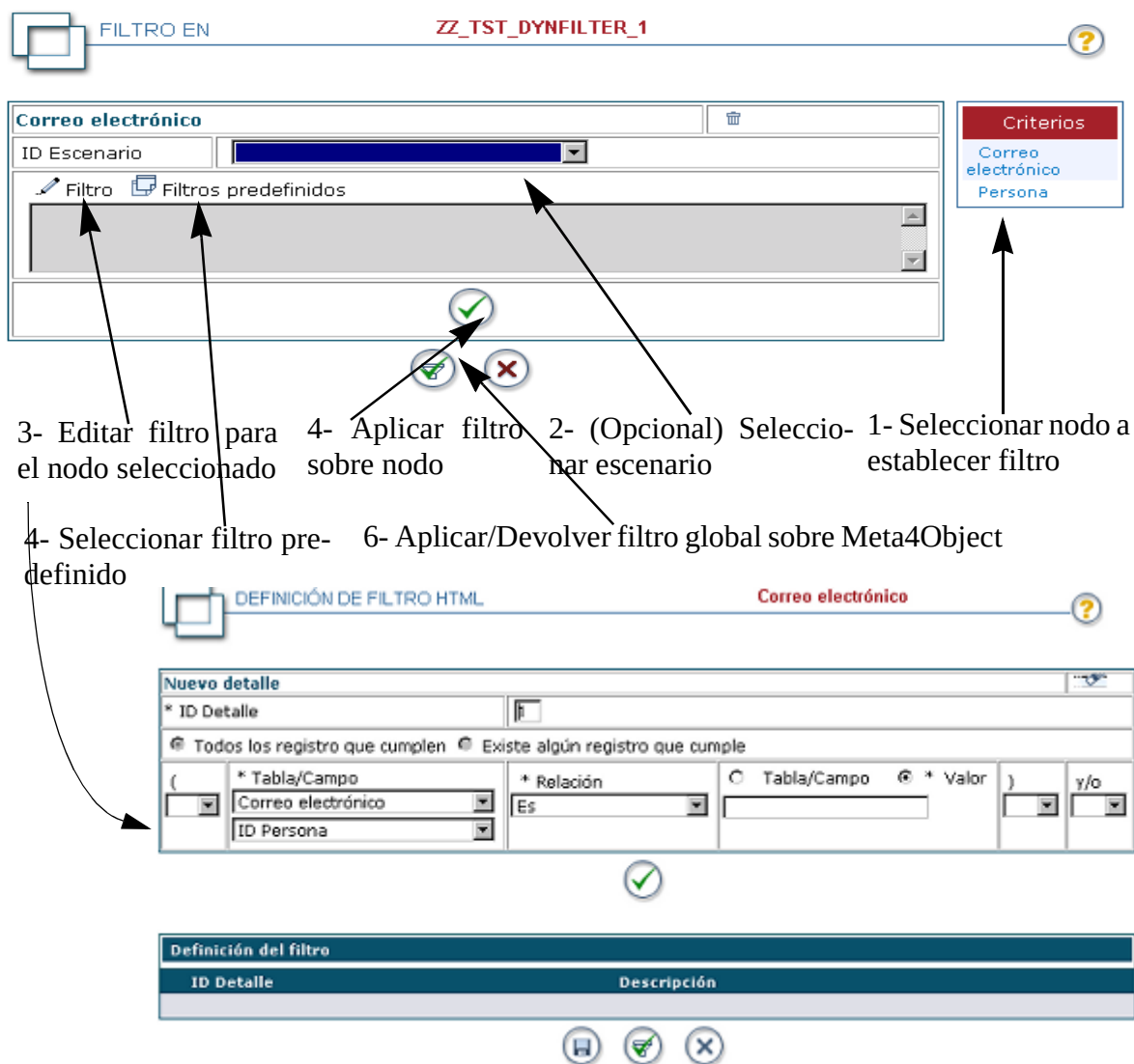
La aplicación permite configurar que un proceso del entorno de cliente ligero solicite los criterios de filtrado para un Meta4Object determinado durante su ejecución. De tal forma que, al ejecutar el proceso, se invoca el sistema de filtro dinámico. Este sistema detecta qué nodos del Meta4Object que se le indique están habilitados para el filtrado dinámico y así se los muestra al usuario.

Sobre estos nodos el usuario podrá establecer los criterios de filtro que desee bien usando filtros predefinidos o bien definiendo nuevos filtros a través del filtro HTML. Para cada nodo que se muestra se toma como tabla base la del nodo de lectura, de forma que los filtros que se elijan o definan irán bien sobre dicha tabla base , o bien sobre alguno de los escenarios disponibles para esa tabla base.

El sistema como salida generará el APISQL de cada uno de los nodos, que podrá aplicarse automáticamente en el propio Meta4Object o guardarse para una posterior aplicación.

No es necesario que cree el filtro en sí, sino que todo lo que tiene que hacer es una llamada desde la página .JSP a la función de filtro dinámico, que después le pedirá los criterios de filtro que correspondan. El sistema muestra el contenido de los criterios de búsqueda o los detalles para que pueda modificarlos o crear uno nuevo.

Figura 36. Filtro dinámico o en ejecución.



Cómo llamar a la función de filtro dinámico

Para invocar al filtro dinámico desde una página JSP, hay que incluir la página *shco_g0/shco_gen_dynfilter_include.jsp* y después llamar a la función genérica de javascript *m4dynfilter()*.

Para más información sobre esta función, sus parámetros de entrada y de salida vea el capítulo Filtros en ejecución del Development Framework.

Cómo usar la salida de la sentencia del filtro dinámico

Una vez generada y ejecutada la sentencia del filtro dinámico, puede usarla para filtrar un Meta4Object. Para eso, puede crear un método LN4 que cargue un Meta4Object determinado con este filtro.

EJEMPLO

```
'[IN] ARG_T3: Meta4Object que se carga
'[IN] ARG_DYN_FILTER_INFO: cadena para el filtro dinámico de serie
'in JIT format - T3!NODE.ITEM=VALUE: ...
' Return: M4_SUCCESS/M4_ERROR

sMeta4ObjectID = ARG_T3
sCode = ARG_DYN_FILTER_INFO

// Dynamically define the Meta4Object Alias for the level-2 call -
also considering the org type. //
DefineInstanceEx(sMeta4ObjectID, sMeta4ObjectID, 0, 1, "",
M4_ORGANIZATION_L2_TYPE_ANY)
sCode = sCode + "iResult = " + sMeta4ObjectID + "!Load
():return(iResult)"
iResult = clcExecuteLN4JIT(sCode)
If (iResult = 0) then
    iResult = M4_ERROR
Else 'OK
    iResult = M4_SUCCESS
Endif
return iResult
```

Uso de gráficos para desarrollar páginas en el entorno Web

5

Acerca de este capítulo

En este capítulo se describen las principales características de los gráficos que se puede crear, sobre todo los que se puede generar en formato HTML, y cómo parametrizar su presentación.

Primero se describen los diferentes tipos de gráficos para después examinar con más detalle los gráficos de dos ejes en HTML. Se describen distintos tipos de resultados y cómo crearlos.

Se asume que el usuario ya tiene conocimientos del Diseñador de Meta4Objects y de la herramienta de Consulta, además de nociones básicas acerca del Diseñador de gráficos.

Introducción

En el objetivo final de incorporar un gráfico en una página HTML intervienen tres tareas independientes en su diseño pero que se interrelacionan en el momento de la ejecución. Estas tareas son las siguientes:

- Diseño del Meta4Object de datos, utilizando el Diseñador de Meta4Objects
- Diseño del gráfico, utilizando el Diseñador de gráficos

Gráficos de uno, dos y tres ejes

Desde el Diseñador de gráficos se pueden generar gráficos de tres tipos:

- Numéricos o gráficos de un eje
- Series o gráficos de dos ejes
- 3D o gráficos de tres ejes

Cada tipo de gráfico tiene una estructura específica en su Meta4Object de datos. Además, cada eje supone un grado más de libertad a la hora de introducir datos. Por ejemplo, para el siguiente conjunto de datos:

Países	Hombres	Mujeres
España	10	15
Francia	30	17
Italia	2	23
China	43	53
Canadá	23	3
México	21	0

- Un gráfico de un eje permite representar el número de hombres y mujeres para un solo país.
- Un gráfico de dos ejes permite representar el número de hombres y mujeres para varios países (n países).
- Un gráfico de tres ejes permite representar n datos (hombres, mujeres, etc.) para n países.



Aunque el Diseñador de gráficos permite diseñar gráficos de uno, dos y tres ejes, en el cliente ligero sólo se admiten los gráficos de dos ejes.

Creación de gráficos de dos ejes en HTML

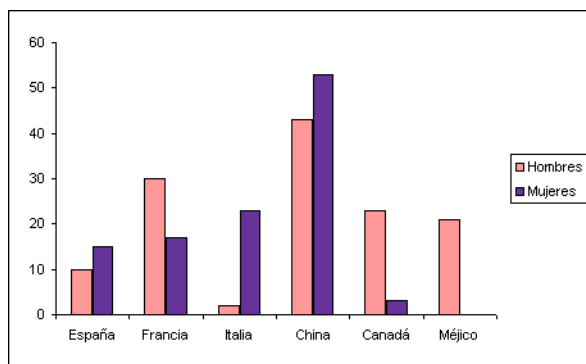
Mediante una serie de tipos de gráficos que se pueden crear en el entorno de cliente ligero para cada tipo de resultados, se describirá cómo se debe:

- Parametrizar el Meta4Object de datos
- Configurar el gráfico partiendo de los datos de dicho Meta4Object

Caso 1: Gráfico de columnas del número de hombres y mujeres en cada país

Partiendo de los datos del ejemplo, se puede crear un gráfico que indique el número de hombres y mujeres de varios países.

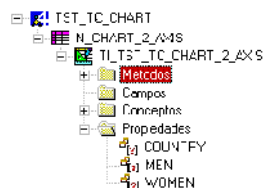
Figura 37. Representación en columnas del número de hombres y mujeres por país



Diseño del Meta4Object

La estructura del Meta4Object de datos, en este caso TST_TC_CHART, para este gráfico debe ser la siguiente:

Figura 38. Estructura del Meta4Object de datos



La lectura de los datos es la siguiente:

- Cada registro del nodo contiene los datos correspondientes a un elemento. El elemento 'COUNTRY' indica el país, y el valor de este elemento se representa en el eje X.
- Los demás elementos representan información del país y se representan en el eje Y. El nombre de cada uno de estos elementos aparece en la leyenda del gráfico.



Todos los elementos que se representan en un gráfico deben pertenecer a un mismo nodo.



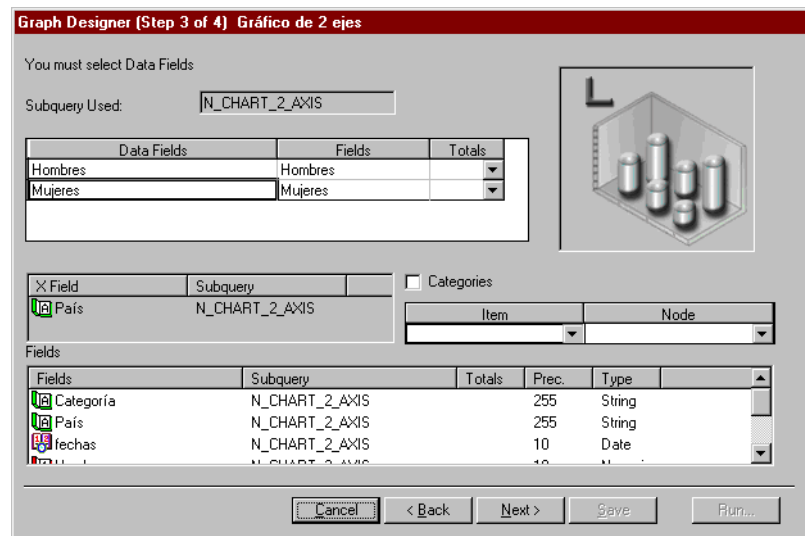
En los gráficos HTML, a causa de una limitación de la salida de nombres (etiqueta Label), no se visualizan los nombres de los ejes X e Y. Esto no ocurre en PeopleNet, donde los gráficos salen correctamente y los nombres de los ejes sí se ven.

Diseño del gráfico

En el Diseñador de gráficos se elige en qué eje se representa cada elemento, el tipo de representación, los colores para cada serie, etc.

1. Al abrir el Diseñador de gráficos, primero indique la consulta con la que se ha creado el Meta4Object de datos que se quiere usar para generar los gráficos.
2. El primer paso es seleccionar un estilo de gráfico. Seleccione 2D para indicar que se creará un gráfico de serie.
3. El segundo paso es seleccionar un campo del Meta4Object de datos para el eje X.
4. El tercer paso es seleccionar un campo del Meta4Object de datos para el eje Y. Después podrá proceder a configurar el gráfico en sí.

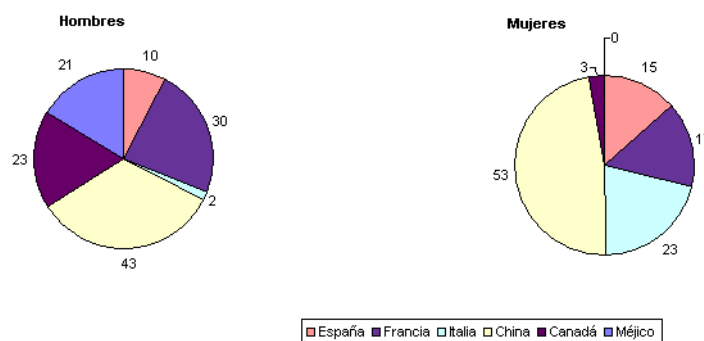
Figura 39. Configuración de los ejes X e Y del gráfico de serie



Caso 2: Gráfico de tarta de hombres y mujeres por países

Para el mismo conjunto de datos se puede elegir diferentes tipos de gráficos. La elección se puede hacer tanto desde el Diseñador de gráficos como desde la página .JSP.

Figura 40. Gráfico de tarta de hombres y mujeres por países



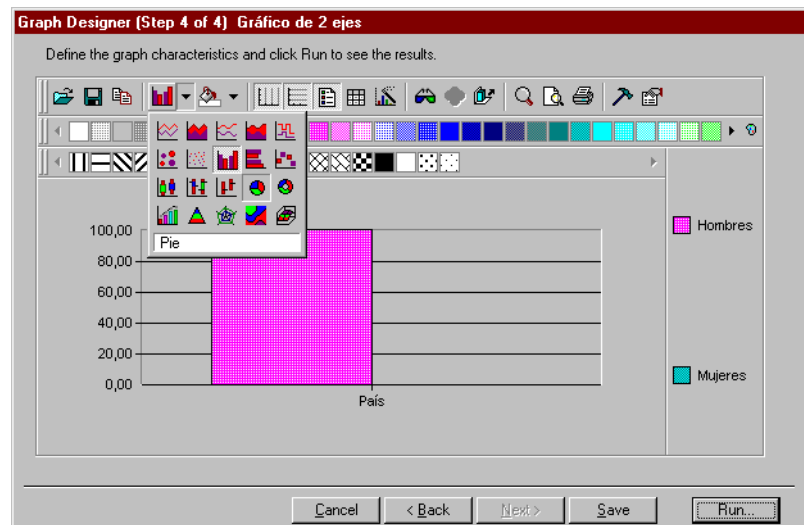
Diseño del gráfico

No hace falta cambiar el Meta4Object de datos que se utilizó en el caso anterior. Simplemente hay que configurar en el Diseñador de gráficos el tipo de gráfico que se quiere utilizar.

Tras concluir el tercer paso, el gráfico ya tiene sus ejes definidos. En el paso cuarto, se puede configurar el gráfico:

- Haga clic en el botón Gallery para seleccionar un tipo de gráfico.

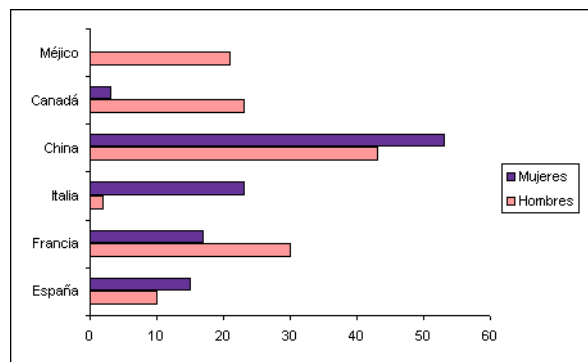
Figura 41. Selección del tipo de gráfico que se va a crear



Caso 3: Conmutación de los ejes X e Y

En algunos casos puede interesar conmutar los ejes X e Y. Por ejemplo, en un gráfico de barras éstas se representarían de forma horizontal.

Figura 42. Representación de la conmutación de los ejes X e Y



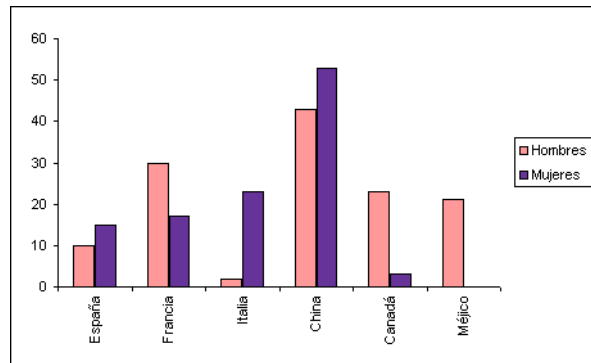
Diseño del gráfico y del Meta4Object de datos

No hace falta cambiar el Meta4Object de datos que se utilizó en el caso anterior ni modificar la configuración del Diseñador de gráficos de los ejemplos anteriores.

Caso 4: Rotación del texto de los ejes

Esta característica permite rotar los textos de los ejes para verlos mejor cuando sean muy grandes.

Figura 43. Rotación del texto de los ejes



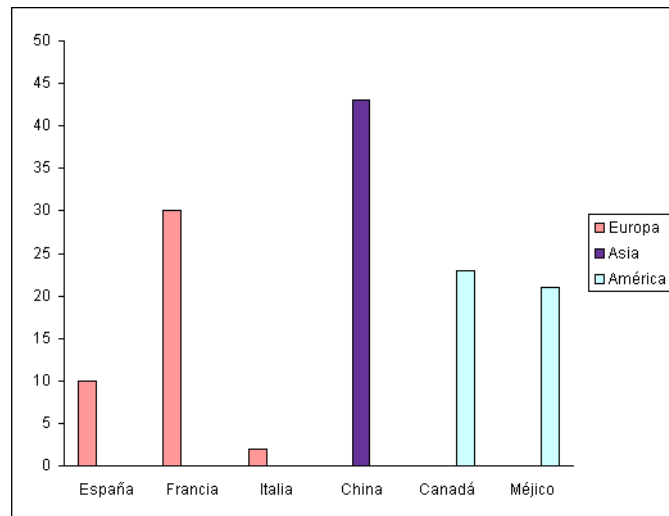
Diseño del Meta4Object de datos y del gráfico

No hace falta cambiar el Meta4Object de datos que se utilizó en el caso anterior ni modificar la configuración del Diseñador de gráficos de los ejemplos anteriores.

Caso 5: Gráfico de categorías

Un gráfico de categorías es aquél en el que se muestran los datos agrupados por conceptos. Por ejemplo, en el siguiente gráfico se muestra el número de hombres en los distintos países, y éstos aparecen agrupados (color) por continentes.

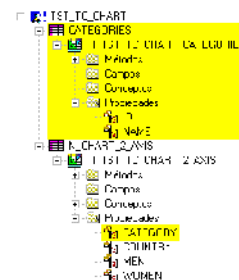
Figura 44. Gráfico de categorías



Diseño del Meta4Object de datos y del gráfico

En el Meta4Object de datos hace falta insertar un nuevo elemento que indique a qué categoría pertenece cada país y un nuevo nodo donde se almacene la lista de categorías.

Figura 45. Modificación de la estructura del Meta4Object de datos para incluir categorías



En este ejemplo, el elemento CATEGORY contendrá los valores (EU, EU, EU, AM, AS, AM) y el nodo CATEGORIES contendrá las parejas (EU, Europa), (AM, América), (AS, Asia). Los datos se leen de la siguiente manera:

- En función de la categoría de cada país, se asigna un color.
- En la leyenda del gráfico aparece el nombre ('NAME') de las distintas categorías.

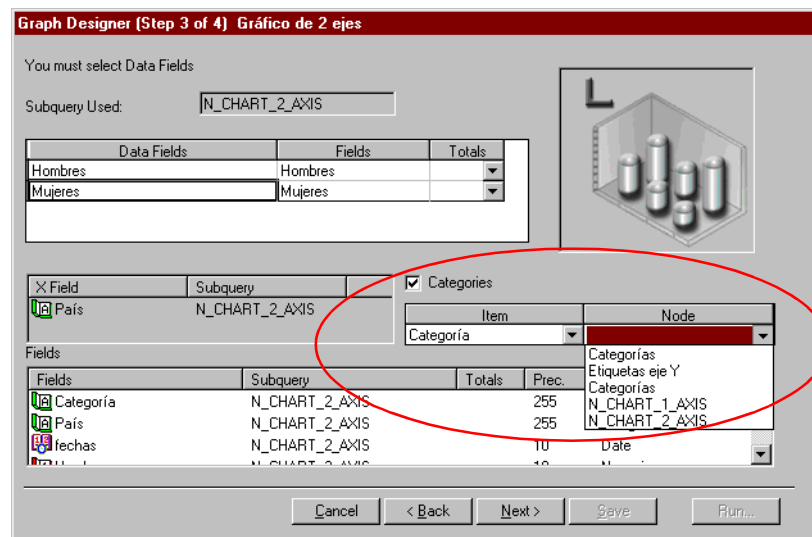


Este tipo de gráfico sólo permite representar una serie en el eje Y.

Tras modificar el Meta4Object, se procederá a configurar el gráfico desde el Diseñador de gráficos en el paso 3:

1. En la pantalla de diseño, active la casilla de verificación Categorías, que indica que se está diseñando un gráfico de categorías.
2. Seleccione el elemento que define la categoría.
3. Seleccione el nodo donde se encuentra el elemento que define la categoría.

Figura 46. Modificación del diseño del gráfico para categorías



Caso 6: Configuración de valores en el eje Y

El eje Y de los gráficos representa por defecto valores numéricos. Si quiere mostrar valores textuales, debe crear una tabla de relación.

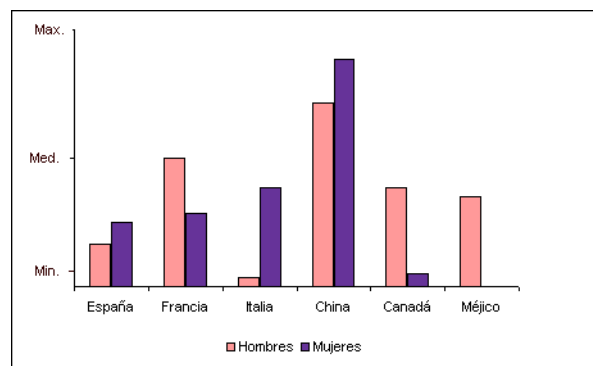
Por ejemplo, si quiere usar los siguientes parámetros:

- Min. para mínimo
- Med. para medio
- Max. para máximo

La tabla de relación tendrá que reflejar lo siguiente:

(valor, texto) - (10, Min.), (20, Med.), (30, Max.)

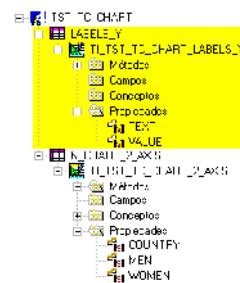
Figura 47. Configuración de valores en el eje Y



Diseño del gráfico y del Meta4object de datos

En el Meta4Object de datos, hay que crear un nuevo nodo que contenga dos propiedades, VALUE y TEXT, donde se insertan los valores de la tabla.

Figura 48. Estructura del Meta4Object de datos para configurar valores en el eje Y

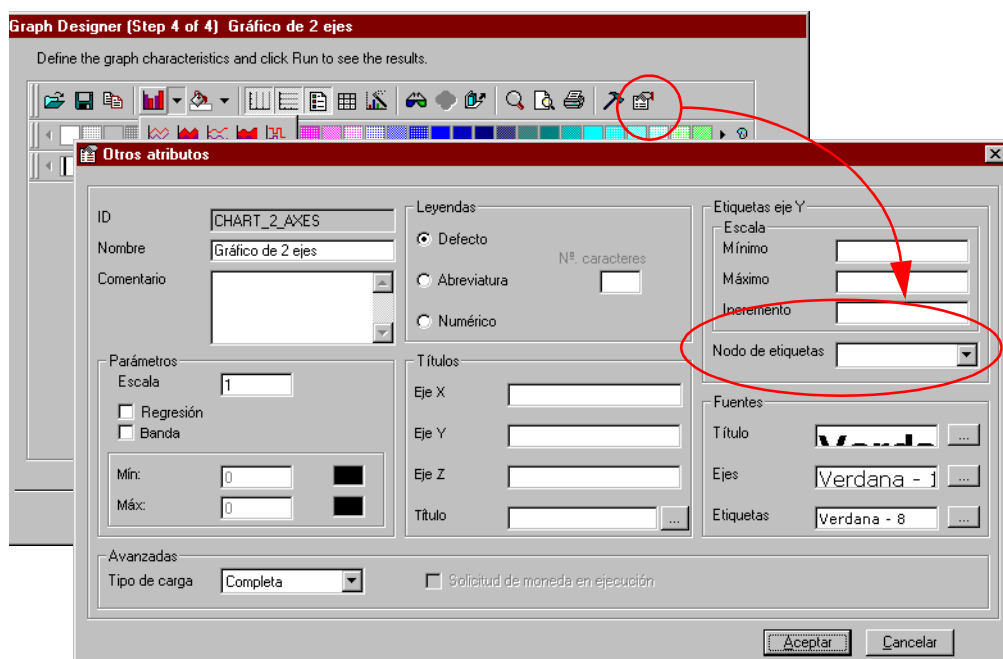


En el ejemplo, el nodo LABELS_Y tendrá las parejas (10, Min.), (20, Med.) y (30, Max.).

En el Diseñador de gráficos, debe indicar el nodo que contiene estas etiquetas. Siga estos pasos:

1. Configure la fuente del contenido de los ejes en el paso 2 del asistente.
2. En la pantalla del paso 3, haga clic en **Atributos**.
3. Seleccione el **Nodo de etiquetas** de la lista.

Figura 49. Configuración del nodo de etiquetas

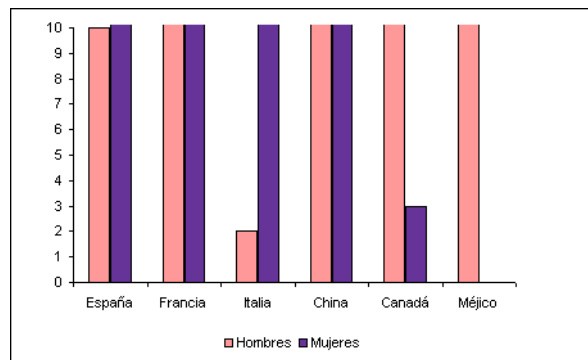


Caso 7: Control de escala del eje Y

En la creación de un gráfico, el sistema elige automáticamente los valores que se muestran en la escala del eje Y. En los ejemplos, el sistema ha elegido los valores 0, 20, 40 y 60.

Sin embargo, existe una característica que permite fijar los valores mínimo, máximo e incremento tanto desde el Diseñador de gráficos como desde la página .JSP. En la figura siguiente se ha fijado el mínimo en 1, el máximo en 10 y el incremento en 1.

Figura 50. Control de escala del eje Y

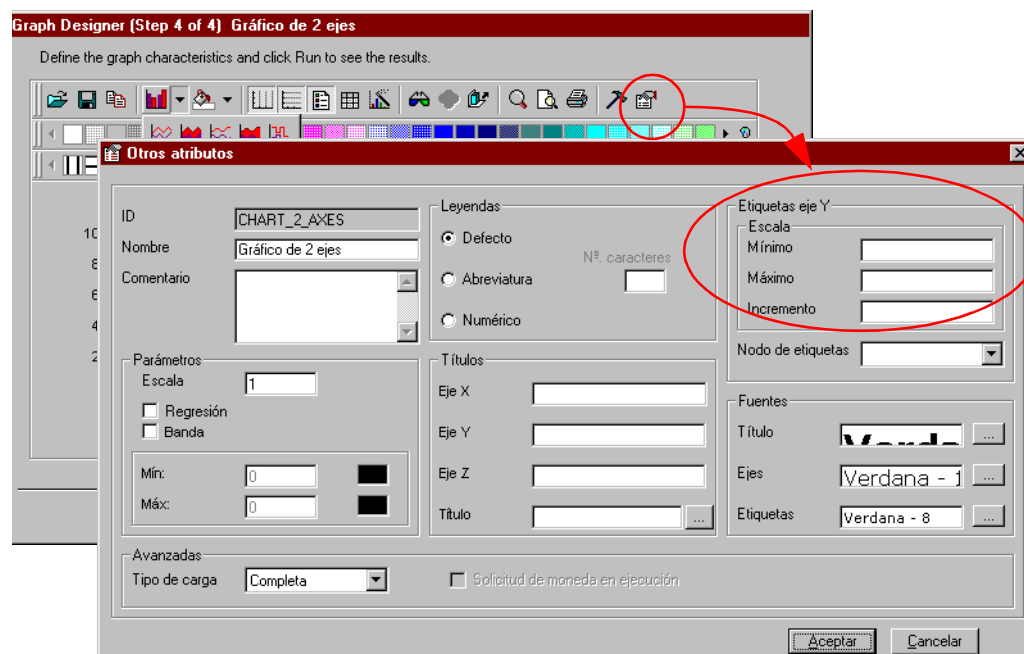


Diseño del gráfico

En el Diseñador de gráficos, debe indicar el nodo que contiene estas etiquetas. Los pasos son:

1. Configure la fuente del contenido de los ejes en el paso 2 del asistente.
2. En la pantalla del paso 3, haga clic en **Atributos**.
3. Introduzca los valores correspondientes en **Escala** para **Etiquetas eje Y**.

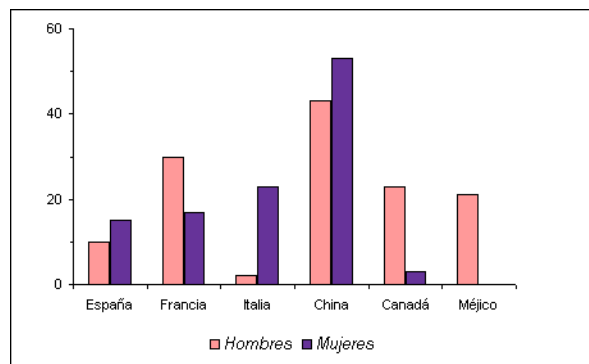
Figura 51. Configuración del control de escala del eje Y



Caso 8: Tipos de fuentes

Se puede especificar el tipo de fuente que se va a utilizar en los textos de un gráfico. Existen tres categorías de fuentes: título, leyendas y etiquetas de los ejes.

Figura 52. Especificación del tipo de fuente de un gráfico

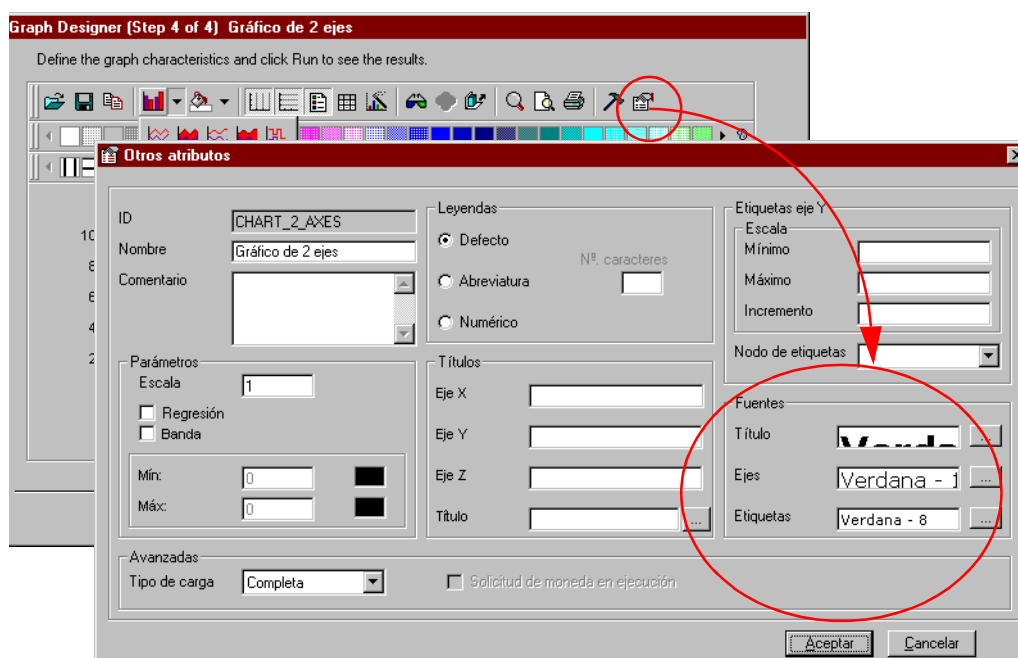


Diseño del gráfico

En el Diseñador de gráficos, siga estos pasos:

1. Configure la fuente del contenido de los ejes en el paso 2 del asistente.
2. En la pantalla del paso 3, haga clic en **Atributos**.
3. Seleccione el **Nodo de etiquetas** de la lista.

Figura 53. Configuración del nodo de etiquetas



Salida HTML de informes

6

Acerca de este capítulo

En este capítulo se describen las principales características de los informes en formato HTML y cómo parametrizar su presentación de acuerdo con la imagen corporativa para incluir estos informes en intranets u otros entornos web.

Primero se presentan las características HTML y a continuación se explican los archivos que componen la plantilla HTML que se usa por defecto, con el objetivo de que el usuario sea capaz de parametrizarla y adaptarla a sus necesidades si lo considera necesario. Es importante señalar que el perfil del usuario que realice esta tarea requerirá conocimientos avanzados de HTML y JavaScript.

Finalmente se incluye un pequeño anexo con todos los parámetros existentes para activar o desactivar las distintas características HTML.

Introducción

La salida HTML no se limita a la generación de una simple página HTML con los datos del informe, sino que crea un conjunto de páginas HTML con la funcionalidad necesaria para construir el marco completo que permite consultar el informe: navegación entre páginas, tabla de contenidos, zooms, búsquedas, etc. A este conjunto de páginas se le llama **Visor HTML**.



El visor HTML muestra un informe en formato HTML puro de alta resolución sin necesidad de usar plug-ins ActiveX o Java.

El motor de informes genera este visor basado en una serie de plantillas HTML completamente parametrizables por el usuario.



Para visualizar correctamente la salida de informes en cliente HTML, es necesario tener configurado el parámetro DPI Setting a 96 DPI. Para configurar dicho valor en Windows, haga clic con el botón derecho del ratón en el escritorio y seleccione la opción "Display Properties" > "Settings" > "Advanced" > "DPI Settings" > "Normal size (96 DPI)".

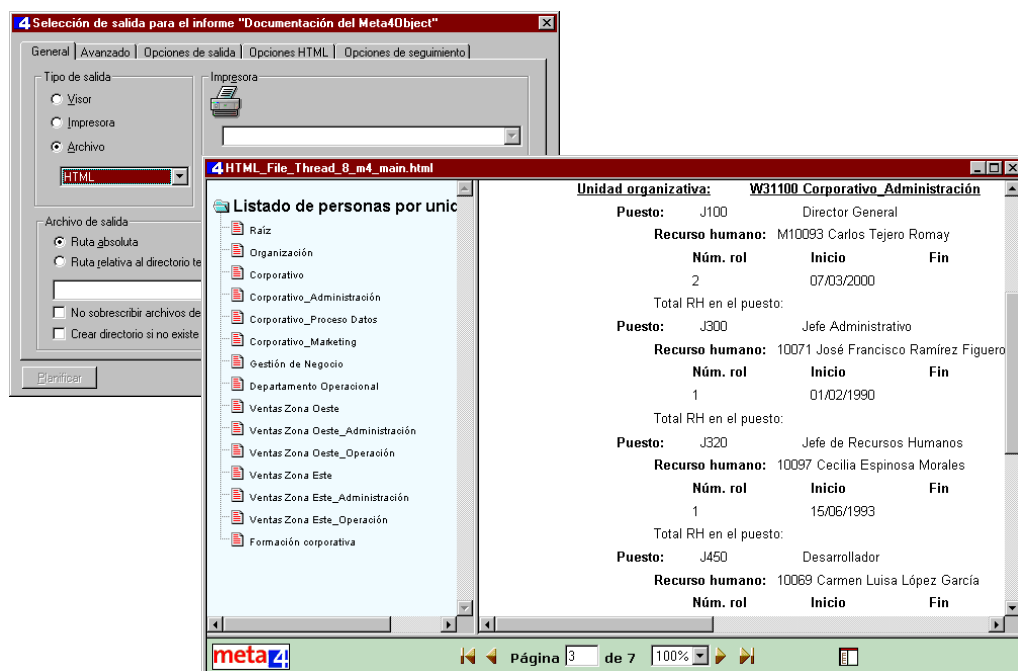
Características de la salida HTML

Cuando el usuario selecciona como salida de un informe el formato HTML, el motor de informes no genera un único archivo HTML con el contenido del informe, sino que además de la página o páginas con los datos, se generan una serie de archivos auxiliares. Estos archivos auxiliares ofrecen la siguiente funcionalidad:

- Navegación entre las distintas páginas
- Aplicación de zoom sobre la vista del informe
- Generación de la tabla de contenidos
- Búsqueda de literales a lo largo del informe
- Parametrización del entorno HTML: marcos, presentación...

Al consultar el informe se puede usar todas estas funcionalidades aunque, según los requisitos de optimización de la visualización, el usuario podrá prescindir de alguna de ellas en la parametrización.

Figura 54. La salida HTML

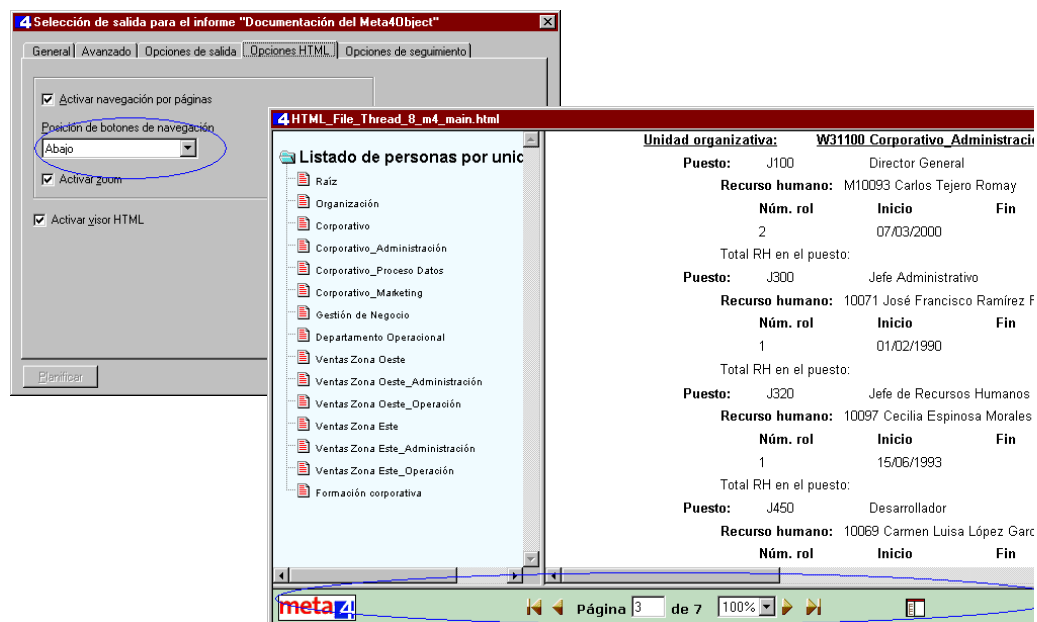


Para habilitar o deshabilitar esta funcionalidad, basta con activarla o desactivarla en las pestañas **Opciones HTML** y **Avanzado** del lanzador de informes o introducir los parámetros adecuados si el informe se genera desde una presentación o desde un entorno de cliente ligero HTML.

Navegación

Desde el visor HTML podrá navegar entre las distintas páginas del informe: ir a una página determinada, pasar a la anterior, a la siguiente, a la primera o a la última.

Figura 55. Navegación



En la plantilla HTML que se usa por defecto, esta funcionalidad se presenta en una barra de navegación que además da acceso a la tabla de contenidos, al zoom y a la búsqueda.

Para habilitar la navegación es necesario activar la casilla de verificación **Activar navegación por páginas** en la pestaña **Opciones HTML**.

De la misma forma, para establecer la barra de navegación en la parte inferior o en la superior se debe seleccionar la opción correspondiente en el cuadro combinado **Posición de botones de navegación** de la pestaña **Opciones HTML**.

Esta barra de navegación, como el resto de archivos, es totalmente parametrizable. Tan sólo se debe respetar una serie de elementos, que son las funciones encargadas de realizar los saltos de páginas, hacer la llamada a la tabla de contenidos o a la búsqueda. En el archivo **m4_menu.html** se encuentra el código correspondiente a la barra de navegación por defecto, que se puede tomar como ejemplo para desarrollar nuevas barras de tareas.

Los elementos que se deben respetar y que aportan la funcionalidad son las siguientes funciones:

- **M4Select(1);**: selecciona como página activa dentro del visor la primera página del informe.
- **Decr();**: muestra la página anterior, siempre que la página activa no sea la primera del informe.
- **M4Select(menu.page.value);**: permite saltar a una página determinada del informe.

Por ejemplo, si se asocia el valor 4 a **menu.page.value**, la página activa dentro del navegador HTML será la número 4.

- **M4Zoom(page.value, zoom.value);**: establece el zoom con que se visualizan la páginas del informe dentro del visor.

Según el ejemplo que se ha mostrado, si **zoom.value** es igual a 2, el zoom que se aplicará sobre el número de la página almacenada en **page.value** será del 200%.

- **Incr();**: muestra la página siguiente, siempre que la página activa no sea la última del informe.
- **M4Select(-1);**: selecciona como página activa dentro del visor la última página del informe.
- **SwitchFrame();**: permite activar o desactivar la visualización de la tabla de contenidos.

El resto de elementos de la barra de navegación (imágenes, botones, fondo, etc.) pueden modificarse con facilidad e incluso pueden añadirse nuevos elementos.

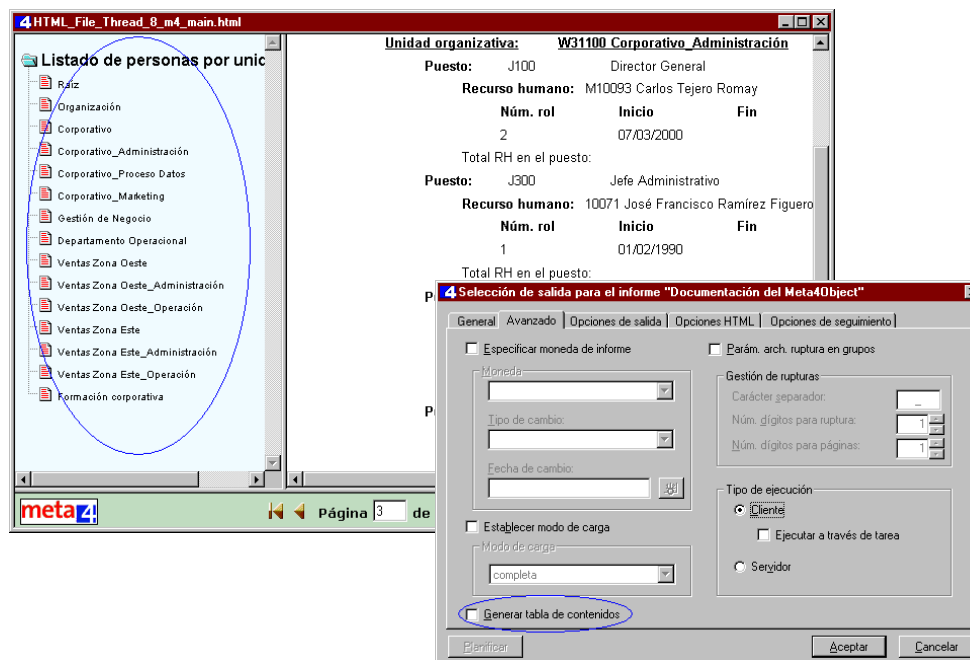
Tabla de contenidos

Para facilitar la navegación y hacer que el acceso a determinados puntos del informe sea más rápido, se ha incluido la posibilidad de utilizar una tabla de contenidos o índice como elemento de navegación.

Ésta no es una característica exclusiva del formato HTML, pues también se pueden incluir tablas de contenidos en formato PDF.

La tabla de contenidos proporciona una vista jerárquica del contenido del informe y permite hacer clic sobre una entrada del árbol para poder ver la página asociada.

Figura 56. Tabla de contenido



El nivel de jerarquía de cada elemento se establece desde la propiedad avanzada **Nivel**, que podrá encontrar dentro de las propiedades avanzadas del campo. Recuerde que esta propiedad sólo es aplicable a campos de tipo estándar, imágenes y códigos de barras que no sean estáticos.

Desde esta propiedad puede establecer qué posición va a tener el campo dentro de la jerarquía del árbol utilizando un valor que sea mayor o igual a 1, ya que el valor 0 está reservado para el nodo principal del árbol y coincide con el nombre del informe.

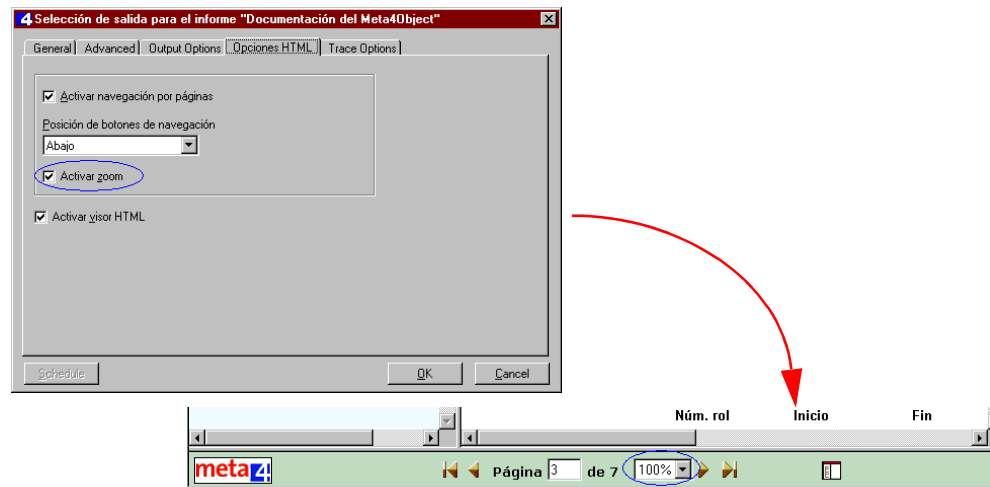
También puede establecer el literal que se mostrará asociado a un nodo dentro del árbol de navegación. Este literal se establece dentro de la propiedad **Contenido** que, al igual que la propiedad **Nivel**, está dentro de las propiedades avanzadas del campo. Como contenido puede establecer un valor literal o crear una expresión con el Editor de expresiones.

Una vez establecidos los distintos niveles jerárquicos de la presentación del informe, active la casilla de verificación **Generar tabla de contenidos** de la pestaña **Avanzado** para ver el informe en HTML. También puede utilizar el parámetro /TOC si está ejecutando el informe desde un entorno de cliente ligero HTML, o TOC a "True" si lo ejecuta desde una presentación.

Zoom

Otro de los elementos que puede utilizar dentro de la barra de navegación del visor HTML es el zoom. Para ello, deberá haber seleccionado previamente la opción **Activar zoom** dentro de la pestaña **Opciones HTML**.

Figura 57. Zoom



Si usa la plantilla HTML por defecto, podrá modificar el zoom en el menú que aparece en la barra de navegación.

Esta opción, aunque es muy útil, puede ralentizar la visualización del informe en el navegador, pues se realiza a través de funciones JavaScript que se ejecutan en la máquina cliente.

Tipos de salida HTML

Se pueden distinguir dos tipos de salidas HTML en función de si se ha activado o no la navegación por páginas:

- **Navegación por páginas desactivada:** se genera un único archivo HTML que contiene todas las páginas del informe. Para delimitar las páginas se introduce una barra horizontal que identifica el final de una página y el comienzo de otra.
- **Navegación por páginas activada:** se generan tantos archivos HTML como páginas tenga el informe. Además, se crea una serie de archivos adicionales cuyo número depende de la parametrización que haya hecho el usuario. Su origen son una serie de plantillas, que no son más que archivos HTML con referencias entre ellos y una serie de palabras reservadas (no HTML), que usará el motor de informes para aportar toda la funcionalidad de la salida HTML, así como para copiar imágenes u otros archivos auxiliares. En este modo de ejecución, si el usuario no especifica ninguna plantilla HTML, el motor de informes usará una predefinida para generar los archivos HTML (visor HTML).

En ambos casos, el motor de informes genera una carpeta *images* al mismo nivel que el directorio raíz, donde se copiarán todos los archivos generales del informe, es decir, imágenes, archivos .js, etc. a los que se haga referencia en varias páginas de un mismo informe.

La salida será distinta si el informe tiene ruptura por grupos, es decir, si se ha especificado una sección para la que se quiere generar salidas HTML independientes por cada registro del nodo asociado. En este caso, el motor de informes genera un directorio por cada grupo, de forma que cada uno de ellos contiene el conjunto de archivos necesarios para que pueda verse de forma independiente al resto.

Parametrización de la salida HTML

Para explicar cómo puede parametrizar el usuario la salida HTML, se describe el conjunto de archivos que usa el motor de informes si el usuario no especifica ninguno.

Antes de empezar, deberá distinguir entre:

- **Plantilla HTML:** conjunto de archivos HTML que, además de las etiquetas propias de HTML, tienen una serie de palabras clave que usa el motor de informes para proporcionar una funcionalidad específica, como navegación, zoom, tabla de contenidos, etc. En toda plantilla HTML tiene que haber un archivo principal. Será por ese archivo por el que se identifique la plantilla. No existe ninguna restricción con respecto al número de archivos que forman la plantilla; dependerá del desarrollador y del configurador del entorno web donde se alojen los informes HTML generados. Cada vez que se ejecute un informe en formato HTML, se deberá especificar una plantilla HTML. En caso contrario, el motor de informes usará una por defecto.
- **Salida HTML:** conjunto de archivos HTML que genera el motor de informes y en los que se encuentran los datos del informe junto a la plantilla HTML indicada. Representa, por tanto, la salida del motor de informes y se puede considerar como una instancia de la plantilla HTML mezclada con datos del informe.

Plantilla HTML

Localización

Las plantillas HTML tienen que localizarse en la ruta especificada en el parámetro de configuración del motor de informes GraphicsPathHTML.

Este parámetro se modifica en:

- Cliente Windows: En el nodo REPORTS del fichero de configuración regmeta4.xml
- Cliente Rich Web: En la entrada REPORTS del fichero de configuración (descargable) M4WizardIn.reg.
- Servidor: En la entrada de registro que se encuentra bajo la rama REPORTS en HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Build\appserver\APPServer\TOOLS\REPORTS. En plataformas Windows en el registro del sistema, en plataformas UNIX en el fichero regmeta4.reg.

Es importante indicar que ahí deben residir todos los archivos que formen la plantilla, incluyendo los archivos de imágenes, Javascript, etc. a los que se haga referencia. No se pueden incluir en subdirectorios.

Selección de una plantilla

Para elegir una plantilla distinta a la que se usa por defecto, el usuario debe indicar al motor de informes el archivo principal de dicha plantilla.

Al tratarse de un parámetro avanzado, no existe una entrada propia en el lanzador de informes. Tenga en cuenta las siguientes indicaciones para establecerlo:

- Si el informe se ejecuta desde una presentación (a través de la tarea M4Throw.CMFThrowProcess), hay que introducir un parámetro indicando el nombre principal del archivo y su extensión (sin la ruta):

```
SetConstStr OTHER_CONFIG_PARAM, "/"
HTML_TEMPLATE_MAIN:miplantilla.html"
```

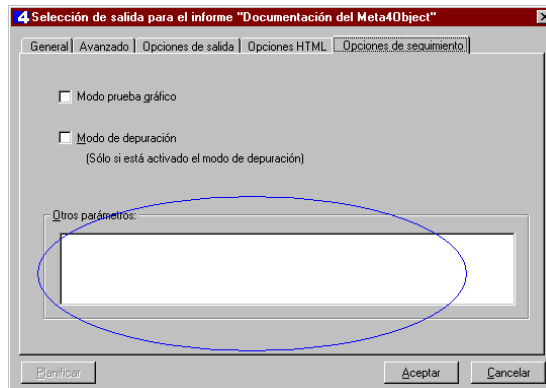
- También se puede seleccionar desde la ventana **Selección de salida para el informe**, dentro del cuadro de texto **Otros parámetros** de la pestaña **Opciones de configuración**. En este caso, el formato del parámetro sería / HTML_TEMPLATE_MAIN:miplantilla.html.
- Si se está ejecutando el informe desde un entorno de cliente ligero HTML usando el M4Tag **executereport**, deberá introducir la misma cadena en el parámetro **OtherParams**.

ExecuteReport

Crea el informe desde el Meta4Object y carga los datos según lo especificado.

```
<m4:executereport
  idreport="XXX"
  sysentence="XXX"
  outputtype="XXX"
  otherparams="XXX"/>
```

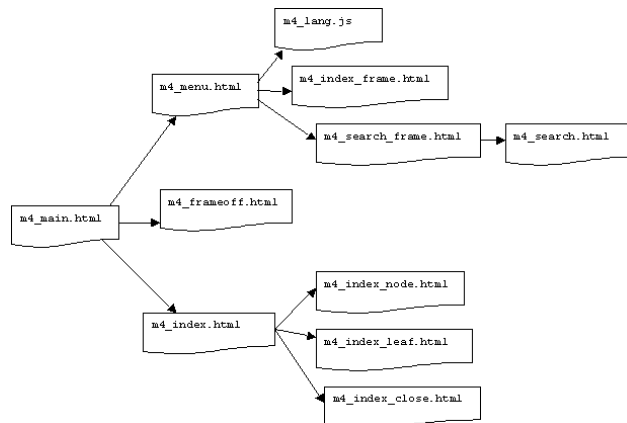
Figura 58. Parámetros avanzados



Contenido de una plantilla

La plantilla HTML que usa el motor de informes por defecto si no se le ha indicado ninguna otra contiene los siguiente archivos:

Figura 59. Estructura de los componentes de una plantilla



- **m4_main.html**: es el archivo principal. Contiene la definición de marcos genéricos y variables para colocar la barra de navegación y la tabla de contenidos. También aquí se establece cuáles son los archivos que componen el resto de la plantilla.
- **m4_menu.html**: contiene la definición para el menú de navegación del informe.
- **m4_frameoff.html**: contiene la definición del marco donde aparecerá el informe HTML.
- **m4_search.html**: contiene el formulario de búsqueda.

- **m4_search_frame.html**: contiene el marco para el formulario de búsqueda.
- **m4_index.html**: contiene la definición de los marcos para las tablas de contenidos del informe.
- **m4_index_close.html**: contiene la definición del cierre de la tabla de contenidos.
- **m4_index_frame.html**: contiene la definición del marco donde aparecerá la tabla de contenidos.
- **m4_index_leaf.html**: contiene la definición de la página del informe en sí.
- **m4_index_node.html**: contiene la definición del nodo al que corresponde una serie de páginas del informe.
- **Archivos de Javascript**:
 - m4_index.js: contiene las instrucciones para gestionar la tabla de contenidos internamente.
 - m4_lang_nn.js: contiene literales traducibles.
donde nn = 2 a 7 y cada número corresponde a un idioma determinado.
- **Imágenes**: m4blank.gif, m4doc.gif, m4fclosed.gif, m4fopen.gif, m4index.gif, m4lastnode.gif, m4left.gif, m4left2.gif, m4logo.gif, m4mlastnode.gif, m4mnode.gif, m4node.gif, m4plastnode.gif, m4right.gif, m4right2.gif, m4root.gif, m4search.gif y m4vline.gif.
- Además, están los siguientes archivos:
 - **m4_html_data_end.html**
 - **m4_html_data_start.html**

Éstos contienen el código que se inserta al principio y al final de todos los archivos HTML con los datos que genera el motor de informe durante la ejecución. De esta forma, se podrá insertar el código para que los archivos HTML adopten el estándar de cada organización. Estos archivos son independientes de la plantillas

Selección de archivos de cabecera y de pie

Al lanzar el informe, el usuario debe indicar al motor de informe los archivos de cabecera o pie de los archivos de datos.

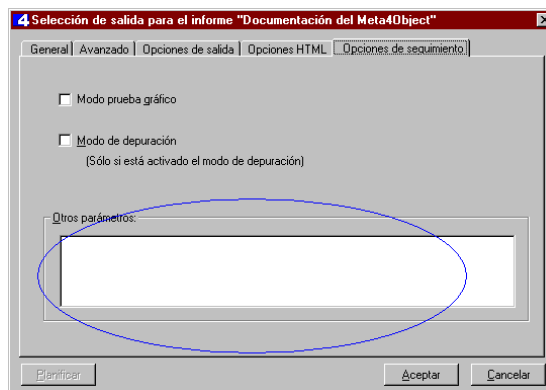
Como ocurre con la plantilla HTML, al tratarse de un parámetro avanzado no existe una entrada propia en el lanzador de informes. Tenga en cuenta las siguientes indicaciones para establecerlo:

- Si el informe se ejecuta desde una presentación (a través de la tarea M4Throw.CMFThrowProcess), hay que introducir un parámetro (no debe incluirse la ruta al archivo, tan sólo su nombre y extensión):


```
SetConstStr OTHER_CONFIG_PARAM, "/  
HTML_TEMPLATE_DATA:archivo_cabecera.html:archivo_pie.html"
```

- También se puede seleccionar desde el cuadro de texto **Otros parámetros** de la pestaña **Opciones de configuración**. En este caso, el formato del parámetro sería /HTML_TEMPLATE_DATA:archivo_cabecera.html:archivo_pie.html.
- Si se está ejecutando el informe desde un entorno de cliente ligero HTML usando el M4Tag executereport, deberá introducir la misma cadena en el parámetro OtherParams.

Figura 60. {Configuración de parámetros



Modificación de una plantilla

Como se ha comentado anteriormente, puede parametrizar la plantilla HTML para que los archivos de salida cumplan los requisitos de su empresa. Esta tarea debe ser desarrollada por usuarios con conocimientos altos de HTML y JavaScript.

Por ejemplo, una de las operaciones más corrientes puede ser cambiar la barra de navegación por otra propia. Para ello, siga estos pasos:

1. Copie todos los archivos de la plantilla por defecto a un directorio temporal y renómbralos poniéndoles, por ejemplo, un prefijo común.
2. Modifique los nuevos archivos y cambie las referencias internas para que usen los nuevos archivos.
3. Copie todos los archivos a la ruta donde indique el parámetro de configuración GraphicsPath.

- ```
href='javascript:M4Select(1);'><img src=<m4{glbfile}={m4left2.gif}m4>
```

```
href='javascript:M4Select(1);'><img src=<m4{glbfile}={primero.gif}m4>
```

Por ejemplo, para el archivo principal **m4\_main.html** se utiliza la palabra clave

[illegible]

---

## Librería de palabras clave

Se pueden definir los siguientes ámbitos en los que son operativas las palabras clave:

- **Global o de informe:** el motor de informes las resuelve al principio de la ejecución.
- **Grupo:** si la ejecución del informe se divide por grupos, genera grupos independientes de archivos por cada uno de ellos. El motor resuelve las palabras clave de este ámbito al finalizar cada grupo.
- **Tabla de contenidos:** se generará una tabla de contenidos por grupo. Existen ciertas palabras que sólo tienen sentido en los archivos de la plantilla HTML donde se define la tabla.

### Globales

#### title

El resultado es un texto con el nombre del informe.

Sintaxis:

```
<m4{title}m4>
```

#### language

El resultado es el ID del lenguaje con el que se ejecuta el informe.

Sintaxis:

```
<m4{language}m4>
```

#### glbfile

Copia, si no existe en destino, el archivo que se indica como argumento en un directorio `./images`, tomando como raíz la ruta especificada como salida del informe.

Devuelve una *url* correspondiente al archivo indicado. Si hubiera ruptura por grupos y todos los grupos usaran las mismas imágenes para la barra de navegación, tendrían que usar este comando para compartir todos las mismas imágenes en destino.

Sintaxis:

```
<m4{glbfile}={nombre_archivo.ext}m4>
```

Ejemplo:

En el archivo de la plantilla *m4\_menu.html*, se utiliza la siguiente referencia a

una imagen:

```
...
<body bgcolor='#cdcdcd' topmargin=5 leftmargin=0 marginwidth=0
marginheight=0>
<DIV STYLE='position: absolute; left: 5px; top: 5px'>
 <img src=<m4{glbfile}={m4logo.gif}m4> border=1>
</DIV>
<!-- LEFT > -->
...
```

En la salida HTML, ese código se convierte en:

```
...
<body bgcolor='#cdcdcd' topmargin=5 leftmargin=0 marginwidth=0
marginheight=0>
<DIV STYLE='position: absolute; left: 5px; top: 5px'>

</DIV>
...
```

Cuando el motor procesa la palabra clave, copia en el destino la imagen `m4logo.gif` y sustituye en el archivo HTML la referencia original por la *url* correspondiente.

## brkfile

Parecida a la palabra clave anterior pero la copia en destino del archivo se produce en el mismo directorio que los archivos de datos. Es decir, serán archivos que difieran para cada grupo (si hay ruptura por grupos). Sólo debe usarse para hacer referencia a archivos de plantilla.

Devuelve una *url* relativa al archivo indicado.

Sintaxis:

```
<m4{brkfile}={nombre_archivo.ext}m4>
```

Ejemplo:

`<m4{brkfile}={M4_INDEX.html}m4>` (se usa en `m4_main.html` para hacer referencia y copiar el archivo que contiene la definición de la tabla de contenidos)

## brkfilelang

Igual que la palabra clave anterior pero con un matiz: el archivo es el que se indica en el argumento pero se concatena el sufijo `_IDLenguaje`, donde `IDLenguaje` es el ID del lenguaje de ejecución. Esto permite cargar diferentes archivos según el idioma de ejecución.

---

Sintaxis:

```
<m4{brkfilelang}={nombre_archivo.ext}m4>
```

Ejemplo:

<m4{brkfilelang}={M4\_LANG.html}m4> (en la salida HTML se traduce por M4\_LANG\_2.html si el ID del idioma de ejecución es el 2)

### **rootnavigation**

Indica cuál será el archivo principal en la navegación, aunque su uso no es obligatorio.

Sintaxis:

```
<m4{rootnavigation}={nombre_archivo.ext}m4>
```

Ejemplo:

En el archivo m4\_main.html se puede ver cómo se establece que él mismo es el archivo raíz de la navegación.

```
<m4{rootnavigation}={m4_main.html}m4>
```

### **firstfile**

El resultado es la *url* de primer archivo de datos de la salida HTML relativa al archivo raíz donde se esté ejecutando el informe o el grupo.

Sintaxis:

```
<m4{firstfile}m4>
```

### **navigation, top\_navigation, bottom\_navigation, toc, zoo, search**

Indican si están o no activadas cada una de las características HTML.

Sintaxis:

```
<m4{navigation}m4>
<m4{top_navigation}m4>
<m4{bottom_navigation}m4>
<m4{toc}m4>
<m4{zoom}m4>
<m4{search}m4>
```

Puede ver su uso en el archivo m4\_main.html.

### **navigation\_array**

Syntaxis:

La lista de los archivos es un dato necesario para procesar correctamente la navegación. Puede ver su uso en el archivo `m4_menu.html`.

**brindex, brindexnode, brindexleaf, brindexclose**

- 1 instancia de la plantilla que indica brkindex
- n instancias de la plantilla que indica brkindexnode, una por nodo
- n instancias de la plantilla que indica brkindexleaf, una por hoja
- 1 instancia de la plantilla que indica brkindexclos

Syntax:

Ejemplo:

[illegible]

```
<m4{brkindexleaf}={m4_index_leaf.html}m4>
<m4{brkindexclose}={m4_index_close.html}m4>
```

```
-->
```

```
...
```

## indexlevel, indexlevelfather, indexstring, indexpage, indexfile, indextarget

Este conjunto de comandos sólo se puede usar en los archivos de la plantilla indicados por brkindexnode y brkindexleaf.

Sintaxis:

```
<m4{indexlevel}m4>
<m4{indexlevelfather}m4>
<m4{indexstring}m4>
<m4{indexpage}m4>
<m4{indexfile}m4>
<m4{indextarget}m4>
```

## Salida HTML

Como se ha dicho anteriormente, la salida HTML es el conjunto de archivos HTML resultado de la ejecución del informe en este formato. El motor de informes usa los archivos de la plantilla junto con los datos propios de informe para generar la salida adecuadamente.

El número de archivos generados depende de las opciones HTML elegidas en el lanzador de informes.

Así, una ejecución que use la plantilla por defecto, con la navegación, el zoom y la tabla de contenidos activados, generaría la salida HTML compuesta por los siguientes archivos:

Figura 61.

Contents of 'reports_20020731_083352'		
Name	Size	Type
images		File Folder
ROOT_m4_frameoff.html	1KB	Microsoft HTI
ROOT_m4_index.html	12KB	Microsoft HTI
ROOT_m4_index_frame.html	1KB	Microsoft HTI
ROOT_m4_lang.js	1KB	JScript File
ROOT_m4_main.html	2KB	Microsoft HTI
ROOT_m4_menu.html	5KB	Microsoft HTI
ROOT_m4_search.html	3KB	Microsoft HTI
ROOT_m4_search_frame.html	1KB	Microsoft HTI
ROOT0.html	80KB	Microsoft HTI
ROOT1.html	52KB	Microsoft HTI





## Referencia rápida de parámetros HTML

En esta sección se resumen los distintos parámetros de la salida HTML, aunque según se lance el informe desde una presentación, el M4Tag executrereport u otro entorno, se hace referencia a ellos de una manera u otra.

### Descripción general

Estos parámetros pueden usarse con esta sintaxis desde

- La etiqueta **executereport**, parámetro **OtherParams**
- Métodos LN4 que invoquen directamente el método C++ ExecuteReport o ExecuteReportServer
- El lanzador de informes, en el cuadro de texto **Otros Parámetros** de la pestaña **Opciones de seguimiento**
- La tarea M4Throw.CMFThrowProcess si usa el parámetro OTHER\_CONFIG\_PARAM

Parámetro	Descripción	Sintaxis
<b>/HTML_NAVIGATION</b>		
	Genera la barra de navegación en la salida HTML. También da acceso a la tabla de contenidos, búsquedas...	<b>/HTML_NAVIGATION:ACTIVE</b> Es el valor por defecto. La barra de navegación se hace visible y se generan tantos archivos HTML como páginas de datos tenga el informe. La barra de navegación se usa para moverse entre páginas. <b>/HTML_NAVIGATION:OFF</b> Se genera un solo archivo HTML que contiene todas las páginas del informe. Para navegar por ellas se debe usar la barra de desplazamiento del navegador.
<b>/HTML_MENU</b>		
	Sólo se aplica si /HTML_NAVIGATION:ACTIVE. Establece la posición de la barra de navegación.	<b>/HTML_MENU:BOTTOM</b> Es el valor por defecto. La barra se sitúa en la parte inferior. <b>/HTML_MENU:TOP</b> Es el valor por defecto. La barra se sitúa en la parte superior.

Parámetro	Descripción	Sintaxis
<b>/TOC, /NTOC</b>		
	Activa o desactiva la tabla de contenidos en la barra de navegación. Para que en la tabla se vean entradas tendrá que haberse parametrizado el informe adecuadamente.	/TOC Activa la tabla y crea un icono para acceder a ella en la barra de navegación estándar. /NTOC Desactiva la tabla de contenido.
<b>/ZOOM, /NZOOM</b>		
	Activa/desactiva el modo zoom.	/ZOOM Activa el modo zoom y presenta en la barra de navegación un cuadro combinado con diferentes tamaños. /NZOOM Desactiva el modo zoom.
<b>/SEARCH, /NSEARCH</b>		
	Activa/desactiva el modo de búsqueda.	/SEARCH Activa el modo de búsqueda. En la barra de navegación aparece un icono que da acceso al formulario de búsqueda. /NSEARCH Desactiva el modo de búsqueda.
<b>/HTML_TEMPLATE_MAIN</b>		
	Selecciona una plantilla HTML.	/HTML_TEMPLATE_MAIN:nombre_archivo.ext Se debe indicar el archivo principal de la plantilla HTML sin especificar la ruta, pues debe estar obligatoriamente localizado en la entrada del Registry <i>GraphicsPathHTML</i> .
<b>/HTML_TEMPLATE_DATA</b>		
	Indica qué archivos se usarán para establecer las cabeceras y pies de los archivos HTML que genera el motor de informes.	/HTML_TEMPLATE_DATA: archivo_cabecera.ext:archivo_pie.txt> Los archivos archivo_cabecera.ext y archivo_pie.ext deben cumplir lo especificado en el parámetro /HTML_TEMPLATE_MAIN.
<b>/WEB</b>		
	Transforma la ruta de salida de un informe a un formato <i>url</i> .	/WEB:B:\temp Este parámetro raramente debe ser usado por el usuario, pues es el M4tag ExecuteReport quien lo calcula al ejecutar el informe.

## Ejecución del informe desde presentaciones

Si ejecuta el informe desde una presentación, puede usar todos los parámetros anteriores, si bien muchos de ellos tienen un formato distinto:

Parámetro	Descripción	Sintaxis
<b>HTML_NAVIGATION</b>		
	Genera la barra de navegación en la salida HTML. También da acceso a la tabla de contenidos, búsquedas....	<pre>SetConstStr HTML_NAVIGATION "ACTIVE"</pre>  Es el valor por defecto. La barra de navegación se hace visible y se generan tantos archivos HTML como páginas de datos tenga el informe. Se usa la barra de navegación para moverse entre páginas.  <pre>SetConstStr HTML_NAVIGATION "OFF"</pre>  Se genera un solo archivo HTML que contiene todas las páginas del informe. Para navegar por ellas se debe usar la barra de desplazamiento del navegador.
<b>HTML_MENU</b>		
	Sólo se aplica si la navegación está activa. Establece la posición de la barra de navegación.	<pre>SetConstStr HTML_MENU "BOTTOM"</pre>  Es el valor por defecto. La barra se sitúa en la parte inferior.  <pre>SetConstStr HTML_MENU "TOP"</pre>  Es el valor por defecto. La barra se sitúa en la parte superior.
<b>TOC</b>		
	Activa o desactiva la tabla de contenidos en la barra de navegación. Para que en la tabla se vean entradas, tendrá que haberse parametrizado el informe adecuadamente.	<pre>SetConstBool TOC True</pre>  Activa la tabla y crea un icono para acceder a la barra de navegación estándar.  <pre>SetConstBool TOC False</pre>  Desactiva la tabla de contenidos. Es el valor por defecto.
<b>HTML_ZOOM</b>		
	Activa/desactiva el modo zoom.	<pre>SetConstBool HTML_ZOOM True</pre>  Activa el modo zoom y presenta en la barra de navegación un cuadro combinado con diferentes tamaños. Es el valor por defecto.  <pre>SetConstBool HTML_ZOOM False</pre>  Desactiva el modo zoom.
<b>HTML_SEARCH</b>		
	Activa/desactiva el modo de búsqueda.	<pre>SetConstBool HTML_SEARCH True</pre>  Activa el modo de búsqueda. En la barra de navegación aparece un icono que da acceso al formulario de búsqueda.  <pre>SetConstBool HTML_SEARCH True</pre>  Desactiva el modo de búsqueda. Es el valor por defecto.

Parámetro	Descripción	Sintaxis
<b>OTHER_CONFIG_PARAMS</b>		
	Es el parámetro comodín para usar cualquier parámetro, pero principalmente los que no tienen uno propio.	La sintaxis es la explicada en la tabla anterior.  <pre>SetConstStr OTHER_CONFIG_PARAM "/" HTML_TEMPLATE_MAIN:miplantill.htm /SEARCH"</pre>  También se puede separar cada parámetro con ## al comienzo y al final y dejando un espacio entre el final de uno y el comienzo del otro:  <pre>SetConstStr OTHER_CONFIG_PARAM "##/ HTML_TEMPLATE_MAIN:miplantill.htm## ##/SEARCH##"</pre>

---

## **EJEMPLO: CÓDIGO OBL QUE EJECUTA UN INFORME EN HTML ESTABLECIENDO UNA SERIE DE PARÁMETROS**

```
BEGIN Action_set set1
 Set = "*C*#SYS_BLOCK_CONTEXT_VAL# := SetConstStr ID_REPORT,
 "SCO_WU_BUDGET";SetConstStr CHANNEL_DATA_NAME , "SCO_WU_BUDGET";
 SetConstStr OUTPUT_TYPE "HTML"; SetConstStr HTML_NAVIGATION
 "ACTIVE"; SetConstStr OTHER_CONFIG_PARAMS
 "HTML_TEMPLATE_MAIN:miplantilla.html" "
END
BEGIN Action_task task1
 Class = "M4setval.setcontextval"
END
BEGIN Action_task task2
 Class = "M4throw.CMFThrowProcess"
 Onok = "actUnload"
 Oncancel = "actUnload"
 BEGIN Evnotify END_OK
 BEGIN Action_notify notif1
 Idevent = "END_OK"
 END
 END
 BEGIN Evnotify END_CANCEL
 BEGIN Action_notify notif2
 Idevent = "END_CANCEL"
 END
 END
END
BEGIN Action_call actUnload
 Sentence = "*O*/Pres2/MiForm.Unload(END_OK) "
END
END
```

---



---

# Parametrización del sistema de publicación en cliente HTML

# 7

## Acerca de este capítulo

En este capítulo se describen las principales características de la publicación de documentos en un entorno HTML o Web y cómo parametrizar dicha publicación para incluir estos documentos en intranets u otros entornos Web para usuarios de determinados perfiles.

El enfoque de este capítulo está en la parametrización de la publicación de documentos o informes para su visualización en un entorno Web. No se entrará en mucho detalle en el proceso de generación de los documentos en sí. En este caso, consulte las ayudas en línea para más información acerca de ello.

## Introducción

El sistema de publicación de documentos permite dejar accesibles documentos para su posterior visualización en un entorno de cliente ligero HTML. Este proceso de publicación establece una seguridad para el acceso a los documentos.

El sistema permitirá obtener la lista de documentos disponibles por usuario, así como realizar ciertas tareas de administración sobre el repositorio. Finalmente permitirá recuperar de forma segura los documentos para visualizarlos, al menos, desde un entorno de cliente ligero HTML

## Procedimiento general

El sistema de publicación de documentos se puede dividir en:

1. Publicación de los documentos en el entorno Windows.
2. Acceso al repositorio de documentos desde el entorno cliente ligero HTML para su visualización, listado de documentos disponibles o borrado de los mismos. Para ello se suministran una serie de páginas JSP que se puede utilizar como plantillas para la parametrización posterior.

## Publicación de documentos

El proceso de publicar documentos consiste en dos fases. La preparación de los documentos a preparar seguido por la ejecución del informe resultante para su publicación en sí.

### Preparación de documentos a publicar

El proceso de publicación está integrado en la ejecución de un informe. Un informe puede generar uno o varios documentos cada uno de los cuales puede estar compuesto por uno o varios archivos. El sistema podrá, en el caso de generar varios documentos, publicarlos cada uno por separado estableciendo diferentes.

Para poder publicar un documento generado tras la ejecución de un informe es necesario prepara de forma adecuada el Meta4Object asociado al mismo.

Previo a la ejecución de un informe, bien en línea o planificado, se podrá indicar si se desea publicar. Este proceso sólo se puede realizar desde un cliente Windows.



---

## Ejecución del informe para la publicación

El proceso de publicación en sí, al estar ligado a la ejecución del informe se realizará en el entorno de su ejecución correspondiente (Windows si es ejecución cliente o la plataforma del servidor de aplicaciones en la ejecución en servidor o planificada).

## Parámetros de publicación

Los parámetros de publicación se pueden establecer en el mismo momento en el que se pide la publicación o bien parametrizarlos para que se obtengan de forma dinámica durante la ejecución del informe, es decir, durante la generación del documento. De esta forma se podrán asociar estos parámetros de publicación a los datos del documento.



Consulte las ayudas en línea para más información acerca de la forma de parametrizar en estos dos modos.

Los parámetros de publicación son:

- Identificador de informe del que proviene el documento
- Clasificación: campo libre de 254 caracteres que se puede usar para guardar información que identifique o clasifique el documento en el repositorio. Existen métodos que permiten recuperar los documentos según los valores de este campo.
- Subclasificación: otro campo exactamente igual que el anterior.
- Fecha de borrado: Fecha GMT en la cual se procederá al borrado automático del documento en el repositorio.
- Seguridad para leer y borrar

Además en el repositorio se almacena por cada documento:

- Usuario y rol que publica
- Fecha y hora de publicación

### Seguridad para leer y borrar

La seguridad definida sobre un documento en el repositorio constará de dos tipos de permisos: lectura y borrado. No existen permisos de publicación. Los permisos de borrado se podrán establecer inicialmente al mismo usuario que realiza la publicación, es decir, el que indica qué se va a publicar.

Por cada documento la seguridad se establecerá de la siguiente forma: sólo podrán acceder a los documentos para visualizarlos o borrarlos los usuarios cuyo identificador de usuario o rol de conexión estén en la lista establecida al publicar el documento.

Un documento en el repositorio podrá no tener seguridad de forma que estará accesible para todos los usuarios.

En el caso de que una ejecución genere varios documentos se podrá establecer la seguridad de forma independiente para cada uno de ellos.

Un usuario con el MSR de ejecución de administrador podrá acceder a cualquier documento del repositorio para su lectura o borrado.

El sistema respecto a la publicación tiene las siguientes restricciones:

- Sólo se podrán publicar informes ejecutados a los formatos HTML y PDF.
- No se podrán realizar publicaciones desde la herramienta Consulta.
- Si el informe no trae datos, es decir, no hay registros en el nodo de la sección principal o todos se anulan por algún filtro en el informe no se realiza la publicación aunque se esté indicando.

Los informes que se van a publicar deberán estar previamente grabados. Si no lo estuvieran y se intentaran publicar el sistema informaría con un mensaje de error.

## Disponibilidad de documentos en el entorno HTML

En un entorno cliente ligero HTML el sistema permitirá obtener listados de documentos disponibles para lectura o borrado para el usuario de la conexión.

El usuario podrá filtrar la lista cumplimentado una condición por uno de los siguientes campos: identificador de informe, la clasificación o la subclasificación.

Al menos el sistema permitirá mostrar como resultado de la consulta los siguientes campos, además de los tres anteriores:

- Nombre del documento (si no se especifica en los parámetros de publicación, el informe lo creará a partir del ID del Informe correspondiente y la fecha y hora de la ejecución del mismo)
- Fecha de publicación

Además suministrará el identificador de documento para poder invocar su visualización o borrado siempre que se tenga el correspondiente permiso.

El sistema permitirá borrar de forma automática los documentos si a la hora de publicar se especificó la fecha de borrado y se ha planificado de forma adecuada la tarea correspondiente.



Consulte el apartado Parametrización del acceso a la publicación en HTML, para más información acerca de dicha tarea.

## Parametrización de la visualización y borrado de documentos en el entorno HTML

En el entorno HTML, desde el listado de documentos disponibles se permite borrar y visualizar documentos. En las páginas tecnológicas que se suministran, para visualizar o borrar se invocan a ciertas páginas que reciben como parámetros el identificador del documento y realizan la tarea correspondiente. Estas páginas se pueden usar como plantillas para una parametrización mayor.



Consulte el apartado Parametrización del acceso a la publicación en HTML, para más información.

Al personalizar estas páginas también puede implicar la personalización de operaciones a efectuar. El sistema ofrece un API a través de métodos de Meta4Object para poder recuperar los documentos para visualizar directamente a través ciertas condiciones de filtrado.

Para su visualización los documentos existen dos mecanismos posibles:

- transferirlos temporalmente al servidor Web a directorios independientes por documento. El sistema borrará todos los archivos transferidos una vez acabada la sesión HTML.
- cargar directamente los documentos en memoria para su visualización. Esta opción está restringida a documentos formados por un único archivo. Al no existir archivos transferidos temporalmente en el servidor web representa un nivel de seguridad más alto que la opción anterior.

Para que la transferencia de archivos entre el servidor de aplicaciones y el servidor Web sea efectiva se tiene que configurar un recurso, un directorio compartido entre ambos según se indica en el guía de *Instalación* y en el apartado Configuración del sistema de e-reporting.

## Gestión de informes publicados en distintos entornos de empresa

La gestión de informes publicados está estrechamente relacionada con la funcionalidad de multisociedad, que implica un nivel de seguridad distinto al de

los roles de aplicación o los usuarios, a pesar de que la asignación de roles o usuarios no implica el uso de la funcionalidad de multisociedad.

El sistema sólo permite el acceso a los informes publicados si:

- No dependen de la empresa.
- Han sido publicados para la misma empresa u organización que se usa como empresa base para la sesión en la que se recuperan.

### **Informes publicados en un entorno de monosociedad**

En los entornos monosociedad, el sistema no almacena información adicional sobre la empresa. Si se publica un informe sin seguridad, estará disponible para todos los usuarios.

### **Informes publicados en un entorno multisociedad**

- Los informes monosociedad son los que utilizan un Meta4Object monosociedad y por lo tanto sólo se ejecutan en el marco de una sociedad determinada.
- Los informes multisociedad son los que utilizan un Meta4Object de datos multisociedad y por lo tanto no sólo se ejecutan para una empresa determinada, sino para todos los datos del Meta4Object, que pueden corresponder a una o más sociedades.

### **Informe monosociedad publicado sin seguridad sobre documentos**

El sistema almacena la sociedad base junto con el documento.

### **Informe monosociedad publicado con seguridad sobre documentos**

El sistema almacena la sociedad base con el documento y muestra dos listas:

- roles de aplicación asociados con esa sociedad
- usuarios, aunque éstos pueden tener roles que no estén asociados con la empresa

El sistema almacena la lista de roles y la de usuarios por separado.

### **Informe multisociedad publicado sin seguridad sobre documentos**

El sistema almacena el documento junto con las empresas asociadas al rol de la empresa base en el momento de la publicación, que son las empresas disponibles para el documento.

### **Informe multisociedad publicado con seguridad sobre documentos**

El sistema almacena el documento junto con las empresas asociadas al rol base en el momento de la publicación, además de las listas de seguridad sobre documentos definidas para el documento publicado.

---

### **Lectura y borrado de documentos publicados**

Todos los documentos, monosociedad o multisociedad, con o sin seguridad, se pueden recuperar y borrar siempre si el RSM del usuario de la sesión es el del administrador.

Un documento sin seguridad sólo se puede leer o borrar si todas las sociedades para las que está disponible ese documento se encuentran entre las disponibles para el rol que accede a la sesión (si se trata de un informe monosociedad, ésta será la única sociedad disponible).

Un documento con seguridad sólo se puede leer o borrar si el rol o usuario de la sesión están incluidos en las listas de seguridad y si todas las sociedades para las que está disponible el documento están disponibles para el rol que accede a la sesión.



No es necesario que la empresa base sea la empresa definida para la sesión o la aplicación, puesto que se puede modificar desde las presentaciones o al ejecutar un informe (sólo si es monosociedad).

## Parametrización de la publicación de documentos

### Parametrización del Meta4Object de datos

Desde el entorno Windows de la aplicación Meta4, para publicar un informe al que puedan tener acceso los usuarios finales desde el cliente web, debe hacer lo siguiente:

1. Adapte el Meta4Object asociado al informe cuyo resultados se desea publicar. El Meta4Object debe tener un nodo libre nuevo que utilice una estructura de nodo específica.
2. Cree un nuevo nodo libre cuyo identificador sea PUB\_DOC\_IMPLEMENTES. Este código identificativo es obligatorio.
3. Herede una estructura de nodo llamada EN\_PUB\_DOC\_IMPLEMENTES para este nodo. Esta estructura de nodo le aportará al Meta4Object de datos todos los métodos que necesita para la publicación.
4. Si desea que los parámetros se asocien de forma dinámica durante la ejecución del informe deberá sobrescribir las reglas de los métodos de la estructura de nodo creado en el paso anterior.
5. Ejecute el informe y defina los parámetros por defecto para publicar el informe:
  - Nombre del documento
  - Clasificación y Subclasificación
  - La fecha de borrado
  - La seguridad sobre los documentos: permisos de lectura y de borrado según roles y usuarios.

En cuanto finaliza este proceso, el sistema publica los resultados en el repositorio de documentos para que estén accesibles desde el entorno cliente ligero HTML.

---

## Nodo libre PUB\_DOC\_IMPLEMENTES

Este nodo contiene todos los métodos que el motor de informes va a invocar en el proceso de publicación.

- NEW\_DOC
- SAVE\_FILE
- END\_DOC

A su vez y a través de nivel 2 invocará a los métodos del Meta4Object DOCUMENT\_REPOSITORY que hace las funciones de repositorio de documentos.

También en este nodo están los métodos que se invocan para recuperar de forma dinámica, es decir, durante la ejecución del informe, los parámetros de publicación.

El motor de los informes invoca el método NEW\_DOC que a su vez invoca estos métodos que recuperan los parámetros de publicación.

Por defecto, estos métodos devuelven el valor almacenado en la propiedad correspondiente. Dichas propiedades han sido rellenas de forma apropiada por el sistema con los valores introducidos en el momento en el que se pidió la publicación. Para su correcto funcionamiento C/S todos tendrán que tener la propiedad avanzada ID Tipo C/S a *Parte Completa*.

Los métodos son:

- GET\_N\_DOCUMENT: devuelve el nombre de documento.
- GET\_CLASIF: devuelve la clasificación.
- GET\_SUBCLASIF: devuelve la subclasificación.
- GET\_EXPIRATION\_DT: devuelve la fecha de borrado.
- GET\_ROLE\_READ\_LIST: devuelve una lista de los roles por sus identificadores, separados por comas.
- GET\_USER\_READ\_LIST: devuelve una lista de los usuarios por sus identificadores, separados por comas.
- GET\_ROLE\_DEL\_LIST: devuelve una lista de los roles con permiso de borrado, separados por comas.
- GET\_USER\_DEL\_LIST: devuelve una lista de los usuarios con permiso de borrado, separados por comas.
- GET\_ORGANIZATION\_LIST: devuelve una lista de los ID de Sociedades separados por comas.

En caso de que se quiere que alguno de los parámetros estuviera asociado a los datos o dependiera de ellos, bastaría con sobrescribir el código de la regla

correspondiente.

Esto es especialmente útil si se desea aplicar una seguridad distinta para cada documento generado en la ejecución. Para ello habría que sobrescribir las reglas en los métodos que devuelven valores para roles, usuarios y sociedades.

Si el informe tiene ruptura, el motor de informes estará en este momento posicionado en el registro correspondiente al grupo que se está procesando que permite la creación de documentos compuestos.

---

### EJEMPLO

En un informe que lista personas agrupadas por sexo, y con una ruptura establecida por sexo. En la ejecución, el motor invocará dos veces al método NEW\_DOC, para cada tipo de sexo.

En este caso se crearán dos documentos y los dos serán publicados en el repositorio de documentos. Por defecto, ambos documentos tendrán los mismos valores, por ejemplo, valores de clasificación, subclasificación, entre otros.

Para establecer como categoría algo que indique si son hombres o mujeres, la regla correspondiente se sobrescribe de la siguiente manera:

```
str = ""
If TST_PUB_DOC_STD_PERSON.STD_ID_GENDER = "F" Then
 str = "Mujeres"
Else
 str = "Hombres"
Endif

return "Clasificación para " + str
```

---

## Parametrización del listado, visualización y borrado de documentos

Todas estas funcionalidades se ofrecen a través de páginas JSP e invocaciones a métodos del Meta4Object que representa al repositorio de documentos.

A continuación se describe los métodos más importantes del Meta4Object que se podrá parametrizar según las necesidades.



---

## Meta4Object DOCUMENT\_REPOSITORY

A parte de realizar las tareas de publicación este Meta4Object ofrece los mecanismos de recuperación con seguridad y transferencia de documentos del servidor de aplicaciones al servidor web para la visualización de los mismos en un navegador.

Este Meta4Object tiene un nodo API\_DOCUMENT\_REPOSITORY con los métodos API que ofrece para esta parametrización en concreto:

### ■ API\_FILTER\_DOCS

Carga en el nodo SRP\_REPORT\_LIST documentos según el filtro que indican los argumentos.

```
'
' Parámetros:
' [IN] ARG_TYPE_QUERY: Valores para la carga
' 0 ----> Carga los documentos con Permisos de lectura
' 1 ----> Carga los documentos con Permisos de Borrado
' 2 ----> Carga los documentos con Per. de lect. y Borr.
' [IN] ARG_FIELD_1: campo por el que se quiere filtrar
' CLASIFFICATION, SUBCLASIFFICATION, ID_REPORT
' [IN] ARG_OPERATOR_1: 1 start, 2, end, 3 cont, ...
' [IN] ARG_VALUE_1:
'
' Return : M4_TRUE/M4_FALSE
'
```

### ■ API\_GET\_DOC

Recupera el documento pasado por argumento si tenemos permisos de lectura sobre el mismo.

El documento se crea en un directorio único dentro del temporal y por sesión: temp/M4ADM\_5132122/DOC\_200207051715/SSP-RECIBO\_052002.htm

La uri al documento se deja en el elemento PROP\_HTTP\_VIEW\_FILE.

En PROP\_HTTP\_VIEW\_ERROR (0 - sin error, 1 - No permiso y 2 - problemas al transferir el archivo)

```
'
' Parámetros:
'[IN] ARG_VIEW_TYPE: 0/1 visualización Http
'[IN] ARG_PATH_TEMP_MAPPING: mapping del appserver al temporal
' del web server + directorio único de la sesión
'[IN] ARG_ID_DOCUMENT:
'[IN] ARG_USER_TEMP_URI: si ARG_VIEW_TYPE = 1, no terminada en
' /
'
' Return : M4_TRUE/M4_FALSE
'
```

#### ■ API\_GET\_DOC\_SAFE

Recupera el documento pasado por argumento si tenemos permisos de lectura sobre el mismo.

Si el documento tiene un sólo archivo se recupera a través de un elemento tipo BLOB para evitar la transferencia de los archivos al temporal.

Si el documento tiene más de un archivo se crea en un directorio único dentro del temporal y por sesión:

temp/M4ADM\_5132122/DOC\_200207051715/SSPRECIBO\_052002.htm

La uri al documento se deja en el elemento PROP\_HTTP\_VIEW\_FILE.

En PROP\_HTTP\_VIEW\_ERROR (0 - sin error, 1 - No permiso y 2 - problemas al transferir el archivo).

```
'
' Parámetros:
'[IN] ARG_VIEW_TYPE: 0/1 visualización Http
'[IN] ARG_PATH_TEMP_MAPPING: mapping del appserver al temporal
' del web server + directorio único de la sesión
'[IN] ARG_ID_DOCUMENT:
'[IN] ARG_USER_TEMP_URI: si ARG_VIEW_TYPE = 1, no terminada en
' /
'[IN] ARG_TASK: tarea cliente ligero
'[IN] ARG_M4O_ALIAS: alias al Meta4Object usado en la jsp
'
' Return : M4_TRUE/M4_FALSE
'
```

---

## ■ API\_DELETE\_DOC

Borra un documento del repositorio comprobando si el usuario actual de la sesión tiene permisos para ello.

```
'
' Parámetros:
' [IN] ARG_ID_DOCUMENT: Id del Documento a borrar
'
' Return : M4_TRUE/M4_FALSE
'
```

## ■ API\_GET\_QUERY\_DOC

Recupera el documento que coincide con los valores pasado por parámetros y sólo debe devolver un documento, en caso contrario en el elemento PROP\_HTTP\_VIEW\_ERROR se deja un 3.

```
'
' Parámetros:
' [IN] ARG_OPERATOR_1: tipo de operador para filtrar por el
' campo ID_REPORT
' [IN] ARG_VALUE_1: valor para filtra por ID_REPORT
' [IN] ARG_OPERATOR_2: tipo de operador para filtrar por el
' campo CLASIFFICATION (1, 2, 3, 4 Y 5)
' [IN] ARG_VALUE_2: valor para filtra por CLASIFFICATION
' [IN] ARG_OPERATOR_3: tipo de operador para filtrar por el
' campo SUBCLASIFFICATION
' [IN] ARG_VALUE_3: valor para filtra por SUBCLASIFFICATION
' [IN] ARG_PATH_TEMP_MAPPING: mapping del appserver al temporal
' del web server + directorio único de la sesión
' [IN] ARG_USER_TEMP_URI: no terminada en /
'
' Return : M4_TRUE/M4_FALSE
'
```

## ■ API\_GET\_QUERY\_DOC\_SAFE

Recupera el documento que coincide con los valores pasado por parámetros y sólo debe devolver un documento, en caso contrario en el elemento PROP\_HTTP\_VIEW\_ERROR se deja un 3.

Utiliza el método API\_GET\_DOC\_SAFE para recuperar de forma más segura los documentos compuestos por un archivo

```
'
' Parámetros:
'[IN] ARG_OPERATOR_1: tipo de operador para filtrar por el
' campo ID_REPORT
'[IN] ARG_VALUE_1: valor para filtra por ID_REPORT
'[IN] ARG_OPERATOR_2: tipo de operador para filtrar por el
' campo CLASIFFICATION (1, 2, 3, 4 Y 5)
'[IN] ARG_VALUE_2: valor para filtra por CLASIFFICATION
'[IN] ARG_OPERATOR_3: tipo de operador para filtrar por el
' campo SUBCLASIFFICATION
'[IN] ARG_VALUE_3: valor para filtra por SUBCLASIFFICATION
'[IN] ARG_PATH_TEMP_MAPPING: mapping del appserver al temporal
' del web server + directorio único de la sesión
'[IN] ARG_USER_TEMP_URI: no terminada en /
'
'[IN] ARG_TASK: tarea cliente ligero
'[IN] ARG_M4O_ALIAS: alias al Meta4Object usado en la jsp
'
' Return : M4_TRUE/M4_FALSE
'
```

---

## Parametrización del acceso a la publicación en HTML

Una vez que estén disponibles los documentos en el repositorio, se puede parametrizar la forma de acceder a ellos desde el cliente ligero. Para ello, se utiliza dos tipos de tareas y también se puede utilizar una serie de páginas tecnológicas como punto de partida de una parametrización.

### Tarea TCPUBDOCS

La tarea de cliente ligero asociada al listado, visualización y borrado de documentos publicados es TCPUBDCOS. A nivel de servicios web las páginas JSP que ofrecen la funcionalidad estén bajo el directorio con el mismo nombre (tcpubdocs).

### Páginas JSP suministradas

Para tener accesible las páginas tecnológicas se debe invocar a la página de listado de documentos publicados desde un entorno securizado con la siguiente ruta:

```
servlet/CheckSecurity/JSP/tcpubdocs/doc_rep_filter_doc.jsp
```

Compruebe que la tarea **tcpubdocs** tiene la seguridad adecuada para que sea accesible por los usuarios que van a utilizar esta funcionalidad.

### Listado de documentos publicados con permiso de lectura o borrado

Las páginas suministradas que se usa para el listado de documentos publicados son:

- **doc\_rep\_filter\_doc.jsp**: Página con los datos traducidos que se utilizarán en la página que muestra el listado (doc\_rep\_filter\_docpage.jsp). Va en los distintos idiomas de ahí que vaya bajo la carpeta del idioma (english/espanol).
- **doc\_rep\_filter\_docjob.jsp**: Página dónde se ejecutan las distintas transacciones con el servidor, para traer la información. Aquí se invocan a los métodos API\_FILTER\_DOCS y API\_DELETE\_DOC. También calcula el PATH\_TEMP\_MAPPING y el TEMP\_URI, o localización del recurso compartido y cadena para devolver como url un documento.

**EJEMPLO**

```

<m4:beginjob/>
 <m4:datadef m4o="DOCUMENT_REPOSITORY" m4name="DocumentRepository"
 />

 <% if (sIdDocToDelete != null) { %>
 <m4:exec
 m4method="DEL_PROCESS:DocumentRepository!API_DOCUMENT_REPOSITORY.A
 PI_DELETE_DOC">
 <m4:param name="ARG_ID_DOCUMENT" value="<%=sIdDocToDelete%>" />
 </m4:exec>

 <% } %>

 <!-- Si no hay filtro trae todos --%>
 <m4:exec
 m4method="CARGA:DocumentRepository!API_DOCUMENT_REPOSITORY.API_FIL
 TER_DOCS">
 <m4:param name="ARG_TYPE_QUERY" value="2" />
 <m4:param name="ARG_FIELD_1" value="<%=sFilterItem%>" />
 <m4:param name="ARG_OPERATOR_1" value="<%=sFilterOperator%>" />
 <m4:param name="ARG_VALUE_1" value="<%=sFilterValue%>" />
 </m4:exec>

 <% if (sSortItem != null) { %>
 <m4:sortitems
 m4name="DocumentRepository!SRP_DOCUMENT_LIST.ordena">
 <m4:param name="<%=sSortItem%>" value="<%=sSortType%>" />
 </m4:sortitems>
 <% } %>

 <m4:outputdef m4alias="DataDocList" >
 <m4:param name="M4NAME0"
 value="DocumentRepository!SRP_DOCUMENT_LIST[*]" />
 </m4:outputdef>

 <m4:endjob/>

```

---

Para indicar que se recuperen todos los documentos con permisos de lectura y de borrado, se usa el argumento:

```
<m4:param name="ARG_TYPE_QUERY" value="2" />
```

- **doc\_rep\_filter\_docpage.jsp**: Página con el código HTML que lista los documentos publicados. Mostrará todos los documentos con permisos de lectura y de borrado permitiendo visualizarlos y borrarlos. Si se intenta visualizar uno sobre el que no se tiene permiso el sistema informará a través de un mensaje en la página HTML.

## Visualización de documentos

Las páginas suministradas que se usan para la visualización de documentos son:

- **doc\_rep\_get\_doc.jsp**: Página con los datos traducidos que se utilizarán si hay algún error de visualización.
- **doc\_rep\_get\_docjob.jsp**: Página con las distintas transacciones con el servidor para traer realizar la transferencias de documentos. Se invoca al método API\_GET\_DOC.

---

### EJEMPLO

```
<m4:beginjob/>

<m4:datadef m4o="DOCUMENT_REPOSITORY" m4name="DocumentRepository"
/>

<%-- Si no hay filtro trae todos --%>
<m4:exec
m4method="DESCARGA:DocumentRepository!API_DOCUMENT_REPOSITORY.A
PI_GET_DOC">
 <m4:param name="ARG_VIEW_TYPE" value="1" />
 <m4:param name="ARG_PATH_TEMP_MAPPING"
value="<%=zpathtempmapping%" />
 <m4:param name="ARG_ID_DOCUMENT" value="<%=ziddocument%" />
 <m4:param name="ARG_USER_TEMP_URI" value="<%=zusertempuri%"
/>
</m4:exec>
```

```

<m4:outputdef m4alias="DataDocList" >
 <m4:param name="M4NAME0"
 value="DocumentRepository!API_DOCUMENT_REPOSITORY[*]" />
</m4:outputdef>
<m4:endjob/>

```

- **doc\_rep\_get\_docpage.jsp**: Página que gestiona el resultado de la transferencia del documento.

## Método alternativo para la visualización de documentos

Como se ha descrito antes, la página **doc\_rep\_get\_doc.jsp** necesita un identificador de documento a visualizar. La página de listado **doc\_rep\_filter\_doc.jsp** invoca a **doc\_rep\_get\_doc.jsp** suministrándole el identificador de documento. Si desea visualizar un documento sin saber su identificador, existe un método alternativo. Consiste en usar el método **API\_GET\_QUERY\_DOC** que devuelve un documento pasándole como argumento un criterio sobre los atributos informe, clasificación y subclasificación. Si ha usado los dos últimos para guardar información que sabe que identifican de forma única al documento dentro del repositorio puede usar este método para recuperar el documento en lugar de usar **API\_GET\_DOC** que le obliga a conocer el identificador de documento.

Para usarlo puede hacer una copia de las 3 páginas **doc\_rep\_get\_doc.jsp** y en la página donde se hacen las transacciones (**doc\_rep\_get\_docjob.jsp**) poner la llamada al nuevo método.

### EJEMPLO

```

<m4:beginjob/>

<m4:datadef m4o="DOCUMENT_REPOSITORY" m4name="DocumentRepository"
/>

<%-- Si no hay filtro trae todos --%>
<m4:exec
m4method="DocumentRepository!API_DOCUMENT_REPOSITORY.API_GET_QUERY_DOC">

 <m4:param name="ARG_OPERATOR_1" value="1" />
 <m4:param name="ARG_VALUE_1" value="SQ6_PUB_PLAN" />

 <m4:param name="ARG_OPERATOR_2" value="1" />

```



---

```

<m4:param name="ARG_VALUE_2" value="014" />

<m4:param name="ARG_OPERATOR_3" value="1" />
<m4:param name="ARG_VALUE_3" value="014" />

<m4:param name="ARG_PATH_TEMP_MAPPING"
value="<%=zpathtempmapping%" />

<m4:param name="ARG_USER_TEMP_URI" value="<%=zusertempuri%"
/>
</m4:exec>

<m4:outputdef m4alias="DataDocList" >
<m4:param name="M4NAME0"
value="DocumentRepository!API_DOCUMENT_REPOSITORY[*]"/>
</m4:outputdef>

<m4:endjob/>

```

---

En el ejemplo se puede ver como se usan los 6 primeros argumentos para pasar tipo de operador (ARG\_OPERATOR\_1, ARG\_OPERATOR\_2 y ARG\_OPERATOR\_3) con el valor 1, que indica "Empieza por ..." y los valores sobre los que hacer el criterio (ARG\_VALUE\_1 sobre ID de Informe, ARG\_VALUE\_2 sobre clasificación y ARG\_VALUE\_3 sobre subclasificación.

Para información más detallada puede consultar el método API\_GET\_QUERY\_DOC.

## Modificación de las páginas JSP

La funcionalidad que se trata lleva una serie de páginas que se han descrito en el apartado anterior y que permiten ser modificadas a nivel de aspecto para que sigan los estándares del producto que las incorpore.

Los dos subprocesos (listado/borrado y visualización) están formados por 3 páginas cada uno para poder separar mejor la parte de comunicación con el servidor y la parte del aspecto de la funcionalidad.

Para tocar el aspecto de uno de los subprocesos se debe modificar en:

1. La página que implementa el aspecto visual. Esta página se identifica fácilmente porque termina en Page.jsp.
2. La página que contiene los literales de traducción. Ésta se encuentra repetida por idioma bajo el directorio que identifica cada idioma.



Todo lo que corresponde con el interfaz de comunicación con el Meta4Object debería mantenerse cómo esta para asegurar el correcto funcionamiento del proceso general.

## Tarea DEL\_EXPIRED\_DOCS

Esta tarea realiza el borrado de documentos de forma automática. Para su correcto funcionamiento, el administrador del sistema debe crear un nuevo grupo de tareas con esta tarea para este fin. Los pasos a seguir son:

1. Seleccione la opción de menú **Herramientas de usuario | Planificador de tareas | Gestionar grupos de tareas**.
2. Cree un nuevo grupo de tareas.
3. En la pestaña Reglas de tiempo seleccione el modo cíclico e indique el intervalo de borrado y la fecha de inicio y fin del borrado cíclico automático.
4. Haga clic en el botón **Siguiente**.
5. En la pestaña Tarea seleccione la tarea llamado DEL\_EXPIRED\_DOCS.
6. Haga clic en el botón **Siguiente**.
7. En la pestaña Parámetros, haga clic en el botón **Siguiente**.
8. En la pestaña Contexto de sesión, complete la parametrización antes de hacer clic en el botón **Finalizar**.

Es importante que esta tarea la realice un administrador porque el proceso de borrado no controla permisos de borrado, únicamente borra los documentos con fecha de borrado menor o igual a la fecha de ejecución de la tarea DEL\_EXPIRED\_DOCS.

---

# Integración con el gestor documental

# 8

## Acerca de este capítulo

En este capítulo se describen las herramientas para integración de la gestión de documentos del gestor documental en un entorno HTML o Web y cómo parametrizar dichas herramientas.

Dicha integración se basa en la ejecución de un include tecnológico y la invocación de una tarea, que se describen detalladamente en los siguientes apartados.

## Introducción

Las opciones de integración con el gestor documental en un entorno HTML o Web incluyen la ejecución de un include con cuatro botones y la llamada a una tarea para clientes Web HTML, tales como el Self-Service del Empleado/Manager (SSE/SSM). Ambos objetos permiten administrar fácilmente los documentos del gestor documental. Así, la tarea ofrece la posibilidad de ver un documento, subirlo al gestor documental y enviarlo a firmar. Por su parte, el include consta de los siguientes elementos:

1. Una etiqueta que se corresponde con el nombre del elemento, que a su vez se corresponde con el identificador del documento.
2. Una caja de texto que va a contener el título y el tipo del documento.
3. Cuatro botones, que permiten realizar las siguientes tareas:
  - Adjuntar un documento.
  - Visualizar documentos.
  - Eliminar documentos.
  - Consultar los datos relativos a la firma de un documento.

El include puede configurarse de forma que se muestre en dos formatos diferentes:

1. Modo formulario
2. Modo tabla

## Integración del include para gestión documental en el SSE/SSM

Para llevar a cabo la integración, en primer lugar es necesario añadir a la página funcional, tanto en modo formulario como en modo tabla, un include de inicialización. Este include va a contener la declaración de todas las variables necesarias para la correcta ejecución del include principal. El include de inicialización debe aparecer una única vez por página:

### – `\tc_docs\tc_doc_initialize_include.jsp` (include de inicialización)

En el include de inicialización están declaradas las siguientes variables:

- **`sgtc_zIDInputIDDOC`**: variable para contener el identificador del campo que corresponde al identificador del documento, dentro del registro funcional al que se quiera añadir el documento.

Por defecto: `String sgtc_zIDInputIDDOC = "SCO_ID_DOC";`

- **`sgtc_zIDInputTITLEDOC`**: variable para contener el identificador del campo que corresponde al título del documento, dentro del registro funcional al que se quiera añadir el documento.

Por defecto: `String sgtc_zIDInputTITLEDOC = "SCO_TITLE_DOC";`

Hay que reseñar que, en modo tabla, es necesario que el `Meta4Object` funcional esté provisto de un elemento que contenga para cada registro el título del documento. Este elemento se rellenará en la carga del `Meta4Object` mediante una llamada de nivel 2:

```
SCO_TITLE_DOC =
SRTC_MANAGE_DOCUMENT!SCO_MANAGE_DOCUMENT.SCO_GET_TITLE_DOCUMENT
(SCO_ID_DOC)
```

Esto es necesario únicamente para trabajar en modo tabla, no en modo formulario.

- **`sgtc_zIDDOC`**: variable para contener el valor del identificador del documento, cuando no haya un documento asociado permanecerá vacía, y cuando haya un documento asociado contendrá el identificador del documento del registro actual.

Por defecto: `String sgtc_zIDDOC = "";`

- **`sgtc_zTITLEDOC`**: variable para contener el valor del título del documento, cuando no haya un documento asociado permanecerá vacía, y cuando haya un documento asociado contendrá el título del documento del registro actual.

Por defecto: `String sgtc_zTITLEDOC = "";`

- **sgtc\_zsubsessionsave:** variable para contener la sesión donde se graba el formulario y se sube el documento, esta variable si permanece vacía el documento se subirá al gestor documental al adjuntarlo, si se rellena con un nombre de sesión, el documento se subirá al gestor al grabar el formulario funcional que contiene el include principal.

Por defecto: `String sgtc_zsubsessionsave = "";`

- **sgtc\_zShowMode:** Para modo formulario se establecerá valor 0, y para modo tabla se establecerá valor 1 (0:modo formulario 1:modo tabla)

Por defecto: `String sgtc_zShowMode = "0";`

- **sgtc\_zReadWrite:** Para modo readonly se establecerá valor 0, y para modo readwrite se establecerá valor 1 (0:modo readonly 1:modo readwrite)

Por defecto: `String sgtc_zReadWrite = "1";`

- **sgtc\_zstylesheet:** variable para contener la página de estilo a utilizar, dependiendo del producto al que se añada el include, la página de estilo será una u otra.

Por defecto: `String sgtc_zstylesheet = "/css/estilo_sse.css";`

- **sgtc\_zNMInputIDDOC:** variable para generar los nombres de los input, por ejemplo: si asignamos a esta variable el literal "SCO\_ID\_DOC". Para modo formulario se generarán los siguientes inputs:

```
"SCO_ID_DOC_ATTACH"
"SCO_ID_DOC_VIEW"
"SCO_ID_DOC_DELETE"
"SCO_ID_DOC_INFO"
"SCO_ID_DOC_TITLE"
```

Y para modo tabla los siguientes:

```
"SCO_ID_DOC_0_ATTACH"
"SCO_ID_DOC_0_VIEW"
"SCO_ID_DOC_0_DELETE"
"SCO_ID_DOC_0_INFO"
"SCO_ID_DOC_0_TITLE"
```

```
"SCO_ID_DOC_1_ATTACH"
"SCO_ID_DOC_1_VIEW"
"SCO_ID_DOC_1_DELETE"
"SCO_ID_DOC_1_INFO"
"SCO_ID_DOC_1_TITLE"
```

Y así sucesivamente para cada una de las filas de la tabla.

Por defecto: `String sgtc_zNMInputIDDOC = "SCO_ID_DOC";`

- **sgtc\_zIDCSSRow**: variable para contener el identificador de estilo para la fila a pintar, en modo formulario se corresponde con el nombre de la clase de estilo que se le da al input del include, y para modo tabla, se corresponde con el nombre de la clase de estilo que se le da a cada una de las filas de la tabla, de esta manera cada vez que se pinte un include dentro de una fila se indicará el estilo de la fila, en muchas tablas las filas que las componen tienen diferentes estilos.

Por defecto: String sgtc\_zIDCSSRow = "fuenteformulario";

A continuación se muestra un ejemplo de declaración de las variables:

### EJEMPLO

```
<%
//variable para contener el identificador del campo que corresponde
al identificador del documento
String sgtc_zIDInputIDDOC = "SCO_ID_DOC";

//variable para contener el identificador del campo que corresponde
al título del documento
String sgtc_zIDInputTITLEDOC = "SCO_TITLE_DOC";

//variable para contener el valor del identificador del documento
String sgtc_zIDDOC = "";

//variable para contener el valor del título del documento
String sgtc_zTITLEDOC = "";

//variable para contener la sesión donde se graba el formulario
funcional que contiene el include
String sgtc_zsubsessionsave = "";

//0:modo formulario 1:modo tabla
String sgtc_zShowMode = "0";

//0:modo readonly 1:modo readwrite
String sgtc_zReadWrite = "1";

//variable para contener la página de estilo a utilizar
String sgtc_zstylesheet = "/css/estilo_sse.css";

//variable para generar los nombres de los input, por ejemplo:
SCO_ID_DOC_1,SCO_ID_DOC_2,...
```

```

String sgtc_zNMInputIDDOC = "SCO_ID_DOC";

//variable para contener el identificador del estilo para la fila a
pintar (solo para modo tabla)
String sgtc_zIDCSSRow = "fuenteformulario";

String ztcsubsesionmanage = "SRTC_MANAGE_DOCUMENT";
String ztcmeta4objectmanage = "SRTC_MANAGE_DOCUMENT";
String ztcnodemanage = "SCO_MANAGE_DOCUMENT";
String ztcmetodomanage = "TITLE:" + ztcmeta4objectmanage + "!" +
ztcnodemanage + ".SCO_GET_TITLE_DOCUMENT";

String sgtc_zText_Title = "";
String sgtc_zButt_Attach = "";
String sgtc_zButt_View = "";
String sgtc_zButt_Delete = "";
String sgtc_zButt_Info = "";

String zlanguser = M4Locale.takeLocale(new
String().valueOf(M4Context.getSession(request).getLanguageID())) .toString();
%>

<%@ include file="/shco_g0/shco_gen_taglib.jsp" %>
<%@ include file="/tc_docs/tc_doc_trans.jsp" %>
<script type="text/javascript" language="Javascript1.5"src="/
translations/m4err_<%=zlanguser%>.js"></script>
<script type="text/javascript" language="Javascript1.2"src="/library/
m4gen.js"></script>
<script type="text/javascript" language="Javascript1.5"src="/library/
m4gen_excep.js"></script>
<script type="text/javascript" language="Javascript1.2"src="/library/
m4doc_include.js"></script>

```

Así, por defecto, el include quedaría configurado para modo formulario, en modo lectura y escritura, y para subir el documento al tiempo que se adjunta.



---

## Configuración del include en modo formulario

Para añadir la funcionalidad de gestión de documentos a cualquier página del producto funcional se dispone de otros dos includes tecnológicos:

1. **\tc\_docs\tc\_doc\_include.jsp**: este include visual contiene la botonera para actuar sobre el gestor documental.

Figura 161. Include para gestión documental integrado en el SSE/SSM en modo formulario

---



2. **\tc\_docs\tc\_doc\_save\_include.jsp**: este include de persistencia se insertará en la página funcional donde se grabe el registro.

---

### EJEMPLO

Por ejemplo, en la página funcional \sse\_g1\ssco\_g1\_p6\_mod.jsp podemos integrar el primer include de la siguiente manera:

```
<%@ include file="../../../tc_docs/tc_doc_initialize_include.jsp" %>

<m4:item m4name="<%=zSCOIDDOC%>" var="sgtc_zIDDOC"
htmlsafe="true"/>
<m4:item m4name="<%=zSCOTITLEDOC%>" var="sgtc_zTITLEDOC"
htmlsafe="true"/>
<input type="hidden" id="<%=sgtc_zNMInputIDDOC%>"
name="<%=sgtc_zNMInputIDDOC%>" value="<%=sgtc_zIDDOC%>"/>

<%@ include file="../../../tc_docs/tc_doc_include.jsp" %>
```

---

Para el correcto funcionamiento de este include es necesario incluir, antes de nada, el include de inicialización de variables:

```
<%@ include file="../../../tc_docs/tc_doc_initialize_include.jsp" %>
```

En este caso no se modificará ninguna variable, ya que se requiere un comportamiento acorde con la declaración por defecto:

```
sgtc_zIDInputIDDOC = "SCO_ID_DOC";
sgtc_zIDInputTITLEDOC = "SCO_TITLE_DOC";
```

```
sgtc_zIDDOC = "";
sgtc_zTITLEDOC = "";
sgtc_zsubsessionsave = "";
sgtc_zShowMode = "0";
sgtc_zReadWrite = "1";
sgtc_zstylesheet = "/css/estilo_sse.css";
sgtc_zNMInputIDDOC = "SCO_ID_DOC";
sgtc_zIDCSSRow = "fuenteformulario";
```

A continuación se deberá rellenar la variable `sgtc_zIDDOC` y `sgtc_zTITLEDOC`.

```
<m4:item m4name="<%=zSCOIDDOC%>" var="sgtc_zIDDOC" htmlspecialchars="true"/>
<m4:item m4name="<%=zSCOTITLEDOC%>" var="sgtc_zTITLEDOC"
htmlspecialchars="true"/>
```

Donde `zSCOIDDOC` contiene el identificador del campo que contiene el id documento para el registro actual, y `zSCOTITLEDOC` contiene el identificador del campo que contiene el título del documento para el registro actual.

Dentro del formulario funcional siempre debe existir un input para contener el identificador del documento, para que así viaje con el formulario cuando éste sea enviado.

```
<input type="hidden" id="<%=sgtc_zNMInputIDDOC%>"
name="<%=sgtc_zNMInputIDDOC%>" value="<%=sgtc_zIDDOC%>"/>
```

Por último, añadimos el include:

```
<%@ include file="../../tc_docs/tc_doc_include.jsp" %>
```

En modo formulario el identificador del documento asociado quedará en la propiedad `value` del input `<%=sgtc_zNMInputIDDOC%>`, y en modo tabla los identificadores de los documentos asociados quedarán en la propiedad `value` de los inputs `<%=sgtc_zNMInputIDDOC%>+"_1"`, `<%=sgtc_zNMInputIDDOC%>+"_2"`,... uno por cada fila de la tabla.

Este include irá situado dentro del formulario de la página funcional.

Por ejemplo, en la página funcional `\mss_g3\espanol\smco_g3_p30_det.jsp`, podremos incluir el primer include de la siguiente manera:

---

## EJEMPLO

```
<%@ include file="../../tc_docs/tc_doc_initialize_include.jsp" %>

<%
//variable para contener la sesión donde se graba el formulario y se
sube el documento (solo para modo formulario)
```

---

```
sgtc_zsubsesionsave = "SSM_GN_INTERVIEW";
%>

<m4:item m4name="<%=zSCOIDDOC%>" var="sgtc_zIDDOC" htmlsafe = "true"/
>
<m4:item m4name="<%=zSCOTITLEDOC%>" var="sgtc_zTITLEDOC"
htmlsafe="true"/>
<input type="hidden" name="SCO_ID_DOC" id="SCO_ID_DOC"
value="<m4:item m4name="<%=zSCOIDDOC%>" htmlsafe = "true"/>"/>

<%@ include file="../../tc_docs/tc_doc_include.jsp" %>
```

---

Para el correcto funcionamiento de este include es necesario seguir los mismos pasos explicados en el ejemplo anterior, pero en este caso se va a rellenar la variable `sgtc_zsubsesionsave` con el nombre de la sesión donde posteriormente se grabará el registro funcional.

Este include irá situado dentro del formulario de la página funcional, y el documento seleccionado se subirá al gestor cuando se grabe el registro actual, este formulario será luego el que se envíe a la página que realiza la persistencia, en nuestro ejemplo:

`\mss_g3\espanol\smco_g3_p30_send.jsp.`

En la página `\mss_g3\espanol\smco_g3_p30_send.jsp`, se añadiría el segundo include `\tc_docs\tc_doc_save_include.jsp`

Para el correcto funcionamiento de este include es necesario declarar con anterioridad la siguiente variable java:

- `sgtc_zNMInputIDDOC`: variable para generar los nombres de los inputs; por ejemplo: si asignamos a esta variable el literal `"SCO_ID_DOC"`:

---

## EJEMPLO

```
<m4:startpage m4task="<%=zsubsesion%>" />

<%
//variable para generar los nombres de los input, por ejemplo:
SCO_ID_DOC
String sgtc_zNMInputIDDOC = "SCO_ID_DOC";
%>

<%@ include file="../../tc_docs/tc_doc_save_include.jsp" %>
```

```

<m4:beginjob/>
<m4:datadef m4o="<%=zmeta4object%>" m4name="<%=zsubsesion%>"/>

<m4:endjob/>
<m4:endpage/>

```

Este include irá situado dentro de <m4:startpage> y antes de <m4:beginjob> que realiza la grabación del registro actual, como se muestra en el ejemplo. Esta subsesión es la que debe ser contenida por la variable java `sgtc_zsubsesionsave`.

El identificador del documento asociado quedará en la variable java `<%=ztcSaveDOCIDC%>`.

En modo formulario se pueden incluir dos o más includes en la misma página, pero deberán tener todos la variable `sgtc_zsubsesionsave` igual a "", es decir, en todos los casos el documento se subirá al gestor cuando se adjunta, o en el caso de que se quiera subir el documento al grabar el formulario, solo uno de los includes podrá estar en modo `readwrite`, mientras el resto de los includes que aparezcan en la página deberán estar en modo `readonly`. Estas son las restricciones que existen a la hora de incluir dos o más includes dentro de una misma página. La variable java `sgtc_zsubsesionsave` es global a toda la página, y por tanto, a todos los includes que contenga.

## Configuración del include en modo tabla

Para añadir la funcionalidad de gestión de documentos, modo tabla, a cualquier página del producto funcional se dispone de un include tecnológico:

`\tc_docs\tc_doc_include.jsp`: Este include contiene la botonera para actuar sobre el gestor documental.

Figura 162. Include para gestión documental integrado en el SSE/SSM en modo tabla

Fecha emisión	Estado	Valido hasta	Título del documento
15-07-2008	Entregado		Título de documento   
15-07-2008	Entregado		Título de documento   
07-05-2009	Entregado		Título de documento   

En modo tabla, el include siempre subirá el documento al gestor cuando se adjunta el documento; de todas formas, el formato tabla está diseñado principalmente para modo lectura.

---

Por ejemplo, en la página funcional \sse\_g1\sco\_g1\_p6.jsp, podremos incluir este include de la siguiente manera:

### EJEMPLO

```
<table class = "tablaestados" cellpadding="0" width="100%">
<tr class="tablaestadosceldatitulo">
<td class = "tablaestadosceldatitulo"> <m4:label
item="SCO_NM_DOC_TYPE" htmlsafe="true" outputdef="<%=znodo%>"/></td>
<td class = "tablaestadosceldatitulo"> <m4:label
item="SCO_DT_EMISSION" htmlsafe="true" outputdef="<%=znodo%>"/></td>
<td class = "tablaestadosceldatitulo"><m4:label
item="SCO_NM_DOC_STATE" htmlsafe="true" outputdef="<%=znodo%>"/></td>
<td class = "tablaestadosceldatitulo" > <m4:label
item="SCO_DT_VALID" htmlsafe="true" outputdef="<%=znodo%>"/></td>

<td class = "tablaestadosceldatitulo"></td>
<td class = "tablaestadosceldatitulo">Título del documento</td>

<td class = "tablaestadosceldatitulo" > </td>
</tr>

<%sgtc_zShowMode = "1";%>
<%sgtc_zReadWrite = "0";%>

<m4:dataloop outputdef="<%=znodo%>">
<m4:current m4varname="current" outputdef="<%=znodo%>"/>
<%zposicion = Integer.valueOf(current).intValue();zcontrol =
zposicion%2;%>
<%if (zcontrol==0){zPaint="";}else{zPaint="2";}%>
<tr>
<td class="fuentevalor<%=zPaint%>"><m4:item item="SCO_NM_DOC_TYPE"
htmlsafe="true" outputdef="<%=znodo%>"/></td>
<td class="fuentevalor<%=zPaint%>"><m4:item item="SCO_DT_EMISSION"
htmlsafe="true" outputdef="<%=znodo%>"/></td>
<td class="fuentevalor<%=zPaint%>"><m4:item item="SCO_NM_DOC_STATE"
htmlsafe="true" outputdef="<%=znodo%>"/></td>
<td class="fuentevalor<%=zPaint%>"><m4:item item="SCO_DT_VALID"
htmlsafe="true" outputdef="<%=znodo%>"/></td>

<m4:item item="<%=sgtc_zIDInputIDDOC%>" var="sgtc_zIDDOC"
htmlsafe="true" outputdef="<%=znodo%>"/>
```

```

<m4:item item="<%=sgtc_zIDInputTITLEDOC%" var="sgtc_zTITLEDOC"
htmlsafe="true" outputdef="<%=znodo%" />

<%sgtc_zNMInputIDDOC = sgtc_zIDInputIDDOC + "_" + zposicion;%>
<%sgtc_zIDCSSRowTable = "fuentevalor" + zPaint;%>

<td class="fuentevalor<%=zPaint%"><input type="hidden"
id="<%=sgtc_zNMInputIDDOC%" name="<%=sgtc_zNMInputIDDOC%"
value="<%=sgtc_zIDDOC%" /></td>

<td class="fuentevalor<%=zPaint%"><%@ include file="../tc_docs/
tc_doc_include.jsp" %></td>

<td class="fuentevalor<%=zPaint%" colspan="3"><a title =
"<%=Tran.getProperty("Button.Delete")%">"
href="javascript:borrar('<m4:item item="SCO_OR_HR_DOC"
htmlsafe="true" outputdef="<%=znodo%" jsafe="true"/>', '<m4:item
item="SCO_ID_DOC_TYPE" jsafe="true" outputdef="<%=znodo%"
htmlsafe="true"/>', '<m4:item item="SCO_DT_EMISSION" jsafe="true"
outputdef="<%=znodo%" htmlsafe="true"/>', '<m4:item
item="SCO_ID_DOC_STATE" jsafe="true" outputdef="<%=znodo%"
htmlsafe="true"/>', '<m4:item item="SCO_DT_VALID" jsafe="true"
outputdef="<%=znodo%" htmlsafe="true"/>');"><img
alt="<%=Tran.getProperty("Button.Delete")%">" src="/iconos/
icono_eliminar_ess_11_12.gif" height="11" width="12" align="right"/
></td>

</tr>
</m4:dataloop>

</table>

```

Para el correcto funcionamiento de este include, es necesario incluir, antes de nada, el include de inicialización de variables:

```
<%@ include file="../tc_docs/tc_doc_initialize_include.jsp" %>
```

Se deben añadir dos nuevas cabeceras a la tabla, una para el identificador del documento (oculta), y otra para el título del documento y los botones.

```

<td class = "tablaestadosceldatitulo"></td>
<td class = "tablaestadosceldatitulo">Título del documento</td>

```

Se deben modificar los valores de las siguientes variables java:

- **sgtc\_zShowMode:** para modo formulario se establecerá valor 0, y para modo tabla se establecerá valor 1 (0:modo formulario 1:modo tabla). En este caso `sgtc_zShowMode = "1";`

- **sgtc\_zReadWrite:** para modo readonly se establecerá valor 0, y para modo readwrite se establecerá valor 1 (0:modo readonly 1:modo readwrite). En este caso `sgtc_zReadWrite = "0"`;

A continuación, ya dentro del dataloop que recorre todos los registros de la tabla se deberá rellenar las variables `sgtc_zIDDOC` y `sgtc_zIDInputTITLEDOC`:

```
<m4:item item="<%=sgtc_zIDInputIDDOC%>" var="sgtc_zIDDOC"
htmlsafe="true" outputdef="<%=znodo%>" />

<m4:item item="<%=sgtc_zIDInputTITLEDOC%>" var="sgtc_zTITLEDOC"
htmlsafe="true" outputdef="<%=znodo%>" />

<m4:item m4name="<%=zSCOIDDOC%>" var="sgtc_zIDDOC" htmlsafe="true"/>
```

Se configuran las variables `sgtc_zNMInputIDDOC` y `sgtc_zIDCSSRow` para generar el nombre de los inputs de la fila actual y el estilo de la fila actual, respectivamente.

1. `<%sgtc_zNMInputIDDOC = sgtc_zIDInputIDDOC + "_" + zposicion;%>`
2. `<%sgtc_zIDCSSRowTable = "fuentevalor" + zPaint;%>`

Por último, dentro de cada fila siempre deben existir dos columnas, una con un input para contener el identificador del documento, y otra con el include.

1. `<td class="fuentevalor<%=zPaint%>"><input type="hidden" id="<%=sgtc_zNMInputIDDOC%>" name="<%=sgtc_zNMInputIDDOC%>" value="<%=sgtc_zIDDOC%>" /></td>`
2. `<td class="fuentevalor<%=zPaint%>"><%@ include file="..tc_docs/tc_doc_include.jsp"%></td>`

En modo formulario el identificador del documento asociado quedará en la propiedad value del input `<%=sgtc_zNMInputIDDOC%>`, y en modo tabla los identificadores de los documentos asociados quedarán en la propiedad value de los inputs `<%=sgtc_zNMInputIDDOC%>+"_1"`, `<%=sgtc_zNMInputIDDOC%>+"_2"`,... uno por cada fila de la tabla.

Este include irá situado dentro del formulario de la página funcional. En el primer include es necesario incluir en el fichero de estilo del producto en que se quiera añadir el include, las siguientes clases, a continuación se muestra el estilo que viene configurado de tecnología en el fichero `style\shco_gen100.css`, en el caso del ejemplo, el producto es el MSS, habría entonces que añadir y configurar estas clases en el fichero `css\estilo_mss.css`:

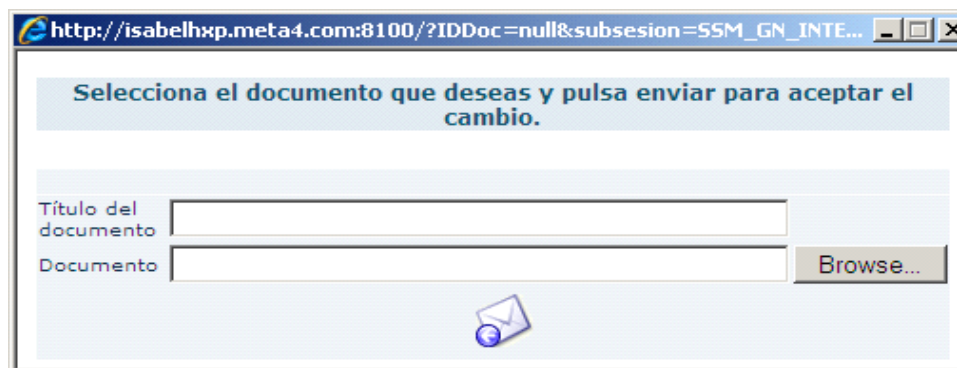
1. [input título del formulario]: `fuenteformulario`
2. [botones del Include]: `fuentebotondotable`

Estas dos clases son específicas del modo tabla. Las indicadas a continuación son válidas para ambos modos:

3. [input y fondo de fila]: `fuentevalor` y `fuentevalor2`

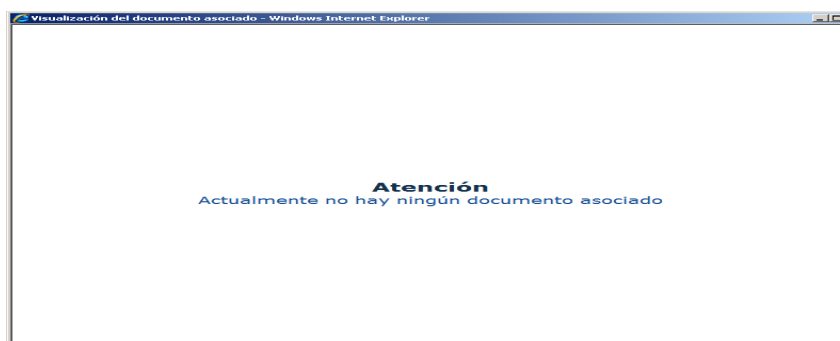
4. [botones del Include]: fuentebotondactable

Figura 163. Pantalla de selección de documento



5. [etiqueta: selecciona el documento que deseas y pulsa...]: fuentecalendario  
 6. [etiqueta título del documento y documento]: fuentecampo  
 7. [botón enviar]: fuenteboton

Figura 164. Mensaje de advertencia de asignación de documento



8. [atención]: fuenteactualizar  
 9. [actualmente no hay ningún...]: fuenteactualizar2

## Configuración del borrado de documentos

Siempre que exista un solo include por página y no sea en modo tabla, subirá el documento al gestor documental cuando se grabe el registro funcional. El borrado de registros, en esta parte, se realiza sobre una tabla, la cual contiene en cada fila un documento. Cada una de estas filas tiene un botón en forma de



---

aspa que elimina el registro, pero no el documento asociado en el gestor, por lo que es necesario eliminar el documento del gestor cuando se borre el registro funcional.

Para ello se dispone del método `SRTC_MTD_DELETE_DOC`, que recibe el identificador del documento a borrar `ARG_ID_DOC`:

```
SRTC_VIEW_DOCUMENT!SRTC_VIEW_DOCUMENT.SRTC_MTD_DELETE_DOC
```

Se ha creado una página de prueba para llamar a este método desde el SSE:

- Página JSP: `tc_docs/tc_doc_delete.jsp`

También se dispone de un include, parecido al que se utiliza para persistir `tc_doc_save_include.jsp`, que sirve para borrar el documento.

- Página JSP: `tc_docs/tc_doc_delete_include.jsp`

Para el correcto funcionamiento de este include es necesario declarar con anterioridad la siguiente variable java:

- **`sgtc_zNMInputIDDOC`**: variable para generar los nombres de los inputs. Por ejemplo, si se asigna a esta variable el literal `"SCO_ID_DOC"`:

---

## EJEMPLO

```
<m4:startpage m4task="<%=zsubsesion%"/>

<%
//variable para generar los nombres de los input, por ejemplo:
SCO_ID_DOC
String sgtc_zNMInputIDDOC = "SCO_ID_DOC";
%>
<%@ include file="../../tc_docs/tc_doc_delete_include.jsp" %>

<m4:beginjob/>
<m4:datadef m4o="<%=zmeta4object%/" m4name="<%=zsubsesion%"/>

<m4:endjob/>
<m4:endpage/>
```

---

Este include va situado dentro de `<m4:startpage>` y antes del `<m4:beginjob>` que realiza el borrado del registro actual.

En los casos en que no sea posible utilizar el include, puede llamarse directamente al método `SRTC_VIEW_DOCUMENT!SRTC_VIEW_DOCUMENT.SRTC_MTD_DELETE_DOC`, pasándole como argumento el identificador del documento a borrar.

## Funciones para mostrar y descargar documentos

Para poder llevar a cabo la visualización y descarga de documentos sin necesidad de utilizar el include, se dispone de sendas funciones, que permiten mostrar y descargar el documento que se les pase como parámetro.

A la primera de ellas, la función `m4opendocument_tech(siddoc)`, se le pasará por parámetro el identificador del documento que se quiera visualizar.

```
// esta funcion muestra el documento que se le pasa como parámetro
function m4opendocument_tech(siddoc)
{
 var spage = "/servlet/CheckSecurity/JSP/tc_docs/tc_doc_view.jsp";
 spage = spage + "?IDDoc=" + siddoc;
 mywindowdoc("view",spage,null);
}
```

Esta función está incluida en la librería `tc_docs/m4doc_include.js`

Por su parte, a la función `m4downloaddocument_tech(siddoc,sfile,sstylesheet)`, que permite descargar un documento, se le pasará por parámetro el identificador del documento que se quiera descargar, el nombre del fichero más su extensión, y la página de estilo a utilizar en la página de descarga. Esta página debe contener las clases `fuentesactualizar` y `fuentesactualizar2`.

En caso de pasar el parámetro del nombre del fichero vacío se descargará con el nombre temporal, y en caso de pasar el parámetro de la página de estilo vacío, la página de descarga se mostrará sin estilo.

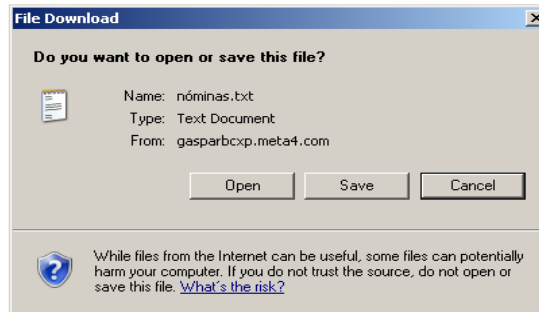
```
// esta funcion descarga el documento que se le pasa como parámetro
function m4downloaddocument_tech(siddoc,sfile,sstylesheet)
{
 var spage = "/servlet/CheckSecurity/JSP/tc_docs/tc_doc_download.jsp";
 spage = spage + "?IDDoc=" + siddoc;
 spage = spage + "&file=" + sfile;
 spage = spage + "&stylesheet=" + sstylesheet;
 mywindowdoc("download",spage,null);
}
```

Esta función está incluida en la librería `tc_docs/m4doc_include.js`.

Figura 165. Mensaje de descarga de documento

http://gasparbcxp.meta4.com:8102/?IDDoc=446&file=nóminas.txt&stylesheet=/css/estilo\_sse.css - D - Windows Internet Ex... 

**Descarga de archivo**  
Pulse aquí para descargar el archivo



## Uso de la tarea para ver un documento y subirlo al gestor documental

Para poder utilizar la tarea para clientes Web HTML (como el SSE/SSM) que permite “Ver, subir documento y enviar a firma”, hay que llamar a la página `tc_docs/tc_doc_view_save_sign.jsp` con un conjunto de parámetros que se describen a continuación:

- **URL\_Doc** (obligatorio): URL del documento a visualizar.
- **Title**: título del documento. Valor por defecto: "Document from ESS/MSS".
- **Author**: autor del documento. Valor por defecto: "ESS/MSS".
- **Abstract**: resumen del documento.
- **DMS**: repositorio de DMS. Valor por defecto: "META4-DMS".
- **Class**: clase, tipo de documento. Ver los tipos de documentos válidos en PeopleNet/Herramientas de desarrollo/Gestión documental/Tipos de documentos.
- **SignNotAllowed**: indica que no se permite la firma del documento. “1”: No permitir la firma del documento. “0”: Permitir firma (valor por defecto).
- **URL\_ERROR**: URL a redirigir cuando el proceso termina con error. Si no viene por defecto se tomará la página de error configurada en el servidor web teniendo en cuenta el idioma de conexión (Configclient.xml).
- **URL\_CANCEL**: URL en caso de cancelación. Si no viene se tomará la página configurada por defecto en el servidor web (Configclient.xml).
- **URL\_SUCCESS**: URL a redirigir cuando el proceso termina con éxito. Si no viene se tomará la página configurada por defecto en el servidor web (Configclient.xml).

Una vez que se ejecuta la tarea el proceso redirigirá a la página adecuada según si la ejecución ha finalizado con éxito o con error.

- Si se ha ejecutado con éxito se redirigirá a la página establecida en el parámetro `URL_SUCCESS`, indicando el identificador del documento (`ID_DOC`) que se ha subido al gestor documental. Si se trabaja con portafirmas externos que dan referencias propias a los documentos, se indicará la referencia externa que haya retornado el portafirmas cuando se solicitó la firma del documento:

`URL_SUCCESS?ID_DOC=Identificador_doc o Ref_externa`

- Si se ha ejecutado con error se redirigirá a la página establecida en el parámetro `URL_ERROR` como página de error, indicando el texto del error (error) y el identificador del documento (`ID_DOC`) si ya se ha subido al gestor documental:

---

URL\_ERROR?error=Error guardando  
documento&ID\_DOC=Identificador\_doc o Ref\_externa

- Si se cancela el proceso de subir el documento y enviar a firmar, se redirigirá a la página establecida en el parámetro URL\_CANCEL

---

## EJEMPLO

A continuación se muestra un formulario que se puede utilizar como ejemplo para llamar a la nueva tarea:

```
<form id="frmViewSaveFile" name="frmViewSaveFile" action="/
servlet/CheckSecurity/JSP/tc_docs/tc_doc_view_save_sign.jsp"
method="post">
<input type="text" size="100" id="URL_Doc" name="URL_Doc"
value=""/>
<input type="text" size="100" id="Title" name="Title" value=""/>
<input type="text" size="100" id="Author" name="Author" value="" /
>
<input type="text" size="100" id="Abstract" name="Abstract"
value=""/>
<input type="text" size="100" id="Class" name="Class" value="" />
<input type="text" size="100" id="DMS" name="DMS" value="" />
<input type="text" size="100" id="SignNotAllowed"
name="SignNotAllowed" value="" />
<input type="text" size="100" id="URL_ERROR" name="URL_ERROR"
value=""/>
<input type="text" size="100" id="URL_CANCEL" name="URL_CANCEL"
value=""/> <input type="text" size="100" id="URL_SUCCESS"
name="URL_SUCCESS" />
</form>
```

---



# Referencia de la biblioteca M4TagLib

# 9

## Semántica de referencia

En este capítulo se describen todas las M4Tags de la biblioteca M4TagLib que se pueden utilizar en el desarrollo de páginas funcionales con tecnología HTML.



Al final del capítulo, se incluye una guía de referencia rápida.

Para describir las etiquetas, se utiliza la sintaxis BNF (Backus Nauer Form). Esta sintaxis, empleada a menudo en la descripción de gramática, consta de reglas. Cada una de éstas tiene una sola variable a la izquierda y varias a la derecha, junto con símbolos terminales.

Las variables (o símbolos no terminales) se representan como palabras normales. Los símbolos terminales aparecen en negrita.

La transición de la regla se representa mediante `::=`. La parte que se encuentra a la izquierda del operador se puede sustituir por la parte derecha.

El símbolo `|` marca las separaciones e indica alternativas.

Siendo `x` un elemento de la parte derecha de la regla, se puede especificar lo siguiente:

- `[x]`: es opcional.
- `(x)*`: aparece cero o más veces.
- `(x)+`: aparece una o más veces.

### EJEMPLO

```
BeginJob ::= <m4:beginjob/>
```

## M4Tags de servicio de datos

Las M4Tags de servicio de datos permiten realizar operaciones básicas con los datos que se desean manejar. Estas M4Tags son:

- M4Tags de servicio de datos básicos
- M4Tags de filtros
- M4Tags de asignación
- Herramientas de servicio de datos

### M4Tags de servicio de datos básicos

#### StartPage

Esta etiqueta indica el comienzo de una página y debe aparecer en las páginas JSP que tienen acceso a datos. Realiza las siguientes funciones:

- Comprueba la autenticación del usuario mediante la *cookie* enviada por el navegador.
- Define la subsesión que se va a usar en la transacción actual con el servidor de aplicaciones.
- Actualiza el tiempo del sistema para controlar la inactividad de los usuarios.
- Redirige a una página de error cuando se producen errores de seguridad.
- Crea el entorno o el contexto de la transacción.

```
StartPage ::= <m4:startpage m4task="subsession_alias"
 [m4preservelevel="0|1|2..."] />
```

#### Parámetros obligatorios:

- **m4task**: parámetro que indica la subsesión que se va a usar.

#### Parámetros opcionales:

- **m4preservelevel**: permite al desarrollador controlar el ciclo de vida en las subsesiones del servidor de aplicaciones. Toma un valor igual o mayor que cero ( $\geq 0$ ).
  - El valor por defecto o el valor 0 crea subsesiones que permanecen activas en el servidor de aplicaciones una vez finalizada la transacción, disponibles para posteriores transacciones.



- El valor 1 crea subsesiones que siempre se limpian cuando finaliza la transacción.
- Si el valor es 2 o superior, se crean subsesiones que se limpian cuando se inicia otra subsesión cuyo valor **m4preservelevel** es mayor que cero.

---

#### EJEMPLO

```
<m4:startpage m4task="subsession_alias" m4preservelevel="2"/>
```

Se crea una subsesión con m4preservelevel 2. Una segunda subsesión, con m4preservelevel distinto de cero, provocará su eliminación del espacio de subsesión en el servidor de aplicaciones. Así, las subsesiones creadas con m4preservelevel 2, "desaparecen".

---

## EndPage

Esta etiqueta indica dónde termina la página y destruye el entorno o el contexto de la transacción.

```
EndPage ::= <m4:endpage/>
```

---

#### EJEMPLO

```
<m4:startpage m4task="subsession_alias"/>
```

```
...
```

```
<m4:endpage/>
```

---

## Page

La etiqueta **Page** tiene la misma función que **StartPage** y **EndPage**. Se recomienda utilizar la etiqueta **Page** en lugar de **StartPage** y **EndPage**.

```
Page ::= <m4:page subsessionid="SubsessionId"
 [preservelevel="X" | keepcontext="(TRUE|FALSE)"]>
 ...
<m4:page/>
```

#### Parámetros obligatorios:

- **subsessionid**: este parámetro es igual que **m4task** para **StartPage**, que indica la subsesión activa.

**Parámetros opcionales:**

- **preservelevel**: este parámetro es igual que **m4preservelevel** para **StartPage**, que permite al desarrollador controlar el ciclo de vida de las subsesiones del servidor de aplicaciones. Toma un valor igual o mayor que cero ( $\geq 0$ ).
  - Un valor nulo o el valor 0 crea subsesiones que permanecen activas en el servidor de aplicaciones una vez finalizada la transacción.
  - El valor 1 crea subsesiones que siempre se limpian cuando finaliza la transacción.
  - Si el valor es 2, se crean subsesiones que se limpian cuando se inicia otra subsesión cuyo valor **m4preservelevel** es mayor que cero.
- **keepcontext**: parámetro booleano que se usa para especifica si se debe eliminar el contexto del servidor Web después de una serie de transacciones.

**BeginJob**

Esta etiqueta indica el comienzo de una transacción.

```
BeginJob ::= <m4:beginjob/>
```

Una transacción es un grupo de operaciones que se ejecutan en el servidor de aplicaciones. Durante una transacción se especifica:

- La definición de Meta4Objects
- Una serie de métodos que se han de ejecutar
- Los datos que se desean obtener y utilizar

**EndJob**

Esta etiqueta marca el final de una transacción. Además, permite controlar el resultado de la transacción: error, advertencia y éxito. Según este resultado, la etiqueta indica una dirección URL determinada.

```
EndJob ::= <m4:endjob
 [[m4alias="exec_alias"]
 m4check="ERROR|WARNING|SUCCESS"
 m4url="url"] />
```

**Parámetros opcionales:**

- **m4alias**: Alias del método que obtiene los mensajes del servidor de aplicaciones cuando se ejecuta.

- **m4url**: parámetro de entrada que indica la dirección URL que se va a ejecutar para controlar el resultado de la transacción. Suele ser la URL de la página de error que contiene código especial para mostrar los mensajes de error.
- **m4check**: parámetro de entrada que indica el resultado de la transacción que se desea controlar.
  - *ERROR*
  - *WARNING*
  - *SUCCESS*

## Job

La etiqueta **Job** tiene la misma función que el par **BeginJob** y **EndJob**. Se recomienda utilizar la etiqueta **Job** en lugar de **BeginJob** y **EndJob**.

```
Job ::= <m4:job [check="(ERROR|WARNING|SUCCESS)"
 alias="DataTagAlias" url="ErrorPage"]>
 ...
</m4:job>
```

### Parámetros opcionales:

Estos parámetros son similares a los utilizados en la etiqueta **EndJob**:

- **alias**: Alias del Meta4Object, que sustituye a **m4alias** en la etiqueta **EndJob**.
- **url**: parámetro de entrada que indica la dirección URL que se va a ejecutar para controlar el resultado de la transacción. Sustituye a **m4url** en la etiqueta **EndJob**.
- **check**: parámetro de entrada que indica el resultado de la transacción que se desea controlar. Sustituye a **m4check** en la etiqueta **EndJob**, y se puede establecer en:
  - *ERROR*
  - *WARNING*
  - *SUCCESS*



La funcionalidad de las etiquetas **Page** y **Job** es muy similar a la de los correspondientes pares descritos anteriormente, pero las etiquetas **Page** y **Job** permiten a los desarrolladores escribir documentos JSP compatibles con XHTML, a diferencia de los pares de etiquetas. No obstante, la notación con los pares sigue funcionando.

**EJEMPLO:**

```
<m4:page subsessionid="subsession_alias">
 <m4:job>
 ...
 </m4:job>
</m4:page>
```

## DataDef

Esta etiqueta proporciona acceso al servidor de aplicaciones y crea la estructura de datos en el servidor Web. Solicita la estructura de datos al servidor de aplicaciones. Cada etiqueta define un Meta4Object, cuyo alias será el nombre de aquella.

```
DataDef ::= <m4:datadef m4o= ["M4ONAME" | "M4ONAME.L2ALIASLIST"]
 m4name="M4OALIAS"
 [m4start_date="...."]
 [m4end_date = "...."]
 [m4id_rsm="...."]
 [m4id_app_role="...."]
 [m4preserve="...."]
 [m4id_organization="...."] />
```

**Parámetros obligatorios:**

- **m4o**: parámetro de entrada que contiene el nombre del Meta4Object.
- **m4name**: parámetro de entrada que identifica la solicitud de datos. Este nombre representa el alias del Meta4Object y se usa en el resto de etiquetas para tener acceso a la estructura de datos.

**Parámetros opcionales:**

- **m4find**: parámetro de entrada que indica si se va a buscar un Meta4Object en el servidor de aplicaciones. Este parámetro le permite acceder al Meta4Object mediante llamadas de nivel 2. Los valores posibles son *TRUE* y *FALSE* (valor por defecto).
- **m4preserve**: si su valor es *TRUE*, se guarda en el servidor de aplicaciones la instancia del Meta4Object de una transacción a otra. Si este Meta4Object se instancia sólo una vez, el parámetro será *FALSE*.

- 
- **m4id\_app\_role**: parámetro de entrada que asigna el rol con que se instancia el Meta4Object. Permite al desarrollador desechar el rol establecido por defecto al iniciar la sesión para un objeto determinado.
  - **m4id\_organization**: parámetro de entrada que asigna la organización con la que se instancia el Meta4Object.
  - **m4start\_date**: parámetro de entrada que asigna la fecha de inicio con que se instancia el Meta4Object.
  - **m4end\_date**: parámetro de entrada que asigna la fecha de fin con que se instancia el Meta4Object.
  - **m4id\_rsm**: parámetro de entrada que asigna el RSM con que se instancia el Meta4Object.

---

### EJEMPLO 1

```
<m4:datadef
 m4o="PERSON_FORM_INTERFACE"
 m4name="person_alias"
/>
```

---

Si se utiliza la etiqueta **DataDef** para hacer una llamada de nivel 2, el atributo de la etiqueta **m4o** puede contener una referencia al alias de nivel 2 del Meta4Object invocado:

```
m4name = "<parentalias>.<L2aliaslist>".
```

Donde L2aliaslist puede ser el alias de nivel 2 o una lista de alias de nivel 2 correspondientes a los distintos niveles de la jerarquía.

---

### EJEMPLO 2

```
<m4:datadef
 m4o="PERSON_FORM_INTERFACE.MAIL"
 m4name="mail_alias"
 m4find="true"/>
```

En este ejemplo se da por hecho que se va a hacer referencia a una instancia del Meta4Object MAIL, llamada MAIL, desde un alias en el Meta4Object PERSON\_FORM\_INTERFACE.

---

## Exec

Esta etiqueta ejecuta el método o concepto de un Meta4Object.

Para los métodos sin argumentos, la etiqueta no tiene cuerpo.

```
Exec ::=
<m4:exec m4method=" [ExecAlias:]DataTagAlias!NODE [REC] .METHOD"/>
```

```
Exec ::= <m4:exec [alias="ExecAlias"]
m4object="DataTagAlias" node="NODE" method="METHOD"/>
```

Para los métodos con argumentos, la etiqueta tiene un cuerpo que contiene tantas etiquetas **Param** como argumentos.

```
Exec ::=
<m4:exec
m4method=" [ExecAlias:]DataTagAlias!NODE [REC] .METHOD">
 <m4:param name="ARGUMENT1" value="value1"/>
 ...
 <m4:param name="ARGUMENTN" value="valueN"/>
</m4:exec>
```

```
Exec ::= <m4:exec [alias="ExecAlias"]
m4object="DataTagAlias" node="NODE" method="METHOD">
 <m4:param name="ARGUMENT1" value="value1"/>
 ...
 <m4:param name="ARGUMENTN" value="valueN"/>
</m4:exec>
```

### Parámetros opcionales:

- **m4method**: indica el método que se va a ejecutar, mediante una notación especial.
  - **ExecAlias**: alias que identifica esta solicitud para la ejecución del método.
  - **DataTagAlias**: alias del Meta4Object.
  - **Node**: nodo del elemento. Si se trata de un método de Meta4Object, puede omitirlo.

- **REC:** se ubica en el registro.
  - **REC:** va a la posición indicada (empezando por 0).
  - **+REC:** avanza posiciones REC.
  - **-REC:** retrocede posiciones REC.
  - **Primero:** va al primer registro.
  - **Último:** va al último registro.

- **METHOD:** nombre del método

Hay una serie de palabras reservadas para el nombre del método, que también se pueden ejecutar con la clase M4Operation.

- **DataTagAlias.LOAD:** carga y descarga Meta4Objects.
- **DataTagAlias.LOADONLY:** carga el Meta4Object.
- **DataTagAlias.UNLOAD:** descarga el Meta4Object.
- **DataTagAlias.PERSIST:** guarda el Meta4Object en el servidor de aplicaciones.
- **DataTagAlias!NODE.NEWRECORD:** crea en el nodo un nuevo registro.
- **DataTagAlias!NODE.DELRECORD:** borra el registro del nodo actual.

## EJEMPLO

```
<m4:exec m4method="LOAD:PERSON.LOAD" />
```

- **alias:** como el ExecAlias de la notación especial, alias que identifica esta solicitud para la ejecución del método.
- **m4object:** como DataTagAlias en la notación especial, alias del Meta4Object.
- **node:** como NODE en la notación especial, nodo del elemento. Es opcional.
- **method:** nombre del método



Puede utilizar el m4method y la notación especial, o los distintos atributos alias, m4object, node y method, según el estilo de código que prefiera. Ambos grupos de atributos juntos son incompatibles.

- **param:** parámetro de entrada que indica los argumentos del método. Habrá tantos parámetros en la etiqueta como argumentos tenga el método.
  - **name:** nombre del argumento.
  - **value:** valor del argumento.

Al indicar parámetros con argumentos nulos, se recomienda usar la palabra *null* en vez de la cadena vacía.

Si los argumentos del método se pasan por referencia en vez de por nombre, el nombre de cada argumento deberá ser numérico. No puede mezclar argumentos por nombre y por referencia en la misma llamada.

## Outputdef

Esta etiqueta define los datos que se desean recuperar. Puede ser un elemento de un registro o todos los datos del nodo.

```
Outputdef ::= <m4:outputdef [m4alias="OutputDefAlias"]>
 <m4:param name="M4NAME0" value="..." />

 <m4:param name="M4NAMEN" value="..." />
</m4:outputdef>

Outputdef ::= <m4:outputdef [m4alias="OutputDefAlias"
 m4object="DataTagAlias" node="NODE" records="REC"/>
```

### Parámetros opcionales:

- **m4alias**: parámetro de entrada que indica el nombre que se asociará a los datos cargados. Si se omite, se asigna un valor por defecto. A menudo se hace referencia a este alias como el **alias del outputdef**.
- **param**: parámetro de entrada que indica los datos que se van a recuperar y que las etiquetas de presentación mostrarán posteriormente.
  - **name**: nombre del parámetro.



El nombre del parámetro no puede ser cualquier nombre, se llama específicamente M4NAME0 o M4NAME1, etc.

- **value**: datos que se van a procesar.
  - **DataTagAlias!Node[REC]**: recupera registros REC.
  - **DataTagAlias!Node[REC].Item**: recupera el elemento del registro REC.
    - **DataTagAlias**: alias del Meta4Object.
    - **Node**: nombre del nodo.



- **REC:** rango de registros que se va a recuperar.
  - **[]:** nada, sólo el registro actual
  - **[A-B]:** rango de registros del A al B.
  - **[\*]:** todos los registros.
  - **[A]:** sólo el registro A.
- **Item:** nombre del elemento.

---

### EJEMPLO

```
<m4:outputdef m4alias="DATA">
 <m4:param name="M4NAME0" value="PERSON!RESULT[FIRST]" />
</m4:outputdef>
```

- 
- **m4object:** como DataTagAlias en la notación especial, alias del Meta4Object del que se obtendrán datos.
  - **node:** como NODE en la notación especial, nombre del nodo del Meta4Object del que se obtendrán datos.
  - **records:** rango de registros que se va a recuperar, como arriba.



Puede utilizar la etiqueta param y la notación especial de los atributos de valores, o los distintos atributos, m4object, node y method, según el estilo de código que prefiera. Ambos grupos de atributos juntos son incompatibles.

---

### EJEMPLO

En este ejemplo se combinan las etiquetas básicas de servicio de datos, en una página JSP que carga los datos de un Meta4Object.

```
<m4:startpage m4task="subsession_alias"/>
<m4:beginjob/>
 <m4:datdedef m4o="SCO_GEO_STRUCTURE" m4name="GEO"/>
 <m4:exec m4method="LOAD:GEO.LOAD"/>
 <m4:outputdef m4alias="PARENT">
 <m4:param name="M4NAME0" value="GEO!STD_MT_COUNTRY[*]" />
 </m4:outputdef>
<m4:endjob/>
<m4:endpage/>
```

## Outputcache

Esta etiqueta define el comportamiento de autoexpiración de la página y obligatoriamente debe ir antes de cualquier tipo de código HTML.

```
Outputcache ::= <m4:outputcache checkexpired="M4O_1,.. M4O_n">
 </m4:outputcache>
```

La cabecera HTTP *Expires* que acabará teniendo la página en llamadas distintas de la primera dependerá de que todos los Meta4Objects indicados sean cacheables en datos (expirará en el futuro) o alguno de ellos no lo sea (expirará en el pasado).

Con este mecanismo se reduce el tiempo necesario para mostrar una página que accede un Meta4Object estático o que no cambia mucho, por ejemplo mediante un comando GET para abrir una página nueva en una ventana nueva.

### Parámetro obligatorio:

- **checkexpired**: parámetro de entrada que indica la lista de los identificadores de los Meta4Objects, separados por comas, para los cuales se tiene que verificar el valor de la propiedad Metadatos accesible en caché.

## Move

Esta etiqueta le sitúa en un registro determinado.

```
Move ::= <m4:move>
 <m4:param name="DataTagAlias_1"
 value="OutputDefAlias_1:NODE[REC] #NODE[REC]" />
 ...
 <m4:param name="DataTagAlias_m"
 value="OutputDefAlias_n:NODE[REC] #NODE[REC]" />
</m4:move>
Move ::= <m4:move outputdef="OutputDefAlias" record="REC">
```

Cuando se utiliza la etiqueta dentro de una transacción (dentro de **Job** o entre **BeginJob** y **EndJob**), se establece la posición en el Meta4Object. Esto se conoce como “movimiento en el servidor o movimiento remoto”. De lo contrario, la etiqueta simplemente define la posición en los resultados recuperados. Esto se conoce como “movimiento en el cliente o movimiento local”.

---

### Parámetros opcionales:

- **param**: indica la acción que se va a realizar.
  - **name**: nombre del Meta4Object.
  - **value**: acción que se va a realizar sobre el Meta4Object.
    - **outputdef\_alias**: alias de OUTPUTDEF.
    - **Node**: nodo que se va a mover.
    - **[REC]**: posición a la que se quiere mover.

### Parámetros opcionales:

- **outputdef**: parámetro que define el alias de los datos que se van a recuperar. Este parámetro sólo se puede usar en un “movimiento local” y debería incluir el nombre de un outputdef definido durante la transacción anterior.
- **record**: posición a la que se quiere mover. Este atributo sólo se puede usar en combinación con el **outputdef** en un “movimiento local”.



Puede utilizar la etiqueta **param** y la notación especial de los atributos de valores, o los distintos atributos, **outputdef** y **record**, según el estilo de código que prefiera. Ambos grupos de atributos juntos son incompatibles.

---

### EJEMPLO

```
<m4:move>
 <m4:param name="GEO" value="PARENT:STD_MT_COUNTRY[0]" />
</m4:move>
```

---

## M4Tags de filtros y ordenación

Estas M4Tags aplican filtros a los datos cargados; de este modo, el acceso a los datos es preciso y flexible, recuperándose sólo los datos solicitados. También le permiten ordenar los datos.

## Filter

Esta etiqueta filtra los datos de un nodo. Los filtros se especifican usando código LN4.

```
Filter ::= <m4:filter
 m4name="DataTagAlias!NODE.FilterAlias"
 m4filter="<%= strM4Filter %>"/>
 strM4Filter ::= <% String strM4Filter="..."; %>
```

### Parámetros obligatorios:

- **m4filter**: parámetro que contiene la cadena que define el filtro que se ha de aplicar.

### Parámetros opcionales:

- **m4name**: nombre del filtro, usando la notación especial:
  - **DataTagAlias** y **NODE** se definen del modo habitual.
  - **FilterAlias**: nombre asignado al filtro.
- **alias**: nombre asignado al filtro (en vez de la notación especial de m4name).
- **node**: nombre del nodo al filtro (en vez de la notación especial de m4name).
- **m4object**: nombre del Meta4Object que contiene el nodo al filtro (en vez de la notación especial de m4name).



Puede utilizar el atributo m4method, o los distintos atributos alias, node y m4object, según el estilo de código que prefiera. Ambos grupos de atributos juntos son incompatibles.

### EJEMPLO

```
<% String strM4Filter="IF STD_N_FIRST_NAME=\"JAMES\" THEN RETURN(1)";
%>

<m4:filter m4name="PERSON!STD_TR_PERSON.FILT" m4filter="<%=
strM4Filter %>"/>
```

---

## RemoveFilter

Esta etiqueta borra el filtro. Si aplica varios filtros a un mismo Meta4Object, deberá borrar cada filtro antes de aplicar uno nuevo, a menos que desee aplicar cada filtro posterior a los datos ya filtrados.

```
RemoveFilter ::= <m4:removefilter
 m4name="DataTagAlias!NODE.FilterAlias"/>
RemoveFilter ::= <m4:removefilter
 alias="FilterAlias" m4object="DataTagAlias" node="NODE"/>
```

### Parámetros opcionales:

- **m4name:** nombre del filtro, usando la notación especial.
  - **DataTagAlias** y **NODE** se definen del modo habitual.
  - **FilterAlias:** nombre asignado al filtro.
- **alias:** nombre asignado al filtro (en vez de la notación especial de m4name).
- **node:** nombre del nodo donde se encuentra el filtro (en vez de la notación especial de m4name).
- **m4object:** nombre del Meta4Object de donde se va a eliminar el filtro (en vez de la notación especial de m4name).



Puede utilizar el atributo m4method, o los distintos atributos alias, node y m4object, según el estilo de código que prefiera. Ambos grupos de atributos juntos son incompatibles.

---

### EJEMPLO

```
<m4:removefilter m4name="PERSON!RESULT.FILTER"/>
```

---

## SortItems

Esta etiqueta ordena los datos de un nodo del servidor; es decir, es un filtro RAM. Para desordenar los datos después de haberlos ordenado, debe quitar el filtro de ordenación. Se puede ordenar sólo por un elemento.

```
SortItems ::= <m4:sortitems
 m4name="DataTagAlias!NODE.SortFilterAlias">
 <m4:param name="ITEMNAME" value="ASC|DESC"/>
</m4:sortitems>
```

### Parámetros obligatorios:

- **m4name**: Nombre del filtro de ordenación. *DataTagAlias* y *NODE* se definen del modo habitual. Este nombre se usa para eliminar el filtro de ordenación.
- **param**: elemento por el que se ordena.
  - **name**: nombre del elemento.
  - **value**: tipo de ordenación.
    - **ASC**: ascendente
    - **DESC**: descendente.

---

### EJEMPLO

```
<m4:sortitems
 m4name="PERSON!RESULT.SORT">
 <m4:param name="STD_ID_NAME" value="ASC"/>
</m4:sortitems>
```

---

---

## EJEMPLO

En este ejemplo se combinan algunas etiquetas de filtros en una página JSP que ordena los datos de un Meta4Object.

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<HTML>
<BODY>
<m4:startpage m4task="subsession_alias"/>
<m4:beginjob/>
 <m4:datdedef m4o="SCO_GEO_STRUCTURE" m4name="GEO"/>
 <m4:exec m4method="LOAD:GEO.LOAD"/>
 <m4:outputdef m4alias="PARENT">
 <m4:param name="M4NAME0" value="GEO!STD_MT_COUNTRY[*]"/>
 </m4:outputdef>
 <m4:sortitems
 m4name="GEO!STD_MT_COUNTRY.SORT">
 <m4:param name="STD_N_COUNTRY" value="ASC"/>
 </m4:sortitems>
<m4:endjob/>
<m4:endpage/>
</BODY>
</HTML>
```

---

## M4Tags de asignación

Estas M4Tags asignan valores a las propiedades indicadas.

### SetItems

Esta etiqueta modifica el valor de una serie de elementos.

```
SetItems ::= <m4:setitems>
 <m4:param name="DataTagAlias_1!NODEA.ITEMA" value="value"/>
 ...
 <m4:param name="DataTagAlias_n!NODEN.ITEMN" value="value"/>
</m4:setitems>
```

#### Parámetros obligatorios:

- **param**: elemento que se va a modificar.
  - **name**: nombre del parámetro.
    - **DataTagAlias**: código identificativo de la etiqueta de datos.
    - **NODE**: nombre del nodo.
    - **[REC]**: selecciona el registro (opcional).
      - **REC**: va a la posición REC, partiendo de 0.
      - **+REC**: avanza posiciones REC.
      - **-REC**: retrocede posiciones REC.
      - **LAST**: va al último registro.
      - **FIRST**: va al primer registro.
    - **Item**: elemento que se va a modificar.
  - **value**: valor que se va a asignar.

#### EJEMPLO

```
<m4:setitems>
 <m4:param name="PERSON!RESULT.STD_ID_NAME" value="JUAN"/>
</m4:setitems>
```



---

## EJEMPLO

En este ejemplo se combinan algunas etiquetas de asignación en una página JSP que modifica los datos de un Meta4Object.

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<HTML>
<BODY>
<m4:startpage m4task="subsession_alias"/>
<m4:beginjob/>
 <m4:datdedef m4o="SCO_GEO_STRUCTURE" m4name="GEO"/>
 <m4:exec m4method="LOAD:GEO.LOAD"/>
 <m4:setitems m4alias="PARENT">
 <m4:param name="GEO!STD_MT_COUNTRY[0].STD_N_COUNTRY"
 value="JAMAICA"/>
 </m4:setitems>
 <m4:exec m4method="SAVE:GEO.PERSIST"/>
<m4:endjob/>
<m4:endpage/>
</BODY>
</HTML>
```

---

## SetFile

Esta etiqueta inserta un archivo en un elemento de un Meta4Object.

```
SetFile ::= <m4:setfile m4blob="BLOB" m4path="PATH"/>
```

### Parámetros obligatorios:

- **m4blob**: indica el elemento, usando la sintaxis especial DataTagAlias!NODE[REC].ITEM.

### Parámetros opcionales:

- **m4path**: indica la ruta del archivo que se va a insertar.

---

## EJEMPLO

```
<m4:setfile m4blob="PERSON!SCO_PERSON.SCO_CURRICULUM"
 m4path="C:\\curriculum.doc"/>
```

---

## SetCurrency

Esta etiqueta asigna valores de elemento tipo moneda y, al mismo tiempo, le permite establecer valores de atributo de moneda; es decir, la fecha de cambio y el tipo de moneda.

```
SetCurrency ::= <m4:setcurrency
 m4name="DataTagAlias!NODE[REC].ITEM
 m4currency_value="..."
 m4currency_id="..."
 [m4currency_date="..."]
 [m4currency_type="..."]/>
```

### Parámetros obligatorios:

- **name**: elemento de tipo moneda que se va a asignar.
- **m4currency\_value**: valor de moneda.
- **m4currency\_id**: código identificativo de moneda.

### Parámetros opcionales:

- **m4currency\_date**: fecha del cambio,
- **m4currency\_type**: tipo de moneda. Si se especifica este campo, también deberá indicar la fecha.

---

### EJEMPLO

```
<m4:setcurrency name="PERSON!ROOT.CURRENCY"
 m4currency_value="10" m4currency_id="1"/>
```

---

## M4Tags de herramientas de servicio de datos

### SetProperty

Esta etiqueta asigna propiedades a M4Operations.

```
SetProperty ::= <m4:setproperty m4property="PROP" m4value="VAL"/>
```

---

## Parámetros obligatorios:

Se pueden establecer las siguientes propiedades de operación del ejecutor XML:

- **m4property:** nombre de la propiedad.
  - **TIMEOUT:** tiempo de espera que una transacción M4XML se estará ejecutando dentro del servidor de aplicaciones antes de que el cliente ligero decida que ha de terminar. Por defecto es 300.000 milisegundos (300 segundos = 5 minutos). Solamente se debe modificar este valor en el caso comprobado de que una operación sea más larga.
  - **DOXML:** este parámetro hace posible recoger directamente el XML devuelto por el servidor de aplicaciones. Si se desactiva, los resultados solamente pueden recogerse utilizando el método de M4Operations `getByteXMLModel()`.
  - **DOPARSE:** este parámetro, si se pone a FALSE, deshabilita el trabajo de cargar el XML retornado por el servidor de aplicaciones en objetos Java. Tiene sentido poner DOPARSE=FALSE sólo en combinación con DOXML=TRUE.
- **m4value:** valor que se va a asignar.
  - *TRUE* o *FALSE* para **DOPARSE** y **DOXML**.
  - Tiempo en milisegundos para **TIMEOUT**.

---

## EJEMPLO

Para modificar el valor de timeout de ejecución XML a 10 minutos (600 segundos) para una operación concreta:

```
<m4:setproperty m4property="TIMEOUT" m4value="600000"/>
```

Para traer directamente el M4XML sin pasar por los métodos de M4Operations/M4Tags e indicar que no es necesaria su carga dentro de objetos Java:

```
<m4:setproperty m4property="DOXML " m4value="TRUE"/>
<m4:setproperty m4property="DOPARSE " m4value="FALSE"/>
```

---

## RemoveM4O

Esta etiqueta elimina de la subsesión un Meta4Object. Es decir, es como si nunca hubiese hecho una solicitud **DataDef** sobre ese Meta4Object en la

subsesión actual. Sólo es necesario indicar el alias del Meta4Object. Se debe utilizar dentro de la etiqueta **<m4:startpage>**.

```
removem4o ::=
<m4:removem4o m4oalias="ALIAS"/>
```

## Param

Esta etiqueta representa un parámetro genérico con nombre y valor. Es una etiqueta auxiliar que se utiliza con otras.

```
Param ::= <m4:param name="NOMBRE" value="VALOR"/>
```

### Parámetros obligatorios:

- **name:** nombre
- **value:** valor

---

### EJEMPLO

```
<m4:param name="EDAD" value="23"/>
```

---

## M4Tags de presentación

Estas M4Tags muestran los datos recuperados del servidor de aplicaciones. Se pueden clasificar en distintos tipos:

- M4Tags de presentación básicos
- M4Tags de iteración
- M4Tags de consulta de traducciones
- Herramientas de presentación

### M4Tags de presentación básicos

#### Item

Esta etiqueta muestra o almacena un valor de elemento en una variable.

```
Item ::= <m4:item
(m4name="[OutputDefAlias]:DataTagAlias!NODE[REC].ITEM" |
outputdef="OutputDefAlias" item="ITEM" record="REC")
[m4varname = "NonDeclaredVariable" |
var = "DeclaredVariable" | vartype = "VariableType" |
m4format="FormatMask" | typename = "FormatType" |
htmlsafe="(true|false)" | jsafe = "(true|false)"]
>
```

#### Parámetros opcionales:

- **m4name**: nombre del elemento, usando la notación especial.
  - **DataTagAlias** y **NODE** se definen del modo habitual.
  - **OutputDefAlias**: sólo se utiliza cuando **OutputDef** tiene un alias.
- **outputdef**: atributo que hace referencia al alias de OutputDef donde se solicitó el elemento (en vez de m4name).
- **item**: nombre del elemento (en vez de m4name).
- **record**: rango de registros que se va a recuperar.



Puede utilizar el atributo m4method, o los distintos atributos, output, item y record, según el estilo de código que prefiera. Ambos grupos de atributos juntos son incompatibles.

- **m4varname**: nombre de la variable de destino donde debería guardarse el contenido del elemento. La etiqueta define una variable Java del tipo `java.lang.String` donde se especifica este atributo, por lo que no debe predeclarar la variable.
- **var**: nombre de la variable de destino donde debería guardarse el contenido del elemento. Esta variable debe declararse previamente en la página. Debe predeclarar una variable y especificar su nombre en este atributo para que se guarde el resultado. El tipo de datos por defecto aceptado para la variable predeclarada es `java.lang.String`.
- **vartype**: tipo de datos para la variable en **var**. Se utiliza para cambiar el tipo de variable por defecto de **var**. Éstos son los posibles valores: *Integer* y *Double*. Le permite utilizar variables predeclaradas de los tipos *Integer* y *Double*.
- **m4format**: máscara que se va a utilizar para formatear este elemento. Consulte la sintaxis de la máscara en el apéndice.
- **typename**: nombre de un tipo de formateo. Entre los tipos de formateo existentes se incluye "DATE", "DATETEXT", "TIMESTAMP", "HOUR", "NUMBER" y "CURRENCY", y los valores para las máscaras asociadas se almacenan en el Meta4Object Configuración de la aplicación.
- **htmlsafe**: especifica si se debe convertir el resultado a un cadena segura HTML. Esto es útil porque si un elemento contiene un carácter como `&`, `<`, `>` o `"`, el navegador puede interpretar estos caracteres como parte de etiquetas HTML y provocar que la página no se vea correctamente. Establézcalo en *TRUE* para convertir los caracteres no seguros a sus equivalentes HTML seguros, de lo contrario, déjelo como *FALSE*. El atributo está establecido por defecto en *FALSE*.



Lo mismo ocurre para los caracteres ISO-8859-1 entre 161 y 255, que se convierten a la cadena equivalente HTML (á -> `&acute;`) y el símbolo del Euro, que se convierte a `&#0128`.

- **jsafe**: especifica si deberían librarse (anteponiendo una barra invertida) cadenas no seguras de Java o JavaScript como `'` (comillas simples), `"` (comillas dobles), `\` (barra invertida), `\n` (nueva línea), `\r` (retorno de carro).
- **linesafe**: permite convertir retornos de carro en su etiqueta (`<BR/>`).

---

## EJEMPLO

```
<m4:item m4name="out_alias:person_alias!SCO_PERSON.SCO_PERSON_ID"/>

<% Integer iPersonID = new Integer(0);
<m4:item outputdef="out_alias" item="SCO_PERSON_ID"
 htmlsafe="false" jsafe="false" typename="number"
 var="iPersonID" vartype="Integer"/>
```

---

## OutputExec

Esta etiqueta muestra el resultado de la etiqueta **Exec** con el alias correspondiente.

```
OutputExec ::= <m4:outputexec
(m4alias="ExecAlias | alias="ExecAlias")
[m4varname = "NonDeclaredVariable" |
var = "DeclaredVariable" | vartype = "VariableType" |
m4format="FormatMask" | typename = "FormatType"]
/>
```

### Parámetros opcionales:

- **m4alias**: alias del método como se define en la etiqueta **Exec**.
- **alias**: alias del método. Utilice alias o m4alias, pero no ambos.
- **m4varname**: nombre de la variable de destino donde debería guardarse el valor devuelto por el método. La etiqueta define una variable Java del tipo `java.lang.String` donde se especifica este atributo, por lo que no debe predeclarar la variable.
- **var**: nombre de la variable de destino donde debería guardarse el valor devuelto por el método. Esta variable debe declararse previamente en la página. Debe predeclarar una variable y especificar su nombre en este atributo para que se guarde el resultado. El tipo de datos por defecto aceptado para la variable predeclarada es `java.lang.String`.
- **vartype**: tipo de datos para la variable en **var**. Se utiliza para cambiar el tipo de variable por defecto de **var**, es decir, *String*. Éstos son los posibles valores: *Integer* y *Double*. Le permite utilizar variables predeclaradas de los tipos *Integer* y *Double*.
- **m4format**: máscara que se va a utilizar para formatear el resultado del método. Consulte la sintaxis de la máscara en el apéndice.

- **typename:** nombre de un tipo de formateo. Entre los tipos de formateo existentes se incluye "DATE", "DATETEXT", "TIMESTAMP", "HOUR", "NUMBER" y "CURRENCY", y los valores para las máscaras asociadas se almacenan en el Meta4Object Configuración de la aplicación.

## EJEMPLO

```
<m4:outputexec m4alias="SHOW"/>
```

## Label

Esta etiqueta muestra el nombre de un nodo o elemento. Esta información incluye la descripción de estos elementos en vez de su código identificativo.

```
Label ::= <m4:label
 (m4name=[OutputDefAlias]:DataTagAlias!NODE.ITEM"
 outputdef="OutputDefAlias" get="item" item="ITEM"/>)
```

```
Label ::= <m4:label
 (m4name=[OutputDefAlias]:DataTagAlias!NODE[REC] "
 outputdef="OutputDefAlias" get="node"/>)
```

### Parámetros opcionales:

- **m4name:** descripción del elemento o el nodo, usando la notación especial. Dependiendo de la descripción, se recuperará la etiqueta del elemento o del nodo.
  - **DataTagAlias!NODE.ITEM:** Recupera la etiqueta **ITEM**.
  - **DataTagAlias!NODE[REC]:** Recupera la etiqueta **NODE** (REC puede ser cualquier valor).
- **outputdef:** parámetro que define los datos que se van a recuperar.
- **get:** Si el atributo get tiene el valor "node", se recupera la etiqueta del nodo al que se hace referencia en el parámetro outputdef. Si el atributo get tiene el valor "item", se utilizará el valor del atributo item como etiqueta del elemento que se ha de recuperar.
- **item :** nombre del elemento que se va a mostrar.



Puede utilizar el atributo m4method, o los distintos atributos alias, node y m4object, según el estilo de código que prefiera. Ambos grupos de atributos juntos son incompatibles.



- **var**: nombre de la variable de destino donde debería guardarse el contenido de la etiqueta. Esta variable debe declararse previamente en la página. Debe predeclarar una variable y especificar su nombre en este atributo para que se guarde el resultado. El tipo de datos por defecto aceptado para la variable predeclarada es `java.lang.String`.
- **vartype**: tipo de datos para la variable en **var**. Se utiliza para cambiar el tipo de variable por defecto de **var**. Éstos son los posibles valores: *Integer* y *Double*. Le permite utilizar variables predeclaradas de los tipos *Integer* y *Double*.
- **htmlsafe**: especifica si se debe convertir el resultado a un cadena segura HTML. Esto es útil porque si un elemento contiene un carácter como `&`, `<`, `>` o `"`, el navegador puede interpretar estos caracteres como parte de etiquetas HTML y provocar que la página no se vea correctamente. Establézcalo en *TRUE* para convertir los caracteres no seguros a sus equivalentes HTML seguros, de lo contrario, déjelo como *FALSE*. El atributo está establecido por defecto en *FALSE*.



Lo mismo ocurre para los caracteres ISO-8859-1 entre 161 y 255, que se convierten a la cadena equivalente HTML (á -> `&acute;`) y el símbolo del Euro, que se convierte a `&#0128`.

- **jsafe**: especifica si deberían librarse (anteponiendo una barra invertida) cadenas no seguras de Java o JavaScript como `'` (comillas simples), `"` (comillas dobles), `\` (barra invertida), `\n` (nueva línea), `\r` (retorno de carro).

---

## Ejemplo

Para el elemento:

```
<m4:label m4name="out_alias:person_alias!SCO_PERSON.N_PERSON_ESP"/>
```

Para el nodo:

```
<m4:label m4name="out_alias:person_alias!SCO_PERSON"/>
```

---

## Count

Esta etiqueta cuenta el número de registros de un nodo.

```
Count ::= <m4:count
 (m4name=" [OutputDefAlias:]DataTagAlias!NODE" |
 outputdef="OutputDefAlias")
 [m4place="(local|remote)"]
 [m4varname = "NonDeclaredVariable" |
 var = "DeclaredVariable" | vartype = "VariableType"]/>
```

### Parámetros opcionales:

- **m4name**: parámetro que especifica el nodo, usando la notación especial.
- **outputdef**: parámetro que especifica el nodo. Se utiliza cuando el recuento es local. Tampoco se puede utilizar junto con **m4name**.
- **m4place**: especifica si el recuento se hace sobre el conjunto local (LOCAL) o remoto (REMOTE). Por defecto, es sobre el remoto.
  - local: número de registros que se han solicitado de un nodo.
  - remote: número de registros que hay realmente en el nodo.
- **m4varname**: nombre de la variable de destino donde debería guardarse el valor del recuento del nodo. La etiqueta define una variable Java del tipo `java.lang.String` donde se especifica este atributo, por lo que no debe predeclarar la variable.
- **var**: nombre de la variable de destino donde debería guardarse el valor del recuento del nodo. Esta variable debe declararse previamente en la página. Debe predeclarar una variable y especificar su nombre en este atributo para que se guarde el resultado. El tipo de datos por defecto aceptado para la variable predeclarada es `java.lang.Integer`.
- **vartype**: tipo de datos para la variable en **var**. Se utiliza para cambiar el tipo de variable por defecto de **var**, es decir, *Integer*. Éstos son los posibles valores: **Double** y **String**. El atributo **vartype** permite utilizar variables predeclaradas de los tipos **Double** y **String**.
- **m4format**: máscara que se va a utilizar para formatear el resultado del recuento del nodo. Consulte la sintaxis de la máscara en el apéndice.
- **typename**: nombre de un tipo de formateo. Entre los tipos de formateo existentes se incluye "DATE", "DATETEXT", "TIMESTAMP", "HOUR", "NUMBER" y "CURRENCY", y los valores para las máscaras asociadas se almacenan en el Meta4Object Configuración de la aplicación.

---

## Ejemplo

```
<m4:count m4name="GEO!STD_MT_COUNTRY"/>
```

---

## Current

Esta etiqueta informa del registro actual de un nodo.

```
Current ::= <m4:current
 (m4name=" [OutputDefAlias:]DataTagAlias!NODE" |
 outputdef="OutputDefAlias")
 [m4varname = "NonDeclaredVariable" |
 var = "DeclaredVariable" | vartype = "VariableType"]/>
```

### Parámetros opcionales:

- **m4name**: parámetro que especifica el nodo, usando la notación especial.
- **outputdef**: parámetro que especifica el nodo. Tampoco se puede utilizar junto con **m4name**.
- **m4varname**: nombre de la variable de destino donde debería guardarse el valor del recuento del nodo. La etiqueta define una variable Java del tipo `java.lang.String` donde se especifica este atributo, por lo que no debe predeclarar la variable.
- **var**: nombre de la variable de destino donde debería guardarse el valor del recuento del nodo. Esta variable debe declararse previamente en la página. Debe predeclarar una variable y especificar su nombre en este atributo para que se guarde el resultado. El tipo de datos por defecto aceptado para la variable predeclarada es `java.lang.Integer`.
- **vartype**: tipo de datos para la variable en **var**. Se utiliza para cambiar el tipo de variable por defecto de **var**, es decir, *Integer*. Éstos son los posibles valores: **Double** y **String**. El atributo **vartype** permite utilizar variables predeclaradas de los tipos **Double** y **String**.

---

## EJEMPLO

```
<m4:current m4name="GEO!STD_MT_COUNTRY"/>
```

---

## GetFile

Esta etiqueta carga un archivo guardado en un elemento de Meta4Object e indica la ruta del archivo en el servidor de aplicaciones.

```
GetFile ::= <m4:getfile m4name="NAME"/>
```

### Parámetros obligatorios:

- **m4name**: nombre del elemento que contiene el archivo.

---

### EJEMPLO

```
<m4:getfile m4name="PERSON!SCO_PERSON.SCO_CURRICULUM"/>
```

---

## GetCurrency

Esta etiqueta recupera valores de elemento tipo moneda y, al mismo tiempo, le permite recuperar los demás atributos de moneda; es decir, la fecha de cambio, el tipo de cambio de la moneda y el identificador de la fuente del tipo de cambio (oficial o legal).

```
GetCurrency ::= <m4:getcurrency
 [m4currency_value=OutputDefAlias:DataTagAlias!NODE[REC].ITEM]
 [m4currency_id=OutputDefAlias:DataTagAlias!NODE[REC].ITEM]
 [m4currency_date=OutputDefAlias:DataTagAlias!NODE[REC].ITEM]
 [m4currency_type=OutputDefAlias:DataTagAlias!NODE[REC].ITEM]
/>
```

### Parámetros opcionales:

- **m4currency\_value**: valor del elemento de moneda.
- **m4currency\_id**: identificador de moneda aplicado al elemento de moneda.
- **m4currency\_date**: fecha de cambio de la moneda
- **m4currency\_type**: fuente de tipo de cambio de la moneda

---

### EJEMPLO

```
<m4:getcurrency m4currency_value="PERSON!ROOT.CURRENCY"/>
```

---

---

## M4Tags de iteración

### DataLoop

Esta etiqueta itera en todos o en algunos de los registros de un nodo. Realiza un movimiento local a cada registro del bucle para permitirle realizar operaciones como mostrar datos utilizando la etiqueta **item**.

```
Dataloop ::= <m4:dataloop outputdef="OutputDefAlias"
 [start = "X" | stop = "Y" | count = "N|*"]
</m4:dataloop>
```

#### Parámetros obligatorios:

- **outputdef**: parámetro que define los datos que se van a recuperar.

#### Parámetros opcionales:

- **start**: parámetro que inicia el bucle en ese punto concreto.
- **stop**: parámetro que finaliza el bucle en ese punto concreto.
- **count**: parámetro que inicia el bucle en todos los registros indicados en la etiqueta **outputdef**, contando hacia delante o hacia atrás en función del valor que se le haya asignado.
  - Si el valor es \*, el recuento incluye todos los registros y se hace hacia delante.
  - Si el valor es -\*, el recuento incluye todos los registros y se hace hacia atrás.

---

#### EJEMPLO

```
<m4:dataloop outputdef="out_alias" start="4" count="*">
 <m4:current outputdef="out_alias"/>:
 <m4:item m4name="out_alias:PERSON!STC_MT_TEST_PERSON.FIRST_NAME"/>
 <m4:item m4name="out_alias:PERSON!STC_MT_TEST_PERSON.LAST_NAME"/>

</m4:dataloop>
```

---

## DataLoopStop

Esta etiqueta detiene la repetición en la etiqueta **dataloop** padre.

```
Dataloop ::= <m4:dataloopstop [outputdef="OutputDefAlias"]>
```

### Parámetros opcionales:

- **outputdef:** Atributo que contiene el alias del OutputDef.

## Iterator

Esta etiqueta muestra una lista de filas parametrizables.

```
Iterator ::= <m4:iterator
 [m4rows=" [X| "*"] "]
 m4node="OutputDefAlias:DataTagAlias!NODE">
 <m4:param name="M4ITEM0" value="ITEMA"/>
 ...
 <m4:param name="M4ITEMN" value="ITEMN"/>
 ... [$CURRENT$] ... [$COUNT$] ... $M4ITEM0$
 ... $M4ITEMN$...
</m4:iterator>
```



La sintaxis de la etiqueta **dataloop** es más sencilla que la de la etiqueta **iterator**, por lo que muchas veces se prefiere el uso de la primera.

### Parámetros obligatorios:

- **m4node:** indica el nodo cuyos datos se muestran.
  - **OutputDefAlias:** alias asignado a la solicitud de datos de la etiqueta **OutputDef**.
  - **DataAlias:** nombre asignado a la etiqueta **DataDef**.
  - **NODE:** nombre del nodo.
- **param:** indica el elemento que se va a mostrar.
  - **name:** nombre asignado al elemento.



El nombre del parámetro no puede ser cualquier nombre, debe denominarse específicamente **M4ITEM0** o **M4ITEM1**, etc.

- **value**: elemento que se va a mostrar.
- **\$name\$**: muestra el elemento que indica el nombre.

#### Parámetros opcionales:

- **m4rows**: indica el número de filas que se va a mostrar. Por defecto, se muestran todas.
  - **\***: muestra todas.
  - **x**: muestra X filas.
- **\$CURRENT\$**: indica el número del registro actual.
- **\$COUNT\$**: indica el número total de registros.

#### EJEMPLO

```
<m4:iterator m4node="PARENT:GEO!STD_MT_COUNTRY">
 <m4:param name="M4ITEM0" value="STD_N_COUNTRY"/>
 $M4ITEM0$

</m4:iterator>
```

## Loop

Esta etiqueta permite realizar bucles. Introduzca cualquier M4Tag, etiqueta HTML o código Java en el cuerpo de la etiqueta. Si no especifica el incremento, el valor por defecto será **+1**.

```
Loop ::= <m4:loop
 from="i" to="j" [step="inc"] | [name = "IndexVarName"] >

</m4:loop>
```

#### Parámetros obligatorios:

- **from**: índice inicial
- **to**: índice final

**Parámetros opcionales:**

- **name:** nombre de la variable de índice que debe ser una cadena predeclarada. Si no está, se utiliza la variable m4lix por defecto.
- **step:** incremento
  - **+N:** incremento positivo.
  - **-N:** incremento negativo.

---

**EJEMPLO**

```
<m4:loop from="0" to="5">
 <m4:item m4name="GEO!STD_MT_COUNTRY[&VAR.m4lix].STD_N_COUNTRY"/>
</m4:loop>
```

---

---

**EJEMPLO**

En este ejemplo se combinan las etiquetas explicadas en este apartado en una página JSP que ejecuta un bucle.

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<HTML>
<BODY>
<m4:startpage m4task="subsession_alias"/>
<m4:beginjob/>
 <m4:datadef m4o="SCO_GEO_STRUCTURE" m4name="GEO"/>
 <m4:exec m4method="LOAD:GEO.LOAD"/>
 <m4:outputdef m4alias="PARENT">
 <m4:param name="M4NAME0" value="GEO!STD_MT_COUNTRY[*]"/>
 </m4:outputdef>
<m4:endjob/>
<m4:loop from="0" to="5">
 <m4:item m4name="GEO!STD_MT_COUNTRY[&VAR.m4lix].STD_N_COUNTRY"/>
</m4:loop>
<m4:endpage/>
</BODY>
</HTML>
```

---



---

## M4Tags de consulta de traducciones

Estas M4Tags realizan consultas de traducciones. Por ejemplo, obtienen el nombre de un código identificativo.

### AddTranslation

Esta etiqueta define una traducción.

```
AddTranslation ::= <m4:addtranslation m4translation="TRANS"/>
```

#### Parámetros obligatorios:

- **m4translation:** identifica el tipo de traducción como *NAME:M4O!NODE.METHODE#ITEM*.
  - **Name:** nombre de la traducción.
  - **M4O!NODE.METHOD:** método que realiza la traducción.
  - **Item:** elemento en que se guarda la traducción.

---

#### EJEMPLO

```
<m4:addtranslation m4translation=
 "TYPE:SCO_TR_X_KN_E_TYPE!SCO_TR_X_KN_E_TYPE
 !VALIDATE#SCO_NM_E_TYPE"/>
```

---

### FireTranslation

Esta etiqueta traduce el código. Se puede traducir uno o más códigos en la misma etiqueta.

```
FireTranslation ::= <m4:firetranslation>
<m4:param name="DATATAGLET!NODE[A].ITEMNAME1"
 value="NAMETRANSL:ARGMETHOD1@DataName!
 NODE[B].ITEM#ARGMETHOD2@DataName!
 [C].NODE_ITEM"/>
<m4:param name="DATATAGLET!NODE[A].ITEMNAME1"
 value="NAMETRANSL:ARGMETHOD1@DataName!
 NODE[Q].ITEM#ARGMETHOD2@DataName!
 [R].NODE_ITEM"/>
...
</m4:firetranslation>
```

**Parámetros obligatorios:**

- **param:** lista de definiciones de elementos.
  - **name:** elemento que se va a traducir.
  - **value:** nombre de la traducción y lista de valores, separados ambos por el carácter @.

---

**EJEMPLO**

```
<m4:firetranslation>
 <m4:param name="KU!SCO_KN_CHECK_KU_ADMINISTRATOR.SCO_E_TYPE_ID"
 value="TYPE:ARG_SCO_E_TYPE_ID#@ARG_SCO_E_TYPE_ID@
 KU!SCO_KN_CHECK_KU_ADMINISTRATOR.SCO_E_TYPE_ID"/>
</m4:firetranslation>
```

---

**GetTranslation**

Esta etiqueta presenta el código traducido de un elemento.

```
GetTranslation ::= <m4:gettranslation m4nametranslation="NAME"/>
```

**Parámetros obligatorios:**

- **m4nametranslation:** nombre del elemento como DATATAG!NODO[REC].ELEMENTO.

---

**EJEMPLO**

```
<m4:gettranslation m4nametranslation=
 "KU!SCO_KN_CHECK_KU_ADMINISTRATOR.SCO_E_TYPE_ID"/>
```

---

---

**EJEMPLO**

En este ejemplo se combinan las etiquetas de traducción en una página JSP que realiza una traducción.

```
<%@ taglib uri="M4Tags" prefix="M4" %>
<%@ page import="java.io.*, java.util.*, java.net.*" %>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html">
<title>Prueba de traducciones</title>
</head>
```

---

```

<body>
<m4:startpage m4task="tc_test"/>
<m4:beginjob/>
 <m4:datadef m4o="SCO_KN_CHECK_KU_ADMINISTRATOR" m4name="KU"/>
 <m4:exec m4method="EXECLOAD:KU.LOADONLY"/>
 <m4:addtranslation
 m4translation="TYPE:SCO_TR_X_KN_E_TYPE!SCO_TR_X_KN_E_TYPE.VALID
ATE
 #SCO_NM_E_TYPE"/>
 </m4:firetranslation>
 <m4:param name="KU!SCO_KN_CHECK_KU_ADMINISTRATOR.SCO_E_TYPE_ID"
value="TYPE:ARG_SCO_E_TYPE_ID@#ARG_SCO_E_TYPE_ID@
 KU!SCO_KN_CHECK_KU_ADMINISTRATOR.SCO_E_TYPE_ID"/>
 </m4:firetranslation>
 <m4:outputdef m4alias="out">
 <m4:param name="M4NAME0"
 value="KU!SCO_KN_CHECK_KU_ADMINISTRATOR[0-5]"/>
 </m4:outputdef>
<m4:endjob/>
<table>
 <m4:tag
m4name="out:KU!SCO_KN_CHECK_KU_ADMINISTRATOR.SCO_E_TYPE.ID"/>:
 <m4:gettranslation
 m4nametranslation="out:KU!SCO_KN_CHECK_KU_ADMINISTRATOR.
SCO_E_TYPE_ID"/>
</table>
<m4:endpage/>
</body>
</html>

```

---

## M4Tags de seguridad y sesión

Las etiquetas de seguridad ejecutan actualizaciones y comprobaciones de seguridad.

### M4Tags de seguridad

#### Logout

Esta etiqueta le permite finalizar una sesión.

```
Logout ::= <m4:logout/>
```

---

#### EJEMPLO

```
<m4:logout/>
```

---

#### CheckLink

Esta etiqueta comprueba la seguridad de un enlace. Este enlace se muestra cuando el usuario conectado cuenta con los permisos necesarios. Un usuario de cliente ligero tiene permisos para acceder a una página si la carpeta que la contiene se llama exactamente igual que una tarea de cliente ligero, que -en caso de estar definida con seguridad- además tiene asociado un rol que está entre los roles que tiene el usuario.

Así, por ejemplo, un usuario tiene permiso para acceder a `/servlet/CheckSecurity/JSP/task/page.jsp` si existe una tarea de código cliente ligero llamada "task" que tiene asociado un rol que está entre los roles del usuario.

#### Parámetros opcionales:

- **CheckSecurityURL**: enlace CheckSecurity que se va a comprobar. Este parámetro es incompatible con el atributo **taskname**, y se utiliza para mostrar un enlace A REF. a un CheckSecurity URL si el usuario tiene permisos para esa tarea.

---

## EJEMPLO

```
<m4:checklink>

</m4:checklink>
```

- **taskname:** identificador de la tarea para la que se van a comprobar los permisos.
  - Si el usuario tiene autorización para la tarea, aparece el cuerpo de la etiqueta.
  - Si el usuario no tiene autorización, no se muestra nada.
  - El atributo taskname prevalece sobre el formato del cuerpo.

---

## EJEMPLO

Sin el atributo taskname, sólo podrá comprobar etiquetas CheckSecurity A HREF.

```
<m4:checklink>

 Texto que se va a mostrar
</m4:checklink>
```

Con el atributo taskname

```
<m4:checklink taskname="tctools">
 |
 Cambio de contraseña
</m4:checklink>
```

---

No debe utilizarse este tag como única seguridad sin crear carpetas asociadas a tareas de cliente ligero, puesto que el tag no impediría la ejecución de una página para la que el usuario no tuviera roles.

Por ejemplo, si queremos impedir que un usuario pueda acceder a la tarea "confidential", y pretendemos utilizar el tag <m4:checklink> para tal fin, escribiendo la siguiente sintaxis:

```
<m4:checklink taskname="confidential">
 http://servlet/CheckSecurity/JSP/task/page.jsp
</m4:checklink>
```

Esto no sería recomendable, puesto que un usuario malintencionado podría escribir directamente http://servlet/CheckSecurity/JSP/task/page.jsp para acceder fraudulentamente a la tarea "task", si no se tiene definida

correctamente la seguridad sobre dicha tarea.

## SetRole

Utilice esta etiqueta para asignar roles. Si no se utiliza, el sistema toma por defecto el rol predeterminado. La etiqueta sólo puede tomar su valor una vez: después de la conexión y antes de la M4Tag **startpage** de la primera página.

```
SetRole ::= <m4:setrole m4role="ROLE"/>
```

### Parámetros obligatorios:

- **m4role**: rol que se va a asignar.

---

### EJEMPLO

```
<m4:setrole m4role="M4ADM">
```

---

## SetOrganization

```
SetOrganization ::= <m4:setorganization
m4organization="ORGANIZATION"/>
```

Utilice esta etiqueta para asignar una organización. Se utiliza después de la conexión y antes de la M4Tag **startpage** de la primera página.

### Parámetros obligatorios:

- **m4organization**: organización que se va a asignar.

---

### EJEMPLO

```
<m4:setorganization m4organization="ORGANIZATION_J">
```

---

## M4Tags de bolsas de sesión

Estas etiquetas le permiten trabajar con bolsas de sesión. Son contenedores en los que se almacenan pares de elemento – valor del objeto de sesión HTTP.

## PutBagValue

Esta etiqueta guarda en la bolsa un par elemento-valor.

```
PutBagValue ::= <m4:putbagvalue
m4key="KEY"
m4value="VALUE"
>
```

### Parámetros obligatorios:

- **m4key**: elemento
- **m4value**: valor

---

**EJEMPLO**

```
<m4:putbagvalue m4key="USER" m4value="JOHN"/>
```

---

## GetBagValue

Esta etiqueta recupera los valores de los elementos.

```
GetBagValue ::= <m4:getbagvalue m4key="KEY" m4varname="PreDecVar"/>
```

**Parámetros obligatorios:**

- **m4key**: clave del elemento.

**Parámetros opcionales:**

- **m4varname**: nombre de la variable en la que se almacena el valor del elemento.

---

**EJEMPLO**

```
<m4:getbagvalue m4key="USER"/>
```

---

## RemoveBagValue

Esta etiqueta elimina un elemento de la bolsa.

```
RemoveBagValue ::= <m4:removebagvalue m4key="KEY"/>
```

**Parámetros obligatorios:**

- **m4key**: elemento.

---

**EJEMPLO**

```
<m4:removebagvalue m4key="USER"/>
```

---



---

## EJEMPLO

En este ejemplo se combinan las etiquetas de bolsas.

### bag01.jsp

Página principal que muestra los datos de un Meta4Object y guarda los valores en la bolsa.

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<%@ page import="java.io.*, java.util.*, java.net.*" %>
<HTML>
<BODY BGCOLOR="#FFFFFF">
Ejemplo del uso de bolsas, toma 1

<m4:startpage m4task="subsession_alias"/>
<m4:beginjob/>
 <m4:datadef m4o="SCO_GEO_STRUCTURE" m4name="GEO"/>
 <m4:exec m4method="LOAD:GEO.LOAD"/>
 <m4:outputdef m4alias="PARENT">
 <m4:param name="M4NAME0"
 value="GEO!STD_MT_COUNTRY[*].STD_N_COUNTRY"/>
 </m4:outputdef>
 <m4:putbagvalue m4key="CLAVE" m4value="VALOR"/>
<m4:endjob/>
enlace
<P>MY DATA:
<m4:item m4name="PARENT:GEO!STD_MT_COUNTRY[0].STD_N_COUNTRY"/>
<m4:endpage/>
</BODY>
</HTML>
```

### bag02.jsp

Página auxiliar que muestra el valor de la bolsa.

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<HTML>
<HEAD><TITLE> Países </TITLE></HEAD>
<BODY BGCOLOR="#FFFFFF">
Ejemplo del uso de bolsas, toma 2

<m4:startpage m4task="subsession_alias"/>
<m4:beginjob/>
 <m4:datadef m4o="SCO_GEO_STRUCTURE" m4name="GEO"/>
 <m4:exec m4method="LOAD:GEO.LOAD"/>
 <m4:outputdef m4alias="PARENT">
```

```
<m4:param name="M4NAME0"
 value="GEO!STD_MT_COUNTRY[*].STD_N_COUNTRY"/>
</m4:outputdef>
Valor de la bolsa: <m4:getbagvalue m4key="KEY"/>
<m4:endjob/>
<P>MY DATA:
<m4:item m4name="PARENT:GEO!STD_MT_COUNTRY[0].STD_N_COUNTRY"/>
<m4:endpage/>
</BODY>
</HTML>
```

---

## ClearBag

```
ClearBag ::= <m4:clearbag/>
```

Esta etiqueta vacía la bolsa. Tenga cuidado con esta operación. Utilícela sólo si quiere finalizar la sesión de usuario.

---

### EJEMPLO

```
<m4:clearbag/>
```

---

---

## Etiquetas de informes

### Informes

#### ExecuteReport

Esta etiqueta accede al Meta4Object que se utiliza para crear el informe, construye el informe a partir del Meta4Object y carga los datos de acuerdo con lo especificado. Cuando finaliza la ejecución, se descarga y se elimina. No es posible pasar a la etiqueta una instancia de un Meta4Object que se creó o cargó anteriormente.

```
ExecuteReport ::= <m4:executereport
 idreport="..."
 sysentence="..."
 [outputtype="..."]
 [otherparams="..." />
```

#### Parámetros obligatorios:

- **idreport**: Identificador del informe que se va a ejecutar.
- **sysentence**: Sentencia que contiene la cadena de parámetros del Meta4Object Sistema de informes.

#### Parámetros opcionales:

- **outputtype**: El tipo de informe que se va a generar, que por defecto es HTML.
- **otherparams**: parámetros auxiliares para el informe.

### EJEMPLO DE USO DE LA M4TAG EXECUTEREPORT, PREDECLARACIÓN DEL TIPO DE SALIDA DEL INFORME, PREPARACIÓN DEL RESULTADO DE LOS DATOS DE ANTEMANO

```

<%
// Definir el tipo de salida de un informe
String sOutputType;
switch (Integer.parseInt(sID_OUTPUT))
{
 case 1:
 sOutputType = "HTML";
 break ;
 case 2:
 sOutputType = "PDF";
 break ;
 case 3:
 sOutputType = "TXT";
 break ;
 default:
 sOutputType = "HTML";
}

%>

<m4:item outputdef="DataPrepare" item="PARAM_STRING"
m4varname="sParameters" />
<script language=javascript>
 {
 var sReportURL = "";
 var sType = "";
 var sAux = "";

//sAux = 'ZZ_QUERY1;ZZ_QUERY1_SCH_TIS$ZZ_QUERY1_SCH_TIS PARAMETER
"Hello";ZZ_QUERY1_SCH_TIS PARAMETRO "Joe ";ZZ_QUERY1_SCH_TIS
PARAMETER1 "Bloggs";$'

// Ejecutar informe y salida
 sReportURL = "<m4:executereport
 idreport=<%=sID_REPORT %>
 syssentence= <%=sParameters%>
 outputtype= <%=sOutputType%>
 otherparams=<%=sREPORTPARAM %> />

```

---

## EJEMPLO DEL USO DE M4TAGS JOB CON EL M4TAG EXECUTEREPORT

```
<%!
 private static String getStringValue(String sValue) {
 return ((sValue == null) || sValue.equals("") ||
 sValue.equals("null")) ? null : sValue;
 }
%>

<%
 String sID_T3 = getStringValue(request.getParameter("txtIdT3"));
 String sID_REPORT =
 getStringValue(request.getParameter("txtIdReport"));
 String sID_OUTPUT =
 getStringValue(request.getParameter("txtIdOutput"));
 String sREPORTPARAM =
 getStringValue(request.getParameter("txtReportParam"));
 String sAllPARAM =
 getStringValue(request.getParameter("txtAllParam"));
 String sID_REPORT_TYPE =
 getStringValue(request.getParameter("txtIdReportType"));
%>

<m4:job>
 <m4:exec alias="Prepare" m4object="PubReportsCL"
 node="API_PUB_REPORTS_CL" method="API_GET_PARAM_STRING">
 <m4:param name="P_ID_T3" value="<%=sID_T3%>" />
 <m4:param name="P_PARAM_STRING" value="<%=sAllPARAM%>" />
 </m4:exec>
 <m4:outputdef m4alias="DataPrepare" m4object="PubReportsCL"
 node="SRP_PARAM_LIST" records="*" />
</m4:job/>
```



Importante: Cuando se ejecuta un recibo en PDF desde el autoservicio del empleado (SSE), no se ven datos si la fecha actual está dentro del periodo de aviso de expiración de la contraseña. Esto sucede solamente con el recibo en PDF; desde el cliente distribuido, la ejecución mediante el M4Tag executereport funciona correctamente, aunque la fecha actual se encuentre dentro del periodo de aviso de expiración. Para evitarlo, basta sustituir `<m4:startpage m4task="REPORTS"/>` por `<m4:startpage m4task="REPORTSEC"/>` en el código que se utiliza para lanzar el recibo del SSE desde la página `sse_g2_rec.jsp`, según se muestra a continuación:

```
<m4:startpage m4task="REPORTSEC"/>
```

```
<script>OpenReports('<m4:executereportsec idreport="SSE_RECIBO_1"
syssentence="<%s=SysSentence%> outputtype="PDF" otherparams "/
#AUTOLOAD:DESIGN:OFF">
```

---

## M4Tags de herramientas

### Xmltransformer

Esta etiqueta convierte el lenguaje XML resultante de **Job** en HTML u otro diferente, como WML. Para ello, utiliza la hoja de estilo XSL indicada en la M4Tag. Para que funcione esta etiqueta, debe haberse establecido la propiedad DOXML en M4ENABLED en la etiqueta **SetProperty**. Debe utilizarse justo después de Job o BeginJob.

```
Xmltransformer ::= <m4:xmltransformer m4xsl="XSL"/>
```

#### Parámetros obligatorios:

**m4xsl:** ruta de acceso a la plantilla XSL.

---

#### EJEMPLO

```
<m4:startpage m4task="TEST"/>
<m4:beginjob/>
<m4:datadef m4o="DEMO_TC_TOOL" m4name="M4ONAME"/>
<m4:exec m4method="LOAD:M4ONAME.LOAD"/>
<m4:outputdef m4alias="M4O_ALIAS">
<m4:param name="M4NAME0" value="M4ONAME!DEMO_TC_TOOL_ROOT[0]"/>
</m4:outputdef>
<m4:setproperty m4property="DOXML" m4value="M4ENABLED"/>
<m4:endjob/>
<m4:xmltransformer m4xsl="/templates/test.xsl"/>
<m4:move>
<m4:param name="M4ONAME" value="M4O_ALIAS:DEMO_TC_TOOL_ROOT[0]"/>
</m4:move>
<m4:iterator m4rows="*"
 m4node="M4O_ALIAS:CHANNEL_NAME!DEMO_TC_TOOL_ROOT">
 <m4:param name="M4ITEM0" value="STD_N_FIRST_NAME"/>
 <m4:param name="M4ITEM1" value="STD_N_FAMILY_NAME_1"/>
 Persona con nombre $M4ITEM0$ y apellido $M4ITEM1$
 </m4:iterator>
<m4:endpage/>
```

---

## Conditional

```
Conditional ::= <m4:conditional
 match="X" [include="Y|*" | exclude="Z|*"] ">
</m4:conditional>
```

Esta etiqueta ofrece un mecanismo flexible para mostrar su cuerpo en función del valor de sus parámetros.

### Parámetros obligatorios:

- **match**: contiene un valor que deben tener los elementos jerárquicos.

### Parámetros opcionales:

- **include**: cuando hay varios valores, éstos se separan mediante comas.
- **exclude**: cuando hay varios valores, éstos se separan mediante comas.



Debe haber al menos uno de los parámetros include o exclude.

Si no coincide ningún otro valor, lo hará el valor asterisco ("\*"). Uno de los parámetros **include** o **exclude** debe tener "\*" como valor. Si no se especifica, su valor por defecto es "\*".

Los elementos individuales de un valor jerárquico de los elementos **include**, **exclude** o **match** deben estar separados por dos puntos (':').

Si el parámetro **match** especifica un valor que coincide con todos los elementos jerárquicos de un valor de **include** o **exclude**, pero también incluye más elementos jerárquicos, se seguirá considerando como una coincidencia.

### EJEMPLO

```
<m4:conditional
include="Europa" exclude="Europe:Alemania,*" match="Europa:Francia">
Esto se mostrará: Francia está en Europa y no se ha excluido.
</m4:conditional>

<m4:conditional
include="Europa" exclude="Europa:Italia,*"
match="Europa:Italia:Roma">
Esto no se mostrará: Roma está en Italia, que ha sido excluida.
</m4:conditional>
```



---

## Input

La M4Tag **Input** introduce una etiqueta input en la página con el atributo “value” que contiene el cuerpo de la tarea.

```
Input ::= <m4:input name="Name" type="Type"
 [value = "Value" | match = "Match" | disabled = "Disabled" |
 readonly = "Readonly" | size = "Size" | maxlength = "MaxLength" |
 src = "Src" | tabindex = "TabIndex" | class = "Class" | style =
 "Style" | onclick = "OnClick" | onblur = "OnBlur" | onchange =
 "OnChange" | onfocus = "OnFocus" | onselect = "OnSelect"] />
```

En la etiqueta **Input**, la presencia de un atributo “value” siempre sobrescribirá cualquier atributo “value” creado del cuerpo.

Si el atributo “type” es “checkbox” o “radio”, el cuerpo de la etiqueta se interpretará del siguiente modo (esto quedará sobrescrito por la presencia del atributo “checked”):

- Si el contenido coincide con el atributo “match”, se activa la casilla de verificación o el botón de radio.
- En caso contrario, la casilla de verificación o el botón de radio no se activa.

### Parámetros obligatorios:

- **name**: nombre de la variable del formulario de entrada, que suele ser una línea de texto.
- **type**: tipo de entrada, como botón, casilla de verificación, archivo, oculto, imagen, contraseña, radio, restablecer, enviar y texto. Para obtener más información, consulte las especificaciones de HTML estándar.

### Parámetros opcionales:

- **value**: valor del objeto Elimina cualquier valor que se haya creado del cuerpo.
- **checked**: si el tipo de entrada es una casilla de verificación (checkbox), indica si debería estar activada por defecto. Si el elemento u objeto debe estar activado, establézcalo en *TRUE*.
- **disabled**: si el tipo de entrada es una casilla de verificación (checkbox) o un botón de radio (radio button), indique *TRUE* si debe estar activado por defecto.
- **readonly**: si el tipo de entrada es texto (text) o una contraseña (passwd), indique *TRUE* si el usuario no tiene permiso para modificarlo.
- **size**: tamaño en caracteres del campo de entrada.
- **maxlength**: indica el número máximo de caracteres permitidos.

- **src**: si la entrada es de tipo imagen (image), indique la fuente. Los URL pueden ser absolutos o relativos.
- **tabindex**: indica el índice de tabulación del elemento de entrada.
- **styleclass**: indica el estilo CSS que se ha de aplicar al elemento de entrada.
- **class**: atributo que no se debe utilizar porque no funciona en Tomcat, tiene el mismo significado que styleclass, y se ha conservado por compatibilidad con JRun.
- **onclick**: controlador de JavaScript para el evento onclick que hace algo cuando el usuario hace clic en ese elemento.
- **onblur**: controlador de JavaScript para el evento onblur que hace algo cuando el usuario deja de estar colocado sobre el elemento.
- **onchange**: controlador de JavaScript para el evento onchange que hace algo cuando el usuario deja de estar colocado sobre el elemento y su valor ha cambiado.
- **onfocus**: controlador de JavaScript para el evento onfocus que hace algo cuando el usuario se coloca sobre ese elemento.
- **onselect**: controlador de JavaScript para el evento onselect que hace algo cuando el usuario selecciona ese elemento.
- **match**: indica el valor con el que debe coincidir el cuerpo antes de que se active la casilla de verificación o el botón de radio.

## Option

La etiqueta option coloca una etiqueta HTML option en la página y establece el parámetro **selected** basándose en parte del cuerpo de la etiqueta.

```
Option ::= <m4:option
 [value = "Value" | selected = "Selected" | match = "Match"]
</m4:option>
```

Si se encuentra el carácter '~' en el cuerpo de la etiqueta, el contenido que haya antes de ese carácter '~' será el parámetro "value". El texto que aparezca después de '~' permanecerá en el cuerpo de la etiqueta.

### Parámetros opcionales:

- **value**: valor de la variable del formulario. Si no se especifica, el contenido del cuerpo de la etiqueta que haya antes del carácter '~' será el valor.

- 
- **selected**: actívelo si se va a seleccionar la opción. Deshará cualquier coincidencia realizada en el cuerpo.
  - **match**: valor con el que debe coincidir el parámetro **value** para que la opción esté seleccionada.

---

**EJEMPLO: USO DE LAS ETIQUETAS INPUT Y OPTION**

```
<table>
<form action="/servlet/login" method="post" name="data">
<tr>
 <td>user</td>
 <td><m4:input type="text" size="20" name="_USER" value="M4ADM"
 readonly="true"/></td>
</tr>
<tr>
 <td>password</td>
 <td><m4:input type="password" size="20" name="_PASSWD"/></td>
</tr>
<tr><td>lang: <select name="_LANG">
<m4:option>2</m4:option>
<m4:option>3</m4:option>
<m4:option>4</m4:option>
</select></td></tr>
<tr><td><m4:input type="submit" name="form-submit" value=" OK "/></td></tr>
</form>
</table>
```

---

## Img

La etiqueta HTML img ha sido encapsulada por la M4Tag **Img**.

```
Img ::= <m4:img
 [src = "Src" | alt = "Alt" | match = "Match" | width = "Width" |
 height = "Height" | style = "Style" | class = "Class"]>
 ...
</m4:img>
```

Es útil cuando la ubicación inicial de la imagen es dinámica. El cuerpo de la etiqueta indica la ubicación de la imagen, es decir, que sigue una notación más coherente con XHTML.

---

### Parámetros opcionales:

- **src**: especifica la ubicación inicial de la imagen. Si se utiliza, se empleará en lugar del contenido del cuerpo de la etiqueta.
- **alt**: especifica texto alternativo para describir la imagen.
- **width**: especifica el ancho de la imagen.
- **height**: especifica el alto de la imagen.
- **style**: especifica la información sobre el estilo de la imagen.
- **class**: indica la clase CSS que se ha de aplicar a la imagen.

---

### EJEMPLO: USO DE LA ETIQUETA **IMG**

Esta etiqueta `img` hace referencia a la ubicación de una imagen:

```
<m4:img
 src="http://www.google.com/images/logo.gif"
 alt="Éste es el logotipo de Google"/>
```

---

## Etiqueta

La M4Tag genérica llamada `Tag` se puede usar para representar cualquier etiqueta HTML. Su uso permitirá ser más coherente con XHTML cuando los valores de los atributos de la etiqueta sean dinámicos.

```
Tag ::= <m4:tag name="HTMLTagName">
 ...
</m4:tag>
```

El cuerpo de esta etiqueta (hasta `~/`) serán los atributos de la etiqueta HTML generada. El texto que aparezca después de `~/` permanecerá en el cuerpo. Si no se encuentra `~/` en el cuerpo, se da por hecho que la etiqueta no tiene cuerpo, por lo que todo el cuerpo se interpreta como atributos de la etiqueta.

### Parámetros obligatorios:

- **name**: nombre de la etiqueta HTML.

---

### EJEMPLO

Uso de `m4:tag`: `<m4:tag name="B"> ~/Esto está en negrita</m4:tag>`

---

## M4Tags de configuración de la aplicación

Estas etiquetas se utilizan para acceder al objeto Java de Configuración de la aplicación (instancia del servidor Web del Meta4Object SAV\_PARAMS).

### GetApplParam

Esta etiqueta recupera el valor de un determinado parámetro de la aplicación.

```
GetApplParam ::= <m4:getapplparam
 section="MY_SECTION"
 key="MY_KEY"
 [output="OUTPUT_TYPE"] />
```

#### Parámetros obligatorios:

- **section:** nombre de la sección
- **key:** nombre de la clave

Cuando se genera una salida, el tipo de salida puede ser:

- **page:** la salida se escribe directamente en la página HTML.
- **jsp:** la salida se deja en un parámetro de contexto JSP.
- **both:** la salida se envía a ambos dispositivos HTML y JSP. Éste es el valor por defecto.

#### EJEMPLOS DE CÓMO OBTENER VALORES Y GENERAR SALIDAS EN DISTINTOS FORMATOS

```
<m4:getapplparam section="FORMAT" key="DATE"/>
<m4:getapplparam section="FORMAT" key="DATE" output="page"/>
<m4:getapplparam section="FORMAT" key="DATE" output="jsp"/>
<m4:getapplparam section="FORMAT" key="DATE" output="both"/>
```

---

## SetApplParam

Esta etiqueta establece un nuevo valor para un determinado parámetro de la aplicación. Este nuevo valor se almacena en el concepto ID\_APP\_USER.

```
SetApplParam ::= <m4:setapplparam
 section="MY_SECTION"
 key="MY_KEY"
 value="MY_NEW_VALUE" />
```

### Parámetros obligatorios:

- **section:** nombre de la sección
- **key:** nombre de la clave
- **value:** nuevo valor

---

### EJEMPLO

```
<m4:setapplparam section="FORMAT" key="NUMBER"
value="<%=sNewParamValue%"/>
```

---

## MSetApplParam

La etiqueta msetapplparam le permite establecer varios parámetros de aplicación en una única transacción.

```
MSetApplParam ::= <m4:msetapplparam section=" MY_SECTION ">
 <m4:applparam key="MY_KEY_1" value="MY_VALUE_1" />
 <m4:applparam key="MY_KEY_2" value="MY_VALUE_2" />
</m4:msetapplparam>
```

### Parámetros obligatorios:

- **section:** nombre de la sección
- **key:** nombre de la clave
- **value:** nuevo valor

---

**EJEMPLO**

```
<m4:msetapplparam section="FORMAT">
 <m4:applparam key="NUMBER" value="<%=sNewParamValue%>" />
 <m4:applparam key="DATE" value="<%=sNewParamValue%>" />
</m4:msetapplparam>
```

---

## DeleteApplParam

Esta etiqueta borra el valor asignado al parámetro de aplicación dado con el concepto ID\_APP\_USER. Después de la ejecución, se asigna el valor de este parámetro al concepto de mayor prioridad .

```
DeleteApplParam ::= <m4:deleteapplparam
 section="MY_SECTION"
 key="MY_KEY"/>
```

### Parámetros obligatorios:

- **section**: nombre de la sección
- **key**: nombre de la clave

---

**EJEMPLO**

```
<m4:deleteapplparam section="FORMAT" key="NUMBER"/>
```

---



---

## EJEMPLO USO DE M4TAGS DE CONFIGURACIÓN DE LA APLICACIÓN EN UNA PÁGINA HTML

```
<html>
<body>
 <center><h1>Etiquetas de configuración de la aplicación:</h1></center>
 <center><h2>GET y SET - Ejemplos.</h2></center>
 <%-- [=====] --%>

 Valor actual del parámetro NUMBER.

section="FORMAT" key="NUMBER" value = <m4:getapplparam
section="FORMAT" key="NUMBER"/>
 <%-- [=====] --%>

 <% String sNewParamValue = new String(new Long(new
java.util.Date().getTime()).toString()); %>
 Establecer nuevo valor del parámetro NUMBER en el valor
 <%=sNewParamValue%> .
 <m4:setapplparam section="FORMAT" key="NUMBER"
value="<%=sNewParamValue%>" />
 <%-- [=====] --%>

 Nuevo valor del parámetro NUMBER.

section="FORMAT" key="NUMBER" value = <m4:getapplparam
section="FORMAT" key="NUMBER"/>
 <%-- [=====] --%>
 <center><h2>DELETE - Ejemplo.</h2></center>
 <%-- [=====] --%>

 <% String sNewParamValue = new String(new Long(new
java.util.Date().getTime()).toString()); %>
 Establecer un valor de usuario del parámetro NUMBER en el valor
 <%=sNewParamValue%> .
 <m4:setapplparam section="FORMAT" key="NUMBER"
value="<%=sNewParamValue%>" />
 <%-- [=====] --%>

 Nuevo valor del parámetro NUMBER.

section="FORMAT" key="NUMBER" value = <m4:getapplparam
section="FORMAT" key="NUMBER"/>
 <%-- [=====] --%>


```

```
Borrar valor de usuario.
<m4:deleteapplparam section="FORMAT" key="NUMBER"/>
<%-- [=====] --%>

Valor por defecto del parámetro NUMBER.

section="FORMAT" key="NUMBER" value = <m4:getapplparam
section="FORMAT" key="NUMBER"/>
<%-- [=====] --%>
<center><h2>MSETAPPLPARAMS - Ejemplo</h2></center>
<%-- [=====] --%>

Valor actual del parámetro NUMBER.

section="FORMAT" key="NUMBER" value = <m4:getapplparam
section="FORMAT" key="NUMBER"/>
<%-- [=====] --%>

<% String sNewParamValue = new String(new Long(new
java.util.Date().getTime()).toString()); %>
Establecer nuevo valor del parámetro NUMBER en el valor
<%=sNewParamValue%> .
<m4:setapplparam section="FORMAT" key="NUMBER"
value="<%=sNewParamValue%>"/>
<%-- [=====] --%>

Nuevo valor del parámetro NUMBER.

section="FORMAT" key="NUMBER" value = <m4:getapplparam
section="FORMAT" key="NUMBER"/>
</body>
</html>
```

---

## Referencia rápida de M4Tags

En este apartado se incluye una guía rápida de todos las M4Tags explicadas a lo largo de este capítulo.

Etiqueta	Descripción	Sintaxis
<b>AddTranslation</b>		
	Define un elemento que se va a traducir.	<code>&lt;m4:addtranslation m4translation="TRANS"/&gt;</code>
<b>BeginJob</b>		
	Inicia una transacción.	<code>&lt;m4:beginjob/&gt;</code>
<b>CheckLink</b>		
	Comprueba la seguridad de un enlace.	<code>&lt;m4:checklink [taskname="TaskIdentifier"]&gt;     &lt;a href="URL"&gt;&lt;/a&gt; &lt;/m4:checklink&gt;</code>
<b>ClearBag</b>		
	Borra la bolsa de un usuario.	<code>&lt;m4:clearbag/&gt;</code>
<b>Conditional</b>		
	Permite mostrar su cuerpo en función del valor de sus parámetros.	<code>&lt;m4:conditional     match="X" [include="Y *"   exclude="Z *"] "&gt; &lt;/m4:conditional&gt;</code>
<b>Count</b>		
	Indica el número de registros.	<code>&lt;m4:count     ( m4name=" [OutputDefAlias:]DataTagAlias!NODE"         outputdef="OutputDefAlias")     [ m4place="(local remote)" ]     [ m4varname = "NonDeclaredVariable"         var = "DeclaredVariable"   vartype = "VariableType" ]/ &gt;</code>
<b>Current</b>		
	Indica la situación actual del registro actual.	<code>&lt;m4:current     ( m4name=" [OutputDefAlias:]DataTagAlias!NODE"         outputdef="OutputDefAlias")     [ m4varname = "NonDeclaredVariable"         var = "DeclaredVariable"   vartype = "VariableType" ]/ &gt;</code>

Etiqueta	Descripción	Sintaxis
<b>DataDef</b>		
	Ofrece acceso a los datos del servidor de aplicaciones Meta4.	<pre> &lt;m4:datadef m4o="M4O"   m4name="ALIAS"   [m4start_date="..."]   [m4end_date="..."]   [m4id_rsm="..."]   [m4find="..."]   [m4id_app_role="..."]   [m4preserve="..."]   [m4id_organization="..."] /&gt; </pre>
<b>DeleteAppiParam</b>		
	Borra el valor asignado al parámetro de la aplicación.	<pre> &lt;m4:deleteapplparam section="MY_SECTION"   key="MY_KEY" /&gt; </pre>
<b>EndJob</b>		
	Finaliza una transacción y comprueba que no hay errores.	<pre> &lt;m4:endjob   [ [ m4alias="exec_alias"]   m4check="ERROR WARNING SUCCESS"   m4url="url" ] /&gt; </pre>
<b>EndPage</b>		
	Finaliza la página JSP.	<pre> &lt;m4:endpage/&gt; </pre>
<b>Exec (sin parámetros)</b>		
	Ejecuta métodos.	<pre> &lt;m4:exec   m4method=" [ExecAlias:]DataTagAlias!NODE[REC].METHOD"     [alias="ExecAlias"] m4object="DataTagAliasAlias"   node="NODE" method="METHOD" ) /&gt; </pre>
<b>Exec (con parámetros)</b>		
	Ejecuta métodos.	<pre> &lt;m4:exec   (m4method=" [ExecAlias:]DataTagAlias!NODE[REC].METHOD"     [alias="ExecAlias"] m4object="DataTagAliasAlias"   node="NODE" method="METHOD") &gt;   &lt;m4:param name="ARGUMENT1" value="value1" /&gt;   ...   &lt;m4:param name="ARGUMENTN" value="valueN" /&gt; &lt;/m4:exec&gt; </pre>
<b>ExecuteReport</b>		

Etiqueta	Descripción	Sintaxis
	Crea un informe, construye el propio informe desde el Meta4Object y carga los datos de acuerdo con lo especificado.	<pre>&lt;m4:executereport     idreport="XXX"     sysentence="XXX"     outputtype="XXX"     otherparams="XXX"/&gt;</pre>
<b>Filter</b>		
	Filtra registros.	<pre>&lt;m4:filter     m4name="DataTagAlias!NODE.FilterAlias"     m4filter="&lt;%= strM4Filter %&gt;"/&gt;</pre>
<b>FireTranslation</b>		
	Realiza traducciones de elementos.	<pre>&lt;m4:firetranslation&gt;     &lt;m4:param name="DataTagAlias!NODE[A].ITEMNAME1"         value="NAMETRANSL:ARGMETHOD1@DataName!         NODE[B].ITEM#ARGMETHOD2@DataName!         [C].NODE_ITEM"/&gt;     &lt;m4:param name="DataTagAlias!NODE[P].ITEMNAME2"         value="NAMETRANSL:ARGMETHOD1@DataName!         NODE[Q].ITEM#ARGMETHOD2@DataName!         [R].NODE_ITEM"/&gt;     ... &lt;/m4:firetranslation&gt;</pre>
<b>GetApplParam</b>		
	Recupera el valor de un determinado parámetro de la aplicación.	<pre>&lt;m4:getapplparam     section="MY_SECTION"     key="MY_KEY"     [output="OUTPUT_TYPE"] /&gt;</pre>
<b>GetBagValue</b>		
	Obtiene un valor de la bolsa del usuario.	<pre>&lt;m4:getbagvalue m4key="KEY" m4varname="PreDecVar"/&gt;</pre>
<b>GetCurrency</b>		
	Obtiene datos de un elemento de tipo moneda.	<pre>&lt;m4:getcurrency  [m4currency_value=OAlias:DataTagAlias!NODE[REC].ITEM] [m4currency_id=OAlias:DataTagAlias!NODE[REC].ITEM]  [m4currency_date=OAlias:DataTagAlias!NODE[REC].ITEM]  [m4currency_type=OAlias:DataTagAlias!NODE[REC].ITEM] /&gt;</pre>
<b>GetFile</b>		

Etiqueta	Descripción	Sintaxis
	Recupera un archivo e imprime su nombre.	<code>&lt;m4:getfile m4name="NAME"/&gt;</code>
<b>GetTranslation</b>		
	Obtiene el código traducido de un elemento.	<code>&lt;m4:gettranslation m4nametranslation="NAME"/&gt;</code>
<b>Img</b>		
	Encapsula la etiqueta HTML img.	<pre> &lt;m4:img [src = "Src"   alt = "Alt"   match = "Match"   width = "Width"   height = "Height"   style = "Style" class = "Class"]&gt; ... &lt;/m4:img&gt; </pre>
<b>Input</b>		
	Introduce una etiqueta input en la página con el atributo "value" que contiene el cuerpo de la tarea.	<pre> &lt;m4:input name="Name" type="Type" [ value = "Value"   match = "Match"   disabled = "Disabled"   readonly = "Readonly"   size = "Size"   maxlength = "MaxLength"   src = "Src"   tabindex = "TabIndex"   class = "Class"   style = "Style"   onclick = "OnClick"   onblur = "OnBlur"   onchange = "OnChange"   onfocus = "OnFocus"   onselect = "OnSelect" ] /&gt; </pre>
<b>Item</b>		
	Indica el valor de un elemento.	<pre> &lt;m4:item m4name=" [OutputDefAlias]:DataTagAlias!NODE[REC].ITEM"   outputdef="OutputDefAlias" item="ITEM" record="REC" ) [ m4varname = "NonDeclaredVariable"   var = "DeclaredVariable"   vartype = "VariableType"   m4format="FormatMask"   typename = "FormatType"   htmlsafe="(true false)"   jsafe="(true false)" ] /&gt; </pre>
<b>Iterator</b>		
	Realiza bucles y escribe valores de varios elementos.	<pre> &lt;m4:iterator [ m4rows=" [X "*"] " ] m4node="OUTDEF:ALIAS!NODE"&gt; &lt;m4:param name="M4ITEM0" value="ITEMA"/&gt; ... &lt;m4:param name="M4ITEMN" value="ITEMN"/&gt; ... [\$CURRENT\$] ... [\$COUNT\$] ... \$M4ITEM0\$ ... \$M4ITEMN\$ ... &lt;/m4:iterator&gt; </pre>

Etiqueta	Descripción	Sintaxis
<b>Job</b>		
	Inicia una transacción con la base de datos. Utilízela en lugar del par <b>&lt;m4:startjob&gt;</b> y <b>&lt;m4:endjob&gt;</b>.	<pre>&lt;m4:job [check="(ERROR WARNING SUCCESS)"         alias="DataTagAlias" url="ErrorPage"] &gt;     ... &lt;/m4:job&gt;</pre>
<b>Label</b>		
	Indica la descripción (nombre) de un elemento o nodo.	<pre>&lt;m4:label ( m4name=[OutputDefAlias]:DataTagAlias!NODE.ITEM"   outputdef="OutputDefAlias" get="item" item="ITEM"/&gt;)  &lt;m4:label ( m4name=[OutputDefAlias]:DataTagAlias!NODE[REC]"   outputdef="OutputDefAlias" get="node"/&gt;)</pre>
<b>Logout</b>		
	Finaliza una sesión activa.	<pre>&lt;m4:logout/&gt;</pre>
<b>Loop</b>		
	Permite la repetición de código en una estructura de bucles.	<pre>&lt;m4:loop from="i" to="j" [ step="inc"]   [ name = "IndexVarName"] &gt;     ... &lt;/m4:loop&gt;</pre>
<b>Move</b>		
	Se sitúa en la posición actual en varios grupos de registros.	<pre>&lt;m4:move&gt;     &lt;m4:param name="DataTagAlias_1"         value="OutputDefAlias_1:NODE[REC]#NODE[REC]"/&gt;     ...     &lt;m4:param name="DataTagAlias_m"         value="OutputDefAlias_n:NODE[REC]#NODE[REC]"/&gt; &lt;/m4:move&gt;  &lt;m4:move outputdef="OutputDefAlias" record="REC"/&gt;</pre>
<b>MSetApplParam</b>		
	Establece un nuevo valor para un determinado parámetro de la aplicación.	<pre>&lt;m4:setapplparam section="MY_SECTION" key="MY_KEY" value="MY_NEW_VALUE"/&gt;</pre>

Etiqueta	Descripción	Sintaxis
<b>Option</b>		
	Introduce una etiqueta HTML option en la página y establece el parámetro <b>selected</b> basándose en parte del cuerpo de la etiqueta.	<pre>&lt;m4:option [ value = "Value"   selected = "Selected"   match = "Match" ] &lt;/m4:option&gt;</pre>
<b>Outputcache</b>		
	Define la autoexpiración de la página.	<pre>&lt;m4:outputcache checkexpired="M4O_1,.. M4O_n"&gt; &lt;/m4:outputcache&gt;</pre>
<b>Outputdef</b>		
	Define la salida de trabajo.	<pre>&lt;m4:outputdef [m4alias="OutputDefAlias"]&gt;   &lt;m4:param name="M4NAME0" value="..." /&gt;   ....   &lt;m4:param name="M4NAMEN" value="..." /&gt; &lt;/m4:outputdef&gt;  &lt;m4:outputdef [m4alias="OutputDefAlias"] m4object="DataTagAlias" node="NODE" records="REC"/&gt;</pre>
<b>OutputExec</b>		
	Muestra el resultado de un método.	<pre>&lt;m4:outputexec ( m4alias="ExecAlias   alias="ExecAlias" ) [ m4varname = "NonDeclaredVariable"   var = "DeclaredVariable"   vartype = "VariableType"   m4format="FormatMask"   typename = "FormatType" /&gt;</pre>
<b>Page</b>		
	Define el entorno de una página.	<pre>&lt;m4:page subsessionid="SubsessionId" [preservelevel="X"   keepcontext="(TRUE FALSE)"]&gt; ... &lt;m4:page/&gt;</pre>
<b>Param</b>		
	Parámetro genérico con nombre y valor.	<pre>&lt;m4:param name="NAME" value="VALUE"/&gt;</pre>
<b>PutBagValue</b>		
	Inserta un par (clave, valor) en la bolsa del usuario.	<pre>&lt;m4:putbagvalue m4key="KEY" m4value="VALUE"/&gt;</pre>



Etiqueta	Descripción	Sintaxis
<b>RemoveBagValue</b>		
	Quita un par (clave, valor) de la bolsa del usuario.	<code>&lt;m4:removebagvalue m4key="KEY"/&gt;</code>
<b>RemoveFilter</b>		
	Quita un filtro de un trabajo.	<code>&lt;m4:removefilter m4name="FilterAlias"/&gt;</code>
<b>Removem4o</b>		
	Quita una definición de Meta4Object del servidor de aplicaciones.	<code>&lt;m4:removem4o m4oalias="DataTagAlias"/&gt;</code>
<b>SetAppiParam</b>		
	Establece un nuevo valor para un determinado parámetro de la aplicación.	<code>&lt;m4:setapplparam section="MY_SECTION" key="MY_KEY" value="MY_NEW_VALUE" /&gt;</code>
<b>SetCurrency</b>		
	Establece datos de moneda.	<code>&lt;m4:setcurrency m4name="DataTagAlias!NODE[REC].ITEM" m4currency_value="..." m4currency_id="..." [ m4currency_date="..." ] [ m4currency_type="..." ] /&gt;</code>
<b>SetFile</b>		
	Inserta un archivo en un elemento de tipo BLOB.	<code>&lt;m4:setfile m4blob="BLOB" m4path="PATH"/&gt;</code>
<b>SetItems</b>		
	Asigna valor a elementos.	<code>&lt;m4:setitems&gt; &lt;m4:param name="DataAlias1!NODEA.ITEMA" value="value"/&gt; ... &lt;m4:param name="DataAliasN!NODEN.ITEMN" value="value"/&gt; &lt;/m4:setitems&gt;</code>
<b>SetOrganization</b>		

Etiqueta	Descripción	Sintaxis
	Asigna una organización.	<code>&lt;m4:setorganization m4organization="ORGANIZATION"/&gt;</code>
<b>SetProperty</b>		
	Asigna la propiedad a M4Operations.	<code>&lt;m4:setproperty m4property="PROP" m4value="VAL"/&gt;</code>
<b>SetRole</b>		
	Asigna un rol.	<code>&lt;m4:setrole m4role="ROLE"/&gt;</code>
<b>SortItems</b>		
	Ordena varios elementos de modo ascendente o descendente.	<pre> &lt;m4:sortitems   m4name="DataTagAlias!NODE.SortFilterAlias"&gt;   &lt;m4:param name="ITEM1" value="ASC DESC"/&gt;   ...   &lt;m4:param name="ITEMN" value="ASC DESC"/&gt; &lt;/m4:sortitems&gt; </pre>
<b>StartPage</b>		
	Inicia una página JSP de Meta4.	<code>&lt;m4:startpage m4task="SubSessionID" [m4preservelevel="..."]/&gt;</code>
<b>Etiqueta</b>		
	Sustitución genérica para cualquier etiqueta HTML	<pre> &lt;m4:tag name="HTMLTagName"&gt; ... &lt;/m4:tag&gt; </pre>
<b>Xmltransformer</b>		
	Establece una propiedad en el entorno de ejecución.	<code>&lt;m4:xmltransformer m4xsl="XSL"/&gt;</code>

---

# Referencia del paquete M4Operations

# 10

## Introducción

En este manual de referencia, se analiza la API incluida en el paquete Java `com.meta4.m4operations`.

Este paquete está constituido por las siguientes clases:

- `M4Operations`
- `OperationException`
- `LogMessage`
- `SortElement`

Además, el paquete incluye las propiedades del archivo `OpenSessionEvents`.

Utilice este paquete para desarrollar funcionalidad en páginas JSP de clientes HTML. Cada clase incluye los siguientes elementos:

- Campos
- Constructores
- Métodos

En este manual no se comentan todos los métodos, campos y recursos, sólo se han incluido los más relevantes.

## Uso de m4operations en páginas JSP

Para utilizar las clases de m4operations en una página JSP, siga estos pasos:

1. Declare el paquete en la página.

---

**EJEMPLO**

```
<%@ page import="com.meta4.m4operations.*" %>
```

---

2. Abra un bloque Java.

---

**EJEMPLO**

```
<%
```

---

3. Declare las variables que se van a usar en la página y a las que se asignarán valores de m4operations.

---

**EJEMPLO**

```
int iActual = 0;
int iTotal = 0;
```

---

4. Rodee el código m4operations con un bloque try-catch para que no se produzcan excepciones.

---

**EJEMPLO**

```
try{
```

---

5. Instancie la clase usando la información del usuario que contiene el objeto request.

---

**EJEMPLO**

```
M4Operations m = new M4Operations(request);
```

---

---

6. Utilice las instancias del paquete `m4operations` creadas en el paso anterior para, por ejemplo, iniciar las variables declaradas.

---

#### EJEMPLO

```
iTotal = m.getCount("", "RECRUITMENT", "ESS_RECRUITMENT_ROOT");
iActual= m.getCurrent("", "RECRUITMENT", "ESS_RECRUITMENT_ROOT");
```

---

7. Cierre el bloque `try-catch` si no va a volver a utilizar el paquete `m4operations`.

---

#### EJEMPLO

```
} catch(Exception e) {}
```

---

8. Cierre el bloque Java.

---

#### EJEMPLO

```
%>
```

---

9. Utilice las variables Java con la sintaxis JSP correspondiente.

---

#### EJEMPLO

```
....
<%
 if ((iTotal > 0) && (iActual != iTotal-1))

>%
....
Número de datos: <%= String.valueOf(iTotal) %>
....
```

---

En el siguiente ejemplo, se utiliza el paquete `m4operations`, en lugar de la `M4Tag count`, para contar el número de registros que hay en un nodo de un `Meta4Object`.

---

**EJEMPLO OPER.JSP**

```
<%@ taglib uri="M4Tags" prefix="m4" %>
<%@ page import="com.meta4.m4operations.*" %>
<HTML>
<HEAD>
<TITLE> Países </TITLE>
</HEAD>

<BODY BGCOLOR="#FFFFFF">
<m4:startpage m4task="MYSUBSESSION"/>
<m4:beginjob/>
 <m4:datadef m4o="SCO_GEO_STRUCTURE" m4name="GEO3"/>
 <m4:exec m4method="LOAD:GEO3!STD_MT_COUNTRY.LOAD"/>
 <m4:outputdef m4alias="PARENT">
 <m4:param
 name="M4NAME0"
 value="GEO3!STD_MT_COUNTRY[*].STD_N_COUNTRY"/>
 </m4:outputdef>
<m4:endjob/>

<%
 int i=0;
 try{
 M4Operations mm = new M4Operations(request);
 i=mm.getCount("PARENT","GEO3","STD_MT_COUNTRY");
 }
 catch(Exception e){}
 out.println("N&umber of records: " + i);
%>

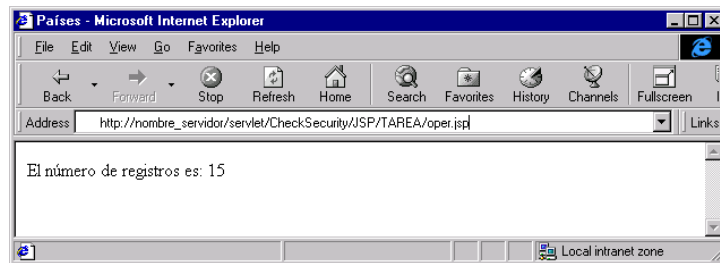
<m4:endpage/>
</BODY>
</HTML>
```

---

---

A continuación se muestra el resultado de este ejemplo:

Figura 164.Resultado de la página oper.jsp



# Clase M4Operations

## Campos

Los campos son constantes y variables.

### Ordenación de campos

Estas constantes indican el tipo de ordenación.

#### ASC

Ascendente.

```
public static java.lang.String
```

#### DESC

Descendente.

```
public static final java.lang.String
```

### Tipos de campo

Estas constantes indican el tipo de campo.

#### M4\_TYPE\_BINARY\_STRING

Campos que incluyen cadenas binarias.

```
public static final byte
```

#### M4\_TYPE\_BLOB

Campos que incluyen objetos de tipo BLOB.

```
public static final byte
```

#### M4\_TYPE\_CURRENCY

Campos que incluyen datos de moneda.

```
public static final byte
```



---

## **M4\_TYPE\_DATE**

Campos que almacenan fechas.

```
public static final byte
```

## **M4\_TYPE\_FIXED\_STRING**

Campos que incluyen cadenas fijas.

```
public static final byte
```

## **M4\_TYPE\_LONG**

Campos que incluyen enteros largos.

```
public static final byte
```

## **M4\_TYPE\_NULL**

Campos que incluyen datos nulos.

```
public static final byte
```

## **M4\_TYPE\_NUMBER**

Campos que contienen números.

```
public static final byte
```

## **M4\_TYPE\_NUM\_VARIANT**

Campos que incluyen números variables.

```
public static final byte
```

## **M4\_TYPE\_TIMESTAMP**

Campos que incluyen datos relativos a la hora.

```
public static final byte
```

## **M4\_TYPE\_VAR\_STRING**

Campos que incluyen cadenas variables.

```
public static final byte
```

## **M4\_TYPE\_VARIANT**

Campos que contienen variables.

```
public static final byte
```

## Campos de evento

Estos campos indican el evento.

### **ON\_ERROR**

Constante que indica que se ha producido un error.

```
public static final byte
```

### **ON\_EVENT**

Variable que especifica el tipo de evento.

```
public static final byte
```

### **ON\_SUCCESS**

Constante que indica que el proceso ha finalizado correctamente.

```
public static final byte
```

### **ON\_WARNING**

Constante que da un aviso.

```
public static final byte
```

## Campos de ejecución

Estos campos ajustan propiedades.

### **M4\_EXECUTOR\_DOPARSE**

Variable que analiza el modelo XML si se le asigna M4\_EXECUTOR\_TRUE.

```
public static final java.lang.String
```

### **M4\_EXECUTOR\_DOXML**

Variable que genera el modelo XML si se le asigna M4\_EXECUTOR\_TRUE.

```
public static final java.lang.String
```

---

### **M4\_EXECUTOR\_FALSE**

Constante que desactiva la generación y el análisis del modelo XML.

```
public static final java.lang.Boolean
```

### **M4\_EXECUTOR\_TIMEOUT**

Variable que especifica el tiempo de inactividad.

```
public static final java.lang.String
```

### **M4\_EXECUTOR\_TRUE**

Constante que activa la generación y el análisis del modelo XML.

```
public static final java.lang.Boolean
```

## **Campos de moneda**

Estas variables especifican valores de elementos moneda.

### **M4\_CURRENCY\_DATE**

Fecha de moneda.

```
public static final java.lang.String
```

### **M4\_CURRENCY\_ID**

Código identificativo de moneda.

```
public static final java.lang.String
```

### **M4\_CURRENCY\_TYPE**

Tipo de moneda.

```
public static final java.lang.String
```

### **M4\_CURRENCY\_VALUE**

Valor de moneda.

```
public static final java.lang.String
```

## Constructores

Los constructores permiten inicializar la clase M4Operations.

### Constructor de página JSP

Constructor que se utiliza en el código de las páginas JSP.

```
public M4Operations (HttpServletRequest
 request)
 throws OperationException
```

#### Parámetros:

- **request**: objeto HttpServletRequest.

### Constructor de etiquetas

Constructor que se utiliza desde las M4Tags.

```
public M4Operations (com.meta4.session.M4SessionManager aio_sess)
 throws OperationException
```

#### Parámetros:

- **aio\_sess**: objeto M4SessionManager que contiene todas las variables de sesión para la tarea actual.

## Métodos

Los métodos realizan diversas operaciones desde la página JSP del cliente HTML.

### addFilter

Filtra los datos de un nodo en la tarea actual.

```
public void addFilter (java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 java.lang.String ai_stLN4filter,
 java.lang.String ai_stNameFilter)
 throws OperationException
```

---

### Parámetros:

- **ai\_stChannelAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.
- **ai\_htLN4filter**: filtro LN4 que se va a aplicar.
- **ai\_stNameFilter**: nombre asignado al filtro.

## addSort

Ordena los datos de un nodo. Se puede ordenar por más de un elemento. Método en desuso.

```
public void addSort(java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 java.util.Hashtantle ai_htItemSort,
 java.lang.String ai_stNameSort)
throws OperationException
```

### Parámetros:

- **ai\_stChannelAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.
- **ai\_stNameSort**: nombre asignado a la ordenación.
- **ai\_htItemSort**: objeto hashtable de los elementos que se van a ordenar y tipo de ordenación (ascendente o descendente).

Ordena los datos de un nodo por un elemento, según especifica el objeto ai\_oItemSort.

```
public void addSort(java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 SortElement ai_oItemSort,
 java.lang.String ai_stNameSort)
throws OperationException
```

### Parámetros:

- **ai\_stChannelAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.
- **ai\_stNameSort**: nombre asignado a la ordenación.
- **ai\_oItemSort**: SortElement que indica el elemento y el tipo de ordenación.

## addTransCode

Define los códigos que se deben traducir en una tarea.

```
public void addTransCode(java.lang.String ai_stItem,
 java.lang.String ai_stTransName,
 java.util.Hashtable ai_htArgMethod)
 throws OperationException
```

### Parámetros:

- **ai\_stItem**: elemento que se va a traducir en el Meta4Object origen.
- **ai\_stTransName**: nombre de la traducción que se va a usar.
- **ai\_htArgMethod**: objeto hashtable de argumentos de la traducción (pares nombre argumento/valor).

## addTransType

Define una traducción.

```
public void addTransType(java.lang.String ai_stTransName,
 java.lang.String ai_stM4O,
 java.lang.String ai_stNode,
 java.lang.String ai_stMethod,
 java.lang.String ai_stDestItems)
 throws OperationException
```

### Parámetros:

- **ai\_stTransName**: nombre de la traducción.
- **ai\_stM4O**: Meta4Object auxiliar.
- **ai\_stNode**: nodo del Meta4Object auxiliar.
- **ai\_stMethod**: nombre del método encargado de la traducción.
- **ai\_stDestitems**: nombre del elemento en que se guarda la traducción.

## beginJob

Inicia una transacción.

```
public void beginJob()
 throws OperationException
```

---

## changePassword

Cambia la contraseña.

```
public int changePassword(java.lang.String ai_stOldPassword,
 java.lang.String ai_stNewPassword)
 throws OperationException
```

### Parámetros:

- **ai\_stOldPassword**: contraseña anterior.
- **ai\_stNewPassword**: contraseña nueva.

### Devuelve:

0 si se ha cambiado correctamente; 1 si la contraseña nueva coincide con el nombre de usuario, y -1 si se ha producido algún error.

## changeCurrency

Establece los elementos de moneda para el Meta4Object de sesión. Para realizar esta operación, el cambio de ID\_CURRENCY emplea la subsesión SESSION.

```
public void changeCurrency(java.lang.String ai_stIdCurrency,
 java.lang.String ai_stDtchange,
 java.lang.String ai_stExType)
 throws OperationException
```

### Parámetros:

- **ai\_stIdCurrency**: código identificativo de moneda.
- **ai\_stIdCurrency**: fecha de cambio. Debe incluirse en formato ISO; por ejemplo, 2000-12-31.
- **ai\_stIdCurrency**: fuente de cambio (oficial o legal).

## checkError

Comprueba errores posteriores a la ejecución de la tarea.

```
public boolean checkError(byte ai_iType)
```

**Parámetros:**

- **ai\_iType**: tipo de evento que se va a comprobar. Consulte la lista de eventos en el apartado *Campos, Campos de evento*.

**Devuelve:**

El valor *TRUE*, si se ha producido un error del tipo especificado.

Comprueba errores posteriores a la ejecución de la tarea:

```
public boolean checkError(byte ai_iType,
 java.util.Vector aio_vMessages)
```

**Parámetros:**

- **ai\_iType**: tipo de evento que se va a comprobar.
- **aio\_vMessages**: vector de mensajes (objeto LogMessage) que se muestra internamente para el evento.

**Devuelve:**

El valor *TRUE*, si se ha producido un error del tipo especificado.

Una vez introducido el alias de un método, comprueba errores posteriores a la ejecución de una tarea.

```
public boolean checkError(byte ai_iType,
 java.lang.String ai_stMethodAlias,
 java.util.Vector aio_vMessages)
```

**Parámetros:**

- **ai\_iType**: tipo de evento que se va a comprobar.
- **ai\_stMethodAlias**: alias de método especificado en una tarea.
- **aio\_vMessages**: mensajes que se muestran internamente para el evento.

**Devuelve:**

El valor *TRUE*, si se ha producido un error del tipo especificado.



---

## createData

Asigna un alias al Meta4Object.

```
public void createData(java.lang.String ai_stAlias,
 java.lang.String ai_stM4O,
 java.util.Hashtable ai_htParams)
 throws OperationException
public void createData(java.lang.String ai_stAlias,
 java.lang.String ai_stM4O,
 boolean ai_bPreserve,
 java.util.Hashtable ai_htParams)
 throws OperationException
public void createData(java.lang.String ai_stAlias,
 java.lang.String ai_stM4O,
 boolean ai_bPreserve,
 java.util.Hashtable ai_htParams,
 java.lang.String ai_sInfoLevel))
 throws OperationException
```

### Parámetros:

- **ai\_stAlias**: alias definido por el usuario.
- **ai\_stM4O**: nombre interno del Meta4Object.
- **ai\_htParams**: lista interna de parámetros de este alias.
  - M4:START\_DATE: fecha de inicio del metadato.
  - M4:END\_DATE: fecha de fin del metadato.
  - M4:ID\_RSM: tipo de RSM del Meta4Object.
  - M4:ID\_APP\_ROLE: rol del Meta4Object.
  - M4:ID\_ORGANIZATION: organización.
- **ai\_bPreserve**: estado de mantenimiento del Meta4Object. Puede tomar los valores *TRUE* o *FALSE*. El valor por defecto es *TRUE*.
- **ai\_sInfoLevel**: el único valor posible es 0, que le permite ver el conjunto de datos y métodos.

## delRegister

Borra el registro actual.

```
public void delRegister(java.lang.String ai_stAlias,
 java.lang.String ai_stNode)
 throws OperationException
```

### Parámetros:

- **ai\_stAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.

## endJob

Finaliza una transacción.

```
public void endJob()
 throws OperationException
```

Para ello ejecuta las operaciones definidas previamente. Se pueden usar las funciones `checkError` y `getLastError` para comprobar si se ha producido un error.

```
public void endJob(java.lang.String ai_stDebugInfo)
 throws OperationException
```

### Parámetros:

- **ai\_stDebugInfo**: información adicional que se guarda en el servidor.

## endTask

Finaliza la tarea actual.

```
public void endTask()
 throws OperationException
```

---

## execMethod

Una vez indicado el alias de un método, devuelve el resultado. Método en desuso.

```
public void execMethod(java.lang.String ai_stMethodAlias,
 java.lang.StringBuffer ao_sbResult,
 java.util.Vector ao_vType)
 throws OperationException
```

### Parámetros:

- **ai\_stMethodAlias:** referencia a un método de una tarea.
- **ao\_sbResult:** resultado del método.
- **ao\_vType:** tipo de resultado del método.

Una vez indicado el alias de un método, devuelve el resultado.

```
public byte execMethod(java.lang.String ai_stMethodAlias,
 java.lang.StringBuffer ao_sbResult)
 throws OperationException
```

### Parámetros:

- **ai\_stMethodAlias:** referencia a un método de una tarea.
- **ao\_sbResult:** tipo de resultado del método (ver el apartado *Campos, Tipos de campo*).

### Devuelve:

Tipo de resultado del método.

## getAuxiliarItem

```
public java.lang.String getAuxiliarItem(java.lang.String
 ai_stItemAlias,
 java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 java.lang.String ai_stReg,
 java.lang.String ai_stItemName)
 throws OperationException
```

## getCount

Devuelve el número de elementos recuperados en el servidor.

```
public int getCount(java.lang.String ai_stItemAlias,
 java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode)
throws OperationException
```

### Parámetros:

- **ai\_stItemAlias**: elemento especificado mediante el método outputDef.
- **ai\_stChannelAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.

### Devuelve:

El número de elementos recuperados en el servidor:

## getCountInClient

Devuelve el número de elementos recuperados en el conjunto local.

```
public int getCountInClient(java.lang.String ai_stItemAlias,
 java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode)
throws OperationException
```

### Parámetros:

- **ai\_stItemAlias**: elemento especificado mediante el método outputDef.
- **ai\_stChannelAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.

### Devuelve:

El número de elementos recuperados en el conjunto local.

---

## getCredential

Obtiene la credencial actual.

```
public java.lang.String getCredential()
 throws OperationException
```

### Devuelve:

La credencial asociada.

## getCurrencyValues

Recupera los datos de un elemento de tipo moneda.

```
public void getCurrencyValues(java.lang.String ai_stItemAlias,
 java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 java.lang.String ai_stReg,
 java.lang.String ai_stItemName,
 java.util.Hashtable ao_htArgCurrency)
 throws OperationException
```

### Parámetros:

- **ai\_stItemAlias**: elemento especificado mediante el método outputDef.
- **ai\_stChannelAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.
- **ai\_stReg**: índice de registro.
- **ai\_stItemName**: nombre del elemento.
- **ao\_htArgCurrency**: objeto hashtable con los datos recuperados.

## getCurrent

Devuelve la posición actual dentro de los datos recuperados.

```
public int getCurrent(java.lang.String ai_stItemAlias,
 java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode)
throws OperationException
```

### Parámetros:

- **ai\_stItemAlias**: elemento especificado mediante el método outputDef.
- **ai\_stChannelAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.

### Devuelve:

El índice actual.

## getFile

Recupera un objeto BLOB guardado como elemento de un Meta4Object.

```
public java.lang.String getFile(java.lang.String ai_stItemAlias,
 java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 java.lang.String ai_stReg,
 java.lang.String ai_stItemName)
throws OperationException
```

### Parámetros:

- **ai\_stItemAlias**: elemento especificado mediante el método outputDef.
- **ai\_stChannelAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.
- **ai\_stReg**: índice de registro.
- **ai\_stItemName**: nombre del elemento.

### Devuelve:

La ruta del archivo en el servidor.

---

## getInfoSessionM4Object

Obtiene información de interés del Meta4Object de sesión. Sólo se puede invocar desde un servidor Web (o sea, si se realiza una petición).

```
public void
getInfoSessionM4Object(javax.servlet.http.HttpServletRequest request,
 java.lang.StringBuffer ao_sbCurrency,
 java.lang.StringBuffer ao_sbExType,
 java.lang.StringBuffer ao_sbOrganization,
 java.lang.StringBuffer ao_sbRole)
 throws RuntimeException
```

### Parámetros:

- **request**: petición.
- **ao\_sbCurrency**: código identificativo de moneda.
- **ao\_sbExType**: fuente de cambio de la moneda (oficial o legal).
- **ao\_sbOrganization**: código identificativo de la organización.
- **ao\_sbRole**: rol.

## getItem

Recupera el valor de un elemento.

```
public java.lang.String getItem(java.lang.String ai_stItemAlias,
 java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 java.lang.String ai_stReg,
 java.lang.String ai_stItemName)
 throws RuntimeException
```

### Parámetros:

- **ai\_stItemAlias**: elemento especificado mediante el método outputDef.
- **ai\_stChannelAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.
- **ai\_stReg**: índice de registro.
- **ai\_stItemName**: nombre del elemento.

**Devuelve:**

El valor asociado al elemento especificado.

## getItemType

Devuelve el tipo interno de un elemento.

```
public java.lang.String getItemType(java.lang.String ai_stItemAlias,
 java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 java.lang.String ai_stItemName)
throws OperationException
```

**Parámetros:**

- **ai\_stItemAlias**: elemento especificado mediante el método outputDef.
- **ai\_stChannelAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.
- **ai\_stItemName**: nombre del elemento.

**Devuelve:**

El tipo de elemento (vea el apartado *Campos, Tipos de campo*).

## getLabel

Devuelve la descripción de un elemento, nodo o Meta4Object.

```
public java.lang.String getLabel(java.lang.String ai_stItemAlias,
 java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 java.lang.String ai_stName)
throws OperationException
```

**Parámetros:**

- **ai\_stItemAlias**: elemento especificado mediante el método outputDef.
- **ai\_stChannelAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.
- **ai\_stName**: nombre del elemento.



---

**Devuelve:**

La descripción de un elemento, nodo o Meta4Object.

**getLastError**

Devuelve el último error durante la transacción interna.

```
public java.lang.String getLastError()
```

**Devuelve:**

El último error.

**getTaskAvailable**

Obtiene un objeto hashtable con los códigos identificativos y nombres de las tareas a las que va a obtener acceso el usuario. Para que funcione este método, se debe haber conectado desde un servidor Web (no independiente).

```
public java.util.Hashtable
getTaskAvailable(javax.servlet.http.HttpServletRequest request)
```

**Parámetros:** objeto request.

**Devuelve:**

Objeto hashtable con el código identificativo y el nombre de la tarea;  
RET\_ERROR\_MALFORMED\_STRING (el alias o el código del Meta4Object no son cadenas válidas); RET\_ERROR\_ELEMENT\_EXIST (el alias que se va a eliminar ya existe en esta ejecución).

**getTransCode**

Escribe la traducción de códigos después de la ejecución de la tarea.

```
public java.lang.String getTransCode(java.lang.String ai_stItemAlias,
 java.lang.String ai_stItemSpec)
throws OperationException
```

**Parámetros:**

- **ai\_stItemAlias:** elemento especificado mediante el método outputDef.
- **ai\_stItemSpec:** elemento que se va a traducir. Debe corresponder al elemento especificado con addTransCode en una tarea anterior.

## getXMLModel

Devuelve el modelo XML.

```
public java.lang.String getXMLModel()
```

### Devuelve:

Si el valor de M4\_EXECUTOR\_DOXML es *TRUE*, devuelve el modelo XML. En caso contrario, devuelve *null*.

## initTask

### Subsesión

Inicia una subsesión. Se utilizó en el *taglet* StartPage.

```
public void initTask(java.lang.String ai_stSubs)
 throws OperationException
```

### Parámetros:

- **ai\_stSubs**: nombre de subsesión o tarea.

### Tarea

Inicia una tarea.

```
public void initTask(java.lang.String ai_stSubs)
 throws OperationException
public void initTask(java.lang.String ai_stSubs,
 java.lang.String ai_stURL)
 throws OperationException
```

### Parámetros:

- **ai\_stSubs**: nombre de tarea.
- **ai\_stURL**: asocia la tarea a una página JSP.

---

## isDataCacheable

Devuelve .

```
public java.lang.String isDataCacheable()
```

### Devuelve:

Devuelve *TRUE* si el valor de la propiedad del Meta4Object Metadatos accesible en caché está definido y vale 1.

## load

Carga un Meta4Object usando su alias.

```
public boolean load(java.lang.String ai_stAlias)
 throws OperationException
```

### Parámetros:

- **ai\_stAlias:** alias del Meta4Object que se va a cargar, definido con el método createData.

### Devuelve:

El estado de la carga:

- *TRUE*: carga correcta.
- *FALSE*: carga errónea.

## login

### Con Parámetros

Inicio de sesión con parámetros.

```
public java.lang.String login(java.lang.String ai_stUser,
 java.lang.String ai_stPass,
 java.lang.String ai_stLanguage,
 java.lang.String ai_stLogonServer,
 java.lang.String ai_stPortServer,
 boolean ai_bRol,
 java.lang.String ai_stCertificate,
 boolean ai_bUseSSL)
 throws OperationException
```

**Parámetros:**

- **ai\_stUser:** usuario.
- **ai\_stPass:** contraseña.
- **ai\_stLanguage:** idioma.
- **ai\_stLogonServer:** nombre descriptivo del servidor de aplicaciones
- **ai\_stPortServer:** puerto del servidor de aplicaciones.
- **ai\_bRol:** si su valor es *TRUE*, elige el rol por defecto.
- **ai\_stCertificate:** certificado SSL.
- **ai\_bUseSSL:** si el valor es *TRUE*, activa el protocolo SSL.

**Devuelve:**

La credencial asociada.

**Con Credencial**

Inicio de sesión mediante una credencial.

```
public java.lang.String login(java.lang.String ai_stLogonServer,
 java.lang.String ai_stPortServer,
 java.lang.String ai_stCredential)
 throws OperationException
```

**Parámetros:**

- **ai\_stLogonServer:** nombre descriptivo del servidor de aplicaciones
- **ai\_stPortServer:** puerto del servidor de aplicaciones.
- **ai\_stCredential:** credencial.

**Devuelve:**

La credencial asociada.

---

## logout

Cierra la sesión actual.

```
public int logout()
```

### Devuelve:

El estado de finalización:

- **0**: salida correcta
- **-1**: salida errónea

## method

Ejecuta un método en la transacción actual.

```
public void method(java.lang.String ai_stMethodAlias,
 java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 java.lang.String ai_stMethod,
 java.util.Hashtable ai_htArgs)
throws OperationException
```

### Parámetros:

- **ai\_stMethodAlias**: alias del método definido por el usuario; se usa posteriormente en execmethod.
- **ai\_stChannelAlias**: alias del Meta4Object.
- **at\_stNode**: nodo del Meta4Object.
- **ai\_stMethod**: nombre del método.
- **ai\_htArgs**: objeto hashtable de argumentos.

## moveData

Mueve datos antes o después de que se ejecute una tarea:

```
public void moveData(java.lang.String ai_stItemAlias,
 java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 java.lang.String ai_stReg)
throws OperationException
```

**Parámetros:**

- **ai\_stItemAlias**: alias del método outputDef.
- **ai\_stChannelAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.
- **ai\_stReg**: destino.

**newRegister**

Crea un nuevo registro.

```
public void newRegister(java.lang.String ai_stAlias,
 java.lang.String ai_stNode)
 throws OperationException
```

**Parámetros:**

- **ai\_stAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.

**outputDef**

Define los datos disponibles después de ejecutar el bloque de transacciones.

```
public void outputDef(java.lang.String ai_stItemAlias,
 java.lang.String ai_stItemSpec)
 throws OperationException
```

**Parámetros:**

- **ai\_stItemAlias**: nombre que se relaciona con los datos que se van a cargar. Si no se especifica, se toma un valor por defecto.
- **ai\_stItemSpec**: datos que se van a recuperar.

---

## **persist**

Guarda el estado actual del Meta4Object.

```
public boolean persist(java.lang.String ai_stAlias)
 throws OperationException
```

### **Parámetros:**

- **ai\_stAlias**: alias del Meta4Object.

### **Devuelve:**

El estado de almacenamiento.

## **removeFilter**

Elimina el filtro y la ordenación establecidos previamente en la definición de una tarea.

```
public void removeFilter(java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 java.lang.String ai_stNameFilter)
 throws OperationException
```

### **Parámetros:**

- **ai\_stChannelAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.
- **ai\_stNameFilter**: alias del filtro que desea eliminar.

## **removeM4OAlias**

Elimina el alias de un Meta4Object y la instancia asociada al mismo.

```
public short removeM4OAlias(java.lang.String ai_stAlias)
 throws OperationException
```

### **Parámetros:**

- **ai\_stAlias**: alias de la instancia de Meta4Object que se va a utilizar.

**Devuelve:**

RET\_OK, RET\_ERROR\_MALFORMED\_STRING (el alias o el código del Meta4Object no son cadenas válidas); RET\_ERROR\_ELEMENT\_EXIST (el alias que se va a eliminar ya existe en esta ejecución).

**setCurrencyValues**

Asigna los datos de un elemento de tipo moneda.

```
public void setCurrencyValues(java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 java.lang.String ai_stReg,
 java.lang.String ai_stItemName,
 java.util.Hashtable ai_htArgCurrency)
 throws OperationException
```

**Parámetros:**

- **ai\_stChannelAlias:** alias del Meta4Object.
- **ai\_stNode:** nodo del Meta4Object.
- **ai\_stReg:** índice de registro.
- **ai\_stItemName:** nombre del elemento.
- **ai\_htArgCurrency:** objeto hashtable con datos de moneda.

**setCurrent**

Asigna la posición dentro de los datos antes de una ejecución (endJob).

```
public int setCurrent(java.lang.String ai_stItemAlias,
 java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 int ai_iPos)
 throws OperationException
```

**Parámetros:**

- **ai\_stItemAlias:** elemento especificado mediante el método outputDef.
- **ai\_stChannelAlias:** alias del Meta4Object.
- **ai\_stNode:** nodo del Meta4Object.
- **ai\_iPos:** nueva ubicación.



---

### Devuelve:

El índice actual después del movimiento.

## setFile

Inserta un archivo como elemento de Meta4Object.

```
public void setFile(java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 java.lang.String ai_stReg,
 java.lang.String ai_stName,
 java.lang.String ai_stPathName)
throws OperationException
```

### Parámetros:

- **ai\_stChannelAlias**: Meta4Object destino.
- **ai\_stNode**: nodo destino.
- **ai\_stReg**: índice destino.
- **ai\_stName**: elemento destino.
- **ai\_stPathName**: ruta local del archivo.

## setItem

Modifica el valor de un elemento en una tarea.

```
public void setItem(java.lang.String ai_stChannelAlias,
 java.lang.String ai_stNode,
 java.lang.String ai_stReg,
 java.lang.String ai_stName,
 java.lang.String ai_stValue)
throws OperationException
```

### Parámetros:

- **ai\_stChannelAlias**: alias del Meta4Object.
- **ai\_stNode**: nodo del Meta4Object.
- **ai\_stReg**: registro que se va a modificar.
- **ai\_stname**: elemento que se va a modificar.
- **ai\_stValue**: valor que se va a insertar.

## setOrganization

Establece la organización para el Meta4Object de sesión, que permite trabajar en modo multisociedad. Para realizar esta operación, el cambio de ID\_ORGANIZATION emplea la subsesión SESSION.

```
public void setOrganization(java.lang.String ai_stIdOrganization)
 throws OperationException
```

### Parámetros:

- **ai\_stIdOrganization**: código identificativo de la organización.

## setProperties

Cambia las propiedades XML.

```
public void setProperties(java.util.Hashtable ai_htProperties)
```

### Parámetros:

- **ai\_htProperties**: tabla de propiedades.

## unLoad

Descarga un Meta4Object usando su alias.

```
public boolean unLoad(java.lang.String ai_stAlias)
 throws OperationException
```

### Parámetros:

- **ai\_stAlias**: alias del Meta4Object que se va a cargar, definido con el método createData.

### Devuelve:

El estado de descarga:

- *TRUE*: descarga correcta.
- *FALSE*: descarga errónea.

---

## Clase OperationException

Esta clase hereda la clase java.lang.Exception.

### Constructores

Lanza las excepciones de la clase M4Operations.

```
public OperationException (java.lang.String ai_stInfo)
public OperationException (java.lang.String ai_stInfo,
 java.lang.String ai_stCode)
```

#### Parámetros:

- ai\_stInfo: información guardada en el mensaje de excepción.
- ai\_stCode: código de la excepción.

### Métodos

#### getCode

Obtiene el código de la excepción.

```
public java.lang.String getcode()
```

## Clase LogMessage

### Constructores

Encapsulan los mensajes internos generados durante la ejecución de la aplicación.

```
public LogMessage()
public LogMessage(com.meta4.m4connect.M4ConnLogMessage logMesg)
public LogMessage(java.lang.String strTitle,
 java.lang.String strMsg,
 java.lang.String strSev,
 java.lang.String strCode)
```

#### Parámetros:

- logMesg: objeto de la clase com.meta4.m4connect.M4ConnLogMessage.
- strTitle: título.
- strMsg: descripción.
- strSev: gravedad. Ejemplo de códigos de gravedad que se usan:
  - com.meta4.M4ConnConstants.LOG\_SEVERITY\_WARNING
  - com.meta4.M4ConnConstants.LOG\_SEVERITY\_ERROR
  - com.meta4.M4ConnConstants.LOG\_SEVERITY\_DEBUG
  - com.meta4.M4ConnConstants.LOG\_SEVERITY\_UNDEFINED
- strCode  
código de error. Se utilizan códigos de mensajes de error numéricos de Meta4® (65536, 65543...).

### Métodos

#### getCode

Recoge el valor del miembro strCode del objeto.

```
public java.lang.String getCode()
```

---

## **getSeverity**

Recoge el valor del miembro strSev del objeto.

```
public java.lang.String getSeverity()
```

## **getTitle**

Recoge el valor del miembro strTitle del objeto.

```
public java.lang.String getTitle()
```

## **getDescription**

Recoge el valor del miembro strMsg del objeto.

```
public java.lang.String getDescription()
```

## Clase SortElement

### Campos

Estas constantes indican el tipo de ordenación.

#### ASC

Ascendente.

```
public static final byte
```

#### DESC

Descendente.

```
public static final byte
```

### Constructores

#### SortElement

Ordena elementos.

```
public SortElement(java.lang.String ai_stItem,
 byte ai_bType)
```

#### Parámetros:

- `ai_stItem`: elemento que se va a ordenar.
- `ai_bType`: tipo. Puede ser *ASC* o *DESC*.

### Métodos

#### getItem

Recoge el valor del miembro `strItem` del objeto.

```
public java.lang.String getItem()
```

# Apéndice: Especificación de datos



La siguiente tabla constituye una referencia rápida de las notaciones empleadas para recuperar un registro de un elemento de un nodo.

Notación	Recupera
A	Elemento A de la instancia de Meta4Object, nodo y registro actuales.
A!B	Elemento de sistema B del Meta4Object A (por ejemplo, A!LOAD).
A[B]	Todos los elementos de la instancia de Meta4Object actual, nodo A, registro B.
A[B].C	Elemento C de la instancia de Meta4Object actual, nodo A, registro B.
A!B[C]	Todos los elementos de la instancia de Meta4Object A, nodo B, registro C.
A!B[+C]	Todos los elementos de la instancia de Meta4Object A, nodo B, registro C (número anteriores al registro actual).
A!B[-C]	Todos los elementos de la instancia de Meta4Object A, nodo B, registro C (número posteriores al registro actual).
A!B[C-D]	Todos los elementos de la instancia de Meta4Object A, nodo B, registros del C al D (enteros, ambos incluidos).
A!B[0]	Todos los elementos de la instancia de Meta4Object A, nodo B, primer registro.
A!B[last]	Todos los elementos de la instancia de Meta4Object A, nodo B, último registro.
A!B[*]	Todo el nodo B de la instancia de Meta4Object A.
A!B[C].D	Elemento D de la instancia de Meta4Object A, nodo B, registro C.
A!B[*].D	Elemento D de la instancia de Meta4Object A, nodo B, todos los registros.
A!B.D	Elemento D de la instancia de Meta4Object A, nodo B, registro actual.





---

# Apéndice: Referencia de sintaxis del formato



## Fuentes

### Sintaxis del formato de horas

Para obtener más información sobre el sintaxis del formato de horas, consulte la página **Java2 API API SimpleDateFormat** en:

<http://java.sun.com/j2se/1.3/docs/api/java/text/SimpleDateFormat.html>

### Sintaxis del formato de números

Para obtener más información sobre el sintaxis del formato de números, consulte la página **Java2 API DecimalFormat** en:

<http://java.sun.com/j2se/1.3/docs/api/java/text/DecimalFormat.html>

### Sintaxis del formato de expresiones de cadena

Para obtener más información sobre el sintaxis del formato de expresiones de cadena, consulte la documentación de la **biblioteca Jakarta-ORO** en:

<http://www.savarese.org/oro/developers/docs/OROMatcher/Syntax.html>